Theoretical Toys

Xiao Liang
https://xiao-liang.com

(Updated on August 23, 2026)

Chapter 1

Approximation

Some Notations. We use $f(x) \sim g(x)$ to denote the fact that $f(x) = g(x)(1 + o(1))$.

1.1 The Uncommon Friends of Big O

1.1.1 Soft-O, Quasi-Linear

The Soft-O notation $\widetilde{O}(n)$ is a variant of the big-O that "ignores" logarithmic factors. Formally, $f(n) \in \widetilde{O}\big(g(n)\big)$ if and only if there exists a constant k s.t. $O\big(g(n)\log^k g(n)\big)$.

Alternatively, one can use the following definition (e.g., see the introduction of [CKPR01]): $\widetilde{O}$

Definition 1.1.1: Soft-O and Soft-Omega

We define $\widetilde{O}$ and $\widetilde{\Omega}$ as follows:

- $f(n) = \widetilde{O}(h(n))$ if there exist constants $c_1, c_2 > 0$ so that for all sufficiently large n, it holds that $f(n) \le c_1 \cdot h(n) \cdot (\log h(n))^{c_2}$.
- $f(n) = \widetilde{\Omega}(h(n))$ if there exist constants $c_1, c_2 > 0$ so that for all sufficiently large n, it holds that $f(n) \ge \frac{c_1 \cdot h(n)}{(\log h(n))^{c_2}}$.

It is worth noting that for any constant k and any ε, $\log^k(n) \in O(n^{\varepsilon})$. Therefore, the soft-O notation is often used to obviate the "nitpicking" within growth-rates that are stated as too tightly bounded for the matters at hand.

Quasi-linear time is defined as $t(n) = \widetilde{O}(n)$; in particular, $t(n) = O(n \log n)$ is called *linearithmic time*. Note that if $t(n)$ is quasi-linear, then $t(n) \in O(n^{1+\varepsilon})$ for every constant $\varepsilon > 0$.

1.1.2 Quasi-Polynomial, Sub-Exponential and Complexity Leveraging

Quasi-polynomial time algorithms are algorithms that run longer than polynomial time, yet not so long as to be exponential time. The worst case running time of a quasi-polynomial time algorithm is $2^{O(\log^c n)}$ for some fixed $c > 0$.

Sub-exponential time is closely related to quasi-polynomial time. The precise definition of "sub-exponential" is not generally agreed upon[1].

[1] See this blog post by Scott Aaronson

Definition 1.1.2: Sub-Exponential Time

Following are two most widely used definition for sub-exponential time:

1. Function $f(n)$ is sub-exponential if $f(n) \in O(2^{n^\varepsilon})$ for every $\varepsilon > 0$.
2. **(Cryptographers'.)** Function $f(n)$ is sub-exponential if $f(n) \in 2^{o(n)}$.

The first one is used in the olden literature of complexity theory, e.g., [BFNW93, Mil01]. The second formalism is more modern, e.g., [IP01, Reg04, Kup05]. Cryptographers seem to prefer the second one. Indeed, the second one captures the running time of the fastest known classical factoring algorithm (as well as that of the fastest known algorithm for graph isomorphism).

Complexity leveraging is a useful technique in cryptography [CGGM00]. It relies on sub-exponential assumptions (the second one in Definition 1.1.2). This technique is best demonstrated by the construction of 2-round SPS ZK protocol from [Pas04, Section 3.5].[2] Xiao: Give an overview of this example? Xiao!

1.1.3 The Iterated Logarithm

The iterated logarithm $\log^*(n)$ can be defined by the following recursive formula:

$$\log^*(n) := \begin{cases} 0 & n \leq 1 \\ 1 + \log^*\big(\log^* n\big) & n > 1 \end{cases}.$$

Intuitively, $y = \log^*(n)$ denotes the number of times the logarithm function must be *iteratively* applied to n before the result is ≤ 1. That is,

$$\underbrace{b^{b^{\cdot^{\cdot^{b}}}}}_{y-1 \text{ times}} \leq n \leq \underbrace{b^{b^{\cdot^{\cdot^{b}}}}}_{y \text{ times}}.$$

Xiao: Talk about [Wee10] as an example. Xiao!

1.2 Useful Asymptotics

1.2.1 Harmonic Numbers

Harmonic number is defined as $H_n = \sum_{i=0}^{n} \frac{1}{i}$. The following exact bound can be proved by the "integral trick".

$$\ln(n+1) \leq H_n \leq \ln(n) + 1.$$

This also implies that H_n is approximately $\ln(n)$, i.e. $H_n \sim \ln(n)$.

The proof of the following fact is left as a simple exercise [Hint: collecting adjacent items in a "binary fashion"]:

$$\lfloor \log n \rfloor + 1 \leq H_n \leq \frac{1}{2}\lceil \log n \rceil + 1$$

The main take-away is: $H_n = \Theta\big(\ln(n)\big)$.

[2] This is the same construction in [Pas03, Section 5]

1.2.2 Some Asymptotics from Taylor Series

Let us recall the following Maclaurin Series (Taylor expansion at the origin point $a = 0$) with some interesting implications (since we are talking about Maclaurin series, imagine that x is very close to 0 in the following):

- $\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \cdots$. (It converges for $x \in (-1, 1]$). This implies that $\ln(1+x) \sim x$ when $x \to 0$.
- $e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots$. (It converges for all $x \in \mathbb{R}$). This implies that $e^x \sim 1 + x$ when $x \to 0$. A quick way to remember this is: this is the exponential version of the above $\ln(1+x) \sim x$.
- $\frac{1}{1-x} = 1 + x + x^2 + x^3 + \ldots$. (It converges for $x \in (-1, 1)$). This implies that $\frac{1}{1-x} \sim 1 + x$ when $x \to 0$.

These examples show how we can get helpful Computer-Science asymptotics from Maclaurin series. More Maclaurin expansion can be found at this Wikipedia page. In the following, we states more useful asymptotics obtained by this approach:

- $\frac{1}{1-\varepsilon} = 1 + \varepsilon \pm O(\varepsilon^2)$
- $(1+\varepsilon)^{\frac{1}{2}} = 1 + \frac{1}{2}\varepsilon \pm O(\varepsilon^2)$

Remark 1.2.1: On the Usage of Big-O

Note that the above use of Big-O notations is different from the standard usage that captures the behavior of an increasing function when x goes to infinity (called "Infinite Asymptotics"). Instead, it is used here to describe a decreasing function on a variable x approaching 0. Such an usage is called "Infinitesimal Asymptotics". See this Wikipedia page for an explanation. We remark that both usages can be unified under the same formal definition of the Big-O notation (via the limit superior).

1.2.3 Stirling's Formula

We want to study the asymptotic behavior of $n!$. We start with the following simply approach.

Taking the logarithm of it and applying the "integral trick" give us the following sharp bounds:

$$n\ln(n) - n + 1 \quad \leq \quad \ln(n!) \quad \leq \quad n\ln(n) - n + 1 + \frac{1}{2}\ln(n), \tag{1.1}$$

where the upper bound requires the clever trick that we collect the extra triangle remainders above the $\ln(n)$ curve to a rectangle that is parallel to y-axis.

Equation (1.1) immediately implies the following sharp bounds:

$$\left(\frac{n}{e}\right)^n e \quad \leq \quad n! \quad \leq \quad \left(\frac{n}{e}\right)^n e\sqrt{n} \tag{1.2}$$

Equation (1.2) also implies:

$$n! = \widetilde{\Theta}\left(\left(\frac{n}{e}\right)^n\right).$$

This result is already very close to the ground truth. Actually, we can show

$$n! = \Theta\left((\frac{n}{e})^n \sqrt{n} \right),$$

by proving that the size of the slivers we dropped in the derivation of the upper bound in Equation (1.1) actually converges to some constant.

Xiao!

Xiao: Prove Stirling's formula:

$$n! \sim \sqrt{2\pi n} (\frac{n}{e})^n \tag{1.3}$$

More exactly, it is

$$n! = \sqrt{2\pi n} (\frac{n}{e})^n \Big(1 + O(\frac{1}{n}) \Big). \tag{1.4}$$

1.3 Bounds for Binomial Coefficients

Useful Equalities for Binomial Coefficients. We first presents a set of widely used equalities regarding binomial coefficients. For all integers n, k, and t such that the following terms are well-defined, we have:

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k} \tag{1.5}$$

$$\binom{n}{k} = \frac{n}{k} \binom{n-1}{k-1} \tag{1.6}$$

$$\binom{n}{k} \binom{n-k}{t} = \binom{n}{t} \binom{n-t}{k} \tag{1.7}$$

The Deathbed Formula. Even if someone asks you about this formula on your deathbed, you should be able to spell it out without thinking.

$$\frac{n^k}{k^k} \leq \binom{n}{k} \leq \frac{n^k}{k!} \leq \frac{n^k}{k^k} \cdot e^k \tag{1.8}$$

Subsets Non-Overlapping. Another useful bound that appears again and again in cryptographic applications is the following one:

Lemma 1.3.1: Subsets Non-Overlapping

Let $k < n$ and $t < (n-k)$. Then, we have

$$\frac{\binom{n-k}{t}}{\binom{n}{t}} \leq \left(1 - \frac{k}{n} \right)^t \text{ and } \frac{\binom{n-k}{t}}{\binom{n}{t}} \leq \left(1 - \frac{t}{n} \right)^k \tag{1.9}$$

Proof. The proof of **??** is rather simple:

$$\begin{aligned}
\frac{\binom{n-k}{t}}{\binom{n}{t}} &= \frac{(n-k)!}{t!(n-k-t)!}\frac{(n-t)!t!}{n!} = \frac{(n-k)!}{(n-k-t)!}\frac{(n-t)!}{n!} \\
&= (n-k)(n-k-1)\cdots(n-k-t+1)\cdot\frac{1}{n(n-1)\cdots(n-t+1)} \\
&= \frac{n-k}{n}\cdot\frac{n-k-1}{n-1}\cdots\frac{n-k-t+1}{n-t+1} = \left(1-\frac{k}{n}\right)\cdot\left(1-\frac{k}{n-1}\right)\cdots\left(1-\frac{k}{n-t+1}\right) \\
&\le \left(1-\frac{k}{n}\right)^t
\end{aligned}$$

Note that Equation (1.7) essentially says that the role of k and t are interchangeable in the fraction considered above. Thus, the above result together with Equation (1.7) gives us the second part of **??**. ■

We show the following simple corollary as an example of the application of Lemma 1.3.1.

Corollary 1.3.2: Subset-Guessing Game

Let $n(\lambda)$ be a polynomial. Let $k(\lambda) = \delta n(\lambda)$ where $0 < \delta < 1$ is a constant. Let $t(\lambda) = \omega(\log \lambda)$ and $t(\lambda) < n(\lambda) - k(\lambda)$. For any computationally-binding commitment scheme Com, no PPT adversary Adv can win the following "subset-guessing" game with non-negligible probability:

1. A challenger samples a random size-t subset $\mathsf{r} = \{b_1, \ldots, b_t\} \subseteq [n]$, and commits to this subset to Adv using Com;
2. Adv then outputs a size-k subset $\{p_1, \ldots, p_k\} \subseteq [n]$;
3. The Adv wins if $\{b_1, \ldots, b_t\} \subset [n] \setminus \{p_1, \ldots, p_k\}$.

Proof. Assume for contradiction that there is a computationally-hiding Com and a PPT Adv that wins in the above game with non-negligible probability. We then show a PPT machine Adv_h that breaks the computationally-hiding property of Com:

1. Adv_h samples independently two random size-t subsets of $[n]$, denoted as $B = \{b_1, \ldots, b_t\}$ and $B' = \{b'_1, \ldots, b'_t\}$. Adv_h sends B_0 and B_1 to the external challenger for the hiding game of Com;
2. Adv_h then internally invokes Adv and relay messages between Adv and the external challenger;
3. After the interaction with the external challenger, Adv will output a set $\{p_1, \ldots, p_k\}$. Adv_h output 1 if and only if $B \subseteq [n] \setminus \{p_1, \ldots, p_k\}$.

In the following, we argue that the following value is non-negligible, which means that Adv_h breaks the hiding of Com:

$$\left|\Pr[\mathsf{Adv}_h = 1 \mid \mathsf{Com}(B)] - \Pr\big[\mathsf{Adv}_h = 1 \mid \mathsf{Com}(B')\big]\right|.$$

First, note that Adv's view in the above game is identical to that in the subset guessing game. It then follows from our assumption that $\Pr[\mathsf{Adv}_h = 1 \mid \mathsf{Com}(B)]$ is non-negligible. Therefore, it suffices to show that $\Pr[\mathsf{Adv}_h = 1 \mid \mathsf{Com}(B')]$ is negligible. Recall that $\mathsf{Alice}dv_h$ outputs 1 if and only if $B \subseteq [n] \setminus \{p_1, \ldots, p_k\}$. However, conditioned on $\mathsf{Com}(B')$ (i.e. the external challenger commits to B'), Adv has no information about B. Thus, $\{p_1, \ldots, p_k\}$ and B are independently distributed. We then have:

$$\Pr\big[\mathsf{Adv}_h = 1 \mid \mathsf{Com}(B')\big] = \frac{\binom{n-k}{t}}{\binom{n}{t}} \leq \left(1 - \frac{k}{n}\right)^t = (1-\delta)^t \tag{1.10}$$

By our choice of parameter, $0 < \delta < 1$ is a constant and $t = \omega(\lambda)$. Therefore, $\Pr[\mathsf{Adv}_h = 1 \mid \mathsf{Com}(B')] = \mathsf{negl}(\lambda)$. This finishes the proof of Corollary 1.3.2. ■

Chapter 2

A Minimalist Treatise on Analysis

2.1 Basic Concepts in Set Theory

Definition 2.1.1: Lower Bound, Supremum, Infimum

Let S be a subset S a partially ordered set $(P, \leq)$:

- A *lower bound* of S is an element a of P such that $\forall x \in S\ [a \leq x]$.
- A lower bound a of S is called an *infimum* (or *greatest lower bound*, or *meet*) of S, denoted as $\inf S$, if for all lower bounds y of S in P, it holds that $y \leq a$.

Similarly,

- An *upper bound* of S is an element a of P such that $\forall x \in S\ [a \geq x]$.
- An upper bound a of S is called an *supremum* (or *least upper bound*, or *join*) of S, denoted as $\sup S$, if for all upper bounds y of S in P, it holds that $y \geq a$.

In the following, we define limits for sets. It is worthing mentioning that there are more than one way to define limits for sets, depending on the mathematical scope you want to study. The definition we show here are the only one that is relevant for measure theory and probability (e.g., see Section 2.4.5). There are more general topological notions of set convergence that we did not focus on.

Definition 2.1.2: Limit Superior, Limit Inferior, Set-Theoretic limit

Suppose that $(A_n)_n$ is a sequence of sets index by $n \in \mathbb{N}$ (that is, this is a countable sequence of sets). Define the following

- Limit superior:
$$\limsup_{n\to\infty} A_n := \bigcap_{n=1}^{\infty} \bigcup_{k=n}^{\infty} A_k.$$
- Limit inferior:
$$\liminf_{n\to\infty} A_n = \bigcup_{n=1}^{\infty} \bigcap_{k=n}^{\infty} A_k.$$

If

$$\liminf_{n\to\infty} A_n = \limsup_{n\to\infty} A_n = A,$$

then, we say that the set-theoretic limit of the sequence$(A_n)_n$ exists and is equal to A, denoted as

$$\lim_{n\to\infty} A_n = A.$$

Xiao: Need to define lim inf and lim sup. They will be particularly important when we talk about measure theory, e.g., Section 2.4.5. Xiao!

2.2 On Mathematical Spaces

2.2.1 Three Major Spaces

Modern mathematics is based on 'abstract spaces,' which are sets of elements that satisfy a carefully selected set of axioms. The nature of the elements is deliberately left unspecified to maintain generality. By choosing different sets of axioms, we obtain different types of abstract spaces. The theory then consists of logical consequences that result from these axioms and are derived as theorems once and for all.

The motivation behind this abstract approach is best explained in the following quotation from [Kre89, Chapter 1]:

> *Mathematicians observed that problems from different fields often enjoy related features and properties. This fact was used for an effective unifying approach towards such problems, the unification being obtained by the omission of unessential details. Hence the advantage of such an abstract approach is that it concentrates on the essential facts, so that these facts become clearly visible since the investigator's attention is not disturbed by unimportant details. In this respect the abstract method is the simplest and most economical method for treating mathematical systems.*
>
> *Since any such abstract system will, in general, have various concrete realizations (concrete models), we see that the abstract method is quite versatile in its application to concrete situations. It helps to free the problem from isolation and creates relations and transitions between fields which have at first no contact with one another.*

The most fundamental spaces we are particularly interested in are (1) Topological Spaces (2) Measure Space, and (3) Metric Space. Out of these three concepts, a topological space is considered the most fundamental, and both metric spaces and measure spaces are (somewhat) built on top of topological spaces. But a metric space and measure space are rather incomparable. (See Section 2.2.2, Section 2.2.3, and Section 2.2.4 for a three-way comparison.)

In the following, we first provide a high-level discussion regarding them and then dive into more details in Section 2.3, Section 2.4, and Section 2.5, respectively.

2.2.1.1 Topological Space

A topological space is arguably the type of space with minimal "structures" that are of interest to non-mathematicians. By abstracting the notion of open sets, topological spaces allow mathematicians to study continuity, convergence[1], connectedness, and other fundamental concepts in a very general and flexible way.

[1]The notion of convergence in a topological space is quite different from our intuitive understanding of the concept. Our naive impression of convergence is more akin to the concept in a metric space where 'distance' is defined. In contrast, topological spaces provide a framework for discussing the convergence of *sequences*, *nets*, and *filters*.

We will be particularly interested in the following concepts in a topological space:

- open/close sets, neighborhood,
- limit points[2], dense subset, separable spaces,
- compactness, continuitiy.

Topological spaces are important for several reasons. Just to name some:

- Generalizing Geometric Concepts: Topological spaces allow the generalization of many geometric and analytic concepts, such as continuity and convergence, without requiring a specific structure like a metric. This makes topology applicable to a wide range of mathematical disciplines, including analysis, geometry, and algebra.
- Algebraic Topology: Topological spaces are the primary objects of study in algebraic topology, where they are used to investigate properties that are invariant under continuous deformations. Tools such as homotopy, homology, and cohomology theories are developed to study topological spaces and their mappings.
- Manifolds: Many spaces of interest in mathematics, particularly in differential geometry and theoretical physics, are manifolds, which are topological spaces that locally resemble Euclidean space. The study of manifolds requires a thorough understanding of topological concepts.

2.2.1.2 Measure Space

Xiao: to do Xiao!

We will be particularly interested in the following concepts in a topological space:

- σ-algebra, Borel sets, Borel-Cantelli Lemma
- Carathéodory's extension theorem
- Lebesgue Measure, Lebesgue integral
- Measure-theoretic probability theory and stochastic process
- L^p space, ℓ^p space, and Hölder's Inequality

2.2.1.3 Metric Space

Xiao: to do Xiao!

We will be particularly interested in the following concepts in a topological space:

- Limit and convergence. Cauchy Sequence. Completeness
- Normed Space. Banach Space.
- Inner product space. Hilbert Space.

2.2.2 Metric Space vs Topological Space

A metric space is always associated with a topological space—with the topology induced by its metric. See Theorem 2.5.2.

[2]Such points are also known as accumulation points or cluster points.

2.2.3 Measure Space vs Topological Space

A measure space and a topological space serve for different purpose, and one is not necessarily the other—A measure space is not always a topological space because a measure space is defined by a σ-algebra and a measure, without any inherent notion of open sets or topology. However, they can be related under certain conditions. Consider the following two examples:

1. Topological Space with a Borel Measure: If you have a topological space $(X, \mathcal{T})$, you can construct a measure space by considering the Borel σ-algebra $\mathcal{B}(X)$, which is generated by the open sets of the topology $\mathcal{T}$. A measure μ defined on this Borel σ-algebra $(X, \mathcal{B}(X), \mu)$ makes it a measure space.

   Example: The Lebesgue measure on $\mathbb{R}$ with the standard topology.

2. Measure Space with an Induced Topology: Conversely, if you start with a measure space $(X, \mathcal{M}, \mu)$, it doesn't inherently have a topology. However, certain sets in $\mathcal{M}$ can induce a topology. For example, if you consider a metric space with a metric that generates a σ-algebra, you can define open sets based on the metric, thereby inducing a topology.

   However, this is not a general rule and depends on additional structure being imposed on the measure space.

2.2.4 Measure Space vs Metric Space

These two concepts of spaces are incomparable:

- Measure Space that is Not a Metric Space: Consider the measure space $(\mathbb{R}, \mathcal{B}(\mathbb{R}), \mu)$, where $\mathcal{B}(\mathbb{R})$ is the Borel σ-algebra on $\mathbb{R}$ and μ is the Lebesgue measure. Without defining a metric, this structure does not inherently include any notion of distance between points in $\mathbb{R}$.

- Metric Space that is Not a Measure Space: Consider the set of rational numbers $\mathbb{Q}$ with the standard metric $d(x, y) = |x - y|$. This forms a metric space, but without a specified measure and σ-algebra, it is not a measure space.

That been said, it is worth noting that in some cases, a measure can be defined using a metric. For example, the Lebesgue measure on $\mathbb{R}$ can be seen as related to the standard metric (but they are still distinct structures).

A measure space and a metric space are distinct mathematical structures with different focuses and applications. A measure space is not inherently a metric space, and vice versa. To better understand this, imagine a map (metric space) with distances between cities (points). Now, you want to measure the area (size) of specific regions on the map (measure space). You might use the distances (metric) to define the shapes of these regions, but the measure itself is a separate concept focusing on the "size."

2.3 Basic Concepts in Topology

Topology is nowadays one of the most widely used mathematical languages. It is important later when we want to discuss advanced topics in probability theory and quantum computing.

Definition 2.3.1: Topology, Open Sets, and Neighborhoods

Let X be a set and let τ be a family of subsets of X. Then τ is called a *topology* on X if:

1. $\emptyset \in \tau$ and $X \in \tau$;
2. Any union of elements of τ is an element of τ;
3. Any intersection of finitely many elements of τ is an element of τ.

If τ is a topology on X, then the pair (X, τ) is called a *topological space*. The members of τ are called *open sets* in X. A subset of X is said to be *closed* if its complement is open.

For a point $x \in X$, N_x is a neighborhood if:

1. $x \in N_x$;
2. N_x is open (w.r.t. the topological space (X, τ)).

Here are some remarks regarding the definition:

- A subset of X may be open, closed, both (a clopen set), or neither. The empty set and X itself are always both closed and open.
- Some mathematicians denote neighborhood for x as a set N_x that contains an open set of X (i.e., N_x itself does not need to be open). This version does not make any effective difference for most discussions in topology.

Xiao: Xiao!

- Interior points, exterior points, boundary points, isolated point, limit points, and closure.
- Separation Axioms T_0 to T_4.

Definition 2.3.2: Topology Generated by Subbasis

For a subset S of the power set of X, the *topology generated by* S (denoted by $\tau(X)$) is defined by the intersection of all the elements (which are sets) in the following family of sets:

$$\{\tau \mid \tau \text{ is a topology on } X \text{ containing } S\}.$$

S is called the *subbasis* of a topology τ on X if $\tau = \tau(S)$.[a]

[a]Some authors also require that S covers X. See this wiki page.

We remark that any intersection of many[3] topologies on X is also a topology on X. Thus, Definition 2.3.2 is well-defined. Also, it is clearly that *coarsest* topology containing S. That is why some authors directly define the topology induced by S as the coarsest topology containing S.

The following lemma give a nice characterization of the induced topology:

[3]Note that there is no requirement on finiteness or countability.

Lemma 2.3.3:

Let $\tau(S)$ denote the topology on X generated by subbasis S. Then,

$$\tau(S) = \{U \subseteq X \mid U = \cup_{V \in F} V \text{ where } F \subseteq S'\},$$

where $S' \coloneqq \{s_1 \cap \ldots \cap s_k \mid s_1, \ldots, s_k \in S, k \in \mathbb{N}\} \cup \{X\}$.

The set S' defined in Lemma 2.3.3 is the collection of all the sets that can be obtained by the intersection of *finitely many* elements in S (and $\cup\{X\}$).[4] Actually, Lemma 2.3.3 is just obtained by following the definition of topology—it simply takes the collection of arbitrary unions of finite intersections of elements of S.

2.3.1 Basis and Local Basis

Definition 2.3.4: Basis and Local Basis

B is a *basis* for (X, τ) iff

1. $B \subseteq \tau$ (i.e., all members of B are open);
2. every open set (i.e., elements in τ) can be expressed as a union of some elements in B.

B is a *local base* for (X, τ) at a point $x \in X$ iff

1. $\forall V \in B\ [x \in V \wedge V \in \tau]$;
2. for any $U \in \tau$ s.t. $x \in U$, there exists $V \in B$ such that $x \in V$ and $V \subseteq U$.

Xiao: Say that this definition is different from the one in wikipedia or some textbooks [Mun84]. But they should be(?) equivalent. Xiao!

Lemma 2.3.5:

Let (X, τ) be a topological space and $B \subseteq \mathcal{P}(X)$. For any $x \in X$, define $B_x = \{U \in B \mid x \in U\}$. Then,

$$B \text{ is a basis of } X \;\Leftrightarrow\; \forall x \in X,\ B_x \text{ is a local basis of } (X, \tau) \text{ at } x.$$

2.3.2 Continuity and Compactness

The concept of continuity of functions we studied in calculus (i.e., real or complex metric space) can be generalized to any topology space.

Definition 2.3.6: Continuity

Let (X, τ_X) and (Y, τ_Y) be topological spaces. Let $f : X \to Y$ be a map. We say that f is *continuous at a point* $x \in X$ if for any $V \in \tau_Y$ for which $f(x) \in V$, there exits $U \in \tau_X$ such that $x \in U$.
We say that f is *continuous* if it is continuous at every point in X. It can be shown that f is continuous if and only if $\forall V \in \tau_Y\ [f^{-1}(V) \in \tau_X]$.

[4] Note that there are debates if $0 \in \mathbb{N}$. Here, we choose to set $0 \notin \mathbb{N}$.

Definition 2.3.7: Compactness

Let (X, τ_X) be a topological space and $K \subseteq X$. K is *compact* in (X, τ_X) if for every collection C of open subsets of X such that $K \subseteq \cup_{U \in C} U$, there is a *finite* subset $F \subseteq C$ such that $K \subseteq \cup_{V \in F} V$. We say that (X, τ_X) is compact space if X is compact in (X, τ_X).

Lemma 2.3.8: Compactness is Preserved by Continuous Map

Let f be a continuous map from (X, τ_X) to (Y, τ_Y). If K is compact in (X, τ_X), then $f(K)$ is compact in (Y, τ_Y).

2.3.3 Dense Subsets

Xiao: need to talk about the concept of Dense Subset. It is used in Solovay-Kitaev Theorem Theorem 4.10.4. Xiao!

Xiao: Talk about separable spaces—a topological space is called separable if it contains a countable, dense subset. Xiao!

See also [Kre89, Chapter 1.3] where a definition of separable spaces (and dense subsets) is given specifically to metric spaces.

2.4 Basic Concepts in Measure Theory

Xiao: List of Resource Xiao!

- The best resource I found that reviews the basic concepts in Measure Theory is this sequence of video lectures.
- One important measure theoretic notion for quantum computing is Haar measure. The work of [Mel24] serves as a good starting point. However, it is mainly about the properties of Haar measure and how to use Haar measure as a tool; it lacks a definitional treatment of Haar measure itself.

2.4.1 σ-Algebra

We first define the most basic concept in Measure Theory—σ-algebra, which is a special case of an algebra.

Definition 2.4.1: Sigma Algebra

Let X be a set and $\mathcal{A} \subseteq \mathcal{P}(X)$. $\mathcal{A}$ is a σ-algebra on X if

1. $X \in \mathcal{A}$;
2. **(Closed under Complementation.)** $\forall A \in \mathcal{A}\ [A^c \in \mathcal{A}]$, where $A^c := X \setminus A$; **and**
3. **(Closed under Countable Unions.)**[a] Let $\{A_i\}_{i \in \mathbb{N}}$ be a countable sequence such that $A_i \in \mathcal{A}$ for all $i \in \mathbb{N}$, then $\cup_{i=1}^{\infty} A_i \in \mathcal{A}$.

[a] Assuming that Conditions 1 and 2 hold, it follows from De Morgan's laws that this condition is equivalent to $\mathcal{A}$ being closed under countable intersections.

Analogous to the case of topological spaces (see Footnote 3), it is easy to see that if $\{\mathcal{A}_j\}_{j\in J}$ is a family of σ-algebras on X, then $\cap_{j\in J}\mathcal{A}_j$ is also a σ-algebra on X. Thus, we have the following analog of Definition 2.3.2. Unfortunately, we do not have an analog of Lemma 2.3.3 that gives clean characterization of the $\mathcal{A}(S)$ defined in Definition 2.4.2.

Definition 2.4.2: σ-Algebra Generated by a Set

For any $S \subseteq \mathcal{P}$, there exists a smallest σ-algebra $\mathcal{A}(S)$ containing S. We call $\mathcal{A}(S)$ the *σ-algebra generated by S*. $\mathcal{A}(S)$ is defined by the intersection of all the elements (which are sets) in the following family of sets:

$$\{\mathcal{A} \mid \mathcal{A} \text{ is a } \sigma\text{-algebra on } X \text{ containing } S\}.$$

Remark 2.4.1. *When considering Definition 2.4.2, you might immediately wonder: Does the intersection of σ-algebras form a σ-algebra? This question becomes even more intriguing when considering the intersection of uncountably many σ-algebras.*

Fortunately, the answer is YES. We summarize this result in Lemma 2.4.3. For a related discussion, see this video.

A related question is whether the union of σ-algebras is still a σ-algebra. Unlike the case of intersection, the answer here is NO. It is possible to construct a counterexample with the union of just two σ-algebras (do this as an exercise!).

Lemma 2.4.3: Intersection of σ-Algebras

The intersection of any collection of σ-algebras, *even if the collection is uncountable*, is itself a σ-algebra.

2.4.2 Borel σ-Algebra

A particularly important type of σ-algebras is Borel σ-Algebras.

Definition 2.4.4: Borel σ-Algebra, Borel Sets, and Borel Measurable

Let (X, τ) be a topological space. The Borel σ-algebra $\mathcal{B}[X]$ of X is defined to be the σ-algebra generated by the collection τ of open subsets of X (i.e., $\mathcal{B}[X] = \mathcal{A}(\tau)$). Elements (which are sets) of $\mathcal{B}[X]$ are called Borel sets. They are said to be Borel measurable.

This wiki page defines Borel algebra differently. Our Definition 2.4.4 is in line with the one used by Tao [Tao11, Definition 1.4.16]. These two versions are equivalent—both of them defines the "minimal" σ-algebra that contains all the open sets of a topological spaces. One of the most famous examples is the Borel algebra $(\mathbb{R}, \mathcal{B}(\mathbb{R}))$ over the real line, which plays a super important for Lebesgue's integration theory.

Definition 2.4.5: Borel Measurable Map

Let (X, τ_X) and (Y, τ_Y) be topological spaces. Let $\mathcal{B}[X]$ and $\mathcal{B}[Y]$ be their corresponding Borel σ-algebras. A function $f : X \to Y$ is Borel measurable if it is $(\mathcal{B}[X], \mathcal{B}[Y])$-measurable (see Definition 2.4.8).

Lemma 2.4.6: Borel Measurability of Continuous Maps

Let (X, τ_X) and (Y, τ_Y) be topological spaces. If $f : X \to Y$ is continuous (as per Definition 2.3.6), then f is Borel measurable (as per Definition 2.4.5).

2.4.3 Measure and Measure Spaces

Starting with a σ-algebra, we can define a measurable space. This space, as its name suggests, *will* allows us to define measures on it (hence, "measurable space" rather than "measure space").

Definition 2.4.7: Measurable Space and Measurable Sets

A *measurable space* $(X, \mathcal{M}_X)$ consists of a set X and a σ-algebra $\mathcal{M}_X$ on X. The elements (which are sets) contained in $\mathcal{M}_X$ are called *measurable sets*.

Analogous to the concept of continuous map (Definition 2.3.6) w.r.t. topological spaces, we can define measurable map for measurable space.

Definition 2.4.8: Measurable Map

Given two measurable spaces $(X, \mathcal{M}_X)$ and $(Y, \mathcal{M}_Y)$, a map $f : X \to Y$ is a $(\mathcal{M}_X, \mathcal{M}_Y)$-*measurable map* if for all $B \in \mathcal{M}_Y$, it holds that $f^{-1}(B) \in \mathcal{M}_X$.

Starting with a measurable space, we now define a measure on it, making it a *measure space*!

Definition 2.4.9: (Countably Additive) Measure and Measure Spaces

Let $(X, \mathcal{B})$ be a measurable space. An *(unsigned) countably additive measure* μ on $\mathcal{B}$, or measure for short, is a map $\mu : B \to [0, +\infty]$ that obeys the following axioms:

1. **(σ-Additivity.)** Whenever $E_1, E_2, \ldots \in \mathcal{B}$ are a countable sequence of disjoint measurable sets, then
$$\mu(\bigcup_{i=1}^{\infty} E_i) = \sum_{i=1}^{\infty} \mu(E_i).$$
2. **(Non-Triviality.)** $\mu(\emptyset) = 0$. (Assuming Condition 1, Condition 2 is equivalent to the following requirement: $\exists E \in \mathcal{B}\ [\mu(E) < \infty]$.

A triplet $(X, \mathcal{B}, \mu)$, where $(X, \mathcal{B})$ is a measurable space and $\mu : B \to [0, +\infty]$ is a countably additive measure, is known as a *measure space*.

Xiao: Xiao!

- Actually, at this moment, we are ready to establish Lebsegue's integration theory. However, this theory (albeit very useful for numerous applications) is not that important for theoretical computer science. Thus, we choose to skip it.
- Completion and complete measure space.
- Constructing outer measure via covering.
- Lebesgue outer measure (and probably Hausdorff outer measure).

- Regular measure.

2.4.4 Outer Measure and Carathéodory's extension theorem

Xiao: Outer measure and Carathéodory's extension theorem ([Tao11, Section 1.7.1] is a great reference for this topic). Here are two interesting applications of Carathéodory's extension theorem: Xiao!

- Used to define Lebesgue measure on the Borel σ-algebra $(\mathbb{R}, \mathcal{B}(\mathbb{R}, \lambda))$, and show its uniqueness. (i.e., extending $\lambda\big((a, b]\big) = b - a$.)

- Define product measure: given two measure spaces $(X_1, \mathcal{A}_1, \mu_1)$ and $(X_2, \mathcal{A}_2, \mu_2)$, first construct a new measurable space $\big(X_1 \times X_2, \sigma(\mathcal{A}_1 \times \mathcal{A}_2)\big)$, where $\sigma(\mathcal{A}_1 \times \mathcal{A}_2)$ is the σ-algebra generated by the Cartesian product $\mathcal{A}_1 \times \mathcal{A}_2$. Then, the corresponding measure is obtained by applying Carathéodory's extension to $\mu(A_1 \times A_2) := \mu_1(A_1) \cdot \mu_2(A_2)$. (In general, this measure is not unique. But it is unique if both μ_1 and μ_2 are finite.)

Xiao: define outer measure and Carathéodory measurebility Xiao!

Theorem 2.4.10: Carathéodory's Extension Theorem

Let $\mu^* : X \to [0, +\infty]$ be an outer measure on a set X, let $\mathcal{B}$ be the collection of all subsets of X that are Carathéodory measurable w.r.t. μ^*, and let $\mu : \mathcal{B} \to [0, +\infty]$ be the restriction of μ^* to $\mathcal{B}$ (thus $\mu(E) = \mu^*(E)$ whenever $E \in \mathcal{B}$). Then, $(X, \mathcal{B}, \mu)$ is a measure space (i.e., $\mathcal{B}$ is a σ-algebra, and μ is a measure).

2.4.5 Borel-Cantelli Lemma

This is an important lemma used (sometimes implicitly) in several lower bounds in cryptography (e.g., [IR89, CFM21]). In the following, we present the lemma in its most general form (i.e., w.r.t. any measure space), although cryptographic or standard TCS applications of it mainly cares about the special case of probability measure.

Lemma 2.4.11: Borel-Cantelli Lemma

Let $(X, \mathcal{F}, \mu)$ be a measure space, and let $\{E_n\}_{n=1}^{\infty}$ be a sequence of $\mathcal{F}$-measurable sets (i.e., $E_n \in \mathcal{F}$ for all n) such that $\sum_{n=1}^{\infty} \mu(E_n) < \infty$. Then,

$$\mu\big(\limsup_{n\to\infty} E_n\big) = 0,$$

where recall from Definition 2.1.2 that $\limsup_{n\to\infty} E_n = \bigcap_{n=1}^{\infty} \bigcup_{k=n}^{\infty} E_k$.

How to interpret Lemma 2.4.11:

- In words, the equation $\mu\big(\limsup_{n\to\infty} E_n\big) = 0$ actually means the following: almost every $x \in X$ is contained in at most finitely many of the E_n (i.e. $\{n \in N : x \in E_n\}$ is finite for almost every $x \in X$).

- When $(X, \mathcal{F}, \mu)$ is a probability measure, the set (which could be called an *event*) $\limsup E_n$ is sometimes denoted $\{E_n \text{ } i.o.\}$, where "*i.o.*" stands for "infinitely often." People usually describe the event $\limsup E_n$ as "$(E_n)_n$ happens infinitely often." This is an awkward and misleading terminology—What happens infinitely often is not really the sets E_n's, but the elements in the set $\limsup E_n$. To see that, let us consider a point $\omega \in \limsup E_n$. First, recall from Definition 2.1.2 that $\limsup E_n = \bigcap_{n=1}^{\infty} \bigcup_{k=n}^{\infty} E_k$; Let us now explain the meaning of $\bigcup$ and $\bigcap$:

– First, note that $\omega \in \bigcup_{n=1}^{\infty} E_n$ for all $n = 1, 2, 3...$
– Then, $\omega \in \bigcap_{n=1}^{\infty} \bigcup_{k=n}^{\infty} E_k$ means that ω is contained in infinitely many set E_n's (i.e., $\omega \in E_n$ for infinitely many $n \in \mathbb{N}$.)

Therefore, $\limsup E_n$ actually denotes a set of ω's that "happen infinitely" (i.e., contained in infinitely many set E_n's). Formally,

$$\limsup_{n\to\infty} E_n = \{\omega \mid \omega \in X \ \wedge \ \omega \text{ is contained in infinitely many } {E_n}'s\}.$$

Thus, what happens infinitely often is not the set E_n, but the elements in the set $\limsup E_n$. Even more accurately, the elements in the set $\limsup E_n$ do not really "happen infinitely often"; rather, they are contained in infinitely many E_n's!

- When $(X, \mathcal{F}, \mu)$ is a probability measure, the theorem asserts that if the sum of the probabilities of all the events in E_n is finite, then the probability of those events happening *infinitely often* (see the above bullet for an accurate explanation) is zero.

  This is summarized as the following Corollary 2.4.12. Typically, this is the most widely used version of Borel-Cantelli lemma in Cryptography.

Corollary 2.4.12: Borel-Cantelli for Probability Spaces

Suppose that $\{E_n\}_{n\in\mathbb{N}}$ is a sequence of events in a probability space. If

$$\sum_{n\in\mathbb{N}} \Pr[E_n] < \infty,$$

then the probability that infinitely many of them occur is 0, or formally,

$$\Pr\left[\limsup_{n\to\infty} E_n\right] = \Pr\left[\bigcap_{n=1}^{\infty} \bigcup_{k=n}^{\infty} E_k\right] = 0.$$

In other words, with probability 1, only a finite number of the events occur.

A Coin-Flipping Example. To better understand the Borel-Cantelli Lemma, let us consider an example of coin-flipping. In particular, consider an infinite sequence of coin flipping:

- Let A_n be the event that the n-th toss is heads.
- Let $P(A_n) = p$ for a (biased) coin that flips to head with probability p.

Note that $\limsup_{n\to\infty} A_n$ in this context means the event that "heads appear infinitely often." This can be seen by a careful analyze of the definition $\limsup_{n\to\infty} A_n := \bigcap_{n=1}^{\infty} \bigcup_{k=n}^{\infty} A_k$:

- **Understanding $\bigcup_{k=n}^{\infty} A_k$:** This union represents the event that at least one of the coin tosses $n, n+1, n+2, \ldots$ results in heads. In other words, starting from the n-th toss onward, there is at least one heads.

- **Understanding $\bigcap_{n=1}^{\infty}\bigcup_{k=n}^{\infty} A_k$:** This intersection means that for every n, there is at least one heads among the tosses $n, n+1, n+2, \ldots$. No matter how far out you go in the sequence of tosses, you will always find a heads eventually.

In summary, for any positive integer n, there must be some $k \geq n$ such that the k-th toss is heads. This means that heads keep appearing no matter how far you go along the sequence of tosses.

First, consider the case $P(A_n) = p = 1/2$. Let us sum the probabilities:

$$\sum_{n=1}^{\infty} P(A_n) = \sum_{n=1}^{\infty} \frac{1}{2} = \infty.$$

Since this sum is infinite, the Borel-Cantelli lemma does not tell us anything in this case.

Next, let us consider the case $P(A_n) = \frac{1}{n^2}$. Let us sum the probabilities:

$$\sum_{n=1}^{\infty} P(A_n) = \sum_{n=1}^{\infty} \frac{1}{n^2}.$$

This is a convergent series (specifically, it converges to $\frac{\pi^2}{6}$). It then follows from the Borel-Cantelli lemma that

$$\Pr\left[\limsup_{n\to\infty} A_n\right] = 0.$$

According to our earlier interpretation of the event "$\limsup_{n\to\infty} A_n$", it means that the probability that "heads appear infinitely often" is 0!

2.4.6 The Second Borel-Cantelli Lemma

We next discuss about the so-called "second Borel-Cantelli lemma." This lemma is also known as the *converse* or *divergence version* of Borel-Cantelli lemma. However, there is a important caveat:

- The Borel–Cantelli lemma holds for general measure space (i.e., Lemma 2.4.11). However, the second Borel–Cantelli lemma applies specifically to probability measures—It is a result within the realm of probability theory and is used to make statements about the behavior of sequences of events under probability measures.

Lemma 2.4.13: The Second Borel-Cantelli Lemma

Let $(\Omega, \mathcal{F}, P)$ be a probability space and let $\{E_n\}$ be a sequence of events in $\mathcal{F}$. If the events $(E_n)_{n=1}^{\infty}$ are pairwise independent and $\sum_{n=1}^{\infty} P(E_n) = \infty$, then it holds that

$$\Pr\left[\limsup_{n\to\infty} E_n\right] = 1.$$

For example, imagine flipping a biased coin with a probability of 0.00001 for heads (H) for infinitely many times. Let E_n denote the event that the n-th flipping gives head. Then, it holds that

$$\sum_{n=1}^{\infty} \Pr[E_n] = \sum_{n=1}^{\infty} 0.00001 = \infty.$$

It then follows from Lemma 2.4.13 that

$$\Pr\left[\limsup_{n\to\infty} E_n\right] = 1.$$

In our context, the event $\limsup_{n\to\infty} E_n$ means that there are infinitely many heads in the infinite sequence of coin flip results. Thus, the above result means that: despite that the individual probability of heads is extremely small (i.e., $p = 0.00001$), if we toss the coin for infinitely many times, it is almost surely (i.e., with probability 1) that we will see heads infinitely often.

2.4.7 L^p Space, ℓ^p Space, and Hölder's Inequality

2.4.7.1 L^p Space

The Lebesgue integral on any measure space gives rise to a natural norm called the L^p norm (or p-norm). A measure space, when perceived with this natural L^p-norm together, forms an important normed measure space. We call it the L^p space (see for Definition 2.4.14 the formal definition). These spaces are fundamental in various areas of mathematical analysis, including functional analysis, harmonic analysis, and partial differential equations.

Definition 2.4.14: L^p Space

Fix a $p \in [1, \infty)$. Given a measure space (X, M, μ), the L^p space $L^p(X, M, \mu)$, or simply $L^p(X)$ when the measure and σ-algebra are understood, consists of all measurable functions $f : X \to \mathbb{R}$ (or $\mathbb{C}$) such that the p-th power of the absolute value of f is (Legesgue) integrable. Formally, the L^p space is defined as:

$$L^p(X) = \left\{ f : X \to \mathbb{R} \;\middle|\; \int_X |f(x)|^p \, d\mu(x) < \infty \right\}$$

The norm in an L^p space, called the p-norm, is defined by:

$$\|f\|_p = \left(\int_X |f(x)|^p \, d\mu(x) \right)^{\frac{1}{p}}$$

Remark 2.4.2 (On the Range of p)**.** *In the above, we only consider $p \in [1, \infty)$. It is worth noting that Definition 2.4.14 can be extended to $p \in [0, \infty]$, with the following caveats:*

- *To define L^∞, one needs to talk about the notions such as essentially bounded functions, essential supremum, and* almost everywhere less than or equal to*. I don't want to include that as they are not so useful form computer scientists.*
- *The definition can be extended to all $0 < p \le \infty$ (rather than just $1 \le p \le \infty$), but it is only when $1 \le p \le \infty$ that $\|\cdot\|_p$ is guaranteed to be a norm (although $\|\cdot\|_p$ is a quasi-seminorm for all $0 < p \le \infty$).*
- *To define L^0, see this wikipedia link.*

2.4.7.2 ℓ^p Space

The ℓ^p space is essentially the "discrete" version of the L^p space defined in Definition 2.4.14. In the former, Lebesgue integral simply degenerates to (potentially infinite) summation, and thus things become easier to understand for people who are not good at math. We now provide the formal definition.

Definition 2.4.15: ℓ^p Space

For $1 \leq p < \infty$, the space ℓ^p consists of all sequences of real or complex numbers $(x_n)_{n\in\mathbb{N}}$ such that the series of the p-th power of the absolute values is convergent. Formally:

$$\ell^p = \left\{ (x_n)_{n\in\mathbb{N}} \;\middle|\; \sum_{n=1}^{\infty} |x_n|^p < \infty \right\}$$

For $p = \infty$, the space ℓ^∞ consists of all bounded sequences of real or complex numbers. Formally:

$$\ell^\infty = \left\{ (x_n)_{n\in\mathbb{N}} \;\middle|\; \sup_{n\in\mathbb{N}} |x_n| < \infty \right\}$$

For $p \in [1, \infty)$, the norm in an ℓ^p space is defined as: $\|x\|_p = \left(\sum_{n=1}^{\infty} |x_n|^p\right)^{\frac{1}{p}}$.
For ℓ^∞, the norm is the supremum norm: $\|x\|_\infty = \sup_{n\in\mathbb{N}} |x_n|$

2.4.7.3 Hölder's Inequality

Hölder's inequality is a fundamental inequality in measure theory and functional analysis. It generalizes the Cauchy-Schwarz inequality and is crucial in the study of L^p spaces. Hölder's inequality provides a bound on the integral (or sum) of the product of two functions in terms of the L^p norms of the functions.

Theorem 2.4.16: Hölder's Inequality

Let (X, M, μ) be a measure space, and let f and g be measurable real- or complex-valued functions on X. Let p and q be real numbers such that $1 \leq p, q \leq \infty$ and $\frac{1}{p} + \frac{1}{q} = 1$. Then, Hölder's inequality states that:

$$\int_X |f(x)g(x)|\, d\mu(x) \leq \left(\int_X |f(x)|^p \, d\mu(x) \right)^{\frac{1}{p}} \left(\int_X |g(x)|^q \, d\mu(x) \right)^{\frac{1}{q}}.$$

Or equivalently, using the notation from Definition 2.4.14, it is

$$\|fg\|_1 \leq \|f\|_p \|g\|_p.$$

Some remarks:

- The integral in is Lebesgue integral, which can be defined for arbitrary measure space.
- When $p = q = 2$, Theorem 2.4.16 is Cauchy-Schwarz inequality.

In the "discrete" ℓ^p space (see Definition 2.4.15), Hölder's inequality has the following format.

Theorem 2.4.17: Hölder's Inequality for ℓ^p Space

Let $x = (x_n)$ and $y = (y_n)$ be sequences of real or complex numbers. Let p and q be real numbers such that $1 \leq p, q \leq \infty$ and $\frac{1}{p} + \frac{1}{q} = 1$. Then, Hölder's inequality states that:

$$\sum_{n=1}^{\infty} |x_n y_n| \leq \left(\sum_{n=1}^{\infty} |x_n|^p \right)^{\frac{1}{p}} \left(\sum_{n=1}^{\infty} |y_n|^q \right)^{\frac{1}{q}}.$$

2.5 Basics Concepts in Metric Space and Functional Analysis

Definition 2.5.1: Metric Space

A *metric space* is a pair (X, d), where X is a set and d is a *metric* on X (or distance function on X), that is, a function $d : X \times X \to \mathbb{R}$ such that for all $x, y, z \in X$, it holds that

1. $d(x, x) = 0$,
2. (Positivity.) If $x \neq y$, then $d(x, y) > 0$.
3. (Symmetry.) $d(x, y) = d(y, x)$.
4. (Triangle Inequality.) $d(x, y) \leq d(x, z) + d(z, y)$.

Theorem 2.5.2: Metric-Induced Topological Space

Every metric space is a topological space with the open sets (in the topological space) being the open balls (in the metric space).

Proof of Theorem 2.5.2. Given a metric space (X, d), we can define a topology on X using the metric. The collection of open sets in this topology is defined as follows:

A subset $U \subseteq X$ is called open if, for every point $x \in U$, there exists an $\varepsilon > 0$ such that the *open ball* $B(x, \varepsilon) \subseteq U$, where the open ball $B(x, \varepsilon)$ is defined as:

$$B(x, \varepsilon) = \{y \in X \mid d(x, y) < \varepsilon\}.$$

It is not hard to show that such a definition of open sets satisfies the requirements of being a topological space.

■

2.5.1 Limit, Cauchy Sequence, and Completeness

Definition 2.5.3: Convergence of a sequence, limit

A sequence (x_n) in a metric space $X = (X, d)$ is said to converge (or to be convergent) if there is an $x \in X$ such that

$$\lim_{n \to \infty} d(x_n, x) = 0.$$

The x is called the limit of (x_n) and we write $\lim_{n \to \infty} x_n = x$, or simply $x_n \to x$.
If (x_n) is not convergent, it is said to be divergent.

Some remarks regarding Definition 2.5.3 follow:

- Note that the limit point x must be in the space X. That is, the sequence cannot convert to some point out of the concerned space.
- Metric space is essentially the "minimal" space where we can talk about limit and convergence. This is because the notion of limit/convergence we first encountered in a real analysis (or calculus) course is about real numbers[5]. A metric space has a metric d defined, which maps (pairs of) elements to the real numbers (see Definition 2.5.1). This allows use to exercise the idea of limit/convergence.

Definition 2.5.4: Cauchy Sequence, Completeness

A sequence (x_n) in a metric space (X, d) is said to be *Cauchy* (or *fundamental*) if for every $\varepsilon > 0$, there is an $N = N(\varepsilon)$ such that for all $m, n > N$, it holds that $d(x_m, x_n) < \varepsilon$.
The space (X, d) is said to be *complete* if every Cauchy sequence in X converges (that is, has a limit which is an element of X).

Xiao: Xiao!

- Define metric space, open/close sets, convergence, Cauchy sequence, and completeness.

  One of the most important properties of Cauchy sequences:

  Lemma 2.5.5:

  Let (X, d) be a metric space and $\{x_n\}_{n\in\mathbb{N}}$ be a Cauchy sequence. If a subsequence $\{x_{n_k}\}_{k\in\mathbb{N}}$ converges to some $x \in X$, then so does $\{x_n\}_{n\in\mathbb{N}}$.

- Normed space and Banach Space.
  - Normed space is obtained by defining a *norm* on a vector space (as per Definition 3.5.2). Note that a vector space itself may not be a metric space. But once a norm is defined on top of it, the norm induces a natural metric, making it a metric space. Also see the first few paragraphs in [Kre89, Chapter 2].
  - The following quotation from [Kre89, Chapter 2.2] is very important

    > *... in many cases a vector space X may at the same time be a metric space because a metric d is defined on X. However, if there is no relation between the algebraic structure and the metric, we cannot expect a useful and applicable theory that combines algebraic and metric concepts. To guarantee such a relation between "algebraic" and "geometric" properties of X, we define on X a metric d in a special way as follows. We first introduce an auxiliary concept, the norm (definition below), which uses the algebraic operations of vector space. Then we employ the norm to obtain a metric d that is of the desired kind. This idea leads to the concept of a normed space. It turns out that normed spaces are special enough to provide a basis for a rich and interesting theory, but general enough to include many concrete models of practical importance. In fact, a large number of metric spaces in analysis can be regarded as normed spaces, so that a normed space is probably the most important kind of space in functional analysis, at least from the viewpoint of present-day applications.*

  - Banach Space is a complete normed space (complete in the metric defined by the norm).

[5] More accurately, it requires the notion of "distance," which is naturally measured by real numbers.

One of the most important Banach Spaces: ℓ^p space.

- Inner-product space and Hilbert space. Directly used in quantum computing.
  - Metric spaces allow us to talk about distance.
  - Normed spaces allow us to talk about distance and length.
  - Inner product spaces allow us to talk bout distance, length, and angle.

  Also see Section 2.5.2.

- Operator norm, bounded operator, and its relation to continuity. The following lemma is especially important

  Lemma 2.5.6: Continuity of Bounded Operators

  Let $(X, \|\cdot\|_X)$ and $(Y, \|\cdot\|_Y)$ be two normed spaces. Let $T : X \to Y$ is a linear operator. Then, the following claims are equivalent:

  - T is continuous (at all points);
  - T is continuous at a single point;
  - T is bounded.

- talk about the concept of "compactness" in metric space (which should be viewed as a special case of Definition 2.3.7).

2.5.2 Motivation for Inner Product Space (and Hilbert Space)

(The following material is taken almost verbatim from the beginning of [Kre89, Chapter 3].)

In a normed space we can add vectors and multiply vectors by scalars, just as in elementary vector algebra. Furthermore, the norm on such a space generalizes the elementary concept of the length of a vector. However, what is still missing in a general normed space, and what we would like to have if possible, is an analogue of the familiar dot product

$$\boldsymbol{a} \cdot \boldsymbol{b} \coloneqq a_1 b_1 + a_2 b_2 + a_3 b_3$$

and resulting formulas, notably the notion of "absolute value" (or "squared modulus")

$$|\boldsymbol{a}| = \sqrt{\boldsymbol{a} \cdot \boldsymbol{a}}$$

and the condition for orthogonality (perpendicularity)

$$\mathbf{a} \cdot \mathbf{b} = 0$$

which are important tools in many applications. Hence the question arises whether the dot product and orthogonality can be generalized to arbitrary vector spaces. In fact, this can be done and leads to inner product spaces and complete inner product spaces, called Hilbert spaces.

Inner product spaces are special normed spaces. Historically they are older than general normed spaces. Their theory is richer and retains many features of Euclidean space, a central concept being orthogonality. In fact, inner product spaces are probably the most natural generalization of Euclidean space, and the reader should note the great harmony and beauty of the concepts and proofs in this field. The whole theory was initiated by the work of D. Hilbert (1912) on integral equations. The currently used geometrical notation and terminology is analogous to that of Euclidean geometry and was coined by E. Schmidt (1908), who followed a suggestion of G. Kowalewski (as he mentioned on page 56 of his paper). These spaces have been, up to now, the most useful spaces in practical applications of functional analysis.

At a high level, an inner product space X is a vector space with an inner product (x, y) defined on it. The latter generalizes the dot product of vectors in three dimensional space and is used to define

1. a norm $\|\cdot\|$ by $\|x\| = \sqrt{\langle x, x\rangle}$,

2. orthogonality by $\langle x, y\rangle = 0$.

A Hilbert space $\mathcal{H}$ is a complete inner product space. The theory of inner product and Hilbert spaces is richer than that of general normed and Banach spaces. Distinguishing features are:

1. representations of $\mathcal{H}$ as a direct sum of a closed subspace and its orthogonal complement (see [Kre89, Theorem 3.3-4]),

2. orthonormal sets and sequences and corresponding representations of elements of $\mathcal{H}$ (see [Kre89, Sections 3.4 and 3.5]),

3. the Riesz representation (see [Kre89, Theorem 3.8-1]) of bounded linear functionals by inner products,

4. the Hilbert-adjoint operator T^* of a bounded linear operator T (see [Kre89, Definition 3.9-1]).

Orthonormal sets and sequences are truly interesting only if they are total (see [Kre89, Section 3.6]). Hilbert-adjoint operators can be used to define classes of operators (self-adjoint, unitary, normal; see [Kre89, Section 3.10]) which are of great importance in applications.

Chapter 3

Algebra from a Modern Point of View

3.1 Pre-Group Concepts

Definition 3.1.1: Magma

A magma (also called "groupoid") is a set M equipped with a binary operation "+" satisfying the following property:

1. **Closure.** M is closed under "+".

Definition 3.1.2: Semigroup

A semigroup is a set S equipped with a binary operation "+" satisfying the following properties:

1. **Closure.** S is closed under "+".
2. **Associativity.** For all $a, b, c \in S$, $(a + b) + c = a + (b + c)$.

Definition 3.1.3: Monoid

A monoid is a set M equipped with a binary operation "+" satisfying the following properties:

1. **Closure.** M is closed under "+".
2. **Associativity.** For all $a, b, c \in M$, $(a + b) + c = a + (b + c)$.
3. **Identity Element.** There is an element e in M such that for all $a \in M$, $a + e = e + a = a$.

The relations among these concepts can be summarized as follows:

- A magma is the most basic algebraic structure (over a set).
- A semigroup is a magma with associativity.
- A monoid is a semigroup with an identity element.

3.2 Groups

Definition 3.2.1: Group

A group is a set G equipped with a binary operation "+" satisfying the following properties:

1. **Closure.** G is closed under "+".
2. **Associativity.** For all $a, b, c \in G$, $(a + b) + c = a + (b + c)$.

3. **Identity Element.** There is an element e in G such that for all $a \in G$,

$$a + e = e + a = a$$

4. **Inverse Element.** For any $a \in G$, there is an element $-a$ in G such that

$$a + (-a) = (-a) + a = e$$

A group is called "Abelian" if it additionally satisfies the following property

5. **Commutativity.** For all $a, b \in G$, $a + b = b + a$.

Xiao: Talk about the relation between Branching Program and Symmetric groups Xiao!
Xiao: Some book uses "factor group" to refer to "quotient group". They are the same. Xiao!

Here is a simple (but very useful) fact of finite group. It gives Euler's theorem when instantiated on group $\mathbb{Z}_n^*$. (The proof is omitted as it is obvious.)

Theorem 3.2.2:

Let G be a finite group of order $m = |G|$. Then $\forall g \in G$, $g^m = 1$.
Specifically, if we set $G = \mathbb{Z}_n^*$ ($n \in \mathbb{N}$), this is the Euler's theorem:

$$\forall a \in \mathbb{Z}_n^*, \quad a^{\phi(n)} = 1 \bmod n.$$

Theorem 3.2.2 gives the following two very important corollaries. The first one is extremely useful for cryptography as it tells a sufficient condition to construct permutation on finite groups. The second one is helpful to compute large exponentiation on finite groups.

Corollary 3.2.1. Let G be a finite group of order $m > 1$. Let $e > 0$ be an integer, and define the function $f_e : G \to G$ by $f_e(g) = g^e$. We have:

$$\mathsf{gcd}(e, m) = 1 \quad \Rightarrow \quad f_e \text{ is bijective}$$

Moreover, if $d = e^{-1} \bmod m$ then f_d is the inverse of f_e.

$\Diamond$

Corollary 3.2.2. Let G be a finite group of order $m > 1$. Then for any $g \in G$ and any integer x,we have $g^x = g^{x \bmod m}$.

$\Diamond$

Other interesting corollaries of Theorem 3.2.2 include:

- Let G be a finite group, and $g \in G$ an element of order i. Then:

$$g^x = g^y \quad \Leftrightarrow \quad x = y \bmod i$$

- Let G be a finite group of order m, and say $g \in G$ has order i. Then $i | m$.

3.2.1 Quotient Groups

The concept of quotient groups plays will play an important role when we talk about quantum computing, especially for the stabilizer formalism for quantum error correction and fault tolerance computation.

We start with the definition of normal subgroups.

Definition 3.2.3: Normal Subgroups

A subgroup N of a group G is called *normal* if for all $g \in G$, it holds that $gN = Ng$ (or equivalently, $gNg^{-1} = N$). We use $N \triangleleft G$ to denote that N is a subgroup of G.

From Definition 3.2.3, it is easy to see that if G is Abelian, then every subgroup of G is a normal subgroup.

The following Definition 3.2.4 says the cosets of a normal subgroup also form a group. We call such groups as quotient groups. Note that G/N is a group only if $N \triangleleft G$; it does not hold if N is an arbitrary subgroup G. Indeed, this was the original motivation to denote *normal* subgroups.

Definition 3.2.4: Quotient Groups

Let $N \triangleleft G$. Then, $G/N := \{gN \mid g \in G\}$ is a group under the operation $(g_1N) \cdot (g_2N) := (g_1g_2)N$. We call groups of the form G/N *quotient groups*.

Here are some interesting properties of quotient groups:

1. If G is finite, then $|G/N| = |G|/|N|$; (In general, $|G/N| = |G : N|$, i.e., the index of N in G.)
2. If G is Abelian, nilpotent, solvable, cyclic, or finitely generated, then so is G/N.
3. Denote $\pi(g) := gN$ for all $g \in G$, then, π is a surjective homomorphism from G to G/N.

3.2.2 Cyclic Groups

Cyclic groups are a type of groups that is of special interest for cryptographers. Several number-theoretic problems are conjectured to be intractable on cyclic groups, while there do exist some non-cyclic groups where these problems are easy.

The first fact we what to stress is that every finite group of prime order is cyclic. This can be regarded as another corollary of Theorem 3.2.2.

Theorem 3.2.5:

If G is a group of prime order p, then G is cyclic. Furthermore, all elements of G except the identity are generators of G.

The following theorem is very important. It shows that $\mathbb{Z}_p^*$ is a cyclic group if p is a prime. Note that is does not follow as a corollary of Theorem 3.2.5. Actually, its proof is very involved but can be found in standard abstract algebra textbooks.

Theorem 3.2.6:

If p is prime then $\mathbb{Z}_p^*$ is a cyclic group of order $p - 1$.

Why does cryptography prefer cyclic groups?

- A cyclic group can be described by a single generator. Also, every element is a generator.

In addition, cyclic groups of *prime order* enjoy additional advantages[1]:

- This is a consequence of the Pohlig–Hellman algorithm, described in Chapter 9, which shows that the discrete-logarithm problem in a group of order q becomes easier if q has (small) prime factors. This does not necessarily mean that the discrete-logarithm problem is easy in groups of nonprime order; it merely means that the problem becomes easier.
- Related to the above, DDH problem is easy if the group order q has small prime factors. For example, in group $\mathbb{Z}_p^*$ with p a prime, discrete log is believed to be hard, but DDH is usually easy. Thus, people have to use subgroups of $\mathbb{Z}_p^*$ of prime order for DDH-based constructions (see Theorem 3.2.7).
- Finding a generator in cyclic groups of prime order is trivial. In contrast, efficiently finding a generator of an arbitrary cyclic group requires the factorization of the group order to be known (see Appendix B.3 of [KL14]).
- When the group order is prime, any nonzero exponent will be invertible, making this computation of multiplicative inverses possible.
- Consider the DDH tuple (g^a, g^b, g^{ab}). For it to be indistinguishable form a random tuple, a necessary is that g^{ab} by itself should be indistinguishable from a uniform group element. One can show that g^{ab} is "close" to uniform (in a sense we do not define here) when the group order p is prime, something that is not true otherwise.

We present a useful theorem w.r.t. the form of subgroups of $\mathbb{Z}_p^*$.

Theorem 3.2.7:
Let $p = rq + 1$ with p, q prime. Then $G := \{h^r \bmod p \mid h \in \mathbb{Z}_p^*\}$ is a subgroup of $\mathbb{Z}_p^*$ of order q.

3.2.3 $\mathbb{Z}_N$, $\mathbb{Z}_N^*$, and RSA

Lemma 3.2.1. Let $a \geq 1$, $n > 1$ be integers. Then a is invertible in $\mathbb{Z}_n$ if and only if $\mathsf{gcd}(a, n) = 1$. $\Diamond$

The RSA Assumption. We first define a set of all integers when are the product of two length-λ primes:

$$Z_\lambda^{(2)} = \{N \mid N = p \cdot q \text{ where } p \text{ and } q \text{ are } \lambda\text{-bit primes.}\}$$

The RAS assumption conjunctures that the following problem is hard: for $N \xleftarrow{\$} Z_\lambda^{(2)}$, e such that $\mathsf{gcd}(e, \phi(N)) = 1$[2] and $y \xleftarrow{\$} \mathbb{Z}_N^*$, the computational task the adversary Adv is to find x such that $x^e = y \bmod N$. The (n, t, ε) hardness of RSA assumption is: no t-time algorithm Adv satisfies:

$$\Pr[\mathsf{Adv}(N, e, y) = x \text{ where } x^e = y \bmod N] > \varepsilon$$

Further discussion regarding the choice of e and other parameters can be found in [KL14].

[1] These are the reasons listed in [KL14]

[2] This requirement is to guarantee that e induces a permutation on $\mathbb{Z}_N^*$ (see Corollary 3.2.1) such that the RAS problem is well defined. Namely, every y has a preimage under $f_e(x) = x^e \bmod N$.

3.2.4 Quadratic Residuosity

Legendre Symbol and Jacob Symbol.

Definition 3.2.8: Legendre Symbol

Let p be an odd prime. The Legendre symbol of an integer a is defined as

$$\left(\frac{a}{p}\right) = \begin{cases} 1 & a \text{ is a QR and } a \neq 0 \bmod p \\ -1 & a \text{ is a QNR} \\ 0 & a = 0 \bmod p \end{cases}.$$

Lemma 3.2.2. Let p be an odd prime. Then $\left(\frac{a}{p}\right) = a^{\frac{p-1}{2}}$. ◊

Definition 3.2.9: Jacobi Symbol

Let N be a positive odd integer. The Jacobi symbol of an integer a is defined as

$$\mathcal{J}_N(a) := \prod_{i=1}^{k} \left(\frac{a}{p_i}\right)^{\alpha_i} = \left(\frac{a}{p_1}\right)^{\alpha_1} \cdot \left(\frac{a}{p_2}\right)^{\alpha_2} \cdots \left(\frac{a}{p_k}\right)^{\alpha_k},$$

where $N = p_1^{\alpha_1} p_2^{\alpha_2} \dots p_k^{\alpha_k}$.

Xiao: Through Section 3.2.4, we define $N = pq$, where p and q are primes of equal length. Xiao!

A tentative outline:

- By Chinese remainder theorem, $\mathbb{Z}_N^* \simeq \mathbb{Z}_p^* \times \mathbb{Z}_q^*$. Denote the isomorphism as $y \leftrightarrow (y_p, y_q)$.
- For $y \in \mathbb{Z}_N^*$ and $y \leftrightarrow (y_p, y_q)$, it can be proved that y is a QR in $\mathbb{Z}_N^*$ if and only if y_p is a QR in $\mathbb{Z}_p^*$ and y_q is a QR in $\mathbb{Z}_q^*$.
- The above implies: each QR $y \in \mathbb{Z}_N^*$ has exactly four square roots.
- Let QR_N set of quadratic residues modulo N. Let QNR_N set of quadratic non-residues modulo N. We have

  $$\frac{|\mathsf{QR}_N|}{|\mathbb{Z}_N^*|} = \frac{|\mathsf{QR}_p| \cdot |\mathsf{QR}_q|}{|\mathbb{Z}_N^*|} = \frac{\frac{p-1}{2} \cdot \frac{q-1}{2}}{(p-1)(q-1)} = \frac{1}{4}.$$

  Note that since $\mathbb{Z}_p^*$ is cyclic, we can easily show that $|\mathsf{QR}_p| = \frac{p-1}{2}$, i.e. half of the elements in $\mathbb{Z}_p^*$ are QRs.
- Also, for $x, y \in \mathbb{Z}_N^*$, we have

  $$\mathcal{J}_N(x \cdot y) = \mathcal{J}_N(x) \cdot \mathcal{J}_N(y) = \mathcal{J}_p(x) \cdot \mathcal{J}_q(x) \cdot \mathcal{J}_p(y) \cdot \mathcal{J}_q(y).$$
- Let $\mathcal{J}_N^+$ (resp. $\mathcal{J}_N^-$) denote the set of elements in $\mathbb{Z}_N^*$ whose Jacobi symbol is $+1$ (resp. -1).

Let QNR_N^+ denote the set of elements in QNR_N whose Jacobi symbol is $+1$. Then we can show the follows:

- $\mathbb{Z}_N^* = \mathcal{J}_N^- \cup \mathcal{J}_N^+$ and $|\mathcal{J}_N^-| = |\mathcal{J}_N^+|$;
- $\mathcal{J}_N^+ = \mathsf{QR}_N \cup \mathsf{QNR}_N^+$ and $|\mathsf{QR}_N| = |\mathsf{QNR}_N^+|$.

- Recall that when the factorization of N is unknown, there is no known polynomial-time algorithm for deciding whether a given x is QR or not. But, somewhat surprisingly, a polynomial-time algorithm is known for computing $\mathcal{J}_N(x)$ without the factorization of N.

- Quadratic residuosity assumption says that it is hard to tell between a random sample from QR and a random sample from QNR^+.

  Definition 3.2.1 (QR assumption)**.** Quadratic residuosity assumption assumes that there exists a generation algorithm Gen such that for all PPT algorithm Adv,

$$\big|\Pr[\mathsf{Adv}(N, \mathsf{qr}) = 1] - \Pr[\mathsf{Adv}(N, \mathsf{qnr}) = 1]\big| \leq \mathsf{negl}(\lambda),$$

  where the probabilities are taken over the following sampling $(N, p, q) \leftarrow \mathsf{Gen}(1^\lambda)$, $\mathsf{qr} \xleftarrow{\$} \mathsf{QR}_N$ and $\mathsf{qnr} \xleftarrow{\$} \mathsf{QNR}_N^+$. $\Diamond$

3.3 Rings

Definition 3.3.1 (Ring)**.** A ring is a set R equipped with two binary operations "+"(usually called *addition*) and $\cdot$ (usually called *multiplication*) satisfying the following properties:

1. R is an Abelian group under "+".
2. R is a monoid under "$\cdot$".[3]
3. The multiplication is distributive with respect to the addition, meaning that:
   - (Left Distributivity) For all $a, b, c \in R$, $a \cdot (b + c) = (a \cdot b) + (a \cdot c)$.
   - (Right Distributivity) For all $a, b, c \in R$, $(b + c) \cdot a = (b \cdot a) + (c \cdot a)$.

$\Diamond$

Definition 3.3.2 (Ideal)**.** A subring A of a ring R is called a (two-sided) ideal of R if for every $r \in R$ and every $a \in A$, both ra and ar are in A. $\Diamond$

Theorem 3.3.1. If A is an ideal of a ring R, then the quotient group R/A is a ring under the following operation:

- **Addition.** $(s + A) + (t + A) = (s + t) + A$
- **Multiplication.** $(s + A) \cdot (t + A) = (s \cdot t) + A$

[3] We remark that some mathematicians prefer to define the ring without multiplicative identity (the unity). So in their definition, R is a semigroup under "$\cdot$", instead of a monoid. But some other mathematicians prefer the current definition. We choose to use the current one because we almost always need the existence of unity. In this book, we put "(with unity)" wherever we want to address it.

$\diamondsuit$

There is special type of ideals defined on commutative rings that we are interested in, especially when we talk about polynomial rings later. It is called principal ideal.

Definition 3.3.3 (Principal Ideal)**.** Let R be a commutative ring (with unity) and let $a \in R$. The set $\langle a \rangle = \{ra \mid r \in R\}$ is an ideal of R. We call it the principal ideal generated by a. $\diamondsuit$

3.4 Fields

Definition 3.4.1: Field

A field is a set F equipped with two binary operations "+" (usually called *addition*) and "·" (usually called *multiplication*) satisfying the following properties:

1. F is an Abelian group under the addition.
2. $F \setminus \{0\}$ form an Abelian group under the multiplication
3. The multiplication is distributive over the addition.

3.5 Modules and Vector Spaces

Definition 3.5.1: Modules

Let R be a ring (not necessarily with unity). A left (resp. right) R-module over R is a set M together with:

1. a binary operation "+" under which M is an Abelian group.
2. a map $R \times M \to M$ (resp. $M \times R \to M$) denoted by "·", such that for all $r, s \in R$ and $m, n \in M$ the following holds:
   (a) $(r + s) \cdot m = r \cdot m + s \cdot m$ (resp. $m \cdot (r + s) = m \cdot r + m \cdot s$)
   (b) $(rs) \cdot m = r \cdot (s \cdot m)$ (resp. $m \cdot (rs) = (m \cdot r) \cdot s$)
   (c) $r \cdot (m + n) = r \cdot m + r \cdot n$ (resp. $(m + n) \cdot r = m \cdot r + n \cdot r$)

   If R has an unity 1, we impose an additional axiom to the map:

   (d) $1 \cdot m = m$ (resp. $m \cdot 1 = m$)

A Remark on the terminology: A *bimodule* is a module that is a left module and a right module such that the two multiplications are compatible. If R is commutative, then left R-modules are the same as right R-modules and are simply called R-modules[4]. Note that Item (d) is optional; modules satisfying it are called unital modules.

One elegant application of modules in cryptography appears in the famous Groth-Sahai [GS08] proof systems. It is not because they use fancy theorems specific to modules; rather, the concept of

[4]To some authors, "R-module" by default means "left R-module", e.g. [DF04].

modules provides a high-level abstract for groups equipped with bilinear maps, thus gives a clear and unified way to interpret their results.

Definition 3.5.2: Vector Space
Let $\mathbb{F}$ be a field. The $\mathbb{F}$-module is called a vector space over the field $\mathbb{F}$.

3.6 Integral Domains

We want to capture all the properties that integers enjoy. If we compare the definition of the ring to the set of integers, two important properties are missing: (1) commutativity and (2) cancellation property. Thus, people propose the concept of integral domain, which plays a prominent role in number theory and algebraic geometry.

Definition 3.6.1 (Unit)**.** we say that an element u of a ring R is a unit (also called "invertible element") if there is another element $v \in R$ such that $uv = vu = 1$. ♢

Definition 3.6.2 (Zero Divisors)**.** In a commutative ring R, $a \neq 0$ is a zero divisor if there is a nonzero element $b \in R$ such that $ab = 0$. ♢

Definition 3.6.3 (Integral Domain)**.** An integral domain is a commutative ring (with unity) that does not have zero divisors. ♢

Certain kinds of integral domain are of our interest. Next, we will list some related concepts and then study them in order.

Definition 3.6.4 (Association)**.** Elements a and b of an integral domain D are called associates if $a = ub$, where u is a unit of D. ♢

Definition 3.6.5 (Reducibility)**.** Let D be an integral domain. A non-zero, non-unit element a is called an irreducible if the following holds:

- whenever a is expressed as a product $a = bc$ with $b, c \in D$, then b or c is a unit.

A non-zero, non-unit element of D that is not irreducible is called reducible. ♢

Definition 3.6.6 (Primes)**.** In an integral domain, a non-zero, non-unit element a is called a prime if the following holds:

- $a|bc$ implies $a|b$ or $a|c$.

♢

3.6.1 Principal Ideal Domain (PID)

Definition 3.6.7 (Principal Ideal Domain)**.** An integral domain D is called a principal ideal domain if every ideal of D has the form $\langle a \rangle$ for some $a \in D$. ♢

Exercise 3.6.1. Here are some simple exercises to help you get a familiar with these concepts.

(a) In an integral domain, every prime is an irreducible.

(b) In a PID, an element is an irreducible if and only if it is a prime.

3.6.2 Unique Factorization Domain (UFD)

We now have the necessary terminology to formalize the idea of unique factorization.

Definition 3.6.8 (Unique Factorization Domain)**.** An integral domain D is a unique factorization domain if the following holds:

1. every non-zero, non-unit element of D can be written as a product of irreducibles of D,
2. the factorization into irreducibles is unique up to associates and the order in which the factors appear.

$\Diamond$

3.6.3 Euclidean Domain (ED) and GCD Domain

Definition 3.6.9 (Euclidean Domain (ED))**.** An integral domain D is called a Euclidean domain if there is a function d (called the measure) from the nonzero elements of D to the nonnegative integers such that:

1. $d(a) \leq d(ab)$ for all nonzero $a, b \in D$
2. if $a, b \in D$ and $b \neq 0$, then there exist elements q and r in D such that $a = bq + r$, where $r = 0$ or $d(r) < d(b)$.

$\Diamond$

From the above definition, it is easy to see that in an ED, the Euclidean algorithm is well defined. Actually, we call it "Euclidean Domain" because it is the integral domain where we can run Euclidean algorithm to compute the unique GCD between any pair of elements.

But we remark that GCD can be defined without referring to Euclidean algorithm. Actually, there is a strictly super-set of ED, called GCD domain, where GCD is defined but may not be unique, and Euclidean algorithm is not admitted.

Definition 3.6.10 (Greatest Command Divisor (GCD))**.** Let R is a commutative ring. We say that $d \in R$ is a greatest common divisor (GCD) of $a, b \in R$ if the following two conditions are satisfied:

1. $d|a$ and $d|b$.
2. For any $c \in R$ with $c|a$ and $c|b$, we have $c|d$.

$\Diamond$

Definition 3.6.11 (GCD Domain)**.** An integral domain D is a GCD domain if for each pair of $a, b \in D \setminus \{0\}$, there exists a greatest common divisor. $\Diamond$

Xiao: Here is my intuition which needs to be verified: GCD in an ED must be unique. But GCD in a GCD domain is not necessarily unique (counter examples?). Xiao!

Theorem 3.6.1 (Relations among different types of Rings)**.** The relations among different types of rings can be summarized as follows:

$$\text{ED} \subset \text{PID} \subset \text{UFD} \subset \text{GCD Domains} \subset \\ \text{Integrally Closed Domains} \subset \text{ Integral Domains} \subset \text{Commutative Rings}$$

Note that all the subset relations are proper. $\diamondsuit$

3.7 Polynomials

Xiao: Xiao!

- characteristic of a ring [Sho08, Page 168]
- Divisibility, units [Sho08, Section 7.1.1]
- Zero divisors and integral domains [Sho08, Section 7.1.2]
- The "Division with remainder property" theorem [Sho08, PAge 177]
- We are familiar with the fact that over the real or the complex numbers, every polynomial of degree k has at most k distinct roots, and the fact that every set of k points can be interpolated by a unique polynomial of degree less than k. Actually, these results extend to much more general, though not completely arbitrary, rings. This generalization is presented in [Sho08, Page 179 -181]
- Ideals and quotient ring [Sho08, Section 7.3], where the most important part is the quotient ring $R[X]/(f)$ defined in [Sho08, Example 7.39 on Page 186]
- Algorithm for polynomial division.
- Definitions for "irreducible polynomials", "associate" [Sho08, Page 428]
- Also need to make a subsection for cyclotomic polynomials and their properties. They are widely used in lattice-based cryptography such as the CKKS FHE scheme [CKKS17] and (Ring/Module)-SIS/LWE based constructions. There are many useful on-line notes for this topic. A good starting point is this note by Yang Li and Kee Siong Ng.

3.7.1 The Ring-Theory Definition

The abstract-algebraic interpretation of polynomials is to consider it as a special ring, i.e. the ring of polynomials. This is perhaps the most mathematically-correct approach to characterize polynomials.

An intuitive way to understand this interpretation is as follows: we start by adding an extra element x (called "indeterminate" or "variable") to a commutative[5] ring R. As we will see, it actually gives us a new ring, which we denote as $R[x]$. Let us consider the form of elements in $R[x]$. Because of the closure property of a ring, for any $a \in R$ and any $i \in \mathbb{N}$, ax^i should also be in $R[x]$, and so is their sum. Therefore, any expression of the form $a_nx^n + a_{n-1}x^{n-1} + \ldots + a_1x^1 + a_0$ should be an element in $R[x]$. This reminds us of the concept of polynomials. Moreover, it is easy

[5] If the ring is not commutative, we will need to distinguish between ax^2 and xax.

to prove that all elements of such a form do form a ring (i.e. all elements in $R[x]$ have such a form). Thus we name $R[x]$ as the "polynomial ring".

Definition 3.7.1 (Polynomial Ring)**.** Let R be a commutative ring. The set of formal symbols

$$R[x] = \{a_n x^n + a_{n-1}x^{n-1} + \ldots + a_1 x^1 + a_0 \mid a_i \in R, n \text{ is a nonnegative integer}\}$$

forms a ring under he natural polynomial addition and multiplication operation, with the natural identity elements for addition and multiplication. ♢

Exercise 3.7.1

Here are some interesting exercises to reveal the relation between a polynomial ring and its underlying ring.

1. If D is an integral domain, then $D[x]$ is an Integral Domain.
2. If F is a field, then $F[x]$ is a Principal Ideal Domain.
3. If F is a field, then $F[x]$ is a Euclidean Domain (with the degree of polynomials as the Euclidean measure).

The reducibility concept of polynomials is just an instantiation of the reducibility of a standard integral domain on an ID of polynomials (see Def. 3.6.5).

Definition 3.7.2: Reducibility of Polynomials over an ID

Let D be an integral domain. A non-zero, non-unit element $f(x) \in D[x]$ is irreducible over D if the following holds:

- whenever $f(x)$ is expressed as a product $f(x) = g(x) \cdot h(x)$ with $g(x), h(x) \in D[x]$, then $g(x)$ or $h(x)$ is a unit in $D[x]$.

A non-zero, non-unit element of $D[x]$ that is not irreducible over D is called reducible over D.

3.7.2 Schwartz-Zipple lemma

A crypto application of Schwartz-Zipple can be found in [KOS18].

But this lemma is widely used in PCP theorem, sum-check protocols and property testing.

An excellent survey of this lemma can be found in this article by Lipton.

Theorem 3.7.3: Schwartz-Zipple Lemma

Suppose that $P(x_1, \ldots, x_n) \in \mathbb{F}[x_1, \ldots, x_n]$ is a non-zero polynomial of total degree d over a field $\mathbb{F}$, and S is a non-empty subset of the $\mathbb{F}$. Then,

$$\Pr\left[P(x_1, \ldots, x_n) = 0\right] \leq \frac{d}{|S|}.$$

3.7.3 The Fundamental Theorem of Algebra

Xiao: Add The Fundamental Theorem of Algebra here

Xiao!

3.7.4 On $\mathbb{F}_{p^n}$: An Application for [Sah99] NIZK

In [Sah99], Sahai used the number of roots of polynomials on finite fields to design a clever mechanism that enjoys the following property: for some parameters ℓ and t, it allows one to sample t (a fixed polynomial) sets of size ℓ, such that no $(t-1)$ sets out of these t sets cover the remaining one. This mechanism is essential to extend the famous [Sah99] non-malleable NIZK to support (bounded) multiple proofs.

Xiao!

Xiao: Add this application here. Abstract from [Sah99].

3.7.5 Shamir's Secret Sharing

We start with the famous Lagrange's interpolation, which is an elegant method to find a polynomial that satisfies a bunch of points.

Algorithm 3.7.1 (Lagrange's Interpolation)**.** Given a set of $k+1$ data points:

$$(x_0, y_0), \ldots, (x_i, y_i), \ldots, (x_n, y_n)$$

where no two x_i's are the same, the interpolation polynomial in the Lagrange form is defined as:

$$L(x) = \sum_{i=0}^{n} y_i \cdot \ell_i(x) \tag{3.1}$$

where each ℓ_i is:

$$\ell_i(x) := \prod_{\substack{0 \le m \le k \\ m \ne i}} \frac{x - x_m}{x_i - x_m} = \frac{(x - x_0)}{(x_i - x_0)} \cdots \frac{(x - x_{i-1})}{(x_i - x_{i-1})} \frac{(x - x_{i+1})}{(x_i - x_{i+1})} \cdots \frac{(x - x_n)}{(x_i - x_n)} \tag{3.2}$$

We have $L(x_i) = y_i$ for all $i \in \{0, \ldots, n\}$. And $L(x)$ is a polynomial of degree at most n.

Lagrange's interpolation can be generalized to any finite field, with the corresponding field operation.

Remark 3.7.4:
We remark that Lagrange's interpolation allow us to recover the whole polynomial express of $L(x)$. Actually, we can also recover $L(x^*)$ at a certain point x^*. To do that, just evaluate $\ell_i(x^*)$'s according to Equation (3.2), and plug them into Equation (3.1). This is a simple observation, but it turns to be very useful for building the Fuzzy IBE scheme in [SW05].

With the understanding of Lagrange's interpolation, we are know ready to present Shamir's Secrete Sharing scheme.

Algorithm 3.7.2 (Shamir's Secrete Sharing)**.** A t-out-of-n secrete sharing scheme can be constructed in the following way.

Given a finite filed $\mathbb{F}$, to share a secrete s:[6]

1. Choose $a_1, \ldots a_{t-1} \xleftarrow{\$} \mathbb{F}$.

2. Define a polynomial $f(x) = s + a_1 x + \ldots a_t x^{t-1}$.

[6]W.l.o.g., we assume $s \in \mathbb{F}$

3. Choose n distinct points $x_1, \ldots, x_n \in \mathbb{F}$.

4. For $i \in [n]$, output $(x_i, f(x_i))$ as the secret share for party P_i.

When t or more parties try to recover the secrete, they can recover the polynomial $f(x)$ using Lagrange's interpolation, and then learn the secrete s from the constant term of $f(x)$.

3.7.6 Verifiable Secret Sharing

Xiao: add VSS Xiao!

3.8 Discrete Fourier Transform

"Young man, in mathematics you don't understand things. You just get used to them."

— John Von Neumann

Usually, I'm against the above saying; but for Fourier transform, I surrender. Xiao!

Xiao:

- Distinguish between the terminologies: "discrete transform", "fast Fourier transform" and "Fourier transform".
- talk about it's application to efficient integer multiplication
- See this YouTube playlist for an amazing series of talks on Fourier Transform (and Fourier analysis in general).
- Also, relates to the quantum Fourier transform in Section 4.6.

The term "Fourier transform" can refer to different things. To avoid confusion, let me summarize it in Table 3.1

Table 3.1: Various Types of Fourier Transform

Terminology	**Signal Continuity**	**Signal Periodicity**
Fourier Transform	continuous	infinite
Fourier Series	continuous	finite
Discrete-Time Fourier Transform	discrete	infinite
Discrete Fourier Transform	discrete	finite

The easiest way to memorize the dimension-N DFT is to view it as a Vandermonde matrix, where the i-th row corresponds to ω_N^{i-1}, where $\omega_N = e^{\frac{2\pi}{N}i}$. I.e.,

$$\mathsf{DFT}_N \coloneqq \begin{bmatrix} (\omega_N^0)^0 & (\omega_N^0)^1 & \cdots & (\omega_N^0)^{N-1} \\ (\omega_N^1)^0 & (\omega_N^1)^1 & \cdots & (\omega_N^1)^{N-1} \\ \vdots & & & \vdots \\ (\omega_N^{N-1})^0 & (\omega_N^{N-1})^1 & \cdots & (\omega_N^{N-1})^{N-1} \end{bmatrix} \tag{3.3}$$

Properties:

- DFT_N is unitary (up to a scaling factor $1/\sqrt{N}$). That is, $\frac{\mathsf{DFT}_N}{\sqrt{N}}$ is unitary. In other words,

$$\mathsf{DFT}_N \mathsf{DFT}_N^\dagger = \mathsf{DFT}_N^\dagger \mathsf{DFT}_N = N \mathbb{1}_N.$$

Quantum Fourier Transform (QFT). It is worth mentioning that the dimension-N quantum Fourier transform (in Section 4.6) are nothing but the quantum-circuit implementation of the unitary map $\frac{\mathsf{DFT}_N}{\sqrt{N}}$.

Fast Fourier Transform (FFT). Fast Fourier transform is an algorithm that computes the $\mathsf{DFT}_N \mathbf{x}$ in a fast way, where $\mathbf{x} \in \mathbb{C}^N$. Note that the naïve implementation cost $O(N^2)$ field operations (in $\mathbb{C}$); in contrast, FFT allows us to do that in $O(N \log N)$ field operations.

3.8.1 DFT vs the Continuous FT (for Signal Processing)

Quick Recap: We start with the Continuous Fourier Transform (CFT):

$$F(f) = \int_{-\infty}^{\infty} f(t) e^{-i2\pi f t} \, dt$$

Then, we sampled the signal $f(t)$ at regular intervals $t = n\Delta t$, giving the sampled discrete signal $x[n] = f(n\Delta t)$. Substituting into the CFT gives:

$$F(f) \approx \sum_{n=0}^{N-1} x[n] e^{-i2\pi f n \Delta t} \Delta t$$

At this point, we discretized the frequency domain as well, setting $f_k = \frac{k}{N\Delta t}$, and arrived at the DFT formula:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-i\frac{2\pi}{N} kn}$$

What Happened to Δt in the Summation? The term Δt does indeed disappear from the final DFT formula. However, this disappearance reflects a **change in units**. Here's what's happening:
1. **In the CFT, $F(f)$ has units of amplitude per Hz:** The integral dt ensures that $F(f)$ represents the amplitude per unit frequency (since $f(t)$ is spread over a continuous spectrum of frequencies).
2. **In the DFT, $X[k]$ is unitless or scaled by Δt:** When transitioning to the DFT, the summation replaces the integral, but the original Δt is absorbed into how the DFT coefficients $X[k]$ are interpreted. Specifically: - The DFT assumes the **frequency resolution** is $\Delta f = \frac{1}{N\Delta t}$, and the spacing in the frequency domain accounts for the missing Δt. - $X[k]$ represents the amplitude of a discrete frequency, not a continuous spectrum.

Where Exactly Does Δt Go? Let's rewrite the CFT and DFT more carefully to track Δt explicitly.

1. **Continuous Fourier Transform (CFT):**

$$F(f) = \int_{-\infty}^{\infty} f(t)e^{-i2\pi ft}\,dt$$

2. **Sampling the Signal:** Sampling $f(t)$ at $t = n\Delta t$ gives:

$$x[n] = f(n\Delta t)$$

   Now approximate the integral as a summation over N samples:

$$F(f) \approx \sum_{n=0}^{N-1} x[n]e^{-i2\pi fn\Delta t}\Delta t$$

   Here, the term Δt is still present, and each term in the summation is scaled by Δt.

3. **Discretizing Frequency:** Set $f = f_k = \frac{k}{N\Delta t}$. Substituting this into the summation:

$$F(f_k) \approx \sum_{n=0}^{N-1} x[n]e^{-i2\pi \frac{k}{N\Delta t}n\Delta t}\Delta t$$

   The Δt in the exponential cancels out:

$$F(f_k) \approx \sum_{n=0}^{N-1} x[n]e^{-i\frac{2\pi}{N}kn}\Delta t$$

4. **Defining the DFT:** The DFT is defined as:

$$X[k] = \sum_{n=0}^{N-1} x[n]e^{-i\frac{2\pi}{N}kn}$$

   Notice that Δt is no longer explicitly present in the formula for $X[k]$. This happens because $X[k]$ inherently scales with Δt when interpreted in the context of the original continuous signal. Specifically: - If you want to interpret $X[k]$ as a scaled version of the continuous Fourier transform $F(f)$, you need to account for the implicit scaling by Δt.

Scaling and Interpretation - The summation in the DFT assumes the signal is sampled at a regular interval Δt. - Δt is absorbed into the frequency resolution $\Delta f = \frac{1}{N\Delta t}$, which determines how the discrete frequencies $f_k = \frac{k}{N\Delta t}$ are spaced. - If you were to reconstruct the continuous Fourier transform from the DFT, you would need to multiply the DFT coefficients by Δt to correctly scale them.

Conclusion. In the transition from the continuous Fourier transform to the discrete Fourier transform: 1. Δt appears explicitly in the summation when approximating the integral but is absorbed into the interpretation of $X[k]$ in the DFT. 2. The DFT coefficients $X[k]$ represent the amplitudes of discrete frequency components, and their scaling implicitly depends on the sampling interval Δt. 3. If you need to directly compare the DFT to the CFT, you must multiply $X[k]$ by Δt to restore the proper scaling.

So, Δt hasn't vanished—it's embedded in the interpretation of the DFT coefficients and the frequency resolution.

Chapter 4

Linear Algebra for Quantum Information Theory

Xiao: The familiarity with the following topics represents minimal background requirements for quantum information theory. A great book for them is [Axl15]. Xiao!

- Define Hermitian matrix, positive semi-definite matrices. And talk about the eigenvalue decomposition of them.
- Spectral decomposition (aka eigen-decomposition) is the factorization of a matrix into a canonical form, whereby the matrix is represented in terms of its eigenvalues and eigenvectors. Only diagonalizable matrices can be factorized in this way. This decomposition captures the essence of density matrix (in quantum computing): every density matrix ρ (i.e. positive semi-definite matrix with trace 1) have a spectral decomposition, where eigenvalues are non-negative and the sum to 1, and the eigenvectors constitute orthonormal basis.
- Hilbert Space. The 2nd chapter of this lecture notes is a good reference.
- Schmidt Decomposition. Define Schmidt decomposition and talk about its application in Uhlmann's Theorem. See this.

  A nice presentation for Schmidt Decomposition can be found at [KLM06, Section 2.7]. Show the example that Schmidt Decomposition makes the computation of partial trace easier (from [KLM06, Section 3.5.2]). Moreover, the way to compute Schmidt Decomposition can be found at [KLM06, Appendix A.7].

4.1 Linear Algebra 101

4.1.1 The Second Nature

Let us make the following our second nature:

1. $\operatorname{tr}(A+B) = \operatorname{tr}(A) + \operatorname{tr}(B)$ and $\operatorname{tr}(A) = \operatorname{tr}(A^T)$.
2. **(Cyclic property of trace.)** $\operatorname{tr}(ABC) = \operatorname{tr}(CAB) = \operatorname{tr}(BCA)$.
3. $\operatorname{tr}(A \otimes B) = \operatorname{tr}(A)\operatorname{tr}(B)$. (Note that $\operatorname{tr}(AB) \neq \operatorname{tr}(A)\operatorname{tr}(B)$)
4. $(AB)^* = A^*B^*$ and $(AB)^\dagger = B^\dagger A^\dagger$.
5. $(A \otimes B)^* = A^* \otimes B^*$, $(A \otimes B)^T = A^T \otimes B^T$, and $(A \otimes B)^\dagger = A^\dagger \otimes B^\dagger$, where "$\dagger$" denotes Hermitian transpose (aka conjugate transpose).
6. The Kronecker product operator "$\otimes$" is both *bilinear* and *associative*.
7. Another way to state Hadamard gate: for $b \in \{0,1\}$, $H|b\rangle = \frac{1}{\sqrt{2}}(|0\rangle + (-1)^b |1\rangle)$.

8. **(Phase Kickback Trick.)** This is also also called “Phase Oracle”. It says, for any classical function $f : \{0, \ldots, d-1\} \to \{0, 1\}$ and any vector $|j\rangle$ *in the computational basis*[1] (i.e., $j \in \{0, \ldots, d-1\}$ for a d-dimensional system), it holds that

$$U_f |j\rangle |-\rangle = (-1)^{f(j)} |j\rangle |-\rangle ,$$

where U_f is the reversible implementation of f, i.e., $U_f |x, b\rangle = |x, b \oplus f(x)\rangle$.

Two main Workhorses from Quantum Computing. This trick is one of the main workhorse from quantum computational power. It is behind the Deutsch [Deu85], Deutsch-Jozsa [DJ92], Bernstein-Vazirani [BV93], and Grover's Search [Gro96] algorithms. (Simon [Sim94] and Shor [Sho94] did not use this trick because the target functions they considered have a different range than $\{0, 1\}$.)

Another workhorse is quantum Fourier transform, which is behind Simon's algorithm, period-finding, discrete algorithms, and hidden subgroup problems. Most importantly, a particularly useful implication of quantum Fourier transform is an algorithm called *phase estimation*. Phase estimation is behind the celebrated Shor's algoritm (i.e., factoring/order-finding)[2], and many applications related to simulating quantum systems.

9. **(The Special QFT $H^{\otimes n}$.)** A workhorse formula appeared in several quantum algorithms:

$$H^{\otimes n} |x_1, \ldots, x_n\rangle = \frac{1}{2^{n/2}} \sum_{y \in \{0,1\}^n} (-1)^{\langle x, y\rangle} |y_1, \ldots, y_n\rangle ,$$

where $x = (x_1, \ldots, x_n)$ and $y = (y_1, \ldots, y_n)$ are binary vectors, and $\langle \cdot, \cdot \rangle$ is the inner product (mod 2)[3]. This is pretty natural—indeed, $H^{\otimes n}$ is just the QFT for the space $(\mathbb{C}^2)^{\otimes n}$.

This is the main magic behind Simon's algorithm [Sim94].

4.1.2 High-Dimension Analog of the Imaginary Unit

Definition 4.1.1: Positive Operators (aka Positive Semi-definite Matrices)

A complex-valued matrix M is a positive semi-definite (resp. definite) matrix if:

- M is Hermitian (thus being normal, thus admitting spectral decomposition).
- All the eigenvalues of M is non-negative (resp. positive).

Positive semi-definite matrices are also called *positive operators*; Positive definite matrices are also called *positive definite operators*.[a]

[a] Some authors refer to positive operators (i.e., semi-definite matrices) as “non-negative operators” (e.g., [Ren08]), to avoid the confusion with positive definite operators

A useful way to see unitary/hermitian/positive operators:

[1] Note that $f(\cdot)$ is only defined on classical string j. It does not make sense to take about $f(x)$ is $|x\rangle$ is a general quantum state, i.e., x cannot be expressed as a classical string.

[2] Shor's algorithm consists of two steps: (1) a classical reduction from factoring to the order-finding problem; (2) a quantum step where phase estimation is used to solve the order-finding problem.

[3] Actually, it does not matter if it is mod 2 or not, as the base is -1.

- Unitary operators U can be viewed as a high-dimension analog of $i = \sqrt{-1}$, because $U^\dagger U = UU^\dagger = I$.
- Hermitian operators H can be viewed as a high-dimension analog of real numbers, because $H = H^\dagger$.
- Positive operators (aka positive semi-definite matrices) can be viewed as a high-dimension analog of non-negative real numbers.
- Positive definite matrices can be viewed as a high-dimension analog of positive real numbers.
- Projectors can be viewed as a high-dimension analog of $\{0, 1\}$ (as $\{0, 1\}$ are the only possible eigenvalues for projectors).
- Density operators (i.e., trace-1 and positive operators) naturally generalizes probability distribution (as their eigenvalues are non-negative and sum up to 1).

The above intuition is well-illustrated by the spectral decomposition of these operators (see Section 4.1.8).

4.1.3 Inner Product Spaces and the Cauchy-Schwarz Inequality

Cauchy-Schwarz Inequality is one of the most widely used inequality. It is worth emphasizing that Cauchy-Schwarz ineqautliy holds in *any inner product space*. Let us first define inner product space formally.

Definition 4.1.2: Inner Product Spaces

Let $\mathcal{V}$ be a vector space over the field $\mathbb{F}$ which is either $\mathbb{R}$ or $\mathbb{C}$. An *inner product* on $\mathcal{V}$ is a map

$$\langle \cdot, \cdot \rangle : \mathcal{V} \times \mathcal{V} \to \mathbb{F}$$

that satisfies the following properties:

1. **Linearity in the second argument.** For any $x, y, z \in \mathcal{V}$ and any $s \in \mathbb{F}$:
$$\langle x, y + z \rangle = \langle x, y \rangle + \langle x, z \rangle \quad \text{and} \quad \langle x, y \rangle = s\langle x, y \rangle.$$
2. **Hermitian Symmetry (or Conjugate Symmetry).** For any $x, y \in \mathcal{V}$, $\langle x, y \rangle = \overline{\langle y, x \rangle}$.
3. **Positive Definiteness.** For any $x \in \mathcal{V}$, if $x \neq \mathbf{0}$, then $\langle x, x \rangle > 0$. (Note that this implies that $\langle x, x \rangle$ must be real, even if $\mathbb{F}$ is $\mathbb{C}$.)

An *inner product space* is a vector space $\mathcal{V}$ over $\mathbb{F}$ (being either $\mathbb{R}$ or $\mathbb{C}$) along with an inner product on $\mathcal{V}$.

Here are some remarks:

- Some authors define linearity in the first argument. But linearity in the second argument seems more natural
- Note that Properties 1 and 2 imply the following rules for the first argument:
$$\langle x + y, z \rangle = \langle x, z \rangle + \langle y, z \rangle \quad \text{and} \quad \langle sx, z \rangle = \overline{s}\langle x, z \rangle.$$

Definition 4.1.3: Induced Norm on a Inner Product Space

Every inner product gives rise to a norm, called the *canonical* or *induced norm*, where the norm of a vector x is denoted and defined by: $\|x\| := \sqrt{\langle x, x\rangle}$.

Remark 4.1.1 (On Positive Definiteness)**.** *Assuming $\langle \mathbf{0}, \mathbf{0}\rangle = 0$ holds (which follows from Property 1), then Property 3 will hold if and only if both Properties 4 and 5 below hold:*

4. **Definiteness (or Point-Separating).** *If $\langle x, x\rangle = 0$, then it must be that $x = \mathbf{0}$.*

5. **Positive Semi-definiteness (or Non-Negative-Definiteness).** $\forall x \in \mathcal{V}$, $\langle x, x\rangle \geq 0$.

Thus, Properties 1 to 5 are satisfied by every inner product.

Theorem 4.1.4: Cauchy-Schwarz Inequality

For all vectors x, y of an inner product space (see Definition 4.1.2), it holds that

$$|\langle x, y\rangle|^2 \leq \langle x, x\rangle \cdot \langle y, y\rangle, \quad (\text{or equivalently, } |\langle x, y\rangle| \leq \|x\| \cdot \|y\|)$$

where $\|\cdot\|$ is the induced norm over the inner product space (see Definition 4.1.3), and $|\cdot|$ denotes the modulus of complex numbers[a].

[a] Note that the output of the inner product operation must be an element in $\mathbb{F}$, which is either $\mathbb{R}$ or $\mathbb{C}$. Thus, $|\langle x, y\rangle|$ is always well defined for any $\langle x, y\rangle$.

4.1.4 Direct Sum

Xiao: Need to talk about the concept of "direct sum". It is important to decompose vector spaces (e.g., in Jordan's Lemma in Section 4.9). A good starting point for this concept is [Axl15, Section 1.C]. Xiao!

4.1.5 Tensor Products of Hilbert Spaces

At the center of quantum computing lies tensor product of vector spaces. The pure-mathematical way to approach this concept involves formal discussion on tensor products of modules. Indeed, vector spaces are just a special type of modules. The reader can find comprehensive explanation for this perspective in [DF04, Chapter 10.4 and 11.5] and this lecture note by Prof. Conrad.

Fortunately, for quantum computing, we only need to focus on a very specific type of the above concept, i.e. tensor product of Hilbert spaces. Before throwing out the definition, let me first motivate it. We know that each isolated quantum system can be represent by a Hilbert space. Now what should we do if we want to describe two (or more) quantum systems? To do that, ideally, we hope to combine spaces V and W in a way that *reserves all the good mathematical properties*. For example, the resulted space would better also be a Hilbert space, which be definition admits some well-defined inner product operation. This turns to be achievable exploiting the Kronecker product operation "$\otimes$".

Definition 4.1.5: Tensor Product of Hilbert Spaces

Let V and W be two Hilbert spaces. The tensor product of them is the Hilbert space $V \otimes W$ whose elements are linear combinations of $|v\rangle \otimes |w\rangle$ where $|v\rangle \in V$, $|w\rangle \in W$ and $\otimes$ is the Kronecker product. (The induced inner product operation is defined in Property 2.)

It is obvious that the above definition is well-defined, i.e. $V \otimes W$ as defined above is indeed a vector space. We now state two important facts about $V \otimes W$:

1. It can be proved that the linear operators on $V \otimes W$ are captured by matrix Kronecker product $\mathbf{A} \otimes \mathbf{B}$, where $\mathbf{A}$ and $\mathbf{B}$ are linear operators on V and W respectively. Namely, for any $|v\rangle \in V$ and $|w\rangle \in W$,
$$(\mathbf{A} \otimes \mathbf{B})(|v\rangle \otimes |w\rangle) = \mathbf{A}|v\rangle \otimes \mathbf{B}|w\rangle .$$
2. It can be proved that $V \otimes W$ allows the following (natural) inner product $\langle \cdot, \cdot \rangle$:
$$\langle \sum_i a_i |v_i\rangle \otimes |w_i\rangle , \sum_j b_j |v'_j\rangle \otimes |w'_j\rangle\rangle = \sum_{i,j} a_i^* b_j \langle v_i | v'_j\rangle \langle w_i | w'_j\rangle .$$

4.1.6 Mixed-Product Property of Kronecker Product

This property is used everywhere in quantum computing/information theory paper. However, I didn't found a place where it is formally addressed. Thus, I will do it here.

This is a general property of the Kronecker product operation.

Lemma 4.1.6: Mixed-Product Property of Kronecker Product

Given matrices A, B, C and D, it holds that
$$(A \otimes B)(C \otimes D) = (AC) \otimes (BD),$$
as long as *one can form the matrix products AC and BD*. That is, the number of columns of A (resp. B) equals the number of rows of C (resp. D).

As a special case of the mixed-product proerpty, we have the following inequality which is particularly useful when computing partial traces:
$$|\alpha_0\beta_0\rangle \langle\alpha_1\beta_1| = (|\alpha_0\rangle \otimes |\beta_0\rangle)(\langle\alpha_1| \otimes \langle\beta_1|) = |\alpha_0\rangle\langle\alpha_1| \otimes |\beta_0\rangle\langle\beta_1| .$$

Another popular use of this property is to factor-out tensor products of states.

- $|a\rangle_A |b\rangle_B = (|a\rangle_A \otimes \mathbb{1}_B)(1 \otimes |b\rangle_B) = (|a\rangle_A \otimes \mathbb{1}_B)|b\rangle_B$.
- $|a\rangle_A |b\rangle_B = (\mathbb{1}_A \otimes |b\rangle_B)(|a\rangle_A \otimes 1) = (\mathbb{1}_A \otimes |b\rangle_B)|a\rangle_A$.
- $|a\rangle_A \otimes \mathbb{1}_B = (\mathbb{1}_A \otimes \mathbb{1}_B)(|a\rangle_A \otimes \mathbb{1}_B)$. We emphasis that $|a\rangle_A \otimes \mathbb{1}_B \neq (\mathbb{1}_A \otimes \mathbb{1}_B)(|a\rangle_A \otimes 1) = |a\rangle_A$. This is a misunderstanding that tends to happen among beginners. **Remember that always check if the dimensions match as w.r.t. MATRIX multiplication. Matrix-scalar multiplication does not count.**
- In summary, we have
$$|a\rangle_A |b\rangle_B = (\mathbb{1}_A \otimes \mathbb{1}_B)(\mathbb{1}_A \otimes |b\rangle_B)|a\rangle_A = (\mathbb{1}_A \otimes \mathbb{1}_B)(|a\rangle_A \otimes \mathbb{1}_B)|b\rangle_B .$$

4.1.7 Projectors and the Completeness Relation for Orthonormal Basis

A very important linear operators is projectors (or projections).

Definition 4.1.7: Projectors

A projector on a vector space V is a linear operator $\mathbf{P} : V \to V$ such that $\mathbf{P}^2 = \mathbf{P}$. On a subspace $\mathcal{W} \subseteq \mathcal{V}$, we say that $\mathbf{P}$ is a *rank-k* projector if there are k eigenvalue-1 eigenvectors of $\mathbf{P}$ falling in $\mathcal{W}$ (i.e., k is the dimension of the intersection space between $\mathcal{W}$ and the space $\mathbf{P}$ projects onto). Note that $\mathbf{P}$ is a rank-d projector in the whole space $\mathcal{V}$ where d is $\mathbf{P}$'s matrix rank.

Here are some properties of projectors:

- The eigenvalues of projectors take values from $\{0, 1\}$: Let λ denote the eigenvalue of a projector $\mathbf{P}$. By definition, $\lambda^2\mathbf{u} = \mathbf{P}^2\mathbf{u} = \mathbf{P}\mathbf{u} = \lambda\mathbf{u}$, which implies that $\lambda \in \{0, 1\}$.
- It is not true that projectors are always Hermitian. Counterexample:

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix} \neq \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix} = A^\dagger.$$

  This counterexample also shows that not all projectors are normal:

$$AA^\dagger = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \neq \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} = A^\dagger A.$$

Definition 4.1.8: Orthogonal and Oblique Projectors

An *orthogonal projector* is a projector that also satisfies $\mathbf{P}^\dagger = \mathbf{P}$ (i.e. Hermitian). A projection matrix that is not an orthogonal projection matrix is called an *oblique projection matrix.*
It is worth noting that an orthogonal projector is always normal, thus supports spectral decomposition (see Definition 4.1.9 and Theorem 4.1.11).

Xiao: Orthogonal projectors are an important class of projectors and enjoy interesting properties. Add more discussion about them. (See here.) Xiao!

Geometrically, projectors represent the projection operation from V to its subspace (depend on P). Namely, if we have a vector $\mathbf{v} \in V$, $\mathbf{P}\mathbf{v}$ is a vector lies in the subspace of V that is defined according to $\mathbf{P}$; $\mathbf{P}^2 = \mathbf{P}$ just reflects the fact that once the vector $\mathbf{v}$ is brought to the subspace, further applications of $\mathbf{P}$ will not move it anymore.

[NC11, Section 2.1.6] takes an alternative (and equivalent) way to define projectors. It considers a dimension-d vector space V and a dimension-k subspace $W \subseteq V$ (where $k \leq d$). By Gram-Schmidt procedure, it is easy to see that there is a set of orthonormal basis $\{|1\rangle, \ldots, |d\rangle\}$ such that $\{|1\rangle, \ldots, |k\rangle\}$ constitutes a set of orthonormal basis for W. Then the projector onto the subspace W can be defined as

$$\mathbf{P} = \sum_{i=1}^{k} |i\rangle \langle i|.$$

Then Definition 4.1.7 simply follows as a property. This approach also reveals the connection between projectors and *completeness relation* for orthonormal vectors.

Xiao: talk about the relation between *completeness relation* for orthonormal vectors and *projectors* to subspace. Useful materials can be found at Section 2.1.6 and Section 2.1.4 of [NC11]. Xiao!

An example: the space $\mathbb{R}^2$ is spanned by orthonormal basis $\{\begin{pmatrix}1\\0\end{pmatrix}, \begin{pmatrix}0\\1\end{pmatrix}\}$. It is easy to check that it satisfies the completeness relation. However, when $\begin{pmatrix}1\\0\end{pmatrix}$ and $\begin{pmatrix}0\\1\end{pmatrix}$ are treated as the orthonormal basis for the subspace $\mathbb{R}^2$ of a larger space $\mathbb{R}^3$, they should be augmented as $\{\begin{pmatrix}1\\0\\0\end{pmatrix}, \begin{pmatrix}0\\1\\0\end{pmatrix}\}$. In this form, they do not satisfies the completeness relation anymore. To fix that, we need to add the third element $\begin{pmatrix}0\\0\\1\end{pmatrix}$ in the set of orthonormal basis for $\mathbb{R}^3$.

4.1.8 Normal Operators and Spectral Decomposition

Definition 4.1.9: Normal Operator

A normal operator on a complex Hilbert space is a continuous linear operator $\mathbf{P}$ such that

$$\mathbf{P}^\dagger \mathbf{P} = \mathbf{P}\mathbf{P}^\dagger.$$

(Since the term "linear operators" can be used interchangeably with "matrices", people also refer to "normal operators" as "normal matrices".)

The following theorem provides an extremely important characterization of normal projectors. It is a special case of the famous Spectral Theorem (see also the discussion in Section 4.1.9).

Theorem 4.1.10: Unitary Diagonalization of Normal Matrices

An operator $\mathbf{A}$ on a complex Hilbert space is normal if and only if it is unitarily diagonalizable. Namely, it can be written as $\mathbf{A} = \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^\dagger$, where $\mathbf{U}$ is a unitary matrix and $\boldsymbol{\Lambda}$ is a diagonal matrix.

Proof. By the Schur decomposition, we can write any complex matrix as $\mathbf{A} = \mathbf{U}\Lambda\mathbf{U}^\dagger$, where $\mathbf{U}$ is unitary and Λ is upper-triangular. This implies that if $\mathbf{A}$ is normal, we must have $\Lambda\Lambda^\dagger = \Lambda^\dagger\Lambda$ (i.e. Λ is also normal). Therefore, Λ must be diagonal: a normal upper triangular matrix is diagonal. The converse is obvious. ∎

Theorem 4.1.11 appears in a slightly different form in [NC11, Section 2.1.6]. Since this form is more quantum-mechanics friendly, we present it in the following.

Theorem 4.1.11: Spectral Decomposition of Normal Operators

A linear operator $\mathbf{P}$ on a vector space V is normal if and only if it is **diagonalizable with respect to some orthonormal basis for** V, i.e. it can be decomposed as:

$$\mathbf{P} = \sum_i \lambda_i |i\rangle \langle i|, \tag{4.1}$$

where $(\lambda_i, |i\rangle)$'s are the eigenvalue-eigenvector pairs of $\mathbf{P}$, and $\{|i\rangle\}_i$ form an orthonormal basis for V.

Remark 4.1.2 (On the Necessity of Orthonormal Basis in Theorem 4.1.11)**.** *We emphasize the "with respect to some orthonormal basis" part in Theorem 4.1.11. In more detail, normal operators are always diagonalizable, but the converse is not true in general: a diagonalizable matrix need not be normal.*

For example, consider

$$A = \begin{pmatrix} 1 & 1 \\ 0 & 2 \end{pmatrix}.$$

The eigenvalues of A are

$$\lambda_1 = 1, \qquad \lambda_2 = 2.$$

Corresponding eigenvectors are

$$v_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad \textit{for } \lambda_1 = 1, \qquad v_2 = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad \textit{for } \lambda_2 = 2.$$

Since A has two linearly independent eigenvectors, it is diagonalizable.

However, A is not normal. Indeed, since A is real, normality means

$$AA^T = A^T A.$$

We compute

$$A^T = \begin{pmatrix} 1 & 0 \\ 1 & 2 \end{pmatrix}.$$

Thus

$$AA^T = \begin{pmatrix} 1 & 1 \\ 0 & 2 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 1 & 2 \end{pmatrix} = \begin{pmatrix} 2 & 2 \\ 2 & 4 \end{pmatrix},$$

whereas

$$A^T A = \begin{pmatrix} 1 & 0 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 0 & 2 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 5 \end{pmatrix}.$$

Therefore,

$$AA^T \neq A^T A,$$

so A is not normal. This shows that diagonalizability alone does not imply normality; what matters in the spectral theorem is diagonalizability with respect to an orthonormal basis.

In terms of projectors, Equation (4.1) can also be written as $\mathbf{P} = \sum_i \lambda_i \mathbf{P}_i$, where λ_i are again the eigenvalues of $\mathbf{P}$, and $\mathbf{P}_i$ is the projector onto the λ_i eigenspace of $\mathbf{P}$. These projectors satisfy the completeness relation $\sum_i \mathbf{P}_i = I$, and the orthonormality relation $\mathbf{P}_i \mathbf{P}_j = \delta_{ij} \mathbf{P}_i$, where δ_{ij} is the Kronecker delta.

Here are some special normal operators (thus enjoying the spectral decomposition):

- **Unitary Operators:** operators represented by matrix U such that $U^\dagger U = I$.
- **Hermitian Operators:** operators represented by matrix H such that $H^\dagger = H$.
- **Positive Operators:** operators represented by positive semi-definite matrices (Definition 4.1.1). Note that these operators are Hermitian by definition. Also, note that density operators are necessarily positive (thus normal, thus diagonalizable).

These special types of normal operators of course enjoy richer properties that general normal operators does not. That can be illustrated by their respective spectral theorems:

Theorem 4.1.12: Spectral Decomposition of Hermitian Operators

Let $\mathbf{H}$ be a Hermitian operator on a dimension-n vector space $\mathcal{V}$. Let its spectral decomposition be $\mathbf{H} = \sum_{i=1}^{n} \lambda_i \mathbf{v}_i$. Then, the following hold:

1. $\forall i \in [n]$, λ_i is real;
2. The collection $\{\mathbf{v}_i\}_{i=1}^{n}$ forms an orthonormal basis for the whole space $\mathcal{V}$. (This is inherited from Theorem 4.1.11.)

Theorem 4.1.13: Spectral Decomposition of Unitary Operators

Let $\mathbf{U}$ be a Hermitian operator on a dimension-n vector space $\mathcal{V}$. Let its spectral decomposition be $\mathbf{U} = \sum_{i=1}^{n} \lambda_i \mathbf{v}_i$. Then, the following hold:

1. $\forall i \in [n]$, $|\lambda_i| = 1$;
2. The collection $\{\mathbf{v}_i\}_{i=1}^{n}$ forms an orthonormal basis for the whole space $\mathcal{V}$. (This is inherited from Theorem 4.1.11.)

4.1.9 Spectral Decomposition and Eigenvalue Decomposition (Diagonalization) in General

As we mentioned earlier, Theorem 4.1.11 is actually a part of the larger topic of eigenvalue decomposition (aka diagonalization).

We first need to define diagonalizable matrices (operators) formally.

Definition 4.1.14: Diagonalizable (and Defective) Matrices

A $n \times n$ matrix $\mathbf{A}$ over a field $\mathbb{F}$ is called diagonalizable (aka nondefective) if there exists an invertible matrix $\mathbf{P}$ such that $\mathbf{\Lambda} = \mathbf{P}^{-1}\mathbf{A}\mathbf{P}$ is a diagonal matrix. A square matrix is called *defective* if it is not diagonalizable.

Definition 4.1.15: Eigenvalue Decomposition

Let A be a diagonalizable matrix (see Definition 4.1.14). Then, its diagonalization $\mathbf{A} = \mathbf{P}\mathbf{\Lambda}\mathbf{P}^{-1}$ (derived from Definition 4.1.14) is called its eigenvalue decomposition.

In the following, we give several equivalent characterizations of diagonalizable (aka eigenvalue-decomposable) matrices. These claims can be proved easily from the definition of the matrix of an operator with respect to a basis.

- An $n \times n$ matrix $\mathbf{A}$ over a field $\mathbb{F}$ is diagonalizable if and only if the sum of the dimensions of its eigenspaces is equal to n, which is the case if and only if there exists a basis of $\mathbb{F}^n$ consisting of eigenvectors of $\mathbf{A}$. If such a basis $\{q_i\}_{i=1}^{n}$ has been found, then $\mathbf{A}$ can be factorized as $\mathbf{A} = \mathbf{P}\mathbf{\Lambda}\mathbf{P}^{-1}$, where $\mathbf{P}$ is the $n \times n$ matrix whose i-th column is the q_i, and $\mathbf{\Lambda}$ is the diagonal matrix whose diagonal elements $\Lambda_{ii} = \lambda_i$. (The matrix $\mathbf{P}$ is known as a *modal matrix* for $\mathbf{A}$.)
- A linear map $\mathbf{T} : V \to V$ is diagonalizable if and only if the sum of the dimensions of its eigenspaces is equal to $\mathsf{dim}(V)$, which is the case if and only if there exists a basis of V

consisting of eigenvectors of $\mathbf{T}$. With respect to such a basis, $\mathbf{T}$ will be represented by a diagonal matrix. The diagonal entries of this matrix are the eigenvalues of $\mathbf{T}$.

All the above can be summarized by the following lemma:

Lemma 4.1.16: Equivalence between Eigendecomposition and Diagonalization

A square matrix has eigendecomposition if and only if it is diagonalizable.

Spectrum Decomposition vs Eigenvalue Decomposition. With the above discussion, Theorem 4.1.11 (and Theorem 4.1.10) has a more natural interpretation: it just says that any $n \times n$ normal matrix A has exactly n orthonormal eigenverctors. In more details, if we put all the orthonormal eigenverctors column by column, they will form a $n \times n$ unitary matrix U; and $\mathbf{A}$ has the eigendecomposition $\mathbf{A} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\dagger$. Note that $\mathbf{U}$ plays the role of $\mathbf{P}$ in Definition 4.1.14 as $\mathbf{U}^\dagger = \mathbf{U}^{-1}$.

In summary, spectrum decomposition is nothing but the eigenvalue decomposition for *normal* matrices (see Definition 4.1.9). In general, eigen-decomposition only ensures that the eigenvectors are *linearly independent*; But they may not be orthogonal. (That is, Definition 4.1.14 only says that P is an *invertible* matrix; But its columns may not be orthogonal). In contrast, spectrum decomposition (i.e., when the concerted matrix is normal) additionally ensures that P is also a orthonormal matrix (i.e., has orthogonal columns).

As a side mark: if we consider the eigenvalue decomposition for a matrix that is *Hermitian*, then not only that the it is a *spectrum* decomposition (because all Hermitian matrices are normal), but additionally the eigenvalues are all real-valued. This is what Theorem 4.1.12 says.

4.1.10 Jordan Normal Form

Then, what can we do with defective matrices (Definition 4.1.14)? The answer is the Jordan normal form [Jor70][4].

Xiao: add Jordan normal form here.

Xiao!

4.1.11 Singular-Value Decomposition

In quantum computing, we often deal with normal matrices. Recall that normal matrices are (unitarily) diagonalizable (Theorem 4.1.10). Thus, by Lemma 4.1.16, they must have spectrum decomposition.

However, under certain circumstances, we need to deal with matrices that are not normal or diagonalizable. What should we do for them? The answer is Singular-Value Decompositon (SVD). SVD can be viewed as a generalization of eigenvalue decomposition (discussed in Sections 4.1.8 and 4.1.9) to arbitrary matrices (even non-square ones).

Definition 4.1.17: Singular-Value Decomposition

Any $m \times n$ complex matrix $\mathbf{M}$ can be factorized as $\mathbf{M} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\dagger$, where $\mathbf{U}$ is an $m \times m$ complex unitary matrix, $\mathbf{V}$ is a $n \times n$ complex unitary matrix, and $\mathbf{\Sigma}$ is a $m \times n$ diagonal matrix with *non-negative real numbers* on the diagonal.
We call $\mathbf{M} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\dagger$ a *singular-value decomposition* (SVD) of $\mathbf{M}$. Some terminologies follow:

- **(Unique) Singular Values.** The diagonal entries $\sigma_i = \Sigma_{ii}$ ($i \in \min\{m, n\}$) are uniquely

[4] Do not confuse this with Jordan's lemma(s) (See Remark 4.9.3).

determined by $\mathbf{M}$ and are known as the singular values of $\mathbf{M}$.

- **Rank.** The number of *non-zero* singular values is equal to the rank of $\mathbf{M}$. (This is indeed another way to define rank for non-square complex matrices.)

- **Left and Right Singular Values.** The columns $\{|v_i\rangle\}_{i\in[m]}$ of $\mathbf{U}$ and the columns $\{|w_i\rangle\}_{i\in[n]}$ of $\mathbf{V}$ are called left singular vectors and *right singular vectors* of $\mathbf{M}$, respectively. They form two sets of *orthonormal bases*. If they are sorted so that the singular values σ_i with value zero are all in the highest-numbered columns, the singular value decomposition can be written as $\mathbf{M} = \sum_{i=1}^{r} \sigma_i |v_i\rangle \langle w_i|$, where $r \leq \min\{m, n\}$ is $\mathbf{M}$'s rank.

Several important properties of SVD follow:

1. The above definition claims that every complex matrix $\mathbf{M}$ must have SVD. This is a fact that can be proven.

2. SVD of a given matrix $\mathbf{M}$ is not unique.

3. If $\mathbf{M}$ is real, then $\mathbf{U}$ and $\mathbf{V}$ can be guaranteed to be real orthogonal matrices; In such contexts, the SVD is often denoted as $\mathbf{M} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\top}$.

4. It follows immediately from Definition 4.1.17 that for any $i \in \{1, 2, \ldots, \min\{m, n\}\}$, it holds that $\mathbf{M}|w_i\rangle = \sigma_i |v_i\rangle$. This geometric property of SVD is a strict generalization of that of spectrum decomposition, i.e., $\mathbf{M}|v_i\rangle = \lambda_i |v_i\rangle$.

5. Xiao: to do... Xiao!

Xiao! Xiao: SVD can provide a more unified point of view and reveal the essence of some operations behind the scene. A great example is when we try to define norms for matrices. (In the following, keep in mind that by definition, singular values are non-negative.)

- The Spectral Norm of a matrix M is actually the infinite norm of the vector consisting of M's singular values (i.e., M's largest singular value).

- The Trace Norm of M is actually the summation of M's singular values.

- Singular Value Decomposition can be viewed as $M = \sum_i s_i |a_i\rangle \langle b_i|$, where $\{s_i\}_i$ are the singular values and $\{a_i\}_i$ and $\{b_i\}_i$ are two sets of orthonormal basis.

Therefore, I find it necessary to present here a formal discussion of SVD.
A good starting point is this video.

4.1.12 Two Ways to Formalize Partial Trace

Definition 4.1.18: Partial Trace

The best way to define partial trace is the following:

$$\mathrm{tr}_B(\rho_{AB}) = \sum_{i=1}^{d} (\mathbb{1}_A \otimes \langle e_i|_B)\rho_{AB}(\mathbb{1}_A \otimes |e_i\rangle_B), \tag{4.2}$$

where $d = \mathsf{dim}(B)$ and $\{|e_i\rangle\}_{i\in[d]}$ is an arbitrary set of basis for B. Some authors prefer the following simplified expression of Equation (4.2): $\mathrm{tr}_B(\rho_{AB}) = \sum_{i=1}^{d} \langle e_i|_B \, \rho_{AB} \, |e_i\rangle_B$.

The formalism in Equation (4.2) makes it clear that partial trace fits into the Kraus-Operator formalism of quantum channels (Theorem 4.5.4). It makes it obvious that partial trace is CPTP (see Section 4.5.3). This is because

$$\mathrm{tr}_B(\rho_{AB}) = (\mathbb{1}_A \otimes \mathrm{tr}_B)\rho_{AB} = \sum_{i=1}^{d}(\mathbb{1}_A \otimes \langle e_i|_B)\rho_{AB}(\mathbb{1}_A \otimes |e_i\rangle_B), \tag{4.3}$$

where we just write down the Kraus operators $\{\mathbb{1}_A \otimes \langle e_i|\}_B$ (see Section 4.5.2). Due to this reason, people also call $\{\mathcal{N}_i \coloneqq \mathbb{1}_A \otimes \langle i|_B\}_i$ (where $\{|i\rangle_B\}_i$ is some orthogonal basis for system B) the *trace-out or (discarding) channel*[5]. It is easy to see that $\{\mathcal{N}_i\}_i$ satisfy the requirements for being Kraus operators, i.e., $\sum_i \mathcal{N}_i^\dagger \mathcal{N}_i = \mathbb{1}_{AB}$. (Note that $\mathcal{N}_i : \mathcal{L}(AB) \to \mathcal{L}(A)$.)

Another way to define partial trace is to formalize it only for product states, and then its effects on general states follows naturally from the linearity of quantum physics.

Definition 4.1.19: Partial Trace (An Alternative Definition)

For any tensor product of rank-one operators *(not necessarily corresponding to a state)* $\rho_{AB} = |x_1\rangle\langle x_2|_A \otimes |y_1\rangle\langle y_2|_B$, let

$$\mathrm{tr}_B(\rho_{AB}) \coloneqq |x_1\rangle\langle x_2|_A \, \mathrm{tr}\left(|y_1\rangle\langle y_2|_B\right) = \langle y_2|y_1\rangle_B \, |x_1\rangle\langle x_2|_A \,. \tag{4.4}$$

To see why Definitions 4.1.18 and 4.1.19 are equivalent, consider a general (i.e., not necessarily product) state ρ_{AB}. It can always be expanded with an orthonormal basis $\{|i\rangle_A, |j\rangle_B\}_{i,j}$ as

$$\rho_{AB} = \sum_{i,j,k,\ell} \lambda_{i,j,k,\ell} \, |i\rangle\langle k|_A \otimes |j\rangle\langle \ell|_B \,, \tag{4.5}$$

where $\lambda_{i,j,k,\ell}$'s are non-negative and sum up to 1.[6] Plugging Equation (4.5) into Equation (4.4), it follows immediately that

$$\mathrm{tr}_B(\rho_{AB}) = \mathrm{tr}_B\Big(\sum_{i,j,k,\ell} \lambda_{i,j,k,\ell} \, |i\rangle\langle k|_A \otimes |j\rangle\langle \ell|_B\Big) = \sum_{i,j,k} \lambda_{i,j,k,j} \, |i\rangle\langle k|_A = \sum_{i,k}\Big(\sum_j \lambda_{i,j,k,j}\Big)|i\rangle\langle k|_A \,. \tag{4.6}$$

Moreover, it is also easy to see that plugging Equation (4.5) into Equation (4.3) will yield the same expression as the RHS of Equation (4.6). Thus, Definitions 4.1.18 and 4.1.19 are equivalent (for density operators).

4.1.13 Unique Linear Extension of Convex Linear Operators

We know that a quantum channel $\mathcal{N}(\cdot) : \mathcal{D}(\mathcal{H}) \to \mathcal{D}(\mathcal{H})$ is always *convex linear*, which is defined formally in Definition 4.1.20.

[5] More accurately, if the whole system is traced out, it is called a discarding channel; if only part of the system is traced out, it is called a trace-out channel

[6] Recall that any density matrix is positive semi-definite (thus normal, thus diagonalizable) and trace-1.

Definition 4.1.20: Convex Linear Maps

A map $\mathcal{N}(\cdot) : \mathcal{D}(\mathcal{H}) \to \mathcal{D}(\mathcal{H})$ is *convex linear* if for any $\rho, \sigma \in \mathcal{D}(\mathcal{H})$ and any $\alpha \in [0, 1]$,

$$\mathcal{N}\big(\alpha\rho + (1-\alpha)\sigma\big) = \alpha\mathcal{N}(\rho) + (1-\alpha)\mathcal{N}(\sigma).$$

Let us also formally defined linear maps.

Definition 4.1.21: Linear Maps

A map $\mathcal{N}(\cdot) : \mathcal{L}(\mathcal{H}) \to \mathcal{L}(\mathcal{H})$ is *linear* if for any $\rho, \sigma \in \mathcal{L}(\mathcal{H})$ and any $\alpha, \beta \in \mathbb{C}$,

$$\mathcal{N}\big(\alpha\rho + \beta\sigma\big) = \alpha\mathcal{N}(\rho) + \beta\mathcal{N}(\sigma).$$

Then the following holds.

Lemma 4.1.22: Unique Linear Extension [HZ11, Proposition 2.30]

For a convex linear operator $\mathcal{N}(\cdot) : \mathcal{D}(\mathcal{H}) \to \mathcal{D}(\mathcal{H})$ as defined above, there exists a unique linear extension $\widetilde{\mathcal{N}}$ of $\mathcal{N}$, whose action is well defined on the space of all operators $X \in \mathcal{L}(\mathcal{H})$ (i.e., on all the linear operators over the Hilbert space $\mathcal{H}$).

See [Wil11, Appendix B] for the proof of Lemma 4.1.22.

One important implication of Lemma 4.1.22 is that: we can assume without loss of generality that a convex linear map is always linear. This does not really relax the assumption, as the former can always be extended to the (unique) latter.

4.1.14 Ajoint

Xiao: The conjugate transpose operation for complex numbers/matrices is actually a special case of the ajoint operation. see [Wil11, Section 4.4.5]. Xiao!

talk how to use it to reverse isometries. (see .e.g, [Wil11, Page 163])

4.1.15 Bloch Coordinates

Pure State on the Bloch Sphere: For a general pure qubit $|\phi\rangle = \cos(\theta/2)|0\rangle + e^{i\psi}\sin(\theta/2)|1\rangle$, the following equality holds:

$$\begin{aligned}
|\phi\rangle\langle\phi| &= \begin{bmatrix} \cos^2(\theta/2) & e^{-i\psi}\sin(\theta/2)\cos(\theta/2) \\ e^{i\psi}\sin(\theta/2)\cos(\theta/2) & \sin^2(\theta/2) \end{bmatrix} && (4.7)\\
&= \frac{1}{2}\begin{bmatrix} 1+\cos(\theta) & \sin(\theta)\big(\cos(\psi) - i\sin(\psi)\big) \\ \sin(\theta)\big(\cos(\psi) + i\sin(\psi)\big) & 1-\cos(\theta) \end{bmatrix} && (4.8)\\
&= \frac{1}{2}(\mathbb{1}_2 + r_x X + r_y Y + r_z Z) && (4.9)\\
&= \frac{1}{2}\begin{bmatrix} 1+r_z & r_x - ir_y \\ r_x + ir_y & 1-r_z \end{bmatrix}, \quad \text{where } \begin{cases} r_x = \sin(\theta)\cos(\psi) \\ r_y = \sin(\theta)\sin(\psi) \\ r_z = \cos(\theta) \end{cases}. && (4.10)
\end{aligned}$$

In the above, $||\mathbf{r} := (r_x, r_y, r_z)||_2 = 1$ is due to the fact that $|\phi\rangle$ is a pure state. It is also worth noting that $\mathbf{r}$ is nothing but the coordinate in the sphere coordinate system (w.r.t. the Block Sphere). See Figure 4.1.

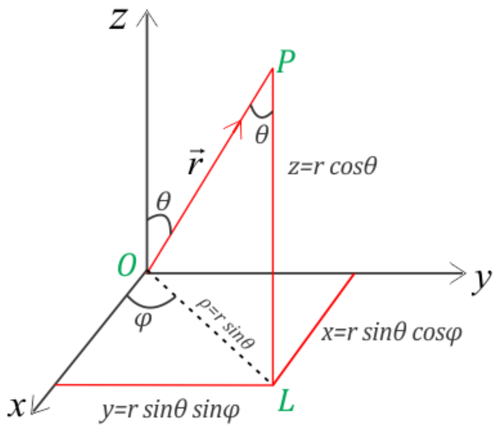

Figure 4.1: Rectangular to Spherical Coordinates

Here are some interesting properties of Bloch coordinate for a general qubit ρ:

1. The RHSs of Equations (4.9) and (4.10) always represent a density operator (of a potentially mixed qubit) as long as $||\mathbf{r}||_2 \leq 1$;
2. $r_x = \mathrm{tr}(X\rho)$, $r_y = \mathrm{tr}(Y\rho)$, and $r_z = \mathrm{tr}(Z\rho)$;
3. The eigenvalues of a general qubit density operator are $\{\frac{1}{2}(1 + ||\mathbf{r}||_2), \frac{1}{2}(1 - ||\mathbf{r}||_2)\}$;
4. A mixture $\{p_j, |\phi_j\rangle\}_j$ with coordinates $\{\mathbf{r}_j\}_j$ gives a density matrix with the Bloch vector $\mathbf{r} = \sum_j p_j \mathbf{r}_j$.

4.2 Useful Geometric Properties of Vectors in Hilbert Space

Xiao: Talk about fidelity, trace distance, and widely-used inequalities that captures several important geometric relations, include Xiao!

- Concepts and inequalities from [AHU19, Section 5.1].

4.3 The Four Postulates of Quantum Mechanics

From a mathematical point of view, the whole area of quantum computing can be derived from the following 4 postulates of quantum mechanics (together with linear algebra for Hilbert spaces). The formalism is taken verbatim from Nielsen and Chuang [NC11], with my comments.

1. **(State.)** Associated to any isolated physical system is a complex vector space with inner product (aka Hilbert space) known as the state space of the system. The system is completely described by its density operator, which is a **trace-one positive operator**[7] ρ, acting on the

[7]This term "positive operator" comes from functional analysis, where positive operators are defined by positive semi-definite matrices. Note that these matrices are Hermitian by definition (Definition 4.1.1).

state space of the system. If a quantum system is in the state ρ_i with probability p_i, then the density operator for the system is $\sum_i p_i \rho_i$.

Rationale behind Postulate 1

Why Hermitian?

- Real Eigenvalues: Hermitian matrices have real eigenvalues. Since the eigenvalues of the density matrix represent probabilities (or in the case of mixed states, the weights of pure states in the mixture), these values must be real numbers. Non-Hermitian matrices can have complex eigenvalues, which would not make sense in the context of probability.

- Physical Observables: In quantum mechanics, any observable (such as position, momentum, or spin) is represented by a Hermitian operator. Since the density matrix is used to calculate expectation values of observables (e.g., $\langle A \rangle = \mathrm{Tr}[\rho A]$ for some observable A), the density matrix itself must be Hermitian to ensure that these expectation values are real and physically meaningful.

- Symmetry: The Hermitian nature of the density matrix also ensures that the probabilities and coherences (off-diagonal elements) in the density matrix are symmetric with respect to complex conjugation. This symmetry is essential for consistent physical interpretation, especially in quantum coherence and interference phenomena.

- Also note that there is a lemma regarding Hermitian matrices:
  - A square matrix H is hermitian if and only if $H = U \Lambda U^\dagger$, where U is unitary and Λ is a diagonal matrix with only real-valued elements.

Why positive semi-definite?

- This requirement arises because the eigenvalues of the density matrix represent probabilities, which must be non-negative. If the density matrix ρ were not positive semi-definite, it could imply negative probabilities, which are not physically meaningful.

  Mathematically, for any vector $|\psi\rangle$, the value $\langle\psi| \rho |\psi\rangle$ must be non-negative (for positive semi-definite ρ's). This condition ensures that any measurement outcome has a non-negative probability, aligning with the principles of probability theory.

Why trace one?

- The trace of the density matrix, $\mathrm{Tr}(\rho) = 1$, ensures that the total probability of all possible outcomes of a quantum measurement sums to one. In quantum mechanics, the trace represents the sum of the eigenvalues of the density matrix, which are the probabilities of the system being in each of its possible pure states.

- Since the density matrix can be expressed as a statistical mixture of pure states $\rho = \sum_i p_i |\psi_i\rangle\langle\psi_i|$, where p_i are probabilities, the sum of these probabilities $\sum_i p_i$ must equal one, reflecting the fact that the system must be in one of the possible states with certainty.

2. **(Evolution.)** The evolution of a closed quantum system is described by a unitary transformation. That is, the state ρ of the system at time t_1 is related to the state ρ' of the system at time t_2 by a **unitary operator** U which depends only on the times t_1 and t_2,

$$\rho' = U\rho U^\dagger.$$

3. **(Measurement.)** Quantum measurements are described by a collection $\{M_m\}$ of measurement operators. These are operators acting on the state space of the system being measured. The index m refers to the measurement outcomes that may occur in the experiment. If the state of the quantum system is ρ immediately before the measurement then the probability that result m occurs is given by

$$p(m) = \operatorname{tr}\left(M_m \rho M_m^\dagger\right),$$

and the state of the system after the measurement is

$$\frac{M_m \rho M_m^\dagger}{\operatorname{tr}\left(M_m \rho M_m^\dagger\right)}.$$

The measurement operators satisfy the **completeness equation**,

$$\sum_m M_m^\dagger M_m = I.$$

This postulate is also known as the *Born rule*.

4. **(Composition.)** The state space of a composite physical system is the tensor product of the state spaces of the component physical systems. Moreover, if we have systems numbered 1 through n, and system number i is prepared in the state ρ_i, then the joint state of the total system is $\rho_1 \otimes \rho_2 \otimes \ldots \otimes \rho_n$.

4.4 POVMs and Projective Measurements

Positive Operator-Valued Measure. POVM measurements is used when we only care about the measurement outputs, but not the post-measurement states. A nice discussion about projective measurements, POVM, and Naimark's dilation theorem can be found here.

Definition 4.4.1: Positive Operator-Valued Measure

A POVM is a set of matrices $\{E_m\}$ on a Hilbert space $\mathcal{H}$ satisfying the following properties

1. Each E_m is a positive semi-definite matrix (aka positive operator, see Definition 4.1.1);
2. $\sum_m E_m = I$.

The probability of measuring m is simply: $p(m) = \operatorname{tr}(E_m \rho)$. Note that Condition 1 ensures that this probability is non-negative; Condition 2 ensures that all the probabilities sum up to 1.

Here are some useful properties of POVMs:

- For a POVM $\{E_m\}$, each E_m is normal, thus admitting spectral decomposition.
- For a POVM $\{E_m\}$, there always exists $\{P_m\}$ such that $P_m^\dagger P_m = E_m$ for all m. To see why, just set $P_m = \sqrt{E_m}$. The square root operation is well defined as E_m is positive semi-definite (see Definition 4.1.1).
- Each E_m is normal (i.e., $E_m^\dagger E_m = E_m E_m^\dagger$), thus admitting spectral decomposition.

Projective Measurements. This is a special case of POVM. Projective measurements are defined by *observables* (see Remark 4.4.1).

Definition 4.4.2: Projective Measurements

A projective measurement is described by an observable M. Since M is Hermitian (thus, normal), M has the following spectrum decomposition (Theorem 4.1.11): $M = \sum_m m P_m$, where each P_m is the projector onto the eigenspace of M with eigenvalue m.
The projective measurement is simply the POVM defined by $\{P_m\}$. (It is not hard to see that $\{P_m\}$ satisfy the two conditions in Definition 4.4.1, because each P_m is the projector onto the *eigenspace* of M with eigenvalue m.)

Remark 4.4.1 (Observables)**.** *Roughly speaking, an observable is a Hermitian operator on the state space of the system being observed. Actually, this is a physical term that I do not fully understand. But it seems that in finite-dimensional Hilbert space, one can just treat observables as Hermitian matrices*[8]*, since observables on finite-dimensional Hilbert spaces are always Hermitian. Roughly, the name "observables" come from the fact that measurements are essentially measuring (thus, "observing") the eigenvalues of observables; the outcome of the measurement is a eigenvalue of the observable, and the state after measurement collapses to the corresponding eigenvector (also called "eigenstate").*

Here is an example distinguish between POVMs and projective measurements. Consider the following measurement:

- Measure $\{|0\rangle\langle 0|, |1\rangle\langle 1|\}$ with probability $\frac{1}{2}$, and measure $\{|+\rangle\langle +|, |-\rangle\langle -|\}$ with probability $\frac{1}{2}$.

This is a POVM specified by $\{E_i\}_{i\in[4]}$, where

$$E_1 = \frac{1}{2}|0\rangle\langle 0|,\ E_2 = \frac{1}{2}|1\rangle\langle 1|,\ E_3 = \frac{1}{2}|+\rangle\langle +|,\ E_4 = \frac{1}{2}|-\rangle\langle -|.$$

It is easy to see that $\{E_i\}_{i\in[4]}$ satisfy the two conditions in Definition 4.4.1. But it cannot be interpreted as a projective measurement (Instead, it is a probabilistic mixture of two projective measurements).

4.5 Quantum Operations (aka Channels) from a Mathematical Point of View

In this section, we present 3 equivalent interpretations of quantum operations (aka channels). A great explanation can be found here.

[8] See this Quora answer.

4.5.1 Stinespring's Representation of Quantum Channels

See this link for the origin of the name.

Definition 4.5.1: Operational Definition of Quantum Channels

As an implication of Schrödinger's equation, quantum operations can be classified as:

1. **unitaries,** which is captured by unitary operations on quantum states.
2. **adding systems,** which is captured by isometry operations (Definition 4.5.3) on quantum states. Isometries can be viewed as an extension of unitaries from a lower-dimension space to a higher-dimension space that preserves length.
3. **partial trace.**

All the above types of operations can be captured by a unitary process happening on a larger system (Hilbert space), followed by a "tracing out" operation. This is sometimes referred to as the "Church of Larger Hilbert Space". This is formally stated as Theorem 4.5.2.

Note that we provide two versions in Theorem 4.5.2. But they are of course equivalent because the isometry in the second "different-space" version can be viewed as a part of the *unitary on the larger space* in the first "same-space" version. In more details, the isometry V in the second version is a rectangular matrix formed from selecting only a few of the columns from the unitary U in the first version. The property $VV^\dagger = \Pi_{BE}$ distinguishes the isometric operation from the unitary one. It states that the isometry takes states in the input system A to a particular subspace of the joint system BE. The projector Π_{BE} projects onto the subspace where the isometry takes input quantum states. (See also [Wil11, Section 5.2.1] for this point of view.)

Theorem 4.5.2: Stinespring Dilation Theorem

Same-Space Version: Let $T : S(H) \mapsto S(H)$ be a completely positive and trace-preserving (CPTP, see Section 4.5.3) map between states on a finite-dimensional Hilbert space H. Then there exists a Hilbert space K and a unitary operation U on $H \otimes K$ such that

$$T(\rho) = \mathrm{tr}_K \left(U(\rho \otimes |0\rangle\langle 0|)U^\dagger \right) \tag{4.11}$$

for all $\rho \in S(H)$, where tr_K denote the partial trace on the K-system.[a]

Different-Space Version: Let $\mathcal{N} : \mathcal{L}(\mathcal{H}_A) \to \mathcal{L}(\mathcal{H}_B)$ be a quantum channel. Let $\mathcal{H}_E$ be a Hilbert space with dimension no smaller than the (Choi rank) of the channel $\mathcal{N}$. An isometric extension or Stinespring dilation of the channel $\mathcal{N}$ is a linear isometry $V : \mathcal{H}_A \to \mathcal{H}_B \otimes \mathcal{H}_E$ such that

$$\forall X_A \in \mathcal{L}(\mathcal{H}_A), \quad \mathcal{N}(X_A) = \mathrm{tr}_E(V X_A V^\dagger).$$

The fact that V is an isometry is equivalent to the following conditions:

$$V^\dagger V = \mathbb{1}_A \quad \text{and} \quad VV^\dagger = \Pi_{BE},$$

where Π_{BE} is a projection of the tensor-product Hilbert space $\mathcal{H}_B \otimes \mathcal{H}_E$.

[a] A remark on the efficiency: It is worth noting that if T is described as a circuit, then there is a dilation U_T represented by a circuit of size $O(|T|)$.

Remark 4.5.1 (Isometry between Hilbert Spaces)**.** *The formal definition of isometry is presented in Definition 4.5.3. But in quantum computing, we usually focus on the special case where the metric spaces are Hilbert spaces. In this context, isometry means a map* $V \in \mathcal{L}(\mathcal{H}, \mathcal{H}')$ *with* $\dim(\mathcal{H}) \leq \dim(\mathcal{H}')$ *such that* $\|[\|_2]|\psi\rangle = \|[\|_2]V|\psi\rangle$ *(or equivalently,* $V^\dagger V = \mathbb{1}_{\mathcal{H}}$*). An isometry can be viewed as a generalization of a unitary, because it maps between spaces of different dimensions but satisfying* $V^\dagger V = \mathbb{1}_{\mathcal{H}}$*. However, it is worth noting that it does not need to satisfy* $VV^\dagger = \mathbb{1}_{\mathcal{H}'}$*. Rather, it satisfies* $VV^\dagger = \Pi_{\mathcal{H}'}$*, where* $\Pi_{\mathcal{H}'}$ *is some projection onto* $\mathcal{H}'$ *(i.e., it is easy to see that* $(VV^\dagger)(VV^\dagger) = VV^\dagger$*).*

Isometries capture both Type 1 and Type 2 operations. A key example to keep in mind is the following isometry: $V|\psi\rangle = |\psi\rangle \otimes |0\rangle$*; for this* V*, it is easy to see that* $V\rho V^\dagger = \rho \otimes |0\rangle\langle 0|$*, exactly as we expected.*

Indeed, isometries are just equivalent to unitaries in terms of being an Kraus operator (see the last part of Section 4.5.2).

Definition 4.5.3: Isometry

Let X and Y be metric spaces with metrics d_X and d_Y. A map $f : X \to Y$ is called an isometry or distance preserving if for any $a, b \in X$ one has: $d_Y\big(f(a), f(b)\big) = d_X(a, b)$.

Xiao: Xiao!

Naimark's Dilation. There is a similar dilation theorem for measurements, which can be understood as an analog of Stinespring's dilation which is for unitaries. This theorem is called Naimark's dilation. It essentially says that any measurement can be viewed as a *projective* measurement on a compound register that includes the original register as a subregister. A formal treatment for this lemma can be found at [Wat18].

4.5.2 Choi-Kraus Decomposition of Quantum Channels

This is also called operator-sum representation of quantum channels. For a rigorous discussion, refer to [NC11, Theorems 8.1 and 8.3]. (For the proof of Theorem 4.5.4, check [Wil11, Section 4.4], which I found cleaner than that in [NC11])

Theorem 4.5.4: Choi-Kraus Decomposition

Let $\mathcal{H}$ and $\mathcal{G}$ be Hilbert spaces of dimension n and m respectively, and Φ be a completely-positive (but not necessarily trace-preserving, see Remark 4.5.2) linear map between $\mathcal{L}(\mathcal{H})$ and $\mathcal{L}(\mathcal{G})$. Then, there are (not necessarily unique) matrices $\{B_i\}_{1 \geq i \geq nm}$ (where $B_i \in \mathcal{L}(\mathcal{H}, \mathcal{G})$ for all $i \in [nm]$) such that for any $\rho \in \mathcal{L}(\mathcal{H})$,

$$\Phi(\rho) = \sum_i B_i \rho B_i^\dagger, \quad \text{and} \quad \sum_i B_i^\dagger B_i \leq \mathbb{1}_{\mathcal{H}}$$

Conversely, any map Φ of this form is a quantum operation, provided that

$$\sum_i B_i^\dagger B_i \leq \mathbb{1}_{\mathcal{H}}. \tag{4.12}$$

Remark 4.5.2 (On Trace Preservation)**.** *If the "$\leq$" sign is replaced with "$=$" in* **??***, Theorem 4.5.4 captures only trace-preserving operations (i.e., CPTP operators); the above formalism (with the "$\leq$"*

sign) captures also non-trace-preserving operations. For more discussion, see [NC11, Section 8.2.3].

We also emphasis that there could exist different Choi-Kraus decompositions of the same quantum channel. The famous depolarizing channel (parameterized by a probability p) serves as a good example:

$$\begin{aligned}\mathsf{Depo}_p(\rho) &= (1-p)\rho + \frac{p}{3}X\rho X^\dagger + \frac{p}{3}Y\rho Y^\dagger + \frac{p}{3}Z\rho Z^\dagger \\ &= \left(1 - \frac{3p}{4}\right)\rho + \frac{3p}{4}\frac{I}{2}\end{aligned}$$

The Choi-Kraus Decomposition for Stinespring's Dilation. The partial-trace in Stinespring's dilation equation Equation (4.11) can be expressed as a sequence of Kraus operators. There is nothing surprising about this as we know that these interpretation of quantum channels are indeed equivalent. But I view this a good exercise for beginners to get familiar with the mixed product property of the Kronecker product (see Lemma 4.1.6). That is,

$$\begin{aligned}\mathrm{tr}_K\left(U_{HK}(\rho_H \otimes |0\rangle\langle 0|_K)U_{HK}^\dagger\right) &= \sum_i (\mathbb{1}_H \otimes \langle i|_k)U_{HK}(\rho_H \otimes |0\rangle\langle 0|_K)U_{HK}^\dagger(\mathbb{1}_H \otimes |i\rangle_k) \\ &= \sum_i (\mathbb{1}_H \otimes \langle i|_k)U_{HK}(\mathbb{1}_H \otimes |0\rangle_K)\rho_H(\mathbb{1}_H \otimes \langle 0|_K)U_{HK}^\dagger(\mathbb{1}_H \otimes |i\rangle_k) \\ &= \sum_i B_i \rho_H B_i^\dagger,\end{aligned}$$

where $\left\{B_i \coloneqq (\mathbb{1}_H \otimes \langle i|_k)U_{HK}(\mathbb{1}_H \otimes |0\rangle_K)\right\}_i$ form a set of Kraus operators as per Theorem 4.5.4. (Note that the second equality relies on the mixed product property of the Kronecker product.) To see that, one can easily compute that $\sum_i B_i^\dagger B_i = \mathbb{1}_H$. This is left as an exercise. (Hint: you will need to use the mix product property several times).

Choi-Kraus Decomposition for Unitary/Isometry Evolution. Unitary evolution is a special kind of quantum channel in which there is a single Kraus operator $U \in \mathcal{L}(H)$ that is unitary, i.e., satisfying $UU^\dagger = U^\dagger U = \mathbb{1}_H$.

Recall that in Remark 4.5.1, we argued that isometries (Definition 4.5.3) are just the generalized unitary between Hilbert spaces of different dimension. Indeed, as in the case of unitary channels, an *isometry channel* V is just a single Kraus operator $V \in \mathcal{L}(H_1, H_2)$ satisfying $V^\dagger V = \mathbb{1}_{H_1}$ (but it may not true that $VV^\dagger = \mathbb{1}_{H_2}$).

4.5.3 Axiomatic Definition of Quantum Operations (or the CPTP formalism)

The presentation of this formalism is based on [Wil11, Section 4.4] and this lecture of MIT 8.371 course, which is equivalent to Theorem 4.5.4 when **??** is restricted to the "=" sign *only*. It starts by asking "what properties should a general quantum operator satisfy?" The following conditions turn out to be complete:

1. **Convex Linearity:** First, quantum mechanics is a linear theory. Moreover, consider the experiment that prepares the ensemble $\{p_x, \rho_x\}$ such that p_x's are the probability distribution for ρ_x's. Then, we want that conducting some experiment on this ensemble will lead to the same result no matter we know the actual state ρ_x before the experiment starts or after the experiment ends (of course, taking also the probability distribution p_x into consideration).

This requires that the channel should be convex linear (Definition 4.1.20). It then follows from the discussion in Section 4.1.13 that we can actually assume *linearity* w.l.o.g.

2. **Hermiticity Preserving:** a hermitian input should lead to a hermitian output. Some textbook omit this property, because we anyway require "Completely Positive" below, and positive operators are Hermitian by definition (Definition 4.1.1).

3. **Trace Preserving:** just as unitaries preserve length, our quantum operations should preserve trace.

4. **Completely Positive:** just like the non-negativity condition on stochastic maps. Here, "Positive" (Definition 4.5.5) means that if ρ is *positive semi-definite*[9], then $\Phi(\rho)$ is positive-semidefinite. However, we need a stronger condition for it to be correct. We need to stipulate that if we act on any part of ρ it should stay positive. That is, if ρ_{AR} is positive semi-definite, then $(\Phi \otimes \mathbb{1}_R)(\rho_{AR})$ should also be positive semi-definite. This is what we mean by "Completely". This is formalized in Definition 4.5.6.

   (As a comparison, transpose is positive but not completely positive. As a side note: the partial positive transpose test (PPT) is one test of an entangled state: if PPT fails, it must be an entangled state)

We remark that [NC11, Section 8.2.4] contains a slightly different version of the above axiomatic representation, which is equivalent to Theorem 4.5.4 (with **??** as it is).

Definition 4.5.5: Positive Linear Map

A linear map $\mathcal{M} : \mathcal{L}(\mathcal{H}_A) \to \mathcal{L}(\mathcal{H}_B)$ is *positive* if $\mathcal{M}(X_A)$ is *positive semi-definite* for all positive semi-definite $X_A \in \mathcal{L}(\mathcal{H}_A)$.

Definition 4.5.6: Completely Positive Linear Map

A linear map $\mathcal{M} : \mathcal{L}(\mathcal{H}_A) \to \mathcal{L}(\mathcal{H}_B)$ is *completely positive* if $\mathbb{1}_R \otimes \mathcal{M}$ is a positive linear map (Definition 4.5.5) for a reference system R of arbitrary size..

4.6 Quantum Fourier Transform

Recall that QFT are nothing but the quantum implementation of $\frac{\mathsf{DFT}_N}{\sqrt{N}}$, where DFT_N is defined in Equation (3.3). Thus, it is straightforward to see its effect on computational basis $\{|x\rangle\}_{x\in\{0,\ldots,N-1\}}$: The Quantum Fourier Transform F_N for N-level quantum system is defined by the following unitary mapping:

$$\forall x \in \{0, 1, \ldots, N-1\}, \quad F_N : |x\rangle \mapsto \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} \omega_N^{x \cdot y} |y\rangle ,$$

[9]This is what we mean by "non-negative"

where the $x \cdot y$ is just the standard integer multiplication, and ω_N is (one of the) N-th roots of unity, i.e. $\omega_N = e^{\frac{2\pi i}{N}}$.[10] For an arbitrary state, the quantum Fourier transform can be written as:

$$\sum_{j=0}^{N-1} x_j |j\rangle \mapsto \sum_{k=0}^{N-1} y_k |k\rangle ,$$

where $y_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j e^{\frac{2\pi i}{N} kj}$, which is exactly the classical discrete Fourier transform.

Remark 4.6.1 (Memorize QFT quickly)**.** *The way I memorize the N-level QFT is to view it as $F_N = \sum_{j=0}^{N-1} \left|\widetilde{j}\right\rangle \langle j|$, where*

- $\{|j\rangle\}_{j\in\{0,\dots,N-1\}}$ *is the computational basis;* **and**
- $\left\{ \left|\widetilde{j}\right\rangle := \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \omega_N^{jk} |k\rangle \right\}_{j\in\{0,\dots,N-1\}}$ *is the Fourier basis.*

QFT as the Hadamard Gate for Qudits. QFE are nothing but the analog of Hadamard gate for qudits. For level-d quantum system (in the previous part, $d = N$), the analog of X and Z gates are defined as follows (both of them are parameterized):

- **The X-Gate.** $\forall x \in \{0,\dots,d-1\}$, $X(x)|j\rangle = |x + j \bmod d\rangle$ for all $j \in \{0,\dots,d-1\}$;

  The spectral decomposition of $X(x)$ is as follows:

  $$X(x) = \sum_{\ell=0}^{d-1} e^{-\frac{2\pi i}{d}\ell x} \left|\widetilde{\ell}\middle\rangle\middle\langle\widetilde{\ell}\right|, \quad \text{with } \left|\widetilde{\ell}\right\rangle = \frac{1}{\sqrt{d}} \sum_{j=0}^{d-1} e^{\frac{2\pi i}{d}\ell j} |j\rangle .$$

  Note that the eigenvalues of $X(x)$ do not depend on the parameter x, but the eigenvectors do. It it also worth noting that the eigenvalues of $X(x)$ (regardless of x) is just the Fourier transformed basis! That is, for all $\ell \in \{0,\dots,d-1\}$,

  $$\left|\widetilde{\ell}\right\rangle = F_d |\ell\rangle = \frac{1}{\sqrt{d}} \sum_{j=0}^{d-1} e^{\frac{2\pi i}{d}\ell j} |j\rangle .$$

- **The Z-Gate.** $\forall z \in \{0,\dots,d-1\}$, $Z(z)|j\rangle = e^{\frac{2\pi i}{d}zj} |j\rangle$ for all $j \in \{0,\dots,d-1\}$;

  As one can expect, $Z(z)$ has diagonal matrix representation with respect to the qudit computational basis states $\{|j\rangle\}_{j\in\{0,\dots,d-1\}}$. It can be proven that $[Z(z)]_{j,k} = e^{\frac{2\pi i}{d}zj}\delta_{j,k}$, where $\delta_{j,k}$ is Kronecker's delta. This immediately reveals its clean spectral decomposition:

  $$Z(z) = \sum_{j=0}^{d-1} e^{\frac{2\pi i}{d}zj} |j\rangle\langle j| ,$$

  where $\{|j\rangle\}_{j\in\{0,\dots,d-1\}}$ is simply the computational basis.

[10] Recall that all the numbers of the following form are the N-th roots of unity: $e^{\frac{2\pi i}{N}k}$ for $k \in \{0, 1, \dots, N-1\}$. Also recall that $i = e^{\frac{\pi i}{2}}$ and $-1 = e^{\pi i}$.

Therefore, the quantum Fourier transform is just the unitary that transform the $Z(z)$ eigenvectors to $X(x)$ eigenvectors. (Recall that the eigenvectors of both $X(x)$ and $Z(x)$ are independent of their respective parameter.) In the 2-level (i.e. qubit) setting, F_2 is just the Hadamard gate (which takes transforms the eigenvectors of Z to that of X).

QFT in Binary Representation. Let $N = 2^n$. For $j \in \{0, 1, \ldots, N-1\}$, consider the following binary representation of integers

$$j = [j_1 j_2 \ldots j_n]_2 = j_1 2^{n-1} + \ldots + j_n 2^0 = \sum_{\ell=1}^{n} j_\ell \cdot 2^{n-\ell};$$

also, consider the following binary representation of fractions

$$0.j = [0.j_\ell j_{\ell+1} \ldots j_m]_2 = \frac{j_\ell}{2} + \frac{j_{\ell+1}}{2^2} + \ldots, \frac{j_m}{2^{m-\ell+1}}.$$

Then, (it follows from some tedious calculation that) the quantum Fourier transform can be written as:

$$\begin{aligned} |j\rangle = |[j_1 j_2 \ldots j_n]_2\rangle \mapsto & \frac{1}{\sqrt{2^n}} \bigotimes_{\ell=1}^{n} \big(|0\rangle + e^{\frac{2\pi i}{2^\ell} j} |1\rangle \big) \\ & = \frac{1}{\sqrt{2^n}} \big(|0\rangle + e^{2\pi i \cdot [0.j_n]_2} |1\rangle \big) \big(|0\rangle + e^{2\pi i \cdot [0.j_{n-1} j_n]_2} |1\rangle \big) \ldots \big(|0\rangle + e^{2\pi i \cdot [0.j_1 j_2 \ldots j_n]_2} |1\rangle \big) \end{aligned} \tag{4.13}$$

It is worth noting that Equation (4.13) is crucial for the *efficient*[11] implementation of QFT. Moreover, there are also important applications of QFT that requires the specific form of Equation (4.13), e.g., phase estimation.

Xiao: Talk about the following simple applications of quantum Fourier Transform Xiao!

- distinguishing pure state and entangled state: perform QFT, and then measure. See [Zha19, Section 1] and [BZ13, Thoerem 4.2].
- the oracle recording paper [Zha19].

4.7 (In)distinguishability of Quantum States

In classical cryptography, the concept of distance is crucial for formal security proofs. According to the security goal, we may use different types of distances, e.g. statistical distance, computational distance (indistinguishability).

Therefore, a crucial step toward quantum information theory and cryptography would be to define a proper distance. One of the most useful definition is trace distance.

Definition 4.7.1: Trace Distance

The trace distance of two density matrices ρ and σ is defined as

$$T(\rho, \sigma) := \frac{1}{2} \operatorname{tr} |\rho - \sigma|,$$

[11] It is worth memorizing that QFT_N can be implemented using a circuit of size $O(\log^2 N) = O(n^2)$.

where for a matrix A, $|A| = \sqrt{A^\dagger A}$.

On the Motivations for Trace Distance. [NC11, Section 9.2] provides explanations on the choice of this concept as a useful measure for the distance among quantum states/systems. But I find that this lecture (with this video) approaches the concept in a more interesting way. It draws analogy between classical probability theory and quantum probability theory, and reveals the measure-theoretical reason behind the concept of trace distance by introducing Schatten norm and dual norm. This video by Prof. O'Donnell shares a similar perspective.

For quantum cryptography or information theory, this concept is useful mainly due to the following two properties (see [NC11, Section 9.2] for more details):

- $T(\rho,\sigma) = \max_P\{\mathrm{tr}\left(P(\sigma-\rho)\right)\}$, where the maximization may be taken alternately over all projectors P, or over all positive operators $P \leq I$. Given the fact that POVM elements are positive operators that are $\leq I$, this property implies that trace distance is equal to the difference in probabilities that a measurement outcome with POVM element P may occur, depending on whether the state is ρ or σ, maximized over all possible POVM elements P.
- **(Contractiveness.)** Suppose Ψ is a trace-preserving quantum operation. Let ρ and σ be density operators. Then $T\big(\Psi(\rho),\Psi(\sigma)\big) \leq T(\rho,\sigma)$. This property says that (trace-preserving) quantum operations can never separate two quantum states farther than their original trace distance.

4.8 Quantum Entropy

Definition 4.8.1: Von Neumann Entropy

The von Neumann entropy of a quantum state ρ is defined as

$$S(\rho) \coloneqq -\,\mathrm{tr}\left(\rho\log(\rho)\right) = -\sum_x \lambda_x \log(\lambda_x),$$

where $\{\lambda_x\}_x$ are the eigenvalues of ρ. We stipulate that $0\log 0 \coloneqq \lim_{x\to 0} x\log x = 0$.

Properties of von Neumann entropy:

1. If ρ is a classical probability distribution, von Neumann entropy boils down to Shannon entropy $H(X) = -\sum_{i=1}^n \Pr(X=x_i)\log\left(\Pr(X=x_i)\right)$.
2. For any ρ on a D-dimension Hilbert space, $S(\rho) \leq \log(D)$, where we have "=" when $\rho = \frac{\mathbb{1}_D}{D}$, i.e., the maximally mixed state.
3. $S(\rho) \geq 0$. It equals 0 when ρ is a pure state.
4. $|S(\rho_A) - S(\rho_B)| \leq S(\rho_{AB}) \leq S(\rho_A) + S(\rho_B)$, where $\rho_A = \mathrm{tr}_B(\rho_{AB})$ and $\rho_B = \mathrm{tr}_A(\rho_{AB})$. The LHS is called *Araki-Lieb Inequality* (or Triangle Inequality), and the RHS is referred to as *sub-additivity.*
5. **Strong Sub-Additivity.** $S(\rho_{ABC}) \leq S(\rho_{AB}) + S(\rho_{BC}) - S(\rho_B)$.
6. If ρ_{AB} is pure, then $S(\rho_A) = S(\rho_B)$.

4.9 Jordan's Lemma [Jor75]

This is an important lemma that has found numerous applications in quantum computing and cryptography.[12]

Lemma 4.9.1: Jordan's Lemma [Jor75]

For any two Hermitian projectors $\mathbf{\Pi}_A$ and $\mathbf{\Pi}_B$ on a Hilbert space $\mathcal{H}$, there exists an *orthogonal*[a] direct-sum decomposition (recall from Section 4.1.4) of $\mathcal{H} = \oplus_j \mathcal{S}_j$ into one-dimensional and two-dimensional sub-spaces $\{S_j\}_j$ (dubbed the *Jordan subspaces*), where each S_j is invariant under both $\mathbf{\Pi}_A$ and $\mathbf{\Pi}_B$. Moreover:

- In each one-dimensional Jordan subspace, $\mathbf{\Pi}_A$ and $\mathbf{\Pi}_B$ act as identity or rank-zero projectors (recall the definition for projectors' rank in Definition 4.1.7); **and**
- In each two-dimensional subspace $\mathcal{S}_j$, $\mathbf{\Pi}_A$ and $\mathbf{\Pi}_B$ are rank-one projectors. In particular, there exist distinct orthogonal bases $\{|v_{j,1}\rangle, |v_{j,0}\rangle\}$ and $\{|w_{j,1}\rangle, |w_{j,0}\rangle\}$ for $\mathcal{S}_j$ such that $\mathbf{\Pi}_A$ projects onto $|v_{j,1}\rangle$ and $\mathbf{\Pi}_B$ projects onto $|w_{j,1}\rangle$.

(include a picture for an exemplary $\mathcal{S}_j$)

[a] We emphasis that direct-sum decompositions do not always gives orthogonal subspaces. This orthogonality is a result of Jordan's Lemma that requires a proof.

Proof. A complete proof is given in Regev's lecture notes. (While [CMSZ21] also gives a proof, their proof does not show that the subspaces $\{\mathcal{S}_j\}_j$ are orthogonal.) ∎

Corollary 4.9.2: Properties of Jordan Decomposition

Using the same notation as in Lemma 4.9.1, the following holds:

1. In $\mathcal{H}$, it holds that $\mathrm{Img}(\mathbf{\Pi}_A) = \mathrm{span}\{|v_{j,1}\rangle\}_j$ and $\mathrm{Img}(\mathbf{\Pi}_B) = \mathrm{span}\{|w_{j,1}\rangle\}_j$.
2. For each j, the vectors $|v_{j,1}\rangle$ and $|w_{j,1}\rangle$ are corresponding left and right singular vectors of the matrix $\mathbf{\Pi}_A\mathbf{\Pi}_B$ with singular value $\sigma_j = |\langle v_{j,1}|w_{j,1}\rangle|$. The same is true for $|v_{j,0}\rangle$ and $|w_{j,0}\rangle$ w.r.t. respect to $(\mathbf{I} - \mathbf{\Pi}_A)(\mathbf{I} - \mathbf{\Pi}_B)$. (cref to the picture mentioned in Lemma 4.9.1. highlight where the singular value s_j is.)

Xiao: to do: Xiao!
Discuss its most recent applications in [NWZ09, CCY20] and [CMSZ21].
The introduction to Jordan's lemma can be found at Section 1.2.3 of Vidick's notes and Regev's notes.

Remark 4.9.3: A Historical Note on Jordan's Lemma(s)

The Jordan's lemma discussed above has nothing to do with Jordan normal form in Section 4.1.10, except that both of them are due to Camille Joran. Jordan normal form first appeared in 1870 [Jor70], while Jordan's lemma (as per) appeared in 1875 [Jor75] (See [Bha96, Chapter VII.8] for a detailed history for this one).

[12] Do not confuse it with the Jordan's lemma in complex analysis.

Also, here is another Jordan's lemma in complex analysis [Jor93] (yes, Camille Jordan again), which is also irrelevant to the Jordan's lemma discussed above.

4.10 Quantum Circuits

4.10.1 Phase-Shift Gates

We use P_θ to denote the phase-shift gate: $P_\theta \coloneqq \begin{bmatrix} 1 & 0 \\ 0 & e^{\theta i} \end{bmatrix}$. We have the following relations:

- Pauli-Z gate: $Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & e^{\pi i} \end{bmatrix} = P_\pi$
- The $\frac{\pi}{8}$-gate $T = \begin{bmatrix} 1 & 0 \\ 0 & e^{\frac{\pi}{4} i} \end{bmatrix} = P_{\pi/4} = \sqrt[4]{Z}$. Note that T is called the "$\frac{\pi}{8}$-gate", though a more proper name might be "$\frac{\pi}{4}$-gate". This is due to some historical reason: People used to express T in the following form: $T = e^{\frac{\pi}{8} i} \begin{bmatrix} e^{-\frac{\pi}{8} i} & 0 \\ 0 & e^{\frac{\pi}{8} i} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & e^{\frac{\pi}{4} i} \end{bmatrix}$, where the $\frac{\pi}{8}$ shows in the diagonal elements.
- The phase gate: $S = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & e^{\frac{\pi}{2} i} \end{bmatrix} = P_{\pi/2} = T^2 = \sqrt{Z}$. Note that in the literature, phase gate might be denoted as P gate (e.g., [BY22]), instead of S. I guess the reason is "P" is the initial letter of the word "phase."
- In general, $P_{\pi/r} = \sqrt[r]{Z}$ for all $r \in \mathbb{R} \setminus \{0\}$.

4.10.2 Gates from a Group-Theory Viewpoint

Following the convention, I use the following notation:

$$\sigma_0 = I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \sigma_1 = X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad \sigma_2 = Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, \quad \sigma_3 = Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Definition 4.10.1: Pauli Group

The n-qubit Pauli group is defined as follows:

$$\mathbf{P}_n \coloneqq \left\{ e^{\frac{\pi}{2} i\theta} \sigma_{j_1} \otimes \ldots \otimes \sigma_{j_n} \mid \forall \theta \in \{0,1,2,3\} \text{ and } \forall j_k \in \{0,1,2,3\} \right\} \tag{4.14}$$

Note that $e^{\frac{\pi}{2} i\theta}$ takes the values ± 1 and $\pm i$ when θ varies in $\{0, 1, 2, 3\}$. We need it in **??** to ensure that $\mathbf{P}_n$ forms a legitimate group. For example, for the 1-qubit case, we have

$$\mathbf{P}_1 = \{\pm I, \pm iI, \pm X, \pm iX, \pm Y, \pm iY, \pm Z, \pm iZ\}.$$

Here are simple properties of the Pauli group:

1. $|\mathbf{P}_n| = 4^{n+1}$.
2. Any $P \in \mathbf{P}_n$ has eigenvalues ± 1 or $\pm i$.

3. For any $P, Q \in \mathbf{P}_n$ either commute ($PQ = QP$) or anti-commute ($PQ = -QP$). Note that physicists use $[P, Q] = 0$ (resp. $\{P, Q\} = 0$) to denote commute (resp. anti-commute).

4. For any $P \in \mathbf{P}_n$, $P^2 = \pm \mathbb{1}_n$.

Definition 4.10.2: Clifford Group

The Clifford group is defined as the group of unitaries that *normalize* the Pauli group:

$$\mathcal{C}_n := \left\{ \mathbf{U} \in \mathcal{U}(\mathbb{C}^{2^n}) \mid \mathbf{U}\mathcal{P}_n\mathbf{U}^\dagger = \mathcal{P}_n \right\}.$$

It is also called the *normalizer* group of $\mathcal{P}_n$, denoted as $N(\mathcal{P}_n)$.

We remark that the Clifford group can be generated by $\{H, S, \mathsf{CNOT}\}$. More accurately,

Theorem 4.10.3: Generators for Clifford Group (or Clifford Gates)

For any $\mathbf{U} \in \mathcal{C}_n$, up to a global phase, $\mathbf{U}$ may be composed from $O(n^2)$ gates from the set $\{H, S, \mathsf{CNOT}\}$. Due to this theorem, people usually call the set $\{H, S, \mathsf{CNOT}\}$ *Clifford gates.*

A proof sketch for Theorem 4.10.3 can be found at [NC11, Section 10.5.2] where the authors discuss the *stabilizer formalism*. (Gates $\{H, S, \mathsf{CNOT}\}$ are also called "normalizer gates" in [NC11] because they normalize the Pauli group.[13])

4.10.3 Universal Quantum Circuits

Xiao: Use the following outline: Xiao!

- In the following, we use T to denote the $\pi/8$ gate; we use S to denote the phase gate. That is,

$$T := \begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}, \qquad S := T^2 = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}.$$

- Show how to implement any circuit using exponentially many gates (see Henry Yuen's lecture).

- **The "$1 + 2$" Universal Set.** All the 1-qubit gates plus one "entangling" 2-qubit gate is universal. This is an exact implementation (i.e., not an approximation). In terms of efficiency, if I'm not wrong, to implement a $2^n \times 2^n$ unitary, we will need $O(n^2 4^n)$ gates.

  A Historical Remark: this fact was stated in [KLM06, Theorem 4.3.3] without proof; its proof is due to Bremner et al. [BDD+02]. [NC11]'s approach is to first talked about the two-level unitary decomposition, and then use the CNOT gate as *the* entangling 2-qubit gate to obtain a universal set. I think [NC11] made this choice because when this book's was originally published (October 2000), the [BDD+02] paper was not published yet.

- Show that it is impossible to use a countable set of gates to implement any circuits. Counting argument (check [KLM06, NC11]). So we have to resort to approximation.

- Notion of approximation ([KLM06, Section 4.3])

- Do the [KLM06, NC11] approach

[13] For some reason that I do not know, [NC11] did not call $\mathcal{C}_n$ the "Clifford Group".

- The most important thing for this section: Solovay-Kitaev theorem. It handles *efficient* approximation of 1-qubit gates with negligibly-close accuracy.

  The following is the version from Henry's lecture:

  Theorem 4.10.4: Solovay-Kitaev Theorem

  Let Γ be a set of 1-qubit unitaries such that:

  1. Γ generates a dense subgroup of $\mathcal{SU}(2)$;
  2. Γ is closed under inverse.

  Then, any 1-qubit unitary in $\mathcal{SU}(2)$ can be ε-approximated by a product of at most $O\big(\log^c \frac{1}{\varepsilon}\big)$ gates from the set Γ, where $c \approx 2$ is a universal constant.

  It is worth noting that $\Gamma := \{H, T\}$ satisfies the requirements in Theorem 4.10.4.

  Here are some explanations regarding Theorem 4.10.4:

  1. A subset S of a space X is dense if **any point in** X can be arbitrarily closely approximated by points in S. Mathematically, S is dense in X if for all $x \in X$ and any $\varepsilon > 0$, there exists some $s \in S$ such that:
     $$||x - s|| < \varepsilon,$$
     where $|| \cdot ||$ is some appropriate measure of distance.
  2. If a set of gates G generates a **dense subgroup** of $SU(2)$, it means that by using gates from G, we can approximate **any matrix in** $SU(2)$ arbitrarily closely. In quantum computing, this implies that any desired single-qubit operation can be implemented to arbitrary precision using only gates from G.
  3. Recall that $SU(2)$ is the group of all 2×2 unitary matrices with determinant 1, which describes all possible single-qubit quantum gates (ignoring global phases). A finite set of gates G cannot exactly generate every single element of $SU(2)$ because:
     - $SU(2)$ is a **continuous group** with infinitely many elements.
     - G, being a finite set, can only produce a countable number of combinations.

     However, if G generates a **dense subgroup** of $SU(2)$, then for any matrix $U \in SU(2)$ and any small $\varepsilon > 0$, there exists a matrix U_{approx} (obtained using gates from G) such that:
     $$||U - U_{\text{approx}}|| < \varepsilon.$$
     This is the key idea behind **universality** in quantum computing: a finite gate set G that generates a dense subgroup of $SU(2)$ can approximate any single-qubit operation to arbitrary precision.
  4. In short, if a gate set G generates a dense subgroup of $SU(2)$, then G can, in principle, approximate any matrix in $SU(2)$ arbitrarily well. However, the Solovay-Kitaev theorem provides additional crucial insights that go beyond just saying "dense approximation is possible." Its significance lies in providing a **constructive** and **efficient** approximation procedure.

- Examples of universal gate sets:

- $\{H, T, \mathsf{CNOT}\}$.
- $\{H, S, \mathsf{Tof}\}$. This set is less preferable than $\{H, T, \mathsf{CNOT}\}$: Tof is a 3-qubit gate but CNOT is only a 2-qubit gate.
- Clifford gates (i.e., $\{H, S, \mathsf{CNOT}\}$) plus the T gate. Recall that S is the phase gate (i.e., T^2). Clifford group is an important concept that need to be discussed. (See Definition 4.10.2 and theorem 4.10.3)

- Also, talk about Toffoli gate and its application in quantum error correction.
  - Toffoli gate is the CCNOT gate. It maps (a, b, c) to $(a, b, c \oplus (a \wedge b))$.
  - Toffoli gate itself is universal. In this sense, it is the quantum analog of NAND gate.

4.11 Magic States, Quantum Teleportation, and Transversal Implementation of the T-gate

Xiao: Xiao!

- In short, the state $|A\rangle = \frac{1}{\sqrt{2}}(|0\rangle + e^{\frac{\pi}{4}i}|1\rangle)$ is called the magic state. With the help of magic states, we can implement the T gate using only a (single) controlled S gate. (Recall that S is in the Clifford group. Thus, it follows that the Clifford generators plus magic states (as auxiliary states) can implement any quantum circuits.

- Start with the discussion at this StackExchagne Q&A.

4.12 The Theoretical Framework for Quantum Error Correction

Xiao: Vidick instructed a Quantum Error Correction course in Semester B 2023 at Weizmann Institute of Science. The course page contains many useful resources. Xiao!

This section is devoted to the following fundamental questions of quantum error correction:

1. How to model quantum error?

2. Why can we even hope to correct quantum error?

3. When can we correct quantum error?

I will not discuss any concrete QECC (e.g., the CCS code or stabilizer code) because there already exist several standard quantum information theory textbooks explaining these topics in a very clean way. However, in contrast, the answers to the above three questions provided in those textbooks are usually obscure or at least not beginner-friendly. So, it behooves me to give a cleaner and easier explanation.

To answer Question 1, we resort to the Kraus Decomposition formalism (Section 4.5.2) of quantum channels. Quantum errors are nothing other than a special type of quantum channels, and the Kraus Decomposition formalism is the most mathematically-convenient model to discuss such channels.

Xiao: Add the definitions of quantum error channels here using the Kraus Decomposition formalism. Xiao!

To answer Question 2, let focus on the case of single-qubit errors. We first remark that correcting Pauli errors (the errors represented by elements in $\{X, Y, Z\}$) is not hard. Also, note that

$\{I, X, Y, Z\}$ form a basis for all the 2×2 matrices. Then, since quantum mechanics is a linear theory, correcting errors captured by $\{X, Y, Z\}$ is equivalent to correcting all single qubit errors. Shor's 9-qubit QEEC is a great illustration of this philosophy.

Xiao: Also need to talk about multi-qubit errors, and the low-order errors in the error expansion... Xiao!

The answer to Question 3 is provided by a famous theorem that pins down the sufficient and necessary conditions for correctable quantum errors. This theorem appears in [NC11], but I personally do not like the way they formalize (and prove) it. Here, I present the version due to Daniel Gottesman [Got10, Theorem 3][14].

Xiao: The following Theorem 4.12.1 is the well-known Knill-Laflamme theorem. I later found that Section 7.2 in Preskill's notes contains a more beginner-friendly explanation if it than the following Gottesman's version. Xiao!

Theorem 4.12.1: Quantum Error-Correction Conditions

A subspace Q is a QECC for the set of $\mathcal{E}$ iff $\forall \left|\psi\right\rangle, \left|\phi\right\rangle \in Q$, $\forall E_a, E_b \in \mathcal{E}$, it holds that

$$\left\langle\psi\right| E_a^\dagger E_b \left|\phi\right\rangle = c_{ab} \left\langle\psi|\phi\right\rangle, \tag{4.15}$$

where each c_{ab} is a complex number that only depends on a and b and does not depend on $\left|\psi\right\rangle$ or $\left|\phi\right\rangle$. We remark that Equation (4.15) implies that $c_{ba} = c_{ab}^*$. That is, $\{c_{ab}\}_{a,b}$ form a Hermitian matrix.

This formalism is of course equivalent to [NC11, Theorem 10.1]. But Theorem 4.12.1 talks about codewords directly, instead of encoding projectors as in [NC11, Theorem 10.1]. So, the current Theorem 4.12.1 makes the underlying intuition more clear:

> *The errors are correctable if they preserve the relative relation between $\left|\psi\right\rangle$ and $\left|\phi\right\rangle$, up to some scalar c_{ab} that only depends on the errors (or the error channel), but not the encoded quantum states.*

The condition encoded by Equation (4.15) can be obtained by the following reasoning. If $\mathcal{E}$ is a set of correctable errors, then for any errors $E_a, E_b \in \mathcal{E}$ and any codewords $\left|\phi\right\rangle$ and $\left|\psi\right\rangle$, the corrupted states must lie in orthogonal subspaces; otherwise, we will not be able to detect the some special errors E_a, E_b when then happen on some special codewords $\left|\phi\right\rangle$ and $\left|\psi\right\rangle$. This is captured by the following requirement:

$$\left\langle\psi\right| E_a^\dagger E_b \left|\phi\right\rangle = c'_a \delta_{ab} \left\langle\psi|\phi\right\rangle, \tag{4.16}$$

where c'_a is a scalar representing the "scaling" errors[15] that are due to the error channel but independent of the quantum states (or codewords). Next, recall that we can actually correct any errors that lie in the linear space spanned by the elements in $\mathcal{E}$. Thus, we can define an arbitrary element from this linear space as $F_a = \sum_s \alpha_{as} E_s$. We can plug F_a and F_b in to Equation (4.16):

$$\left\langle\psi\right| F_a^\dagger F_b \left|\phi\right\rangle = \sum_{s,t} \alpha_{as}^* \alpha_{bt} \left\langle\psi\right| E_s^\dagger E_t \left|\phi\right\rangle = \sum_d \alpha_{ad}^* \alpha_{bd} c'_d \left\langle\psi|\phi\right\rangle,$$

which gives us Equation (4.15) with $c_{ab} = \sum_d \alpha_{ad}^* \alpha_{bd} c'_d$.

[14] Actually, Theorem 4.12.1 is stated differently from [Got10, Theorem 3]. The original [Got10, Theorem 3] is a special case of Theorem 4.12.1 where $\left|\psi\right\rangle = \left|\phi\right\rangle$ and $E_a = E_b$. But (surprisingly) one can show that these two versions are equivalent.

[15] Note that this means we also captures non-unitary errors.

4.12.1 Bounds for QECC

The quantum analogue of the Hamming bound for classical error correction code.

Lemma 4.12.2: Quantum Hamming Bound

Let $\mathcal{C}$ be a $((n, k, 2t+1))_D$ non-degenerate code in a D-dimension Hilbert space. Then, $\sum_{s=0}^{t} \binom{n}{s}(D^2-1)^s \leq D^{n-k}$. For the particular case of $D = 2$, it holds that $\sum_{s=0}^{t} \binom{n}{s} 3^s \leq 2^{n-k}$.

The quantum analog of the singleton bound for classical error correction code (Lemma 11.2.1).

Lemma 4.12.3: Quantum Singleton Bound

Let $\mathcal{C}$ be a $((n, k, d))$ code. Then $k \leq n - 2(d-1)$.

Xiao: The proof for Lemma 4.12.3 is a good example for the application of the quantum Von Neumann entropy. Xiao!

The quantum analog of the Gilbert-Varshmov bound for classical error correction code (Lemma 11.2.1).

Lemma 4.12.4: Quantum Gilbert-Varshamov Bound

Assume n, k, and d satisfy the condition $\sum_{s=0}^{d-1} \binom{n}{s}(D^2-1)^s \leq D^{n-k}$. Then, there exits a $[[n, k, d]]_D$ QECC on D-dimension Hilbert space.

4.13 Supplementary Readings for Quantum Computation and Information

General Quantum Computing.

- The bible [NC11].
- The textbook by Phillip Kaye, Raymond Laflamme, and Michele Mosca [KLM06].

Quantum Information Theory.

- For quantum information theory, [Wil11] is a great reference. (It is published by Cambridge University Press as [Wil17].).
- The Theory of Quantum Information by John Watrous.
- Quantum Entropies: the textbook by Tomamichel [Tom16].
- For quantum min, max, and conditional entropies: [KRS09].
- Quantum Leftover Hash Lemma: [RK05].
- A must-read paper for understanding entropy argument in cryptography: [BF10].

On QKD Protocols. Resources for the security proof of QKD protocols (and useful quantum information-theory tools for cryptography):

- Renato Renner's Ph.D. thesis [Ren08], where smooth min and max entropies are proposed.
- Marco Tomamichel's Ph.D. thesis [Tom12].
- [TL17] and the references therein.

Quantum Error Correction and Stabilizer Formalism.

- Lecture notes by Steven M. Girvin [Gir23].
- A short introduction by Daniel Gottesman [Got10].
- Daniel Gottesman's PhD thesis (link)
- The paper [ABOEM17].

Quantum Complexity.

- [GHLS15] is a great introductory book for quantum Hamiltonian complexity.
- Interactive Proofs with Quantum Devices (by Thomas Vidick).
- The Complexity of Quantum States and Transformations (by Scott Aaronson)
- Quantum Proofs (By Thomas Vidick and John Watrous)

On Physics.

- The "derivation" of Shrödinger's equation can be found at [Mar21].

Great Video Resources.

- Intro to Quantum Computing (by Henry Yuen)
- Quantum Complexity (by Henry Yuen)
- The quantum complexity course by Sevag Gharibian.
- The best lecture for BB84 QKE by Ramona Wolf.
- The 11th BIU Winter School on Cryptography
- Video course by Artur Ekert at https://www.arturekert.org/iqis.
- Daniel Gottesman has recorded lectures on both Quantum Information and Error-Correct Codes&Fault-Tolerant Computation. Check his homepage.

Chapter 5

Probability Theory

5.1 The Second Nature

The following results are used so often that they almost become the second nature of cryptographers:

- Let A, B, and C be three events. Then,

$$\begin{cases} \big|\Pr[A] - \Pr[B]\big| \geq 2r \\ \Pr[C] \geq 1 - r \end{cases} \quad \Rightarrow \quad \big|\Pr[A|C] - \Pr[B|C]\big| \geq r$$

Averaging Argument. The following claim follows from a simply averaging argument. We will dive into this type of argument in more detail in Section 5.4.

Lemma 5.1.1. Let A and B be two events. Let c be a constant. Assume that $\Pr[A] \geq 1 - \delta$ and $\Pr[B] = \frac{1}{c}$. It then follows that

$$\Pr[A \mid B] \geq 1 - c \cdot \delta.$$

$\diamondsuit$

5.2 The General Disjunction Rule of Events

Everyone is familiar with the following disjunction rule of two events:

$$\Pr[A_1 \vee A_2] = \Pr[A_1] + \Pr[A_2] - \Pr[A_1 \wedge A_2],$$

which can be straightforwardly demonstrated via Venn diagram.

In the following, we show the (less-familiar) extension of the above rule to n events.

$$\Pr[A_1 \vee \ldots \vee A_n] = \sum_{k=1}^{n} (-1)^{k-1} \sum_{\substack{\{i_1, \ldots, i_k\} \\ \in C_{k,n}}} \Pr[A_{i_1} \wedge \ldots \wedge A_{i_k}], \tag{5.1}$$

where $C_{k,n}$ is the set of all ordered k-uples $i_1 < \ldots < i_k$ of $[n]$.

Equation (5.1) can be easily proved by induction. But the writing could be painful, thus omitted. In the following we show the case for $n = 3$ to provide some intuition.

$$\Pr[A_1 \vee A_2 \vee A_3] = \sum_{i=1}^{3} \Pr[A_i] - \big(\Pr[A_1 \wedge A_2] + \Pr[A_1 \wedge A_3] + \Pr[A_2 \wedge A_3]\big) + \Pr[A_1 \wedge A_2 \wedge A_3]$$

5.3 Union Bound and the Probabilistic Method

Xiao: Put the definition/derivation here... Xiao!

An exemplary application of union bound and the probabilistic method is the proof of the following lemma, which is an important step in obtaining the famous Nisan-Wigderson PRG.

Lemma 5.3.1: Overlapping Subsets [NW94][1]

Let an (ℓ, k, d)-design be a set $\mathcal{I} = \{I_1, \ldots, I_m\}$, where each I_i is a size-k subset of $\{1, 2, \ldots, \ell\}$ such that any $|I_i \cap I_j| \leq d$ for all $i \neq j$.
If $\ell \geq 10k^2/d$, then there is an (ℓ, k, d)-design that achieves $m = 2^{d/10}$ and can be constructed in deterministic time $2^{O(\ell)}$.

Proof. On inputs ℓ, k, d with $\ell > 10k^2/d$, our algorithm A will construct an (ℓ, k, d)-design $\mathcal{I}$ with $2^{d/10}$ sets using the simple greedy strategy:

> Start with $\mathcal{I} = \emptyset$ and after constructing $\mathcal{I} = \{I_1, \ldots, I_m\}$ for $m < 2^{d/10}$, search all subsets of $[\ell]$ and add to $\mathcal{I}$ the first k-sized set I satisfying the following condition:
>
> $$\forall j \in [m],\ |I \cap I_j| \leq d. \tag{5.2}$$

Clearly, A runs in $\mathsf{poly}(m)2^\ell = 2^{O(\ell)}$ time and so we only need to prove it never gets stuck. In other words, it suffices to show that if $\ell = 10n^2/d$ and $\{I_1, \ldots, I_m\}$ is a collection of k-sized subsets of $[\ell]$ for $m < 2^{d/10}$, then there exists an k-sized subset $I \subseteq [\ell]$ satisfying Equation (5.2). This can be shown by probability method. Namely, we do so by showing that if we pick I at random by choosing independently every element $x \in [\ell]$ to be in I with probability $2k/\ell$.

Since the expected size of I is $2k$, it follows from Chernoff bound that

$$\Pr[|I| \geq k] \leq 0.9. \tag{5.3}$$

Since the expected size of intersection $I \cap I_j$ is $2k^2/\ell < d/5$ for all $j \in [m]$, it follows again from Chernoff bound that

$$\forall j \in [m],\ \Pr[|I \cap I_j| \geq d] \leq 0.5 \cdot 2^{-d/10}.$$

Because $m \leq 2d/10$, the above inequality together with union bound implies:

$$\forall j \in [m],\ \Pr[|I \cap I_j| < d] \geq 1 - 0.5 \cdot 2^{-d/10} \cdot 2^{d/10} = 0.5. \tag{5.4}$$

?? and **??** implies that with probability at least $0.9 \cdot 0.5 = 0.45$, the set I will simultaneously satisfy Equation (5.2) and have size at least k. Since we can always remove elements from I without damaging Equation (5.2), this completes the proof. ■

5.3.1 Johnson-Lindenstrauss Lemma

An nice application of the union bound is in the famous Johnson-Lindenstrauss lemma, which states that any set of n points in a high-dimensional space can be embedded into a low-dimensional space (of dimension $O(\log n)$) while preserving the pairwise distances between the points up to a small

[1] This formalization and proof are taken from [AB09].

distortion. The proof of this lemma relies on the union bound to show that with high probability, the random projection used in the embedding will preserve the distances for all pairs of points.

The Core Idea. Imagine you have a massive cloud of points in a very high-dimensional space—say, a million dimensions. The Johnson–Lindenstrauss (JL) lemma tells you something almost magical: you can project all those points down into a *much* lower-dimensional space, and the **distances between every pair of points are approximately preserved**. The distortion can be made as small as you like, and the number of dimensions you need in the target space depends only on the *number of points*, not on the original dimension.

Formal Statement: The Johnson-Lindenstrauss lemma can be stated as follows:

Lemma 5.3.2: Johnson-Lindenstrauss

Given n points in $\mathbb{R}^d$ (where d can be astronomically large) and a tolerance $0 < \varepsilon < 1$, there exists a linear map

$$f : \mathbb{R}^d \to \mathbb{R}^k$$

where

$$k = O\left(\frac{\log n}{\varepsilon^2}\right)$$

such that for **every** pair of points u, v in the original set:

$$(1 - \varepsilon)\,\|u - v\|^2 \;\le\; \|f(u) - f(v)\|^2 \;\le\; (1 + \varepsilon)\,\|u - v\|^2.$$

In words: all pairwise distances are preserved up to a factor of $(1 \pm \varepsilon)$.

How the Proof Works (Intuition). The proof is surprisingly simple and relies on **random projections**. Here is the recipe.

1. **Pick a random matrix.** Construct a $k \times d$ matrix R whose entries are drawn i.i.d. from a Gaussian $\mathcal{N}(0,\, 1/k)$. The map is then simply $f(x) = Rx$.

2. **Analyse a single pair.** For any fixed vector $w = u - v$, the projected vector Rw is a sum of independent random variables. By concentration of measure (specifically a variant of the Chernoff/Hoeffding bound for chi-squared random variables), the squared norm $\|Rw\|^2$ concentrates tightly around its expectation $\|w\|^2$. The probability that the distortion exceeds ε is bounded by something like
$$2\exp\bigl(-c\,\varepsilon^2\,k\bigr).$$

3. **Union bound over all pairs.** There are $\binom{n}{2} < n^2$ pairs of points. If we choose $k = O(\log n/\varepsilon^2)$, then the failure probability per pair is less than $1/n^2$, so by the union bound all $\binom{n}{2}$ distances are simultaneously preserved with positive (in fact high) probability.

That is it. A random projection works, and we do not even need to see the data to construct it.

Why Is It So Important? The Johnson-Lindenstrauss lemma is one of the most influential results in modern computer science and data science for several deep reasons.

1. **It breaks the curse of dimensionality.** In machine learning and data analysis, data often lives in spaces with thousands or millions of dimensions (think word embeddings, genomic data, image features). Most algorithms have running times or storage costs that depend on the dimension d. The Johnson-Lindenstrauss lemma says you can reduce d to $O(\log n)$—which is *tiny*—while faithfully preserving the geometric structure (distances) that most algorithms depend on. This makes intractable problems tractable.

2. **The target dimension is independent of the original dimension.** This is the truly shocking part. Whether your data starts in $\mathbb{R}^{100}$ or $\mathbb{R}^{10^9}$, the number of dimensions you need after projection depends only on how many points you have and how much distortion you will tolerate. A dataset of a million points can be faithfully embedded in roughly $O(\log 10^6) \approx 40$–50 dimensions regardless of where it started.

3. **The projection is data-oblivious.** Unlike PCA or autoencoders, which need to see the data first, a JL projection is just a random matrix. This means it can be computed in advance, works in streaming settings, and is trivially parallelizable. It also means there are strong privacy benefits—you can project data without knowing what it looks like.

4. **It underpins a huge number of algorithms.** The Johnson-Lindenstrauss lemma is a workhorse behind many landmark results, including approximate nearest neighbor search (locality-sensitive hashing), fast approximate algorithms for clustering and classification, compressed sensing, sketching algorithms for streaming data (e.g., the Count-Sketch), and randomized numerical linear algebra (e.g., fast approximate matrix multiplication and regression).

5. **It reveals something fundamental about high-dimensional geometry.** The lemma is a manifestation of the *concentration of measure* phenomenon: in high dimensions, random projections are remarkably well-behaved. It tells us that high-dimensional Euclidean space is, in a sense, much more "compressible" than we might naïvely think.

A Concrete Example: Suppose you have $n = 1{,}000{,}000$ documents, each represented as a TF-IDF vector in $\mathbb{R}^{100{,}000}$. You want to find nearest neighbours, but working in 100,000 dimensions is expensive.

With $\varepsilon = 0.1$ (10% distortion), the Johnson-Lindenstrauss lemma guarantees you can project down to roughly

$$k \approx \frac{24 \ln(10^6)}{0.1^2} \approx \frac{24 \times 13.8}{0.01} \approx 33{,}120$$

dimensions (and in practice, even fewer often suffice). In those $\sim$33,000 dimensions, every pairwise distance is within 10% of its original value, so your nearest-neighbour queries return essentially the same results—but much faster.

Key Takeaway. The Johnson–Lindenstrauss lemma tells us that **random projection is an absurdly effective dimensionality reduction technique**. It is simple, fast, requires no knowledge of the data, comes with provable guarantees, and the resulting dimension depends only on the number of points—not the ambient dimension. It is one of those rare results that is both theoretically beautiful and immediately practical.

5.3.2 Bonferroni's Inequality

The following inequality can be considered as the counterpart of the union bound.

Lemma 5.3.3: Bonferroni's Inequality
$\Pr[A_1 \wedge \ldots \wedge A_n] \geq \Pr[A_1] + \ldots + \Pr[A_n] - (n-1)$.

5.4 Averaging Argument

Consider the following simple fact: if the average of a set real numbers $\{a_i\}_{i\in[n]}$ is some c, then there must exist some $a_i \geq c$ (or $a_i \leq c$). This fact with some of its variants turns out to be very helpful in many cryptographic scenarios, especially in the security proof of protocols where non-uniform argument is used.

Before we present the most crypto-friendly version, see an example for how powerful this kind of argument can be (even) at its simplest form:

- Erdos argument for no-monochromatic-clique graph.

There are several interesting variants of this argument, see Appendix A.2.2 of [AB09]. In the following, we show a popular one that always appears when a proof wants to make use of the auxiliary (or random) tape of non-uniform Turing machines.

Lemma 5.4.1: Averaging Argument

If $X \in [0,1]$ and $E[X] = \mu$, then $\forall c < 1$ the following holds:

$$\Pr[X \leq c\mu] \leq \frac{1-\mu}{1-c\mu} \tag{5.5}$$

Remark 5.4.1. *As one may already realized, the averaging argument shown in Lemma 5.4.1 has a similar flavor of the famous Markov Inequality (Lemma 5.7.1). Indeed, both of them say that a sampled random variable cannot differ too much from its expectation. Some papers refer to Markov inequality as the "averaging argument".*

A Toy Example. An interesting application (of Lemma 5.4.1) that has some counter-intuitive implication: suppose you took a lot of exams, each with the score range $[1, 100]$. If your average score was 90, then in $\geq \frac{1}{2}$ fraction of these exams you scored ≥ 80.

The following corollary (of Lemma 5.4.1) is ubiquitous in the security proof of crypto protocols.

Corollary 5.4.1 (Averaging Argument)**.** If $a_1, a_2, \ldots, a_n$ are numbers in the interval $[0,1]$ whose average is ρ, then at least $\frac{\rho}{2}$ of the a_i's are at least as large as $\frac{\rho}{2}$. $\diamondsuit$

5.4.1 Exemplary Applications of the Averaging argument

Applications in Non-Uniform Argument. We demonstrate the usage of Corollary 5.4.1 by the following abstracted scenario. Consider a adversary Adv in some security reduction. Imagine that we want to build a machine $\mathcal{B}$ such that if Adv does something specific (w.l.o.g., say "outputting 1") with probability p^* (e.g., breaks the security property we are proving), $\mathcal{B}$ can make use of Adv to break some underlying assumptions with some probability polynomially-related to p^*. In this procedure, $\mathcal{B}$ may (internally) run Adv up to some point and save the current state of Adv as st^*, to which we usually refer as "freeze machine Adv at st^*" . Later, it may start Adv (directly) from st^* to finish the remaining steps, or to perform some specific operation (e.g., rewinding the steps after st^*).

A common task in this scenario is to estimate the probability that Adv outputs 1 when being stated from st^*. For example, if rewinding is the concerned operation, this probability determines how many rewindings (the expected running time) are necessary for Adv to output 1 (again).

According to our assumption, we have $\Pr[\mathsf{Adv} = 1] = p^*$. But this probability is taken over all the randomness used by Adv, which might include the random tape of Adv, the distribution of the input and so on. Since we now freeze Adv at st^*, the probability of outputting 1 is not p^* anymore. We actually target the following conditional probability

$$\Pr[\mathsf{Adv} = 1 \mid \text{starting from } \mathsf{st}^*].$$

More formally, we should use S to denote the random variable that describe the possible status of Adv up to the "freezing point". We use Sup to denote the support of S . We are interested in the case where $\mathrm{S} = \mathsf{st}^*$. Let us consider the following decomposition:

$$\begin{aligned}\Pr[\mathsf{Adv} = 1] &= \sum_{\mathsf{st} \in \mathsf{Sup}} \Pr[\mathsf{Adv} = 1 \wedge \mathrm{S} = \mathsf{st}] \\ &= \sum_{\mathsf{st} \in \mathsf{Sup}} \Pr[\mathsf{Adv} = 1 \mid \mathrm{S} = \mathsf{st}] \cdot \Pr[\mathrm{S} = \mathsf{st}].\end{aligned}$$

The idea behind the proof of Corollary 5.4.1 tells us a useful fact that there are at least $p^*/2$ fraction of Sup such that from Adv resuming from these states will output 1 with probability at least $p^*/2$.[2] Using our notation, it means that there exists a subset $\mathsf{Sup}' \subset \mathsf{Sup}$ such that the following holds:

(i) $|\mathsf{Sup}'| \geq \frac{p^*}{2} |\mathsf{Sup}|$, **and**

(ii) $\forall \mathsf{st} \in \mathsf{Sup}'$, $\Pr[\mathsf{Adv} = 1 \mid \mathrm{S} = \mathsf{st}] \geq \frac{p^*}{2}$.

In the common setting, p^* is usually noticeable. The above says that if $\mathcal{B}$ picks st uniformly at random, then with noticeable probability, the remaining part of Adv will finish outputting 1 (or satisfy some requirement) with noticeable probability. This usually suffices to finish the security reduction.

For a concrete example of the above approach, see the proof of soundness for the famous BJY protocol (a 4-round ZKAoK from any OWFs) [BJY97, Lemma 4.3].

Applications to "Truncated" Executions. In some scenarios, we need to do security reduction with an expected polynomial time adversary. This could be potentially problematic as the cryptographic assumptions are usually stated w.r.t. PPT adversaries. To address this problem, we can truncate the target adversary when it goes beyond some pre-fixed polynomial running time, and still hope to finish the reduction successfully with non-negligible probability.

?? contains a detailed discussion (with concrete examples) about how to use averaging argument in such a "truncating" argument.

Applications in the Black-Box Separation Literature. Averaging argument is widely used in the proof of black-box separation results. In these applications, it usually appears in the following form (e.g., [HR04, CLMP12, Haj18]).

[2]Note that I mean to say that following the same proof technique used to prove Corollary 5.4.1, we can prove the concerned fact. But this fact does not following immediately from Corollary 5.4.1 per se.

Let X and Y be random variables. Let $\mathsf{Event}_{X,Y}$ be some event depend on X and Y. Assume it is known that

$$\Pr_{X,Y}[\mathsf{Event}_{X,Y}] \geq 1-\varepsilon. \tag{5.6}$$

Then, the averaging argument[3] is used to show that for $c > 1$

$$\Pr_X\left[\Pr_Y[\mathsf{Event}_{X,Y}] \geq 1 - c\cdot\varepsilon\right] \geq 1-\frac{1}{c}, \tag{5.7}$$

and also,

$$\Pr_X\left[\Pr_Y[\mathsf{Event}_{X,Y}] \geq 1 - \frac{1}{c}\right] \geq 1-c\cdot\varepsilon, \tag{5.8}$$

This procedure is so common that authors usually omit the details. Here, I provide a detailed derivation that may help a beginner to see the concrete math.

To do that formally, assume for contradiction that **??** does not hold. That is, there are a $\delta < 1-\frac{1}{c}$ fraction of X such that $\Pr_Y[\mathsf{Event}_{X,Y} \mid X] \geq 1-c\cdot\varepsilon$. Call this δ fraction of X "good". Then, we have

$$\begin{aligned}
\Pr_{X,Y}[\mathsf{Event}_{X,Y}] &= \Pr_Y[\mathsf{Event}_{X,Y} \mid X \text{ is good}]\cdot\Pr_X[X \text{ is good}] + \Pr_Y[\mathsf{Event}_{X,Y} \mid X \text{ is bad}]\cdot\Pr_X[X \text{ is bad}]\\
&\leq 1\cdot\delta + (1-c\cdot\varepsilon)\cdot(1-\delta)\\
&= 1 - c\cdot\varepsilon + c\cdot\varepsilon\cdot\delta\\
&= 1 + c\cdot(\delta-1)\varepsilon\\
&< 1-\varepsilon \qquad\qquad (\text{since } \delta < 1-1/c).
\end{aligned}$$

This contradicts **??**, thus finishing the proof.

If we replace the $\mathsf{Event}_{X,Y}$ with its negation, we will get the following version of averaging argument:

$$\Pr_{X,Y}[\mathsf{Event}_{X,Y}] \leq \varepsilon \;\Rightarrow \begin{cases} \Pr_X\left[\Pr_Y[\mathsf{Event}_{X,Y}] \leq c\cdot\varepsilon\right] \geq 1-\frac{1}{c} \\ \Pr_X\left[\Pr_Y[\mathsf{Event}_{X,Y}] \leq \frac{1}{c}\right] \geq 1-c\cdot\varepsilon \end{cases}. \tag{5.9}$$

<u>Another Proof.</u> There is another way to derive **??** from **??**, using Markov's inequality (Lemma 5.7.1). First, observe that

$$\Pr_{X,Y}[\mathsf{Event}_{X,Y}] = \mathop{\mathbb{E}}_X\left[\Pr_Y[\mathsf{Event}_{X,Y}]\right] \quad \Big(\text{and } \Pr_{X,Y}[\neg\mathsf{Event}_{X,Y}] = \mathop{\mathbb{E}}_X\left[\Pr_Y[\neg\mathsf{Event}_{X,Y}]\right]\Big).$$

Thus, it follows from **??** that

$$\begin{aligned}
&\mathop{\mathbb{E}}_X\left[\Pr_Y[\neg\mathsf{Event}_{X,Y}]\right] \leq \varepsilon\\
\Rightarrow &\Pr_X\left[\Pr_Y[\neg\mathsf{Event}_{X,Y}] \geq c\cdot\varepsilon\right] \leq \frac{1}{c} \qquad (\text{by Markov's Inequaltiy as in Lemma 5.7.1})\\
\Rightarrow &\Pr_X\left[\Pr_Y[\neg\mathsf{Event}_{X,Y}] \leq c\cdot\varepsilon\right] \geq 1-\frac{1}{c}\\
\Rightarrow &\Pr_X\left[\Pr_Y[\mathsf{Event}_{X,Y}] \geq 1-c\cdot\varepsilon\right] \geq 1-\frac{1}{c},
\end{aligned}$$

[3] As mentioned in Remark 5.4.1, "averaging argument" in this context usually refers to Markov Inequality

where the last inequality is exactly **??**.

5.4.2 Another Example: Hardness Amplification from Weak OWFs to OWFs

Xiao: Another good exemplary application of the averaging argument is the amplification from weak OWFs to (ordinary) OWFs [Yao82]. This lecture note by Rafael Pass contains a good presentation of it. Xiao!

5.5 The Coupling Argument: A Note on Its Core Inequality

5.5.1 Introduction

A *coupling argument* is a proof technique borrowed from probability theory and used pervasively in the analysis of randomized algorithms, Markov chains, and cryptographic security proofs. Its purpose is to compare two random processes (or two probability distributions, or two events arising in two different experiments) by constructing them jointly on a single probability space in a carefully chosen way.

The key conceptual move is this: instead of analyzing two distributions μ and ν separately and then comparing them, one builds a joint distribution π on pairs (X, Y) such that

$$X \sim \mu, \qquad Y \sim \nu,$$

i.e., π has the correct marginals. Such a π is called a *coupling* of μ and ν. The freedom lies in how X and Y are correlated under π, and a clever choice of correlation often makes it easy to argue that $X = Y$ with high probability.

Why the construction helps. Once X and Y are placed on the same probability space, the distance between μ and ν can be controlled by the probability of disagreement:

$$\|\mu - \nu\|_{\mathrm{TV}} \;\le\; \Pr_{(X,Y)\sim\pi}[X \neq Y].$$

At the level of events, the analogous inequality is

$$\big|\Pr[E_1] - \Pr[E_2]\big| \;\le\; \Pr[E_1 \neq E_2], \tag{5.10}$$

Xiao: Prove the first inequality using the second inequality? which is the *core inequality* underlying every coupling argument. Here E_1 and E_2 are events defined on the common (coupled) probability space, and $\{E_1 \neq E_2\}$ denotes the symmetric difference $E_1 \triangle E_2$. Xiao!

Typical use in cryptography. In cryptographic hybrid proofs, one often wishes to bound

$$\big|\Pr[\mathcal{A} \text{ wins in } \mathbf{H}_1] - \Pr[\mathcal{A} \text{ wins in } \mathbf{H}_2]\big|,$$

where $\mathbf{H}_1$ and $\mathbf{H}_2$ are two experiments differing in some local way (e.g., a single oracle is patched at one point). The standard recipe is:

1. **Couple** the two experiments by running them on the *same* underlying randomness (same function f, same coins r, b, D, same auxiliary input φ, etc.), differing only in the local change.
2. Observe that the win-events E_1, E_2 can disagree only when some intermediate quantity (e.g., the adversary's first-stage output) differs between the two runs.

3. Apply the core inequality (5.10) to bound the hybrid distance by the probability of that intermediate disagreement.

4. Bound that probability by analyzing the local source of divergence (e.g., the probability the adversary queries the patched point).

The cleverness of any coupling argument lies in step 1: choosing how to share randomness so that the right-hand side of (5.10) becomes small and tractable. The inequality itself, however, is elementary, and we now state and prove it.

5.5.2 The Core Inequality

Lemma 5.5.1 (Coupling Inequality for Events)**.** Let E_1 and E_2 be events on a common probability space $(\Omega, \mathcal{F}, \Pr)$. Then

$$\left|\Pr[E_1] - \Pr[E_2]\right| \;\le\; \Pr[E_1 \neq E_2],$$

where $\{E_1 \neq E_2\} := E_1 \triangle E_2 = (E_1 \cap \overline{E_2}) \cup (\overline{E_1} \cap E_2)$. $\Diamond$

Proof. We give two short proofs.

Proof 1 (via indicators). Let $\mathbf{1}_E$ denote the indicator random variable of an event E, so $\Pr[E] = \mathbb{E}[\mathbf{1}_E]$. By linearity of expectation,

$$\Pr[E_1] - \Pr[E_2] = \mathbb{E}[\mathbf{1}_{E_1} - \mathbf{1}_{E_2}]\,.$$

For each outcome $\omega \in \Omega$, the value $\mathbf{1}_{E_1}(\omega) - \mathbf{1}_{E_2}(\omega) \in \{-1, 0, +1\}$, and it is nonzero precisely when $\omega \in E_1 \triangle E_2$. Hence pointwise,

$$\left|\mathbf{1}_{E_1}(\omega) - \mathbf{1}_{E_2}(\omega)\right| = \mathbf{1}_{\{E_1 \neq E_2\}}(\omega).$$

Applying $|\mathbb{E}[X]| \le \mathbb{E}[|X|]$ yields

$$\left|\Pr[E_1] - \Pr[E_2]\right| = \left|\mathbb{E}[\mathbf{1}_{E_1} - \mathbf{1}_{E_2}]\right| \le \mathbb{E}\big[\left|\mathbf{1}_{E_1} - \mathbf{1}_{E_2}\right|\big] = \Pr[E_1 \neq E_2].$$

Proof 2 (set-theoretic). Decompose each event according to whether the other occurs:

$$\begin{aligned} \Pr[E_1] &= \Pr[E_1 \cap E_2] + \Pr\big[E_1 \cap \overline{E_2}\big], \\ \Pr[E_2] &= \Pr[E_1 \cap E_2] + \Pr\big[\overline{E_1} \cap E_2\big]. \end{aligned}$$

Subtracting,

$$\Pr[E_1] - \Pr[E_2] = \Pr\big[E_1 \cap \overline{E_2}\big] - \Pr\big[\overline{E_1} \cap E_2\big].$$

Both terms on the right are nonnegative, so the absolute value of their difference is at most their sum:

$$\left|\Pr[E_1] - \Pr[E_2]\right| \le \Pr\big[E_1 \cap \overline{E_2}\big] + \Pr\big[\overline{E_1} \cap E_2\big] = \Pr[E_1 \triangle E_2] = \Pr[E_1 \neq E_2].$$

■

Remark 5.5.1. *The lemma is trivial in isolation: it merely says that two probabilities can differ by at most the mass on which the two events disagree. Its power in coupling arguments comes from the freedom to* construct *the joint probability space so that the disagreement event $\{E_1 \neq E_2\}$ is rare and structurally simple to analyze. Designing this coupling is the creative part of the argument; inequality* (5.10) *is the mechanical step that converts the design into a quantitative bound.*

We now show how the core inequality

$$\big|\Pr[E_1] - \Pr[E_2]\big| \;\le\; \Pr[E_1 \neq E_2]$$

implies the distributional version

$$\|\mu - \nu\|_{\mathrm{TV}} \;\le\; \Pr_{(X,Y)\sim\pi}[X \neq Y].$$

Before stating the proof, we briefly recall the measure-theoretic notation, which is essential for understanding why the argument works.

Probability spaces and the symbol ω. In measure-theoretic probability, every random experiment is modeled by a *probability space* $(\Omega, \mathcal{F}, \Pr)$, where:

- Ω is the *sample space* — the set of all possible "raw outcomes" of the experiment;
- an element $\omega \in \Omega$ is one specific outcome, i.e., one complete realization of all the randomness in the experiment;
- $\mathcal{F}$ is the collection of events (measurable subsets of Ω);
- $\Pr$ assigns probabilities to events.

A *random variable* taking values in some space $\mathcal{X}$ is then *a function*

$$X : \Omega \to \mathcal{X},$$

and $X(\omega)$ is the value that X takes when the underlying outcome is ω. If X and Y are two random variables defined on the *same* probability space, both are functions of the same ω, and the statement $X(\omega) = Y(\omega)$ means that, for this particular outcome ω, the two functions return the same value. The event $\{X = Y\}$ is the set of all such outcomes:

$$\{X = Y\} \;:=\; \{\omega \in \Omega : X(\omega) = Y(\omega)\}.$$

Probabilists often drop ω from the notation and simply write "$X = Y$" or "$X \in A$", with the understanding that both random variables are evaluated at the same (implicit) outcome ω.

Setup. Let μ, ν be probability distributions on a measurable space $(\mathcal{X}, \mathcal{F})$, and let π be any *coupling* of them: a joint distribution on $\mathcal{X} \times \mathcal{X}$ such that, under $(X, Y) \sim \pi$, the marginals satisfy $X \sim \mu$ and $Y \sim \nu$. The total variation distance is

$$\|\mu - \nu\|_{\mathrm{TV}} \;=\; \sup_{A \in \mathcal{F}} \big|\mu(A) - \nu(A)\big|.$$

Proof. Work on the joint probability space $(\Omega, \mathcal{F}_\Omega, \Pr)$ underlying the coupling π, on which both X and Y are defined as functions $X, Y : \Omega \to \mathcal{X}$. Fix an arbitrary measurable set $A \in \mathcal{F}$, and define the two events

$$E_1 := \{X \in A\} = \{\omega \in \Omega : X(\omega) \in A\}, \qquad E_2 := \{Y \in A\} = \{\omega \in \Omega : Y(\omega) \in A\}.$$

By the marginal property of the coupling,

$$\Pr[E_1] = \mu(A), \qquad \Pr[E_2] = \nu(A).$$

Step 1: Apply the core inequality. Since E_1 and E_2 live on the common (coupled) probability space, the core inequality gives

$$\left|\mu(A) - \nu(A)\right| \;=\; \left|\Pr[E_1] - \Pr[E_2]\right| \;\leq\; \Pr[E_1 \neq E_2].$$

Step 2: Compare the disagreement event to $\{X \neq Y\}$. We claim that $\{E_1 \neq E_2\} \subseteq \{X \neq Y\}$. To see this, fix any outcome $\omega \in \Omega$ and observe the pointwise implication

$$X(\omega) = Y(\omega) \quad\Longrightarrow\quad \big(X(\omega) \in A \iff Y(\omega) \in A\big) \quad\Longrightarrow\quad \mathbf{1}_{E_1}(\omega) = \mathbf{1}_{E_2}(\omega).$$

In words: if, for this particular outcome ω, the two random variables happen to take the same value, then either both values are in A or both are outside A — they cannot disagree about membership in A. Contrapositively, the events E_1 and E_2 can disagree at ω only if $X(\omega) \neq Y(\omega)$. Therefore

$$\{E_1 \neq E_2\} \;\subseteq\; \{X \neq Y\},$$

and by monotonicity of probability,

$$\Pr[E_1 \neq E_2] \;\leq\; \Pr[X \neq Y].$$

Step 3: Combine and take the supremum. Chaining the two inequalities,

$$\left|\mu(A) - \nu(A)\right| \;\leq\; \Pr[X \neq Y]$$

holds for every $A \in \mathcal{F}$. The right-hand side does not depend on A, so taking the supremum over A yields

$$\|\mu - \nu\|_{\mathrm{TV}} \;=\; \sup_{A \in \mathcal{F}} \left|\mu(A) - \nu(A)\right| \;\leq\; \Pr_{(X,Y)\sim\pi}[X \neq Y]. \qquad \blacksquare$$

Remark 5.5.2. *The proof shows that the distributional TV bound is just the event-level inequality applied uniformly over all measurable sets, plus the trivial observation that* if the random variables themselves agree at ω, then so do all events derived from them at ω. *This is why proving the event version suffices to obtain the more general distributional version: the latter is a uniform consequence of the former.*

5.6 Berry-Esseen Theorem

Xiao: Check these resources: Xiao!

- Ryan O'donnell's lecture
- This Wikipedia page.

Recently, Tomaszewski's Conjecture was resolved [KK20]. Berry-Esseen inequality plays an important role in the final proof.

5.7 Concentration Bounds

5.7.1 Markov Inequality

The common form of Markov Inequality is shown below:

Lemma 5.7.1: Markov Inequality

If X is a nonnegative random variable and $a > 0$, then the probability that X is at least a is at most the expectation of X divided by a, i.e.,

$$\Pr[X \geq a] \leq \frac{E(x)}{a}.$$

Let $a = c \cdot E(X)$ (where $c > 0$); then, we can rewrite the previous inequality as:

$$\Pr[X \geq c \cdot E(X)] \leq \frac{1}{c}.$$

The following version is Markov inequality is useful if one want to lower-bound the value of a random variable. It can be proved by applying Markov's inequality (Lemma 5.7.1) to $X = B - Y$. Note that X is a non-negative random variable as $Y < B$.

Corollary 5.7.2: The Reverse Markov Inequality

Let Y be a random variable that is never larger than $B \in \mathbb{R}$. Then, for all $a < B$,

$$\Pr[Y \leq a] \leq \frac{B - E(Y)}{B - a}.$$

The following is a widely used argument in cryptography. It is so standard that many authors refer to it without a proof. In [DGH+19], the authors formalize it under the name "Markov Inequality for Advantages"[4]. The reason why it is called "Markov Inequality" remains mysterious to me. Maybe it is because the proof and the intuition behind this bound goes in the same sense as the standard Markov Inequality?

Theorem 5.7.1 (Markov Inequality for Advantages)**.** Let $A(Z)$ and $B(Z)$ be two random variables depending on a random variable Z and potentially additional random choices. Assume that

$$\big|\Pr_Z[A(Z) = 1] - \Pr_Z[B(Z) = 1]\big| \geq \varepsilon \geq 0.$$

Then

$$\Pr_Z\big[\big|\Pr[A(Z) = 1] - \Pr[B(Z) = 1]\big| \geq \frac{\varepsilon}{2}\big] \geq \frac{\varepsilon}{2}.$$

◊

Proof. The idea is to condition the event on $\big|\Pr[A(Z) = 1] - \Pr[B(Z) = 1]\big| \geq \frac{\varepsilon}{2}$. Let

$$a = \Pr_Z\big[\big|\Pr[A(Z) = 1] - \Pr[B(Z) = 1]\big| \geq \frac{\varepsilon}{2}\big].$$

We have $\varepsilon \leq a \times 1 + (1 - a) \times \frac{\varepsilon}{2}$. Since $0 \leq 1 - a \leq 1$, we obtain $\varepsilon \leq a + \frac{\varepsilon}{2}$. The inequality now follows. ∎

5.7.2 Chebyshev Inequality

Xiao: Chebyshev's inequality played an important role in the hidden-bits model NIZK setting, e.g. Xiao!

[4] This is the first place where I saw such a formalism But it is possible that it already appeared somewhere else.

[FLS90, GIK+23].

5.7.3 Chernoff Bound

Xiao: Wikipedia has an exhaustive explanation for this topic. Xiao!

The lecture of Prof. O'donnell also gives a great presentation for Chernoff Bounds

Theorem 5.7.2 (Chernoff Bound)**.** Let X_i be i.i.d. random variables such that $0 \le X_i \le 1$. Let $\mu = \mathbb{E}[\sum_i X]$. For and $\varepsilon > 0$, we have

- $\Pr[X \le (1-\delta)\cdot\mu] \le \exp\big(-\frac{\delta^2\mu}{2}\big)$
- $\Pr[X \ge (1+\delta)\cdot\mu] \le \exp\big(-\frac{\delta^2\mu}{2+\delta}\big)$

♢

Interestingly, if we know that $L \le \mu \le H$, the following bounds hold:

- $\Pr[X \le (1-\delta)\cdot L] \le \exp\big(-\frac{\delta^2 L}{2}\big)$
- $\Pr[X \ge (1+\delta)\cdot H] \le \exp\big(-\frac{\delta^2 H}{2+\delta}\big)$

The following corollary of Chernoff bound is taken from Prof. O'donnell's notes for his lecture on Chernoff Bound. The poof was left as an exercise.

Lemma 5.7.1 (Sampling Lemma)**.** Let μ be the unknown mean for a random variable $0 \le X \le 1$. Let $x_1, \ldots, x_n$ be n independent samples of X. Let $\hat{\mu}$ be the empirical mean of x_i's, i.e. $\hat{\mu} := \frac{x_1+\cdots+x_n}{n}$. Then for any $0 < \varepsilon, \delta < 1$ such that $n \ge \frac{3\ln(1/\delta)}{\varepsilon^2}$, the following holds:

$$\Pr\big[|\hat{\mu} - \mu| \le \varepsilon\big] \ge 1 - \delta.$$

♢

Theorem 5.7.3 (Chernoff Bound)**.** Xiao: Put the general form here .... ♢ Xiao!

Xiao: Xiao!

Theorem 5.7.4 (Chernoff Bound (Upper Tail))**.** Let $X = \sum_{i=1}^n X_i$, where $X_i = 1$ with probability p_i and $X_i = 0$ with probability $1-p_i$, and all X_i's are independent. Let $\mu = \mathbb{E}[X] = \sum_{i=1}^n p_i$. Then the following holds for any $0 < \delta < 1$:

$$\Pr[X \ge (1+\delta)\cdot\mu] \le \exp\big(-\frac{\delta^2\mu}{2+\delta}\big)$$

♢

Probably the most widely used form of Chernoff bound is the following one:

Corollary 5.7.1. Let $X_1, \ldots, X_n$ be independent variables with $0 \le X_i \le 1$ for all $1 \le i \le n$, denote $\mu = \mathbb{E}[\frac{\sum_{i=1}^n X_i}{n}]$. Then, for any $\varepsilon > 0$,

$$\Pr\big[\big|\frac{\sum_{i=1}^n X_i}{n} - \mu\big| \ge \varepsilon\big] \le 2^{-\varepsilon^2\cdot n} \tag{5.11}$$

♢

The take-away from Chernoff bound is very simple: The empirical mean of a bunch of independent random variables is approaching the expectation (of the mean) in an exponentially fast manner.

5.7.4 An Exemplary Application of Concentrations Bounds: the Goldreich-Levin Theorem

Xiao: the proof of Goldreich-Levin theorem serves as an beautiful example where Markov's inequatliy, Chebyshev's inequality, and Chernoff bound are used. So, now is a prefect time to present this famous theorem. Use the version from Trevisan's lecture notes. Xiao!

My favorite explanation of Goldreich-Levin theorem is Bellare's lecture notes.

5.7.5 Hoeffding Inequality

Hoeffding's Inequality can be regarded as the most general (at least for TCS applications) concentration bound for *independent* random variables. In particular, Chernoff's inqaulity can be recovered as a special case of Hoeffding's inequality. A clean derivation of Hoeffding's inequality can be found in this lecture notes.

Theorem 5.7.5 (Hoeffding's Inequality [Hoe63])**.** Let $X_1, \ldots, X_n$ be independent variables with $b_i \le X_i \le a_i$ for all $1 \le i \le n$, denote $\mu = \mathbb{E}[\frac{\sum_{i=1}^n X_i}{n}]$. Then, for any $\varepsilon > 0$, we have:

$$\Pr\big[\big|\frac{\sum_{i=1}^n X_i}{n} - \mu\big| \ge \varepsilon\big] \le 2 \cdot e^{-\frac{2\cdot\varepsilon^2\cdot n^2}{\sum_{i=1}^n (b_i - a_i)^2}} \tag{5.12}$$

The following single-side form also holds:

$$\Pr\big[\frac{\sum_{i=1}^n X_i}{n} - \mu \ge \varepsilon\big] \le e^{-\frac{2\cdot\varepsilon^2\cdot n^2}{\sum_{i=1}^n (b_i - a_i)^2}} \tag{5.13}$$

$\Diamond$

Xiao: Include the version from [BF10, Section 2]. Xiao!

5.8 Stochastic Process

5.8.1 Doob's Martingale

Xiao: Doob's Martingale. A good source can be found at [GLM23]. Xiao!

5.9 Coupon-Collection Problems

Lemma 5.9.1. Suppose that there are m different types of coupons, and each time one obtains a coupon of type i ($1 \le i \le m$) with probability $\frac{1}{n}$, where $n \ge m$ is a parameter. Note that with probability $1 - \frac{m}{n}$, one does not obtain any coupon (or obtains an "empty coupon"). Then the expected number of coupons one need amass before obtaining k ($1 \le k \le m$) different types of non-empty coupons is

$$n \cdot (H_m - H_{m-k}) \tag{5.14}$$

where $H_t \coloneqq 1 + \frac{1}{2} + \ldots + \frac{1}{t}$ is the t-th harmonic number for $t \in \mathbb{N}$. $\Diamond$

Proof. Let $X(k)$ denote the number of coupons collected before k different types of coupons is attained. We need to compute $E[X(k)]$. we define X_j ($j = 0, 1, ..., k-1$) to be the random variable representing the number of additional coupons that need be obtained after j distinct types have been collected in order to obtain another distinct type, and we note that

$$X(k) = X_0 + X_1 + \ldots + X_{k-1}.$$

When j distinct types of coupons have already been collected, a new coupon obtained will be of a distinct type with probability $(m-j)/n$. Therefore

$$\Pr[X_j = k] = \frac{m-j}{n}\left(\frac{n-m+j}{n}\right)^{k-1} \quad k \geq 1$$

or, in other words, X_j is a geometric random variable with parameter $(m-j)/n$. Hence, $\mathbb{E}[X_j] = \frac{n}{m-j}$ implying that

$$\begin{aligned}\mathbb{E}[X(k)] &= \frac{n}{m-k+1} + \frac{n}{m-k+2} + \ldots + \frac{n}{m-1} + \frac{n}{m} \\ &= n(H_m - H_{m-k})\end{aligned}$$

where H_t is the t-th harmonic number for $t \in \mathbb{N}$. ■

Xiao!

Xiao: We can use the above lemma in the following way: in our setting, n is the total nubmer of challenges, m is the set of "good" challenges (i.e. the prover will answer), k is the number of distinct challenges needed to extract a valid witness. If k is a polynomial and m is a super-polynomial on security parameter λ, we can assume that $m - k + 1 \geq m/2$. Then the expected running time of the knowledge extractor can be bounded as

$$\begin{aligned}\mathbb{E}[X(k)] &= \frac{n}{m-k+1} + \frac{n}{m-k+2} + \ldots + \frac{n}{m-1} + \frac{n}{m} \\ &\leq n\frac{k}{m-k+1} = k \cdot \frac{n}{m-k+1} \leq k \cdot \frac{n}{2m} = 2k\frac{n}{m} = \mathsf{poly}(\lambda)\end{aligned}$$

Since both k and $\frac{n}{m}$ are polynomials of λ, the expected running time $\mathbb{E}[X(k)]$ is also upper-bounded by a polynomial of λ.

Chapter 6

Entropies

6.1 Divergence

Xiao: talk about Kullback–Leibler divergence (also called relative entropy). check this video by Wilde. Xiao!

6.2 Shanon Entropy and Friends

Xiao: Xiao!

- First, $H(X) = -\sum_x p_X(x) \log \big(p_X(x)\big)$.
- **Joint-Entropy.** This is just Shanon's entropy for a joint distribution:

$$H(X,Y) = -\sum_{x,y} p_{X,Y}(x,y) \log \big(p_{X,Y}(x,y)\big).$$

- **Conditional Entropy.** Deriving conditional entropy $H(X|Y)$. First, consider the entropy conditioned on each instantiation of Y: $H(X|Y=y) = -\sum_x p_{X|Y}(x|y) \log \big(p_{X|Y}(x|y)\big)$. Then, the conditional entropy can be defined as:

$$H(X|Y) = \sum_y p_Y(y)\big(H(X|Y=y)\big) = -\sum_{x,y} p_{X,Y}(x,y) \log \big(p_{X|Y}(x|y)\big).$$

- **Mutual Information.** $I(X:Y) = H(X) - H(X|Y) = H(Y) - H(Y|X)$. Intuitively, it represents the information about X you can infer from Y.

  This quantities is crucial in Shanon's theorem: The capacity of a classical channel $\mathcal{N}$ is given by $C(\mathcal{N}) = \max_{p_X(x)} I(X:Y)$.

- Basic properties:
  - Conditioning does not increase entropy: $H(X|Y) \le H(X)$.
  - $H(X,Y) = H(X) + H(Y|X) = H(Y) + H(X|Y)$. (Just follows by definition.)

6.2.1 Min Entropy

Entropy is a fundamental concept in information theory, measuring the unpredictability of a random variable. While Shannon entropy provides an average measure of uncertainty, **min-entropy** captures the *worst-case* unpredictability, which is especially important in cryptography and randomness extraction.

However, min-entropy can sometimes be too strict, leading to the introduction of **smooth min-entropy**, which allows small variations in probability distributions to provide a more stable entropy measure. This document explores both concepts with motivations, definitions, and examples.

Min-Entropy. We first define the min-entropy:

Definition 6.2.1 (Min-Entropy)**.** Let X be a discrete random variable with probability mass function $p(x)$. The **min-entropy** of X is defined as:

$$H_{\min}(X) = -\log_2 \max_x p(x). \tag{6.1}$$

$\Diamond$

The min-entropy measures the uncertainty of the most likely outcome of X, meaning it represents the best possible guessing strategy for an (computationally unbounded) adversary.

Example 6.2.1 (Fair Coin Toss)**.** Consider a fair coin where:

$$P(X = \text{Heads}) = 0.5, \quad P(X = \text{Tails}) = 0.5.$$

The min-entropy is:

$$H_{\min}(X) = -\log_2(0.5) = 1 \text{ bit}.$$

This means an optimal guessing strategy succeeds with probability 50%.

Example 6.2.2 (Biased Dice)**.** Consider a 6-sided die where one face (say, ”6”) appears with probability 0.5, while the others appear with probability 0.1. The min-entropy is:

$$H_{\min}(X) = -\log_2(0.5) = 1 \text{ bit}.$$

Despite having six possible outcomes, the min-entropy reflects the fact that the die is highly biased.

6.2.2 Smoothing the Min-Entropy

Motivation. Min-entropy can be too strict in real-world applications, particularly in:

- Cryptography: Noise in randomness sources can make min-entropy estimates unreliable.
- Password Security: Some users pick highly predictable passwords, but most do not.
- Quantum Cryptography: Measurement errors can affect entropy estimates.

To mitigate these issues, *smooth min-entropy* was introduced, which allows small modifications to the probability distribution to provide a more stable measure.

Definition 6.2.2 (Smooth Min-Entropy)**.** For a small smoothing parameter ε, the **ε-smooth min-entropy** of X is defined as:

$$H^{\varepsilon}_{\min}(X) = \max_{\widetilde{X} \in B_\varepsilon(X)} H_{\min}(\widetilde{X}), \tag{6.2}$$

where $B_\varepsilon(X)$ is a set of probability distributions that are ε-close to X in total variation distance. $\Diamond$

This means we ignore rare, extreme cases and compute min-entropy on a slightly modified distribution.

6.2.2.1 Example: Password Guessing Attacks and Entropy Measures

In a password-based authentication system, security relies on users selecting passwords that are hard to guess. Ideally, attackers should require a large number of attempts to correctly guess a password.

However, in practice, many users choose predictable passwords such as "password," "123456," or "qwerty." Attackers exploit this by prioritizing common passwords in dictionary attacks.

Let's assume users create passwords from a dictionary of 1 million possible words. Ideally, if passwords were chosen uniformly at random:

$$H_{\min}(X) = -\log_2(10^{-6}) = 20 \text{ bits} \tag{6.3}$$

This means an attacker would need to try, on average, 2^{20} (about 1 million) guesses.

However, in reality:

- Some passwords are much more common
- Suppose "password" appears 10% of the time: $P(X = \text{"password"}) = 0.1$

Applying min-entropy:

$$H_{\min}(X) = -\log_2(0.1) \approx 3.32 \text{ bits} \tag{6.4}$$

What Does This Mean? An attacker needs about $2^{3.32} \approx 10$ guesses to break an account. This is extremely weak security!

Why Min-Entropy Can Be Misleading. Min-entropy focuses only on the most likely password.

Let's "smooth out" the top 10% of users who chose "password." The most probable password after smoothing out the top 10% might now have probability

$$P(X) = 0.005 \quad (0.5\%).$$

Applying the smooth min-entropy formula:

$$H^{\varepsilon}_{\min}(X) = -\log_2(0.005) \approx 7.64 \text{ bits}.$$

What Does This Mean? Instead of assuming that all users are vulnerable after just 10 guesses, smooth min-entropy suggests that for most users, an attacker would need about

$$2^7 = 128$$

guesses before success. This is still weak, but significantly better than the 10 guesses predicted by min-entropy.

In summary, min-entropy says

> "The entire system is extremely weak (only 3.32 bits of security)."

This is misleading because it assumes every user is equally vulnerable.

Instead, smooth min-entropy says:

> "The weakest 10% of users are at extreme risk, but the remaining 90% have more security (closer to 7.64 bits)."

This provides a more realistic assessment of security. Thus, smooth min-entropy helps us differentiate between high-risk users and the overall security of the system.

( ) Update upto

6.3 Quantum Entropies

Xiao: To do... Xiao!

6.3.1 Conditional Max and Min Entropies

Xiao: See [Ren08, Chapter 3]. Xiao!

Xiao: There seems to exist different ways to define conditional smooth min-entropy. The most easy-to-understand one appears in [VV12, Section 2]. But that formalism is different from, e.g., [Ren08, MLDS+13, MS16]. However, it seems that these difference is only cosmetic—all of them are meant to enable the extraction of randomness in the presence of quantum adversaries; In this sense. they functions equivalently. Xiao!

6.3.2 Entropic Uncertainty Relation

Xiao: To do. see [Wol21, Chapter 3.3] Xiao!

6.3.3 Rényi's Family of Entropies

The following definition is taken from [MS16, Section 3.2], which is in turn taken from [MLDS+13].

Definition 6.3.1: Quantum Rényi Divergence and Entropy

Let ρ be a density matrix on $\mathbb{C}^n$. Let σ be a positive semi-definite matrix on $\mathbb{C}^n$ whose support[a] contains the support of ρ. Let $\alpha > 1$ be a real number. Then,

$$d_\alpha(\rho\|\sigma) \coloneqq \mathrm{Tr}\left[\left(\sigma^{\frac{1-\alpha}{2\alpha}}\rho\sigma^{\frac{1-\alpha}{2\alpha}}\right)^\alpha\right]^{\frac{1}{\alpha-1}}.$$

More generally, for any[b] positive semi-definite matrix ρ' whose support is contained in the support σ, let

$$d_\alpha(\rho'\|\sigma) \coloneqq \mathrm{Tr}\left[\frac{1}{\mathrm{Tr}[\rho']}\left(\sigma^{\frac{1-\alpha}{2\alpha}}\rho'\sigma^{\frac{1-\alpha}{2\alpha}}\right)^\alpha\right]^{\frac{1}{\alpha-1}}.$$

Then, the Rényi divergence is defined as $D_\alpha(\rho'\|\sigma) \coloneqq \log d_\alpha(\rho'\|\sigma)$.
For a density operator ρ, the (unconditional) Rényi entropy is defined by

$$H_\alpha(\rho) \coloneqq D_\alpha(\rho\|I) = -\frac{1}{\alpha-1}\log\mathrm{Tr}[\rho^\alpha].$$

[a]For linear operators, "support" refers to the space which is orthogonal to its kernel (equivalently, the space spanned by the columns of the matrix).

[b]I.e., those that are not necessarily trace-1 (in other words, density operators).

Chapter 7

Number Theory

7.1 Prime Number Distribution

Xiao: Xiao!

- A useful link
- Gauss's bound
- Chebyshev bound
- Bertrand's postulate
- Talk about the relation between prime number distribution and Reimann Hypothesis

7.2 Euler's Totient Function

Xiao: Euler's theorem Xiao!

(Note that if $N = p \cdot q$, where p and q are two primes, then once we know $\phi(N)$, it is easy to factor N. To do that,

1. Note that

$$\begin{aligned} &\phi(N) = (p-1)(q-1) = N - (p+q) + 1 \\ \Rightarrow &p + q = N + 1 - \phi(N) \end{aligned} \tag{7.1}$$

2. p and q can be easily solved from Equ. 7.1 and $N = p \cdot q$.

In summary, computing $\phi(N)$ is equivalent to factorizing N when N is the product of two primes. (The other direction is trivial, i.e. it is easy to compute $\phi(N)$ given the factorization of N.)

)

Xiao: repeated squaring Xiao!

Xiao: Fermat's little theorem Xiao!

Xiao: Chinese remainder theorem Xiao!

7.3 Quadratic Residues

Lemma 7.3.1 (text)**.** Let $p > 2$ be prime. Every quadratic residue in $\mathbb{Z}_p^*$ has exactly two square roots. $\Diamond$

Th Quadratic Residuosity (QR) assumption was originally formalized in [GM84], to construct the well-known Goldwasser-Micali PKE scheme. Another very simple and interesting application is given by Kushilevitz and Ostrovsky [KO97], where they build the first computational Private Information Retrieval protocol in the single database setting with sub-linear communication complexity. More specifically, they achieve communication complexity $O(n^{\varepsilon})$ for any $\varepsilon > 0$, where n is the size of the database.

Xiao!

Xiao: There are two constructions in [KO97]. They start with the first construction which achieves communication complexity $O(n^{0.5+\varepsilon})$. Based on the first construction, they build their final scheme which achieves communication complexity $O(n^{\varepsilon})$. But the first construction is very simple. It can be presented here as a good demonstration of the power of QR assumption.

7.4 Composite Residues

This assumption gives the well-know Paillier [Pai99] and Dåmgard-Jurik [DJ01] cryptosystems. This assumption relies on the group $\mathbb{Z}^*_{N^2}$, where N is the product of two equal-length primes. The following theorem summarize important properties of $\mathbb{Z}^*_{N^2}$ to our interest.

Theorem 7.4.1. Let $N = pq$, where p, q are distinct odd primes of equal length. Then:

1. $\mathsf{gcd}(N, \phi(N)) = 1$.
2. For any integer $a \geq 0$, we have $(1+N)^a = (1+aN) \bmod N^2$.
   As a consequence, the order of $(1+N)$ in $\mathbb{Z}^*_{N^2}$ is N.
3. $\mathbb{Z}_N \times \mathbb{Z}^*_N$ is isomorphic to $\mathbb{Z}^*_{N^2}$, with isomorphism $f : \mathbb{Z}_N \times \mathbb{Z}^*_N \to \mathbb{Z}^*_{N^2}$ given by
   $$f(a, b) = (1+N)^a \cdot b^N \bmod N^2$$
   where the operation in $\mathbb{Z}_N \times \mathbb{Z}^*_N$ is defined as $(a_1, b_1) \cdot (a_2, b_2) = (a_1 + a_2, b_1 \cdot b_2)$.

$\diamondsuit$

Define the subset of N-th residues in $\mathbb{Z}^*_{N^2}$ as $\mathsf{Res}(N^2)$, using Theorem 7.4.1, we can show that very element in $\mathsf{Res}N^2$ is of the form (a, b) if written in the isomorphic group $\mathbb{Z}_N \times \mathbb{Z}^*_N$. Moreover, this characterization is sufficient. We summarize this in the following corollary.

Corollary 7.4.1. Let $N = pq$, where p, q are distinct odd primes of equal length. Denote the set of N-th residues modulo N^2 by $\mathsf{Res}(N^2)$. Then:

$$\mathsf{Res}(N^2) < \mathbb{Z}^*_{N^2} \quad \text{and} \quad \mathsf{Res}(N^2) \cong \{(0, b) \mid b \in \mathbb{Z}^*_N\} < \mathbb{Z}_N \times \mathbb{Z}^*_N$$

$\diamondsuit$

We are now ready to present the decisional composite residuosity (DCR) assumption. Intuitively, this assumption conjecctures that it is infeasible to distinguish a uniform element of $Z^*_{N^2}$ from a uniform element of $\mathsf{Res}(N^2)$. Formally,

Assumption 7.4.1 (DCR Assumption)**.** let $\mathsf{GenModulus}$ be a polynomial-time algorithm that, on input 1^λ, outputs (N, p, q) where $N = pq$, and p and q are λ-bit primes. The DCR assumption is that there exist a $\mathsf{GenModulus}$ algorithm such that for any PPT adversary Adv, the following holds:

$$\big| \Pr[\mathsf{Adv}(N, a) = 1] - \Pr[\mathsf{Adv}(N, b) = 1] \big| \leq \mathsf{negl}(\lambda)$$

where $a \xleftarrow{\$} \mathsf{Res}(N^2)$ and $b \xleftarrow{\$} \mathbb{Z}^*_{N^2}$.

7.5 Chinese Remainder Theorem

Xiao: Discuss the Chinese Remainder Theorem Xiao!

Xiao: he following materials are helpful: Xiao!

- CRT statement in Theorem 2.6 [Sho08, Page 21], Theorem 2.8 [Sho08, Page 29], Theorem 4.5 [Sho08, Page 81], and whole Section 4.4 [Sho08, Page 82].

Chapter 8

Hash Functions

useful links for this chapter:

- CMU Algorithms in the Real World course

Xiao: Here (or may be at the end of this chapter, discuss about the difference and relation between IT-secure hashing and cryptographic hashing.) Xiao!

The following paragraph is quoted from [HL18], which gives many examples for the application of cryptographic hashing:

- *Cryptographically secure hash functions are a fundamental building block in cryptography. Some of their most ubiquitous applications include the construction of digital signature schemes [NY89], efficient CCA-secure encryption [BR93], succinct delegation of computation [Kil94], and removing interaction from protocols [FS87]. In their most general form, hash functions can be modeled as "random oracles" [BR93], in which case it is heuristically assumed that an explicitly described hash function H (possibly sampled at random from a family) behaves like a random function, as far as a computationally bounded adversary can tell.*

8.1 Collision Resistant Hash Family

Collision Resistant Hash Functions, usually denoted as CRHF, was first formalized explicitly by Damgård [Dam88].

Xiao: collision resistant hash families Xiao!

Definition 8.1.1 (Collision Resistant Hash Families)**.** (to be done ...)

$\Diamond$

8.1.1 Merkle Hashing Trees and the Extraction Lemma

The following formalism of Merkle hashing trees is taken from [HHPS11].

Denote by $MT_{h,n}(X)$ the binary Merkle tree over string X using n-bit leaves and the hash function $h : \{0,1\}^{2n} \leftarrow \{0,1\}^n$. For each node k in the tree $n \in MT_{h,n}(X)$, we denote by v_k the value associated with that node. That is, the value of a leaf is the corresponding block of X, and the value of an intermediate node $n \in MT_{h,n}(X)$ is the hash $v_k = h(v_\ell \| v_r)$ where v_ℓ, v_r are the values for the left and right child of k, respectively. Xiao: (If one of the children of a node is missing from the tree then we consider its value to be the empty string.) Xiao!

For a leaf node $x \in MT_{h,n}(X)$, the sibling path of x consists of the value v_x and also the values of all the siblings of nodes on the path from x to the root. Given the index of a leaf $x \in MT_{h,n}(X)$ and a sibling path for x, we can compute the values of all the leaves on the x-to-root path itself in a bottom-up fashion by starting from the two leaves and then repeatedly computing the value of a parent as the hash of the two children values.

We say that an alleged sibling path $P = (v_x, v_{k_0}, v_{k_1}, \ldots, v_{k_i})$ is valid with respect to $MT_{h,n}(X)$ if i is indeed the height of the tree and the root value as computed on the sibling path agrees with the root value of $MT_{h,n}(X)$. Note that in order to verify that a given alleged sibling path is valid, it is sufficient to know the number of leaves and the root value of $MT_{h,n}(X)$. We also note that any two different valid sibling paths with respect to the same Merkle tree imply in particular a collision in the hash function.

Merkle Tree Proof Protocol and an Extraction Lemma. Merkle trees play an important role in interactive arguments (i.e. computationally sound proofs). It can be used in the scenario where an PPT prover P wants to hash a long string and later proves to a verifier V that the hashing is honestly done w.r.t. some preimage string, without disclosing the string in full. We present this protocol (due to [Kil92]) in Protocol 8.1.1.

Protocol 8.1.1: Merkle Tree Hash-and-Prove

Let $\mathcal{H}$ be a collision-resistant hash family. The protocol where the prover hash-and-proves w.r.t. a string X proceeds in the following way:

1. The verifier V samples a function $h \xleftarrow{\$} \mathcal{H}$ and sends it to the prover.
2. The prover P builds Merkle tree $MT_{h,n}(X)$ using h received from V. It then sends the root value v and the number of leaves s to V.
3. V samples uniformly at random u distinct numbers $(p_1, \ldots, p_u) \in [s]^u$, indicating the leaves it wants to verify. V sends these values to S.
4. The prover replies with these leaves specified by $(p_1, \ldots, p_u)$ and with a sibling path for each one of them, and the verifier accepts if all these sibling paths are valid.

The soundness of Protocol 8.1.1 is captured by the following lemma, which basically says that any PPT prover P^* that manages to convince the verifier with good probability must know (in the sense of *argument of knowledge*) the preimage string.

Lemma 8.1.2: Merkle Tree Extraction [HHPS11]

There exists a black-box extractor K with oracle access to a Merkle-tree prover that has the following properties:

- For every prover P and $v \in \{0,1\}^*$, $s, u \in \mathbb{N}$, and $\delta \in [0,1]$, $K^P(v, s, u, \delta)$ makes at most $u^2 s(\log(s) + 1)/\delta$ calls to its prover oracle P;
- Fix any hash function h and input string X with s leaves of n-bits each, and let v be the root value of $MT_{h,n}(X)$. Also fix some $u \in \mathbb{N}$ and a prover's remaining strategy $P^* = P^*(h, X, u)$ for Step 3 and Step 4 (that may depend on h, X and u). Then if P^* has probability at least $(1-\alpha)^u + \delta$ of convincing the verifier in the Merkle-tree protocol $MTP_h(v, s, u)$ (for some $\alpha, \delta \in (0,1]$), then with probability at least $1/4$ (over its internal randomness) the extractor $K^{P^*}(v, s, u, \delta)$ outputs values for at least a $(1-\alpha)$-fraction of the leaves of the tree, together with valid sibling paths for all these leaves.

Note that the proof of Lemma 8.1.2 does not rely on collision resistance of the hash function, it is merely a information-theoretical result. But it is usually used in conjunction with the collision-resistance property of hash functions to establish cryptographic results such as computational soundness or argument of knowledge property.

8.2 Universal One-way Hash Family

Another useful cryptographic (thus based on hardness assumptions) hashing is the *universal one-way hashing*. It was proposed in [NY89]. Xiao: More discussion and applications can be found there.. Roughly speaking, the definition starts with Adv picking a input x_1 before it learns the function. Then we sample a function from the family and give it to Adv. The goal of Adv is to find a second input x_2 which shares the same image as that of x_1, under the sampled hash function. Xiao!

Definition 8.2.1 (Universal One-Way Hash Family)**.** Xiao: UOWHF Xiao!

◊

8.3 Universal Hash Family

This notion of universal hashing, which bounds the collision probability of a hash function in a statistical sense, dates back to [CW79, WC81].

Definition 8.3.1: Universal Hash Family

A family $\mathcal{H} = \{h_k\}_k$ of hash functions from domain $\mathcal{D}$ to range $\mathcal{R}$ is *universal* if $\forall x_1 \neq x_2 \in \mathcal{D}$,

$$\Pr[h_k \xleftarrow{\$} \mathcal{H} : h_k(x_1) = h_k(x_2)] \leq \frac{1}{|\mathcal{R}|} \tag{8.1}$$

Such families exist if $|\mathcal{R}|$ is a power of two (see [CW79]). Moreover, there exist universal hash families which take strings of length n as input and contain $2^{O(n)}$ hash functions; therefore it takes $O(n)$ bits to specify a hash function from such a family ([WC81]). Thus, when discussing communication of hash functions, we can assume that both the sender and the recipient are aware of the family from which a hash function has been chosen, and that the transmitted data consists of $O(n)$ bits used to specify the hash function from the known family.

A simple example of universal hash family from $\mathcal{D} = \{0,1\}^k$ to $\mathcal{R} = \{0,1\}^n$ is

$$h_A(x) = A \cdot x$$

where $A \xleftarrow{\$} \{0,1\}^{n \cdot k}$ is interpreted as a $n \times k$ matrix and x is interpreted as a $k \times 1$ vector. The calculations are done modulo 2.

8.4 Pair-wise Independent Hash Family

Definition 8.4.1 (Pair-wise Independent Hash Family)**.** A family of hash functions $\mathcal{H}$ is *pairwise independent* if $\forall x_1 \neq x_2 \in \mathcal{D}$ and $\forall y_1, y_2 \in \mathcal{R}$,

$$\Pr[h_k \xleftarrow{\$} \mathcal{H} : h_k(x_1) = y_1 \wedge h_k(x_2) = y_2] = \frac{1}{|\mathcal{R}|^2} \tag{8.2}$$

◊

Note that in equation (8.1) for universal hash family, the probability is bounded by "≤". But in the equation (8.2), the symbol is "=". Actually, some authors also use "≤" when defining pair-wise hash family. It does not matter that much since, in applications, it usually suffices the purpose once the collision probability is $\frac{1}{|\mathcal{R}|}$. I guess the tradition of using "=" is the following: the concept of pair-wise independent hashing is analogous to the concept of independence in probability theory, i.e. $\Pr[A \wedge B] = \Pr[A] \cdot \Pr[B]$, where "=" symbol is used.

Xiao: I haven't checked whether there exist an construction that achieves probability strictly smaller than $\frac{1}{|\mathcal{R}|}$ Xiao!

Xiao: Give an exmple of pair-wise independent hashing. e.g. $h_{a,b}(x) = ax + b$ Xiao!

Pair-wise independence can be generalized to the following concept of *k-wise independence*.

Definition 8.4.1: k-wise Independent Hash Families

A family of hash functions $\mathcal{H} = \{h_i\}_i$ is *k-wise independent* if $\forall x_1 \neq \ldots \neq x_k \in \mathcal{D}$ and $\forall y_1, \ldots, y_k \in \mathcal{R}$, it holds that

$$\Pr[h_k \xleftarrow{\$} \mathcal{H} : h_i(x_1) = y_1 \wedge \ldots \wedge h_i(x_k) = y_k] = \frac{1}{|\mathcal{R}|^k} \tag{8.3}$$

Here are some obvious facts about k-wise independent hash family

Fact 8.4.1. Suppose $\mathcal{H}$ is a k-wise independent hash family for $k \geq 2$. Then

1. $\mathcal{H}$ is also $(k-1)$-wise independent.
2. For any $x \in \mathcal{D}$ and $y \in \mathcal{R}$, $\Pr[h \xleftarrow{\$} \mathcal{H} : h_i(x) = y] = \frac{1}{|\mathcal{R}|}$.
3. $\mathcal{H}$ is universal.

Xiao: Mention that [WC81] gives q-wise independent hash function for any q. Xiao!

Remark 8.4.1 (On the ambiguous usage of "2-universal")**.** *Usually, k-wise independent hash family is also called "k-universal" hash family [WC81], and the one given in Definition 8.3.1 is called "universal". But there are a few authors referring to Definition 8.3.1 as "2-universal", namely "universal" and "2-universal" are simply different names for the same property to them. To make the situation even more confusing, some researchers refer to Definition 8.3.1 as "weakly 2-universal" and they refer to "pair-wise independent" as "strongly 2-universal". And when they say "2-universal", they by default mean "weakly 2-universal", i.e. Definition 8.3.1. One of such authors is Vadhan [Vad12].*

8.5 Bloom Filter

Xiao: discuss Bloom filter here Xiao!

Chapter 9

Pseudorandomness

9.1 Leftover Hash Lemma

In this section, we play with one of the most important lemma – Leftover Hash Lemma (LHL). Introduced first in [ILL89], it has since found numerous applications in the realms of complexity theory/quantum computing/(randomized) algorithm/information theory/cryptography. To give a few examples from cryptography, LHL was used to build Leakage-Resilient Encryption [HLWW13], Deterministic Encryption [BFO08], Fully Homomorphic Encryption [Gen09] and Program Obfuscation [BLMZ18] etc.

Roughly, LHL says that a universal hash function constitutes a good randomness extractor, "smoothing out" an input distribution to nearly uniform on its range, provided that the former has sufficient min-entropy. LHL can be generalized to the average conditional min-entropy setting [DORS03].

Definition 9.1.1 (Statistical Distance)**.** Let X and Y be two random variables with range U. Then the statistical distance between X and Y is defined as

$$\Delta(X,Y) = \frac{1}{2}\sum_{u\in U}\big|\Pr[X=u]-\Pr[Y=u]\big| \tag{9.1}$$

For $\varepsilon \geq 0$, we also define the notion of two distributions being ε-close:

$$X \approx_{\varepsilon} Y \Leftrightarrow \Delta(X,Y) \leq \varepsilon.$$

$\Diamond$

Definition 9.1.2 (Min-Entropy)**.** The min-entropy $\mathsf{H}_\infty(X)$ of a random variable X is defined as

$$\mathsf{H}_\infty(X) = -\log\Big(\max_x\big\{\Pr[X=x]\big\}\Big) = \min_x\Big\{-\log\big(\Pr[X=x]\big)\Big\}. \tag{9.2}$$

If $\mathsf{H}_\infty(X) \geq k$, we call X a k-source.

$\Diamond$

Lemma 9.1.1 (Leftover Hash Lemma [ILL89])**.** Let $\mathcal{H} = \{h_i\}_{i\in\mathcal{I}}$ be a universal hash family[1] $\mathcal{I}\times\mathcal{D}\to\mathcal{R}$ with $|\mathcal{D}| = 2^n$ $|\mathcal{R}| = 2^\ell$ for some $n,\ell > 0$. Let $\mathsf{Ext}(x,i) = h_i(x)$. For any random variable X on support $\mathcal{D}$, the following holds:

$$\Delta\Big(\big(\mathsf{Ext}(X,U_\mathcal{I}),U_\mathcal{I}\big),\big(U_\mathcal{R},U_\mathcal{I}\big)\Big) \leq 2^{-\left(\frac{\mathsf{H}_\infty(X)-\ell}{2}+1\right)} \tag{9.3}$$

[1] For the definition of universal hash family, refer to Def. 8.3.1.

or equivalently (but easier to interpret),

$$\ell \leq \mathsf{H}_\infty(X) - c \quad \Rightarrow \quad \Delta\Big(\big(\mathsf{Ext}(X, U_\mathcal{I}), U_\mathcal{I}\big), \big(U_\mathcal{R}, U_\mathcal{I}\big)\Big) \leq 2^{-(\frac{c}{2}+1)} \tag{9.4}$$

where $U_\mathcal{I}$ and $U_\mathcal{R}$ are uniform distributions on $\mathcal{I}$ and $\mathcal{R}$ respectively.

In particular, to achieve statistical distance ε, we need to set

$$\ell \leq \mathsf{H}_\infty(X) - 2\log\big(\frac{1}{\varepsilon}\big) + 2$$

◊

Proof. A simple but elegant proof is given in Reyzin's lecture notes. ■

Remark 9.1.1. *It seems different people formalize LHL in different way. Reyzin's version (link) assumes universal hashing, but Rubinfiel's version (link) assumes 2-universal hashing. Also, [PW08] also assumes pairwise independent hash function. Xiao: Is it true that if we assume pairwise indenpedent hash function, then we will only need $\ell \leq \mathsf{H}_\infty(x) - 2\log\big(\frac{1}{\varepsilon}\big)$?* Xiao!

How to Explain LHL to a Kid. Typically, we use hash functions for compression. Smaller range of a hash family (thus more compression achieved) means more information loss on the input. LHL takes advantage of this property to build *randomness extractors* (see Section 9.2) from universal hash families in the following way: even if the input distribution has low entropy, by hashing it with a *uniformly chosen* member from a universal hash family with proper compression rate, we can always "smooth" it to an (almost) uniform output. Parametrically, for an input with min-entropy k, if we compress it to c bits shorter than k, i.e. the output has length $m = k - c$, the joint distribution of the output and the hash key will $(\frac{1}{2})^{\frac{c}{2}+1}$-close to uniform distribution. Thus, the statical distance is exponentially small on the amount compressed below the min-entropy.

Xiao: Also, talk about the average-min entropy and the extended LHL. Check [BFO08], Reyzin's lecture notes and Yu Yu's lecture notes. Xiao!

Definition 9.1.3 (Conditional Min-Entropy and Average Min-Entropy)**.** Let A, B be random variables. The conditional min-entropy $\mathsf{H}_\infty(A \mid B = b)$ is defined as

$$\mathsf{H}_\infty(A \mid B = b) = -\log\Big(\max_a \Big\{\Pr[A = a \mid B = b]\Big\}\Big) = \min_a \Big\{-\log\Big(\Pr[X = a \mid B = b]\Big)\Big\}. \tag{9.5}$$

The average min-entropy $\widetilde{\mathsf{H}}_\infty(A \mid B)$ is defined as

$$\widetilde{\mathsf{H}}_\infty(A \mid B) = -\log\Big(\mathbb{E}_B\Big[\max_a\{\Pr[A = a \mid B]\}\Big]\Big) = -\log\Big(\mathbb{E}_B\Big[2^{-\mathsf{H}_\infty(A \mid B=b)}\Big]\Big). \tag{9.6}$$

◊

The following is a very important lemma that characterize the relations among min-entropy, conditional min-entropy and average min-entropy.

Lemma 9.1.2 (Relations among Entropies [DORS03, DRS04])**.** Let A, B, C be random variables. Then

(a) For any $\delta > 0$, the following holds with probability (over the choice of b) at least $(1-\delta)$:

$$\mathsf{H}_\infty(A \mid B = b) \geq \widetilde{\mathsf{H}}_\infty(A \mid B) - \log\left(\frac{1}{\delta}\right) \tag{9.7}$$

(b) If B has at most 2^λ possible values, then

$$\widetilde{\mathsf{H}}_\infty\big(A \mid (B, C)\big) \geq \widetilde{\mathsf{H}}_\infty\big((A, B) \mid C\big) - \lambda \geq \widetilde{\mathsf{H}}_\infty(A \mid C) - \lambda. \tag{9.8}$$

In particular,

$$\widetilde{\mathsf{H}}_\infty(A|B) \geq \mathsf{H}_\infty\big((A, B)\big) - \lambda \geq \mathsf{H}_\infty(A) - \lambda. \tag{9.9}$$

$\Diamond$

Lemma 9.1.3 (Generalized LHL [DORS03, DRS04])**.** Let $\mathcal{H} = \{h_i\}_{i \in \mathcal{I}}$ be a universal hash family[2] $\mathcal{I} \times \mathcal{D} \to \mathcal{R}$ with $|\mathcal{D}| = 2^n$ $|\mathcal{R}| = 2^\ell$ for some $n, \ell > 0$. Let $\mathsf{Ext}(x, i) = h_i(x)$. For any random variable X on support $\mathcal{D}$ and Y, the following holds:

$$\Delta\Big(\big(\mathsf{Ext}(X, U_\mathcal{I}), Y, U_\mathcal{I}\big), \big(U_\mathcal{R}, Y, U_\mathcal{I}\big)\Big) \leq 2^{-\left(\frac{\widetilde{\mathsf{H}}_\infty(X \mid Y) - \ell}{2} + 1\right)} \tag{9.10}$$

or equivalently (but easier to interpret),

$$\ell \leq \widetilde{\mathsf{H}}_\infty(X \mid Y) - c \quad \Rightarrow \quad \Delta\Big(\big(\mathsf{Ext}(X, U_\mathcal{I}), Y, U_\mathcal{I}\big), \big(U_\mathcal{R}, Y, U_\mathcal{I}\big)\Big) \leq 2^{-(\frac{c}{2} + 1)} \tag{9.11}$$

where $U_\mathcal{I}$ and $U_\mathcal{R}$ are uniform distributions on $\mathcal{I}$ and $\mathcal{R}$ respectively.

In particular, to achieve statistical distance ε, we need to set

$$\ell \leq \widetilde{\mathsf{H}}_\infty(X \mid Y) - 2\log\big(\frac{1}{\varepsilon}\big) + 2$$

$\Diamond$

9.2 Randomness Extractors

Definition 9.2.1 (Randomness Extractor [NZ96])**.** Let the seed U_r be uniformly distributed on $\{0,1\}^r$. We say that a function $\mathsf{Ext} : \{0,1\}^n \times \{0,1\}^r \to \{0,1\}^\ell$ is a $(n, m, \ell, \varepsilon)$-strong extractor if, for all random variable X on $\{0,1\}^n$ with $\mathsf{H}_\infty(X) \geq m$, the following holds:

$$\Delta\Big(\big(\mathsf{Ext}(X, U_r), U_r\big), \big(U_\ell, U_r\big)\Big) \leq \varepsilon$$

$\Diamond$

Remark 9.2.1 (Strong vs. Standard Extractor)**.** *Note that the extractor defined here is called* strong *extractor. The "standard" extractor only requires that* $\mathsf{Ext}(X, U_r)$ *is close to uniform. The above version is called "strong" as it additionally requires the* U_d *part to be public. Usually, the strong version here is more widely used in cryptography.*

[2] For the definition of universal hash family, refer to Definition 8.3.1.

Definition 9.2.2 (Average-Case Extractor [DORS03, DRS04])**.** Let the seed U_r be uniformly distributed on $\{0,1\}^r$. We say that a function $\mathsf{Ext} : \{0,1\}^n \times \{0,1\}^r \to \{0,1\}^\ell$ is an average-case $(n, m, \ell, \varepsilon)$-strong extractor if, for all pairs of random variables (X, Y) such that X has support $\{0,1\}^n$ and $\widetilde{\mathsf{H}}_\infty(X \mid Y) \geq m$, the following holds:

$$\Delta\Big(\big(\mathsf{Ext}(X, U_r), Y, U_r\big), \big(U_\ell, Y, U_r\big)\Big) \leq \varepsilon$$

$\Diamond$

Theorem 9.2.1 (Worst-Case to Average-Case Extractors [DORS03, DRS04])**.** For any $\delta > 0$, if Ext is a $(n, m - \log\big(\frac{1}{\delta}\big), \ell, \varepsilon)$-strong extractor, then Ext is also an average-case $(n, m, \ell, \varepsilon + \delta)$-strong extractor. $\Diamond$

Proof. The proof trivially follows from Lemma 9.1.2-(a). ∎

Remark 9.2.2 (Interpreting LHL in term of Extractors)**.** *By simple calculations on the parameters, one can interpret LHL in the following way:*

- *LHL says that universal hash families are $(n, m, \ell, \varepsilon)$-strong randomness extractors whenever $\ell \leq m - 2\log\left(\frac{1}{\varepsilon}\right) + 2$.*
- *Generalized LHL says that universal hash families are average-case $(n, m, \ell, \varepsilon)$-strong randomness extractors whenever $\ell \leq m - 2\log\left(\frac{1}{\varepsilon}\right) + 2$.*

9.3 Expander Graphs

Xiao: add expander graphs here Xiao!

Chapter 10

Lattices

Many parts of this Chapter is taken from the marvelous survey of Peikert [Pei15]. I only pick the basic and widely-used materials. For an advanced and complete discussion on this topic, refer to [Pei15].

10.1 Basic Concepts

Dual Lattices. Given a lattice $\mathcal{L}$, it is easy to see that the set of points whose inner products with the vectors in $\mathcal{L}$ are all integers constitutes a lattice. Such a lattice is called dual lattice of $\mathcal{L}$, usually denoted as $\mathcal{L}^*$.

Definition 10.1.1 (Dual Lattice)**.** The dual (sometimes called reciprocal) of a lattice $\mathcal{L} \subseteq \mathbb{R}^n$ is defined as:

$$\mathcal{L}^* = \{\mathbf{v} : \langle \mathbf{v}, \mathcal{L} \rangle \subseteq \mathbb{Z}\}$$

Moreover, if $\mathbf{B}$ is a basis of $\mathcal{L}$, then $\mathbf{B}^{-\mathrm{T}} = (\mathbf{B}^{-1})^{\mathrm{T}} = (\mathbf{B}^{\mathrm{T}})^{-1}$ is a basis of $\mathcal{L}^*$. $\Diamond$

For example, $(c\mathcal{L})^* = c^{-1}\mathcal{L}$.

Xiao: Define the n-th successive minima. Xiao!

10.2 The "Hard-Core" on Lattices

In this section, we define several hard problems on lattices and briefly survey the known results for their complexity. We remark that these problems are not related to crypto applications directly, roughly because they do not have "nice" algebraic structures that can be employed by crypto. However, they are important because they provide hardness. In Section 10.3, we will introduce lattice problems that are directly related to crypto; the hardness of those problems are established by reductions to the problems in this section.

10.2.1 The Shortest Vector Problem

The most basic and important problem on lattices is the shortest Vector Problem (SVP). This problem has been here since the 18th century, attracting attentions from famous mathematicians including Gauss and Minkovski.

Definition 10.2.1 (Shortest Vector Problem)**.** Given an arbitrary basis $\mathbf{B}$ of some lattice $\mathcal{L} = \mathcal{L}(\mathbf{B})$, find a shortest nonzero lattice vector, i.e., a $\mathbf{v} \in \mathcal{L}$ for which $\|\mathbf{v}\| = \lambda_1(\mathcal{L})$. $\Diamond$

This question has been open for hundreds of years. But until today, we still do not have a solution. One important result for this question is the following theorem given by Minkovski, which upper-bounds the solution.

Theorem 10.2.1 (Minkowski's First Theorem)**.** For any lattice $\mathcal{L}$, we have $\lambda_1(\mathcal{L}) \leq \sqrt{n} \cdot \det(\mathcal{L})^{1/n}$. $\Diamond$

There are several other theorems of this kind, e.g. Hermite's Theorem, Gauss Heuristic. See [HPSS08] for more discussion.

Although the SVP problem is very fascinating, more closely related to modern cryptography is the approximate version of SVP (and also some other problems on lattices of similar flavor). We now summarize them in the following.

Definition 10.2.2 (Approximate SVP Problem)**.** For lattice dimension parameter n, $\mathsf{gapSVP}_{\gamma(n)}$ is a promise (decisional) problem. On input $(\mathcal{L}, d)$, where $\mathcal{L}$ is a n-dimensional lattice and d is real number, output:

- YES: if $\lambda_1(\mathcal{L}) \leq d$,
- NO: if $\lambda_1(\mathcal{L}) > \gamma(n) \cdot d$

$\Diamond$

Definition 10.2.3 (Approximate Shortest Independent Vector (SIVP) Problem)**.** Given a basis $\mathbf{B}$ of a full-rank n-dimensional lattice $\mathcal{L} = \mathcal{L}(\mathbf{B})$ (i.e. $\mathbf{B}$ is a $n \times n$ full-rank matrix), output a set $S = \{s_i\} \subset \mathcal{L}$ of n linearly independent lattice vectors where $\|s_i\| \leq \gamma(n) \cdot \lambda_n(L)$ for all i. $\Diamond$

Definition 10.2.4 (Bounded-Distance Decoding)**.** Given a basis $\mathbf{B}$ of an n-dimensional lattice $\mathcal{L} = \mathcal{L}(\mathbf{B})$ and a target point $t \in \mathbb{R}^n$ with the guarantee that $\mathsf{dist}(t, \mathcal{L}) < d \coloneqq \lambda_1(\mathcal{L})/(2\gamma(n))$, the bounded-distance decoding problem BDD_γ is to find the unique lattice vector $v \in \mathcal{L}$ such that $\|t - \mathbf{v}\|_< d$. $\Diamond$

Algorithms and complexity.[1] The above lattice problems have been intensively studied and appear to be intractable, except for very large approximation factors. Known polynomial-time algorithms like the one of Lenstra, Lenstra, and Lovász [LLL82] and its descendants (e.g., [Sch87] with [AKS01] as a subroutine) obtain only slightly sub-exponential approximation factors $\gamma = 2^{\Theta(n \log\log n/\log n)}$ for all the above problems. Known algorithms that obtain polynomial $\mathsf{poly}(n)$ or better approximation factors, such as [Kan83, AKS01, MV10, ADRS15], either require super-exponential $2^{\Theta(n \log n)}$ time, or exponential $2^{\Theta(n)}$ time *and* space. There are also time-approximation tradeoffs that interpolate between these two classes of results, to obtain γ approximation factors in $2^{\widetilde{\Theta}(n/\log\gamma)}$ time [Sch87]. Importantly, the above also represents the state of the art for quantum algorithms, though in some cases the hidden constant factors in the exponents are somewhat smaller (see, e.g. [**?**]). By contrast, the integer factorization and discrete logarithm problem (in essentially any group) can be solved in polynomial time using Shor's quantum algorithm [Sho99].

On the complexity side, many lattice problems are known to be NP-hard (sometimes under randomized reductions), even to approximate to within various sub-polynomial $n^{o(1)}$ approximation factors. E.g., for the hardness of SVP, see [Ajt98, Mic98, Kho04, HR07]. However, such hardness is not of any direct consequence to cryptography, since lattice-based cryptographic constructions so far rely on polynomial approximation problems factors $\gamma(n) \geq n$. Indeed, there is evidence that for factors $\gamma(n) \geq \sqrt{n}$, the lattice problems relevant to cryptography are not NP-hard, because they lie in $\mathsf{NP} \cap \mathsf{coNP}$ [GG98, AR04].

[1] This part is taken verbatim from [Pei15]

Summary for the Hardness of SVP: In summary, $\mathsf{gapSVP}_{\gamma(n)}$ gets progressively easier as $\gamma(n)$ gets larger and larger. The complexity of $\mathsf{gapSVP}_{\gamma(n)}$ in terms of $\gamma(n)$ is as follows:

- Between 1 and $\mathcal{O}(1)$: NP-hard (based on a randomized reduction).
- Between $\mathcal{O}(1)$ and $2^{(\log n)^{1-\varepsilon}}$: Quasi-NP-hard (based on a randomized reduction).
- $\sqrt{n/\log n}$: Both NP and coAM (based on a quasi-polynomial-time randomized reduction).
- $\sqrt{n}$: Both NP and coNP (which is the same class as problems such as factoring).
- $\mathcal{O}(n)$: Here we get an implication of one-way functions. There is a gap at which point we get PKE and FHE, etc.
- $2^{n \log\log n/\log n}$: BPP (polynomial-time).

Moreover, known $\mathsf{gapSVP}_{\gamma(n)}$ algorithms run in nearly exponential time in terms of lattice dimension.

10.3 Crypto-Friendly Lattice Problems

10.3.1 Short Integer Solution (SIS)

Short Integer Solution. The short integer solution (SIS) problem was first introduced in the seminal work of Ajtai [Ajt96], and has served as the foundation for one-way and collision-resistant hash functions, identification schemes, digital signatures, and other "minicrypt" primitives (but not public-key encryption).

Definition 10.3.1 (Short Integer Solution)**.** For a lattice of dimension n, a number of samples m, a modulus q, and a norm bound β, the *Short Integer Solution* problem $\mathsf{SIS}_{n,m,q,\beta,}$ is to first sample $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n\times m}$ and then ask for a nonzero integer vector $\mathbf{z} \in \mathbb{Z}^m$ such that

$$\mathbf{A}\mathbf{z} = \mathbf{0} \in \mathbb{Z}_q^n \quad \text{and} \quad \|\mathbf{z}\| \leq \beta.$$

$\Diamond$

Theorem 10.3.1 (Hardness of SIS)**.** For any $m = \mathsf{poly}(n)$, any $\beta > 0$, and any sufficiently large $\beta \leq q/\mathsf{poly}(n)$, solving $\mathsf{SIS}_{n,q,\beta,m}$ with non-negligible probability is at least as hard as solving gapSVP_γ and SIVP_γ on arbitrary n-dimensional lattices (i.e., in the worst case) with overwhelming probability, for some $\gamma = \beta \cdot \mathsf{poly}(n)$. $\Diamond$

Here are some remarks on the harness parameters:

- For $\beta \geq q$, the SIS problem is easy: simply setting $\mathbf{z} = (q, 0, \ldots, 0)^{\mathrm{T}}$ gives us $\mathbf{A}\mathbf{z} = \mathbf{0} \bmod q$.
- β and m have to be large enough to guarantee the existence of a solution. This is the case whenever $\beta \geq \sqrt{n \log q}$ and $m \geq n \log q$. This is because of the following pigeonhole argument: first, we can assume without loss of generality that $m = n \log q$.[2] Then because there are

[2] This is because once we can solve SIS for $\mathbf{A}_{n\times m}$, we can easily extend the solution when more rows are appended at the end of $\mathbf{A}$: simply append 0's at the end of the solution vector.

more than q^n vectors $x \in \{0,1\}^m$, there must be two distinct x, x' such that $\mathbf{A}\mathbf{x} = \mathbf{A}\mathbf{x}' \in \mathbb{Z}_q^n$, so their difference $z = x - x' \in \{0, 1, -1\}^m$ is a solution with $\|\mathbf{z}\|_{\leq} \beta$ for $m = n \log q$ and $\beta \geq \sqrt{n \log q}$.

- After a line of work[MR04, GPV08], the state-of-the-art value for the hardness parameters are $\gamma = \beta \cdot \widetilde{O}(\sqrt{n})$ and $\beta \leq q/\widetilde{O}(\sqrt{n})$.
- [MP13] achieve $\beta \leq q/n^{\varepsilon}$ for any constant $\varepsilon > 0$. But the γ is somewhat subtle: it can depend on the norm of the SIS solution in the ℓ_∞ norm.

10.3.2 Learning with Error (LWE)

Xiao: [BCM+18, Section 2.3] also contains a clean summarization of the LWE assumption. Xiao!

The learning with errors (LWE) problem was defined by Regev [Reg05].

Definition 10.3.2 (Decisional LWE Problem [Reg05])**.** The $\mathsf{LWE}_{n,q,\chi,m}$ problem is two distinguish the following tow distributions:

$$(\mathbf{A}, \mathbf{s}^{\mathrm{T}}\mathbf{A} + \mathbf{e}^{\mathrm{T}} \bmod q) \text{ and } (\mathbf{A}, \mathbf{u}^{\mathrm{T}})$$

where $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n\times m}$, $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^{n\times 1}$, $\mathbf{e} \leftarrow \chi^{m\times 1}$ and $\mathbf{u} \xleftarrow{\$} \mathbb{Z}_q^{m}$. ♢

Different presentations of the hardness reduction of (average-case) LWE assumption to (worst-case) lattice problems exist in the literature. The one presented here (taken from [GSW13]) is probably the clearest one.

Definition 10.3.3 (B-Bounded Dsitributions)**.** A distribution ensemble $\{\chi_n\}_{n\in\mathbb{N}}$, supported over the integers, is called B-bounded if

$$\Pr_{e \leftarrow \chi_n}[|e| > B] \leq \mathsf{negl}(n)$$

♢

The following theorem shows the reduction from the LWE problem to the GapSVP problem, which is critical for all LWE-based cryptosystem. This idea is originated from [Reg05], and refined in [Pei09, MM11, MP12]. The version presented here is stated as Corollary 2.1 from [Bra12].

Theorem 10.3.2 (Hardness of LWE)**.** Let $q = q(n) \in \mathbb{N}$ be either a prime power or a product of small (size $\mathsf{poly}(n)$) distinct primes, and let $B \geq \omega(\log n) \cdot n$. Then there exists an efficient sampleable B-bounded distribution χ such that if there is an efficient algorithm that solves the average-case LWE problem for parameters n, q, χ, then:

- There is an efficient *quantum* algorithm that solves $\mathsf{gapSVP}_{\widetilde{O}(nq/B)}$ on any n-dimensional lattice.
- If $q \geq \widetilde{O}(2^{n/2})$, then there is an efficient *classical* algorithm for $\mathsf{gapSVP}_{\widetilde{O}(nq/B)}$ on any n-dimensional lattice.

In both cases, if one also considers distinguishers with sub-polynomial advantage, then we require $B \geq \widetilde{O}(n)$ and the resulting approximation factor is slightly larger than $\widetilde{O}(n^{1.5}q/B)$. ♢

Modulus-to-Noise Ratio. The value q/B usually arouses concerns regarding the efficiency of constructions , so people refer to it as "modulus-to-noise ratio".

Discrete Gaussian Distribution. The most widely-used error distribution to construct hard LWE problem is the discrete version of Gaussian distribution. It is the distribution over $\mathbb{Z}$ where the probability of x is proportional[3] to $e^{-\pi(|x|/\sigma)^2}$, where σ is the width parameter. The hardness of LWE w.r.t. discrete Gaussian distribution is stated as the following theorem. A discrete Gaussian with parameter σ is $B = \sigma$ bounded, except with negligible probability.

Theorem 10.3.3 (Hardness of LWE w.r.t. Discrete Gaussian [Reg05])**.** For any $m = \mathsf{poly}(n)$, any modulus $q \leq 2^{\mathsf{poly}(n)}$, and any (discretized) Gaussian error distribution χ of parameter $\sigma = \alpha \cdot q \geq 2\sqrt{n}$ where $0 < \alpha < 1$, solving the decisional $\mathsf{LWE}_{n,q,\chi,m}$ problem is at least as hard as quantumly solving $\mathsf{gapSVP}_{\widetilde{O}(n/\alpha)}$ and $\mathsf{SIVP}_{\widetilde{O}(n/\alpha)}$ on arbitrary n-dimensional lattices. ◊

Note that the exact values of m (the number of samples) and q (the modulus) play essentially no role in the ultimate hardness guarantee (apart from the lower bound for $q \geq 2\sqrt{n}/\alpha$). However, the approximation factor $\gamma = \widetilde{O}(n/\alpha)$ degrades with the modulus-to-noise ration $\sigma/q = 1/\alpha$. For gapSVP_γ and SIVP_γ, the best known (classical or quantum) algorithms for these problems run in time $2^{\widetilde{O}(n/\log \gamma)}$, and in particular they are conjectured to be intractable for $\gamma = \mathsf{poly}(n)$.

10.3.3 Learning with Rounding

LWE problem is inherently randomized. But there are some crypto primitives (e.g. PRF) that prefers deterministic hardness assumption. To address this issue, [BPR12] proposed a deterministic version of LWE, called "learning with rounding" (LWR), and showed how to reduce it to LWE. LWR is used to build efficient PRF based on hard lattice problems, and plays a important role in other applications such as watermarking [KW17, KW19] and trapdoor hash functions [DGI+19].

Definition 10.3.4 (Rounding Function)**.** For integers $p \geq q \geq 2$, the rounding function $\lfloor \cdot \rceil_p : \mathbb{Z}_q \to \mathbb{Z}_p$ is defined as:

$$\lfloor x \rceil_p = \left\lfloor \frac{(x \bmod q)}{q} \cdot p \right\rceil \bmod p$$

This notion extends to vectors and matrices component-wisely. ◊

Definition 10.3.5 (Learning with Rounding)**.** For a distribution D_s on $\mathbb{Z}_q^{n\times 1}$, the learning with rounding problem $\mathsf{LWR}_{n,q,p,m}^{D_s}$ problem is two distinguish between the following tow distributions:

$$(\mathbf{A}, \lfloor \mathbf{s}^{\mathrm{T}}\mathbf{A} \rceil_p) \text{ and } (\mathbf{A}, \mathbf{u}^{\mathrm{T}})$$

where $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n\times m}$, $\mathbf{s} \xleftarrow{D_s} \mathbb{Z}_q^{n\times 1}$ and $u \xleftarrow{\$} \mathbb{Z}_p^{m\times 1}$ ◊

Definition 10.3.6 (Hardness of LWR)**.** Let χ be any efficiently sampleable B-bounded distribution over $\mathbb{Z}$, and let $p \leq \frac{q}{B\cdot n^{\omega(1)}}$. Then for any distribution D_s on $\mathbb{Z}_q^n$, solving decision $\mathsf{LWR}_{n,q,p,m}^{D_s}$ is at least as hard as solving decision $\mathsf{LWE}_{n,q,\chi,m}^{D_s}$, i.e. the $\mathsf{LWE}_{n,q,\chi,m}$ problem where the secret vector $\mathbf{s}$ comes from the same distribution D_s. ◊

[3] "Proportional" means that one needs to normalize the value such that the probability for each $x \in \mathbb{Z}$ sum up to 1.

10.4 Ring-SIS and Ring-LWE

Xiao: Xiao!

- A good starting point is:
  - the lecture notes made by Stephens-Davidowitz for Vaikuntanathan's course (lectures 5 and 6), **plus**
  - this Youtube video by Afred Menezes
- Also need to create a subsection for Module-SIS an Module-LWE

Remark regarding the idea lattice. Section 3.1 of Vaikuntanathan's course (lectures 5) mentioned a way to embed an idea $\mathcal{I}$ of the polynomial ring $\mathbb{Z}_q/\langle x^n+1\rangle$ into the lattice $\mathbb{Z}^n$. such an embedding will reveal that the ideal $\mathcal{I}$ is actually a strange lattice whose successive minima are all equal. This part is a little bit hard to understand. Let me explain:

- Work in the ring $R = \mathbb{Z}[x]/(x^n-1)$ (or a similar cyclic ring), where multiplication by x acts as a linear map on R. Embed R into $\mathbb{Z}^n$ via the trivial embedding that sends

  $$a_0 + a_1x + \cdots + a_{n-1}x^{n-1} \;\longmapsto\; (a_0, a_1, \ldots, a_{n-1}) \in \mathbb{Z}^n,$$

  and equip $\mathbb{Z}^n$ with the standard ℓ_2 norm and inner product.

  Under this embedding, "multiplication by x" corresponds to the $n \times n$ permutation matrix that cyclically shifts coordinates:

  $$x \cdot (a_0, a_1, \ldots, a_{n-1}) \;=\; (a_{n-1}, a_0, a_1, \ldots, a_{n-2}).$$

  Permutation matrices are orthogonal, hence they preserve the ℓ_2 norm and the standard inner product. Therefore, for any $y \in R$ (viewed as a vector in $\mathbb{Z}^n$),

  $$\|x \cdot y\|_2 \;=\; \|y\|_2 \quad \text{and more generally} \quad \|x^k \cdot y\|_2 \;=\; \|y\|_2 \quad \text{for all } k = 0, 1, \ldots, n-1.$$

  Consequently, the vectors $y, xy, x^2y, \ldots, x^{n-1}y$ all have the same length.

10.4.1 Ring-SIS

Xiao: Old stuff. Need to remove. It is even overly inaccurate... Xiao!

In the standard SIS problem defined in previous section, the underlying sets sets are $\mathbb{Z}^n$ and $\mathbb{Z}_q^n$. Roughly speaking, Ring-SIS is the ring version of the standard SIS problem, i.e. the underlying sets are rings R and R_q (corresponding to $\mathbb{Z}^n$ and $\mathbb{Z}_q^n$ in the standard SIS setting, respectively). People care bout this ring version because it usually provides more efficient cryptographic constructions, due to the "richer" algebraic structure of the underlying rings. Of course, the analysis of harness assumptions requires more careful analysis, as the "richer" structures of rings admits more attacks than that for a standard SIS.

Definition 10.4.1 (Ring-SIS)**.** Let R be a ring, equipped with some norm $\|\cdot\|$. For a positive integer q, denote the quotient ring R/qR as R_q. For $\mathbf{a} \xleftarrow{\$} R_q^m$, the Ring-SIS problem $\mathsf{R\text{-}SIS}_{q,m,\beta}$ is to find a nonzero vector $\mathbf{z} \in R^m$ of norm $\|\mathbf{a}\| \leq \beta$ such that:

$$F_{\mathbf{a}}(\mathbf{z}) = \langle \mathbf{a}, \mathbf{z} \rangle = 0 \in R_q.$$

$\diamondsuit$

Remark 10.4.1 (Harness of Ring-SIS)**.** *The hardness of Ring-SIS depends on the choice of the underlying ring R and the norm $\|\cdot\|$. A typical choice is to set R to be the so-called* rank-n ring of convolution polynomials $\mathbb{Z}[x]/\langle x^n - 1 \rangle$*, in which case R_q will be $\mathbb{Z}_q[x]/\langle x^n - 1 \rangle$.*

When the ring is of the form $\mathbb{Z}[x]/\langle f(x) \rangle$ where $\mathsf{deg}(f) = n$, the preferred norm is the so-called canonical embedding $\sigma : \mathbb{Z}[x]/\langle f(x) \rangle \to \mathbb{C}^n$ *from algebraic number theory. This embedding maps each ring element $r \in R$ to the vector $(r(\alpha_1), \ldots, r(\alpha_n)) \in \mathbb{C}^n$, where the $\alpha_i \in \mathbb{C}$ are the n complex roots of $f(X)$.*

Such choice of the underlying ring and the norm has several advantages. As the reason is advanced and complicate, we do not provide further discussion. We refer the readers to Section 4.3 in [Pei15].

10.4.2 Ring-LWE

Xiao: old stuff. Need to remove... Xiao!

Just like SIS vs Ring-SIS, the LWE problem also has a ring version called Ring-LWE.

Definition 10.4.2 (Decisional Ring-LWE Problems)**.** The decisional Ring-LWE problem $\mathsf{R\text{-}LWE}_{q,\chi,m}$ is to distinguish the following two distributions:

$$(\mathbf{a}, s \cdot \mathbf{a} + \mathbf{e} \bmod q) \text{ and } (\mathbf{a}, \mathbf{u})$$

where $\mathbf{a} \xleftarrow{\$} R_q^m$, $s \xleftarrow{\$} R_q$, $\mathbf{u} \xleftarrow{\$} R_q^m$ and $\mathbf{e} \leftarrow \chi^m$. $\diamondsuit$

As that case of Ring-SIS, the hardness of Ring-LWE depends on the choice of the underlying ring and the error distribution. This was investigated in the work of [LPR10], where they pick a *cyclotomic* ring and a special error distribution[4]. We will not provide further discussion here. Next, we will only list the hardness reduction theorem from Ring-LWE to gapSVP. We refer the readers to [LPR10, LPR13, AP13] and Section 4.4 of [Pei15] for more details.

Theorem 10.4.1 (Hardness of Ring-LWE [LPR10])**.** For any $m = \mathsf{poly}(n)$, cyclotomic ring R of degree n (over $\mathbb{Z}$), and appropriate choices of modulus q and error distribution χ of error rate $\alpha < 1$, solving the $\mathsf{R\text{-}LWE}_{q,\chi,m}$ problem is at least as hard as quantumly solving the gapSVP_γ problem on arbitrary ideal lattices in R, for some $\gamma = \mathsf{poly}(n)/\alpha$. $\diamondsuit$

10.5 Two Critical Equations for Lattice-Based Crypto

The materials presented in this part is based on the excellent talks by Hoeteck Wee (link) and David Wu (link).

Xiao: Explain how to derive and apply the follow two equations Xiao!

$$\mathbf{C}_1, \ldots, \mathbf{C}_n \mapsto \mathbf{C}_{f(x)}$$
$$\big[\mathbf{C}_1 - x_1\mathbf{G} \mid \ \ldots \ \mid \mathbf{C}_n - x_n\mathbf{G}\big]\mathbf{H}_{f,x} = \mathbf{C}_f - f(x)\mathbf{G}$$

[4] Actually, [LPR10] uses a certain fractional ideal $R^\vee$ that is dual to R

10.6 The CKKS Fully Homomorphic Encryption Scheme

One major implication of LWE is fully homomorphic encryption (FHE), and the most practical construction of FHE is the Cheon-Kim-Kim-Song (CKKS) scheme [CKKS17].

CKKS encrypts scaled polynomial representations of complex vectors, supports homomorphic addition and multiplication, and maintains small approximation error—enabling secure real-number computation under encryption. It is good for the following tasks for example:

- Supports approximate arithmetic suitable for real-valued computations.
- Vectorized (SIMD) operations over many encrypted numbers.
- Post-quantum security due to RLWE assumption.
- Predictable, bounded precision loss.

But note that it is not intended for exact modular arithmetic (rounding error is inherent), and each multiplication consumes modulus levels and precision (and thus requiring parameter tuning and error budgeting).

We provide here a high-level overview of their elegant ideas.

10.6.1 The Construction

CKKS is based on the Ring Learning With Errors (RLWE) problem.

- Let N be a power of two (e.g., $N = 2^{15}$).
- Define the cyclotomic polynomial $\Phi_{2N}(x) = x^N + 1$.
- Define the ring:
$$R = \mathbb{Z}[x]/\langle\Phi_{2N}(x)\rangle, \quad R_q = R/qR.$$

An element of R_q is a polynomial

$$a(x) = a_0 + a_1 x + \cdots + a_{N-1}x^{N-1} \bmod q.$$

The security rests on the hardness of distinguishing RLWE samples

$$(a,\, a \cdot s + e)$$

from random, where s and e are small polynomials.

Key Generation:

$$\text{Sample } s \leftarrow \text{small}, \quad a \leftarrow R_q \text{ uniform}, \quad e \leftarrow \text{small}.$$

Define

$$b = -a \cdot s + e.$$

Then:

$$\text{Public key: } pk = (b, a), \quad \text{Secret key: } sk = s.$$

Encoding Real or Complex Numbers. CKKS encodes a vector of complex numbers into a polynomial through the *canonical embedding*

$$\mathbb{C}^{N/2} \leftrightarrow R \otimes \mathbb{C}.$$

Each "slot" holds one complex entry. To preserve fractional precision, plaintexts are multiplied by a scaling factor Δ (for example, 2^{40}) before encryption.

Check this blog for a detailed tutorial on CKKS encoding/decoding for plaintext.

Encryption. Given a plaintext polynomial $m \in R$, sample small noise terms u, e_1, e_2, and compute:

$$c_0 = b \cdot u + e_1 + \Delta \cdot m,$$
$$c_1 = a \cdot u + e_2.$$

The ciphertext is:

$$ct = (c_0, c_1).$$

Decryption

$$m' = \frac{c_0 + c_1 \cdot s}{\Delta} \bmod q.$$

Since $b = -a \cdot s + e$, the terms involving a cancel and one obtains:

$$c_0 + c_1 \cdot s = \Delta \cdot m + \text{small noise}.$$

After division by Δ, the recovered value $\hat{m}$ satisfies

$$\hat{m} \approx m.$$

Homomorphic Operations Let ciphertexts $ct_1 = (c_0, c_1)$ and $ct_2 = (d_0, d_1)$ represent plaintexts m_1 and m_2.

- **Addition:**

  $$ct_{\text{add}} = (c_0 + d_0,\ c_1 + d_1),$$

  which decrypts to $m_1 + m_2$.

- **Multiplication:**

  $$ct_{\text{mul}} = (c_0 d_0,\ c_0 d_1 + c_1 d_0,\ c_1 d_1).$$

  This produces three components. A relinearization step (key switching) reduces it back to two components, and then a **rescaling** operation divides by the scale Δ to control precision:

  $$\text{Rescale}(ct_{\text{mul}}) \Rightarrow \text{approx. } m_1 \cdot m_2.$$

We provide a detailed explanation in Section 10.6.2.

Modulus Chain and Rescaling CKKS uses a descending chain of moduli

$$q_L > q_{L-1} > \cdots > q_0.$$

Operation	Mathematical effect	Purpose
KeyGen	Sample (s, a, e), set $b = -as + e$	RLWE hardness basis
Encode	Vector $\to$ polynomial $\times\Delta$	Represent reals/complex
Encrypt	$(c_0, c_1) = (bu + e_1 + \Delta m,\ au + e_2)$	Confidentiality
Decrypt	$(c_0 + c_1 s)/\Delta$	Approximate plaintext
Add	Component-wise addition	Homomorphic addition
Multiply	Poly. mult. + relinearize + rescale	Homomorphic multiplication
Rotate	Apply Galois automorphism	Permute SIMD slots

Table 10.1: Summary for CKKS FHE

After each multiplication, the ciphertext modulus drops to the next level, and coefficients are divided by Δ. This keeps the ciphertext size fixed and stabilizes precision.

Rotation and Conjugation CKKS also supports “slot rotations” and complex conjugation using existing Galois automorphisms of the cyclotomic ring. Additional *Galois keys* allow homomorphic rotation or permutation of encrypted vectors (used for SIMD-style computation).

Summary. We summarized the structure of the construction in Table 10.1.

10.6.2 Relinearization, Rescaling, Modulus switching, and Key switching,

In lattice-based fully homomorphic encryption schemes such as CKKS, ciphertexts are polynomials over the ring

$$R_q = \mathbb{Z}_q[x]/\langle x^N + 1\rangle.$$

Each ciphertext typically has two or more components. As we apply homomorphic operations, both the *size* of the ciphertext (number of components) and its *noise magnitude* increase. The four procedures discussed here manage this complexity and maintain decryption correctness.

Relinearization $\to$ reduce ciphertext size after multiplication.
Key switching $\to$ change the encryption key used for a ciphertext.
Rescaling $\to$ control the numerical scale and noise growth.
Modulus switching $\to$ lower the modulus to keep noise proportionally small.

10.6.2.1 Relinearization

When two ciphertexts

$$ct_1 = (c_0, c_1), \quad ct_2 = (d_0, d_1)$$

are multiplied, the ciphertext result has three components:

$$ct_{\text{mult}} = (c_0 d_0,\ c_0 d_1 + c_1 d_0,\ c_1 d_1).$$

However, the original decryption formula

$$m = c_0 + c_1 s$$

expects only two components. The product now decrypts using

$$c_0d_0 + (c_0d_1 + c_1d_0)s + c_1d_1s^2,$$

which includes a term involving s^2. To continue future operations efficiently, we must *remove* the dependence on s^2 and express everything again in the standard two-component form.

Main Idea. We precompute a ciphertext which encrypts s^2 *under the same key*:

$$ek^{(2)} = (b^{(2)}, a^{(2)}) = (-a^{(2)}s + e^{(2)} + s^2,\ a^{(2)}).$$

Then, when $c_1d_1s^2$ appears, we replace s^2 using this encryption:

$$s^2 \approx b^{(2)} + a^{(2)}s.$$

Substituting and combining terms gives new coefficients

$$(c_0', c_1') = (c_0d_0 + c_1d_1b^{(2)},\ c_0d_1 + c_1d_0 + c_1d_1a^{(2)}),$$

which again has only two components and decrypts with standard key s. This transformation is called **relinearization**.

Important: The above is just a simplified explanation of the CKKS's idea of re-linearization. For an explanation that is more close the original CKKS, see this link.

10.6.3 Rescaling and Modulus Switching

CKKS represents approximate real numbers by multiplying them by a scaling factor Δ. After a homomorphic multiplication, two ciphertexts with scale Δ produce a new ciphertext with scale Δ^2. If we continue multiplying, the scale increases exponentially, making later divisions or comparisons meaningless.

Rescaling. After each multiplication, we **rescale** the ciphertext to bring its scale back to Δ. Let the ciphertext result have modulus q and scale Δ^2. We choose a smaller modulus $q' = q/\Delta$ and divide all coefficients of the ciphertext by Δ (rounding to nearest integer):

$$c_i' = \left\lfloor \frac{c_i}{\Delta} \right\rceil \bmod q'.$$

The effect on the encoded message:

$$m' = \frac{c_0' + c_1's}{\Delta} \approx m_1 \cdot m_2,$$

and the scale returns to approximately Δ. Note that rescaling reduces the available modulus size and therefore the remaining "computation levels."

Modulus Switching. If we suppose we must do L multiplications, with a scale Δ, then we will define q_L as $q_L = \Delta^L \cdot q_0$ with $q_0 \geq \Delta$, which will dictate how many bits we want before the decimal part. For each $\ell \in [L]$, we set $q_\ell = \Delta^\ell \cdot q_0$.

To "drop" to a smaller modulus $q_{i-1} < q_i$, we scale all ciphertext components proportionally:

$$c'_j = \left\lfloor \frac{q_{i-1}}{q_i} c_j \right\rceil \bmod q_{i-1}.$$

This preserves the plaintext ratio $(c_0 + c_1 s)/q_i$ up to rounding error but keeps noise proportionally bounded.

Effect: The ciphertext now lives in ring $R_{q_{i-1}}$, with smaller modulus and slightly higher noise, but equivalent underlying plaintext.

Relation to Rescaling. In schemes like BFV or BGV (for integer arithmetic), *modulus switching* is used directly to control noise. In CKKS, *rescaling* also changes the modulus, but it specifically normalizes the *scale factor* of approximate numbers. Hence, rescaling can be viewed as a *numerically-motivated* modulus switch.

Important: Again, the above is just a simplified explanation of the CKKS's idea of rescaling and modulus switching. For an explanation that is more close the original CKKS, see this link.

10.6.4 Key Switching

Sometimes we need to reinterpret a ciphertext encrypted under one secret key s_{old} as if it were encrypted under another key s_{new}—for example, when performing rotations in CKKS or sharing data between users.

Suppose we have a ciphertext component c_1 that multiplies s_{old}. We create **key switching keys** (also called *evaluation keys*) that encrypt powers of the old key in a decomposition over the new key:

$$KS = \{(b_i, a_i) = (-a_i s_{\text{new}} + e_i + \gamma_i s_{\text{old}},\ a_i)\}_i$$

for small base decomposition digits γ_i.

Then, given a polynomial c_1, we decompose it in the same base:

$$c_1 = \sum_i \gamma_i c_{1,i}.$$

Using the precomputed switching keys, we can rebuild equivalent components that are consistent with the new secret key s_{new}:

$$(c'_0, c'_1) = \left(c_0 + \sum_i c_{1,i} b_i,\ \sum_i c_{1,i} a_i \right).$$

Now the ciphertext (c'_0, c'_1) decrypts correctly under s_{new}. This process is called **key switching**.

Important: Relinearization is a special case of key switching where $s_{\text{old}} = s^2$ and $s_{\text{new}} = s$.

10.7 More to add...

Xiao: Here are some other important topics that we may want to add in the future: Xiao!

- Lattice trapdoors and their applications in cryptography.

- Flooding and noise smudging techniques
- Gaussian sampling and tail bounds
- Regev and Peiker's quantum and classical reductions from LWE to worst-case lattice problems.

10.8 Supplementary Readings

Here are some resources for further reading:

- Lattices Algorithms and Applications by Daniele Micciancio.
- Lattices in Computer Science by Oded Regev.
- Lattices, Learning with Errors and Post-Quantum Cryptography by Vinod Vaikuntanathan.
- Lattices in Cryptography by Chris Peikert.
- The textbook by Goldwasser and Micciancio [MG02].
- Here is a recent paper by Chen, Liu, and Zhandry [CLZ22] containing a nice summary of SIS, LWE, and DCP.

Chapter 11

Coding Theory

11.1 Basic Concepts

Definition 11.1.1:

An $(n,k,d)_q$ code is a function $C : \Sigma^k \to \Sigma^n$ such that:

- $|\Sigma| = q$;
- For every $x, x' \in \Sigma^k$, $\mathsf{dist}_H(C(x), C(x')) \geq d$, where $\mathsf{dist}_H(\cdot,\cdot)$ is the hamming distance.

In particular, we use $[n,k,d]_q$ to denote a *linear* $(n,k,d)_q$ code.

Xiao: talk about code rate (or information rate) R. Fractional Hamming distance. δ-distance code. Xiao!

11.2 The Bounds

Lemma 11.2.1: Singleton Bound

Let C be a $(n,k,d)_q$ code. Then $d \leq n-k+1$

Proof. Let $C' : \Sigma^k \to \Sigma^{n-d+1}$ be the projection of C to the first $n-d+1$ coordinates. That is, $C'(x)$ contains the first $n-d+1$ entries of $C(x)$. We see that C' must be an injective function, because if $C'(x) = C'(x')$ with $x \neq x'$, then $C(x)$ and $C'(x)$ can differ in at most $d-1$ coordinates, contradicting the fact that C has minimum distance at least d. But if C' is injective then its range must be at least as large as its domain, and so $n-d+1 \geq k$. ■

Definition 11.2.2: MDS Codes

An $(n,k,d)_q$ code is Maximum Distance Separable (MDS) if $d = n-k+1$ (i.e., the equality is achieved in the singleton bound).

Lemma 11.2.1 (Gilbert-Varshamov Bound)**.** For every n and $\frac{d}{n} < \frac{1}{2}$ there is a $[n,k,d]_2$ code such that

$$k \geq n \cdot \left(1 - H_2(\frac{d}{n})\right) - \Theta(\log n)$$

where $H_2(\cdot)$ is Shannon's binary entropy.

In terms of code rate and δ distance, the Gilbert bound shows that for any $\delta < \frac{1}{2}$, when n is large enough, there always exists a $[n,k,d]_2$ code such that:

$$\mathsf{R} \geq 1 - H_2(\delta) - o(1)$$

$\diamondsuit$

Proof. To do the proof, we first need to recall some implications from Stirling's formula. Stirling's approximation gives

$$n! = \Theta\big(\sqrt{n} \cdot (\frac{n}{e})^n\big)$$

This implies:

$$\binom{n}{k} = \Theta\Big(\sqrt{\frac{n}{k(n-k)}} \cdot (\frac{n}{k})^k \cdot (\frac{n}{n-k})^{n-k}\Big)$$

which implies:

$$\log \binom{n}{k} = n \cdot H_2(\frac{k}{n}) + \Theta(\log n)$$

Xiao: to be done from Luca's Lecture notes... Also, check the Gilbert-Varshamov bound in Chapter 19.2 in [AB09]. ■ Xiao!

11.3 Linear Codes

Given a specific tuple of (n, k, d), there are so many ways to design a $[n, k, d]$ code. One special type of codes draws our attention due to its clean format and rich theoretical implications. Such kind of codes is linear code (on $\mathbb{F}_2$). Its linearity admits the application of the beautiful theory of linear algebra.

Basic Concepts. The codeword of a $[n, k, d]_2$ linear code can be treated as a dimension-k linear subspace of $\mathbb{F}_2^n$. Then, any set basis $\{g_1, \ldots, g_k\}$ for the codeword space are called generators of this linear code. We say $\mathcal{C}$ is of length-n and rank-k (or dimension-k, written as $dim(\mathcal{C}) = k$), since $\mathcal{C}$ actually forms a dimension-k subspace of the vector space $\mathbb{F}_2^n$.

Let $\mathcal{C}$ be a $[n, k, d]_2$ linear code. Any codeword $\mathbf{c} \in \mathcal{C}$ can be expressed as a matrix-vector multiplication $\mathbf{G}^T\mathbf{x}$, where $\mathbf{G}$ is a $k \times n$ matrix whose i-th row is g_i.[1] $\mathbf{G}$ is called the *generator matrix* for $\mathcal{C}$. The *parity check matrix* is the matrix $\mathbf{H}$ such that $\mathbf{H}\mathbf{G}^T = 0$. It has the property that any vector $\mathbf{c}$ is a valid codeword (i.e., $\exists \mathbf{v}$ s.t. $\mathbf{c} = \mathbf{G}^T\mathbf{v}$) if and only if $\mathbf{H}\mathbf{c} = 0$. Due to this property, the value $\mathbf{H}\mathbf{c}$ is called the "syndrome" of $\mathbf{c}$.

Remark 11.3.1 (The Standard Form of Generator Matrices)**.** *The generator matrix* $\mathbf{G}$ *is in the standard form if* $\mathbf{G} = \begin{bmatrix} \mathbb{1}_k & \mathbf{P}_{k\times(n-k)} \end{bmatrix}$. *Then, the corresponding parity check matrix is given by* $\mathbf{H} = \begin{bmatrix} -\mathbf{P}^T_{k\times(n-k)} & \mathbb{1}_{n-k} \end{bmatrix}$.

The *dual code* of $\mathcal{C}$ is defined as $\mathcal{C}^\perp := \big\{\mathbf{x} \mid \mathbf{x} \cdot \mathbf{z} = 0 \ \forall \mathbf{z} \in \mathcal{C}\big\}$. Here are some properties of dual codes:

1. The generator matrix $\mathbf{G}$ of $\mathcal{C}$ is always the parity matrix of $\mathcal{C}^\perp$.
2. $(\mathcal{C}^\perp)^\perp = \mathcal{C}$.
3. $dim(\mathcal{C}) + dim(\mathcal{C}^\perp) = n$.

[1] Traditionally, people prefer to set the dimension of $\mathbf{G}$ to be $k \times n$ (i.e., the code then consists of the row-spanned space of $\mathbf{G}$), instead of $n \times k$. Some authors prefer to use the "row-vector" representation, where a message is interpreted as a row vector $\mathbf{x}^T$ and the encoding procedure is then $\mathbf{x}^T\mathbf{G}$. Throughout this book, we use the "column-vector" representation: we representation a message by a column vector $\mathbf{x}$, and use $\mathbf{G}^T\mathbf{x}$ for encoding.

11.4 Walsh-Hadamard Code

Walsh-Hadamard code is a $[2^n, n, 2^{n-1}]_2$ code. Given any message $m \in \{0,1\}^n$

$$\mathsf{WH}(m) = (\langle m, [1]_2 \rangle, \langle m, [2]_2 \rangle, \ldots, \langle m, [2^n]_2 \rangle),$$

where $[i]_2$ is the binary representation of i, and "$\langle \cdot, \cdot \rangle$" is the inner product modulo 2.

11.5 Reed-Solomon Code

Reed-Solomon code makes use of larger alphabet size to achieve better distance and rate. It is a $[n, k, n-k+1]_q$ code where:

- the alphabet is a size-q field $\mathbb{F}_q$; **and**
- $k \le n \le q$.

For a message $m = (m_0, \ldots, m_{k-1}) \in \mathbb{F}_q^k$, its codeword is computed as follows:

1. Treat $m = (m_0, \ldots, m_{k-1})$ as degree-$(k-1)$ polynomial in $\mathbb{F}_q[x]$:

$$m(x) = m_0 + m_1 \cdot x + m_2 \cdot x^2 + \ldots + m_{k-1} \cdot x^{k-1}.$$

2. Evaluate $m(x)$ on n prefixed points $\{x_1, \ldots, x_n\}$.[2]

3. Output the evaluations as the codeword for m, i.e.

$$\mathsf{RS}(m) = \big(m(x_1), \ldots, m(x_n)\big).$$

Vandermonde Matrix Representation. The encoding procedure of Reed-Solomon code can be expressed as a Vandermonde linear transformation. For example, if we use $\{x_1, \ldots, x_n\}$ as the set of evaluation points, then for any $m = (m_0, \ldots, m_{k-1})$,

$$\mathsf{RS}(m) = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^{k-1} \\ 1 & x_2 & x_2^2 & \cdots & x_2^{k-1} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^{k-1} \end{bmatrix}_{n \times k} \cdot \begin{bmatrix} m_0 \\ m_1 \\ \vdots \\ m_{k-1} \end{bmatrix}_{k \times 1} = \begin{bmatrix} m(x_1) \\ m(x_2) \\ \vdots \\ m(x_n) \end{bmatrix}_{n \times 1}$$

Xiao: Need to talk about the Folded Reed-Solomon code [GR08]. It is used in the recent impressive work by Yamakawa and Zhandry [YZ22]. Xiao!

11.6 Coding theory in general

Xiao: More coding theory stuff Xiao!

11.7 Non-malleable code

Xiao: On non-malleable code Xiao!

11.7.1 Splite-state non-malleable code

11.8 Randomized encoding (used in [KOS18])

Xiao: Xiao!

- the plain definition can be satisfied by Yao's garbled circuits.
- But there is an adaptive version. It can be constructed by equivocal commitments plus garbled circuits. See the reference in [KOS18].
- [Lin17, Chapter 1] contains a good introduction to randomized encoding.

[2] Common choices for the set of evaluation points include $\{0, 1, \ldots, n-1\}$, $\{0, 1, \alpha, \alpha^2, \ldots, \alpha^{n-2}\}$, or $\{\alpha^0, \alpha^1, \ldots, \alpha^{n-1}\}$, where α is the primitive element of $\mathbb{F}_q$.

Chapter 12

(Classical) Complexity Theory

12.1 The Basics

Traditionally, Complexity Theory cares about constructible functions. Take time-constructible functions as an example (similar reason applies to space-constructible). For such functions, a TM "knows" the time bound under which it is operating, simply by "looking at" the description of the function. These functions are usually considered natural. Most importantly, several theorems only holds (provably) for such functions. A typical example is the time hierarchy theorem, whose proof requires that the TM must determine in $O(f(n))$ time whether an algorithm has taken more than $f(n)$ steps. Time-constructibility is thus proposed to formulate these natural functions.

Definition 12.1.1 (Time Constructibility)**.** A function $f : \mathbb{N} \mapsto \mathbb{N}$ is time-constructible if there is a TM M that computes the function $1^n \mapsto [f(n)]_2$ in $O\big(f(n)\big)$ time, where $[f(n)]_2$ denotes the binary representation of the number $f(n)$. $\Diamond$

Remark 12.1.1. *Here are some remarks:*

1. *Usually, a Turing machine uses a binary alphabet. If we use such a TM to compute the function $1^n \to f(n)$, the output is by default in binary representation. In the above definition, we put $[f(n)]_2$ mainly to make this requirement explicit for the machines which do not use a binary alphabet. But this does not matter much since other alphabet can be converted into a binary one without much blowing-up in time complexity.*

2. *Some textbook define time constructibility only for functions $f(n) > n$. That is to allow the algorithm time to read its input.*

3. *There is a definition called "fully time-constructible functions". It is the same as Definition 12.1.1 except that the computation should be done in exactly $f(n)$ time, instead of $O\big(f(n)\big)$.*

Definition 12.1.2 (Space Constructibility)**.** A function $f : \mathbb{N} \to \mathbb{N}$ is space-constructible if there is a TM that computes the function $1^n \to [f(n)]_2$ in $O(f(n))$ space, where $[f(n)]_2$ denotes the binary representation of the number $f(n)$. $\Diamond$

In some scenarios such as computing on low storage machines, the space resource can be a bottleneck of computation power. Thus people also care about the class of language captured by deterministic/non-deterministic logarithm space. Three potential problems arise when we want to investigate L and NL:

1. The input already occupies linear space.

2. The machine may not have enough space to write down the full output.

3. Since $\mathsf{NL} \subseteq \mathbf{P}$, NL may not be "closed" under Karp reduction.

For the first problem, we do not count the space occupied by the input. In addition, we usually (e.g. in Savitch's theorem 12.5.1) restrict ourselves to space complexity $f(n) \geq \log(n)$, such that we have enough space to write down the index of the input position that we want to access. For the last two problems, people propose implicitly-computable functions and log space reduction as shown in the following definitions.

Definition 12.1.3 (Implicit Logspace Computability)**.** A function $f : \{0,1\}^* \to \{0,1\}^*$ is implicitly logspace computable, if the following holds

(1) $\forall x \in \{0,1\}^*$, $\exists c$ s.t. $|f(x)| \leq |x|^c$ (i.e. f is polynomially bounded).

(2) The languages $L_f = \{(x,i) \mid f(x)_i = 1\}$ and $L'_f = \{(x,i) \mid i \leq |f(x)|\}$ are in L.

$\Diamond$

Remark 12.1.2. *The definition of logspace computability may seem confusing at first glance.*

1. *The definition of logspace computability may seem confusing at first glance. But all it wants to say is that the function can be computed using log space. This requirement boils down to the two languages in the second item of the definition: (i) $L_f \in \mathsf{L}$ means each bit of the output can be computed in log space; (ii) $L'_f \in \mathsf{L}$ means the total length of the output can be computed in log space. Also, note that the first item in the definition is to restrict us to functions the polynomial output size. Without it, a function with exponential size of output may also satisfy the requirement in (2).*

2. *Do not confuse it with the concept of space/time constructible functions (Definition 12.1.1 and 12.1.2). Indeed, space/time constructibility is more about the properties of functions, instead of machines. It is proposed to capture all the "interesting" and natural functions people care about, ruling out functions that are troublesome to analysis (Fortunately, those cases are usually rare and unnatural). In contrast, implicit-logspace computability is more about the machines. Since logspace machine do not have enough space to write down the full output, people thus propose this class of function to allow meaningful discussion for such machines.*

Definition 12.1.4 (Logspace Reduction and NL-Completeness)**.** A language A is logspace reducible to language B, denoted $A \leq_{\mathsf{log}} B$, if there exists a implicitly logspace computable function $f : \{0,1\}^* \to \{0,1\}^*$ such that for all $x \in \{0,1\}^*$,

$$x \in A \quad \Leftrightarrow \quad f(x) \in B$$

We say that $B \in \mathsf{NL}$ is NL-complete if for every $A \in \mathsf{NL}$, $A \leq_{\mathsf{log}} B$. $\Diamond$

Definition 12.1.1: Various Types of Reductions

Karp Reduction: let X and Y be decisional problems. A polynomial-time computable function f is called a Karp reduction from X to Y if, for every x, it holds that $x \in X$ if and only if $f(x)\ inY$.

The following quote from [Gol08] explains the relation between Karp reduction and Turing (aka Cook) reduction: "Thus, syntactically speaking, a Karp-reduction is not a Cook-reduction, but it trivially gives rise to one (i.e., on input x, the oracle machine makes query

$f(x)$, and returns the oracle answer). Being slightly inaccurate but essentially correct, we shall say that Karp-reductions are special cases of Cook-reductions."

Levin Reduction: let R and R' be relations for two search problems. Let

$$S_R = \{x : \exists y \text{ s.t. } (x,y) \in R\}, \quad S_{R'} = \{x' : \exists y' \text{ s.t. } (x',y') \in R'\}.$$

A pair of polynomial-time computable functions, f and g, is called a Levin reduction from R to R' if f is a Karp reduction from S_R to $S_{R'}$, and for every $x \in S_R$ and $y' \in R'(f(x))$ it holds that $\big(x, g(x,y')\big) \in R$, where $R'(x') = \{y' : (x',y') \in R'\}$.

Levin Reduction can be viewed as a generalization of Karp reduction to search problems. We will use this type of reduction in Theorems 12.4.1 and 12.6.2.

Turing Reductions: if A is Turing reducible to B in polynomial time, then $A \subseteq P^B$. This reduction is usually denoted as $A \leq_T^p B$. This type of reduction is also called "**Cook reduction**". It is useful for $\#P$ completeness.

Parsimonious Reductions: a Parsimonious reduction R from problem X to problem Y is a reduction such that for any instance x of X, the number of solutions to x is equal to the number of solutions to problem $R(x)$, which is an instance of Y.

Theorem 12.1.1 (Efficient Universal Turing Machine [HS66])**.** There exists a TM U such that for every $x, \alpha \in \{0,1\}^*$, $U(x,\alpha) = M_\alpha(x)$, where M_α denotes the TM represented by α. Moreover, if M_α halts on input x within T steps then $U(x,\alpha)$ halts within $c \cdot T \log T$ steps, where $c > 0$ is a number depending only on

- M_α's alphabet size
- M_α's number of tapes
- M_α's number of states

$\Diamond$

12.1.1 Oblivious TM, Configuration Graphs and Snapshots

Xiao: FiXme updates up to here (To be done ...) Xiao!

12.1.2 Transformations between Different Computational Models

Need to formalize these folklore claims

- Any Boolean circuit can be transformed into an equivalent arithmetic circuit over any field, with at most a constant-factor blowup in size.
- Any circuit Cir is not layered it can easily be transformed into a layered circuit Cir' with a small blowup in size.
- If a computer program runs in time $T(n)$ on a RAM with at most $S(n)$ cells of memory, then the program can be turned into a (layered, fan-in 2) arithmetic circuit of depth not much more than $T(n)$ and width of about $S(n)$. [Each layer of the circuit represent a

configuration of the RAM execution. So the circuit has depth $\approx T$ and width $\approx S$.]

- There exist a transformation from the time-T space-S RAM computation to a circuit of depth $S \log T$ and size $2^{\Theta(S)}$. (See [Tha20, Section 5.4].)
- **RAM-to-CirSAT.** Any RAM program, running in time T and outputting y on input x, can be efficiently transformed into an instance (C, x, y) of arithmetic circuit satisfiability (i.e. $\exists w$ s.t. $C(x, w) = y$), where the circuit C has size close to T and depth close to $\log T$, and the witness w is of size $\approx T$. (See [Tha20, Section 5.5].)

12.2 Boolean Circuits

Definition 12.2.1: Boolean Circuits

For every $n \in \mathbb{Z}$, an n-input single-output Boolean circuit is a directed acyclic graph with n sources (vertices with no incoming edges) and one sink (vertex with no outgoing edges). All non-source vertices are called gates and are labeled with one of $\wedge$, $\vee$ or $\neg$ (i.e., the logical operations OR, AND, and NOT). The vertices labeled with $\vee$ and $\wedge$ have fan-in (i.e., number of incoming edges) equal to 2, and the vertices labeled with $\neg$ have fan-in 1. The size of Cir, denoted by $|\mathsf{Cir}|$, is the number of vertices in it.

Remark 12.2.2: On the number of fan-out

Note that, in Definition 12.2.1, we do not put any restriction on the fan-out of sources, i.e. one input can go to several gates. But, traditionally, other gates (except for the input gates) have fan-out 1.
It does not make too much difference to allow more fan-out. However, the fan-in is usually clearly stipulated as in Definition 12.2.1. Two remarks follow:

- For circuits whose fan-in ≤ 2, its number of edges cannot be too big even if arbitrary fan-out number is allows. Assume that the number of total gates is m for such circuit. Then the total number of its edges is bounded by $2m$.
- Boolean formulae are just circuits where each gate has fan-in equal to 1.

12.2.1 Boolean Formulas, CNFs and Universal Gates

We first distinguish between Boolean circuits (functions) from Boolean formulas.

Definition 12.2.3: Boolean Formulas

A Boolean circuit is a Boolean formula if the fan-out of each gate is no ≤ 1.

- *If you are a cryptographer, memorize the following claim: polynomial-size Boolean formulas are equivalent to* NC^1 *(see this lecture note for a proof. See [GVW12, AV19] for applications).*

Fact 12.2.1. NAND and Tof (Toffoli) gates:

- A Toffoli gate is defined as $\mathsf{Tof}(a, b, c) = \big(a, b, \mathsf{XOR}\big(\mathsf{AND}(a, b), c\big)\big) = (a, b, ab \oplus c)$. It can be used to compute AND, NOT, and XOR as follows:

– $\mathsf{AND}(a,b)$: $\mathsf{Tof}(a,b,0) = (a,b,ab)$;

– $\mathsf{NOT}(a)$: $\mathsf{Tof}(1,1,a) = (1,1,1\oplus a) = (1,1,\neg a)$;

– $\mathsf{XOR}(a,b)$: $\mathsf{Tof}(1,a,b) = (1,1,a\oplus b)$.

- A NAND-gate is defined as: $\mathsf{NAND}(a,b) = \neg(a\wedge b) = \neg a \vee \neg b$. We can convert a AND and OR gate to a NAND gate in the following way:

  – $\mathsf{NOT}(a) = \mathsf{NAND}(a,a)$

  – $\mathsf{AND}(a,b) = \neg\mathsf{NAND}(a,b) = \mathsf{NAND}\big(\mathsf{NAND}(a,b), \mathsf{NAND}(a,b)\big)$

  – $\mathsf{OR}(a,b) = \mathsf{NAND}(\neg a, \neg b) = \mathsf{NAND}\big(\mathsf{NAND}(a,a), \mathsf{NAND}(b,b)\big)$

- $\mathsf{NAND}(a,b) = c$ if and only if $a+b+2c-2 \in \{0,1\}$. This simple obervation helps in building NIZK and NIWI in [GOS06b, GOS06a].

Theorem 12.2.4: Boolean Functions to CNFs

For every Boolean function $f : \{0,1\}^\ell \to \{0,1\}$ there is an ℓ-variable CNF formula ϕ of size $\ell \cdot 2^\ell$ such that $\phi(x) = f(x)$ for every $x \in \{0,1\}^\ell$, where the size of a CNF formula is defined to be the number of $\wedge$ or $\vee$ symbols it contains.

Proof. We give a constructive proof, building such a CNF formula ϕ explicitly. For a variable $v \in \{0,1\}^\ell$, it is easy to construct a ℓ-variable "characteristic function" $C_v(z_1,\ldots,z_\ell)$ such that $C_v(z_1,\ldots,z_\ell)$ only consists of disjunctions among z_i's and $\overline{z}_i$'s, and satisfies the following requirement:

$$C_v(z_1,\ldots,z_\ell) = \begin{cases} 1, & \text{if } z_1\|\ldots\|z_\ell = v \\ 0, & \text{if } z_1\|\ldots\|z_\ell \neq v \end{cases}$$

Then the following formula satisfies all the property specified in the theorem:

$$\phi(z_1,\ldots,z_\ell) = \bigwedge_{v:f(v)=1} C_v(z_1,\ldots,z_\ell).$$

■

A very important implication of Theorem 12.2.4 is that: $\{\mathsf{AND},\mathsf{NOT},\mathsf{OR}\}$ form a universal set for Boolean circuits. According to Fact 12.2.1, we know that $\{\mathsf{NAND}\}$ or $\{\mathsf{Tof}\}$ is also universal for Boolean circuits.

Corollary 12.2.5: Universal Gates for Boolean Circuits

$\{\mathsf{AND},\mathsf{NOT},\mathsf{OR}\}$ is a universal set of gates for Boolean circuits. So is $\{\mathsf{AND},\mathsf{XOR}\}$, $\{\mathsf{NAND}\}$, or $\{\mathsf{Tof}\}$.

Proof (An alternative proof of Corollary 12.2.5). We provide a different proof for Corollary 12.2.5 without invoking Theorem 12.2.4.

Xiao: Outline of this proof

Xiao!

- Any Boolean function $f : \{0,1\}^\ell \mapsto \{0,1\}$ can be viewed as a multi-linear polynomial from $\mathbb{Z}_2^\ell \mapsto \mathbb{Z}_2$.

- It can be viewed as a polynomial because f is a finite-value function. Recall that any finite-value function can be interpolated by a polynomial; It is (multi-)linear because $b^2 = b$ for any $b \in \{0, 1\}$ (i.e., degree higher than 1 does not make any contribution).

- Therefore, to perform any computation, we only need the ability to perform addition and multiplication on $\mathbb{Z}_2$, which are exactly AND and XOR.

■

The following theorem is not surprising, given that 3SAT is NP complete. Also, the proof is not hard. However, its proof contains very useful tricks for converting circuit gates to formula clauses (more accurately, 3CNF clauses).

Theorem 12.2.6: CKT-SAT to 3CNF

There is a polynomial-time reduction from CKT-SAT to 3CNF. (This theorem is w.r.t. 2-fan-in, unbounded fan-out circuits.)

Proof. (See the second half of this video.
The key trick is the following formula from mathematical logics:

$$a \Rightarrow b \quad \Leftrightarrow \quad \neg a \vee b.$$

Using this formula, we get that

- For AND gates: $w_3 = w_1 \wedge w_2$ can be converted to $(\neg w_3 \vee w_1) \wedge (\neg w_3 \vee w_2) \wedge (\neg w_1 \vee \neg w_2 \vee w_3)$. The proof goes as follows:

$$\begin{aligned} & w_3 = w_1 \wedge w_2 \\ \Leftrightarrow\ & (w_3 \Leftrightarrow w_1 \wedge w_2) \\ \Leftrightarrow\ & (w_3 \Rightarrow w_1 \wedge w_2) \wedge (w_1 \wedge w_2 \Rightarrow w_3) \\ \Leftrightarrow\ & \big(\neg w_3 \vee (w_1 \wedge w_2)\big) \wedge \big(\neg(w_1 \wedge w_2) \vee w_3\big) \\ \Leftrightarrow\ & (\neg w_3 \vee w_1) \wedge (\neg w_3 \vee w_3) \wedge (\neg w_1 \vee \neg w_2 \vee w_3) \end{aligned}$$

- For NOT gates: $w_2 = \neg w_1$ can be converted to $(w_1 \vee w_2) \wedge (\neg w_1 \vee \neg w_2)$. Its proof goes as the above.

- For OR gates: $w_3 = w_1 \vee w_2$ can be converted to $(\neg w_3 \vee w_1 \vee w_2) \wedge (\neg w_1 \vee w_3) \wedge (\neg w_2 \vee w_3)$. Its proof goes as the above.

More generally, any propositional formula involving 3 variables can be converted to a CNF.) ■

12.2.2 NC and AC circuits

Definition 12.2.7: The class NC

For every d, a language $\mathcal{L}$ is in NC^d if $\mathcal{L}$ can be decided by a family of Boolean circuits $\{\mathsf{Cir}_n\}$ where Cir_n satisfies the following requirements:

- it is of $\mathsf{poly}(n)$ size, **and**
- it is of $O(\log^d n)$ depth.

The class NC is $\cup_{i \geq 0} \mathsf{NC}^i$.

Definition 12.2.8: The class AC

For every d, a language $\mathcal{L}$ is in $\mathbf{AC}^d$ if $\mathcal{L}$ can be decided by a family of Boolean circuits $\{\mathsf{Cir}_n\}$ where Cir_n satisfies the following requirements:

- it is of $\mathsf{poly}(n)$ size, **and**
- it is of $O(\log^d n)$ depth, **and**
- its OR and AND gates are allowed to have unbounded fan-in.

The class $\mathbf{AC}$ is $\cup_{i\geq 0}\mathbf{AC}^i$.

Fact 12.2.2. $\mathsf{NC}^i \subseteq \mathbf{AC}^i \subseteq \mathsf{NC}^{i+1}$, where the inclusion is known to be strict for $i = 0$. (Proof: Unbounded (but $\mathsf{poly}(n)$) fan-in can be simulated using a tree of OR/AND gates of depth $O(\log n)$.)

Fact 12.2.3. The following problems (more accurately, their language version) are in $\mathbf{AC}^0$:

1. Binary addition (with carry bits) of 2 n-bit binary strings.

Here are some interesting lower-bounds for $\mathbf{AC}^0$:[1]

1. Depth-2 circuits are either DNFs or CNFs.
2. Every Boolean function can be computed by a (exponential-size) DNF and also by a CNF. (This is just Thm. 12.2.4.)
3. Depth-2 $\mathbf{AC}^0$ circuits for PARITY have size at least $\Omega(2n)$.
4. If Cir is an $\mathbf{AC}^0$ circuit of size s, depth d computing PARITY, then $s \geq 2^{\Omega(n^{\frac{1}{d-1}})}$.

   We remark that this famous theorem says $\mathbf{AC}^0$ circuit of constant depth and subexponential size[2] cannot compute PARITY. It is a highly non-trivial result. The proof exploits Håstad's switching lemma.

The following problems (more accurately, their language version) are in NC^0:

1. Binary addition (with carry bits) of 3 n-bit binary strings.
2. This class circuits turn out to be super-important for the recent breakthrough in building indistinguishability obfuscation (see [JLS21] and the references therein). One important property is that all such circuits have constant locality, i.e., each bit of the output can depend on at most a constant number of the input bits. Therefore, any NC^0 circuit can be expressed as a constant degree multi-variate polynomial over a ring (e.g., $\mathbb{Z}_p$).
3. Building PRGs in NC^0 is a problem the draws intensives attention. See the discussion and references in [JLS21, Setion 1].

The following problems (more accurately, their language version) are in NC^1:

1. Binary addition (with carry bits) of n length n-bit binary strings.

[1] for more details, check this lecture note

[2] Note that some authors define subexponential size as $\cap_{\varepsilon>0}\mathbf{DTIME}(2^{n^\varepsilon})$, instead of $2^{o(n)}$. Here, the "subexponential" means $\cap_{\varepsilon>0}\mathbf{DTIME}(2^{n^\varepsilon})$.

2. Integer multiplication (of 2 n-bit numbers).

3. Integer division.

4. Inner product.

5. Matrix Multiplication (of 2 $n \times n$ matrices where each entry is of size n).

6. Parity checking: $\mathsf{PARITY} = \{x : x \text{ has an odd number of 1s}\}$

7. Evaluation of polynomial size Boolean formulas. We note that this result is non-trivial. Actually, we can show that polynomial-size Boolean formulas are equivalent to NC^1 (see this lecture note).

The following problems (more accurately, their language version) are in NC^2:

1. $\boldsymbol{A}^n$ of $n \times n$ matrix $\boldsymbol{A}$ where each entry of size n.

2. Determinant of $n \times n$ matrix.

3. Solving the linear system $\boldsymbol{Ax} = \boldsymbol{b}$, where A is a non-singular $n \times n$ matrix, b an n dimensional column vector. (The algorithm is non-trivial. See this lecture notes.)

12.2.3 Branching Program

Theorem 12.2.1 (Barrington's Theorem)**.** (To do ...) $\Diamond$

12.3 Hierarchy: A Fresh Perspective

12.3.1 TM Hierarchy

Theorem 12.3.1: Time Hierarchy Theorem [HS65]

If f and g are time-constructible functions satisfying $f(n) \log f(n) = o(g(n))$, then

$$\mathbf{DTIME}(f(n)) \subsetneq \mathbf{DTIME}(g(n))$$

Proof. (The idea is to use the diagnization technique on a language involving simulating a universal Turing machine for $f(n)$ steps. It can be viewed as a scale-down version of the proof for HALT is undecidable.

For more details, refer to this lecture and P62 on Arora&Barak) ■

Theorem 12.3.1 (Non-Deterministic Time Hierarchy Theorem [Coo73])**.** If f and g are time-constructible functions satisfying $f(n+1) = o(g(n))$, then

$$\mathsf{NTIME}(f(n)) \subsetneq \mathsf{NTIME}(g(n))$$

$\Diamond$

Proof. (to be done. P63 on Arora&Barak) ■

The following theorem is the space analogue to Theorem 12.3.1. Note that it does not have the $f(n)$ factor that appears in Theorem 12.3.1. This is essentially due to the fact that the universal Turing machine consumes only $S(n)$ space to simulate a $S(n)$-space machine.thm:space-hierarchy

Theorem 12.3.2: Space Hierarchy Theorem [SHL65]
If f, g are space-constructible functions satisfying $f(n) = o\big(g(n)\big)$, then $$\mathsf{SPACE}\big(f(n)\big) \subsetneq \mathsf{SPACE}\big(g(n)\big)$$

Theorem 12.3.2 (Collapse of PH)**.** PH has the following properties of collapse:

(1) For every $i \geq 1$, $\sum_i^{\mathsf{P}} = \prod_i^{\mathsf{P}}$ implies $\mathsf{PH} = \sum_i^{\mathsf{P}}$.

(2) $\mathbf{P} = \mathsf{NP}$ implies $\mathsf{PH} = \mathbf{P}$.

$\diamondsuit$

Proof. We only need to prove the second item. The same argument extends to the first item $i > 1$.
Assume $\mathbf{P} = \mathsf{NP}$, we prove $\mathbf{P} = \mathsf{PH}$ by induction on i that $\sum_1^{\mathsf{P}} \subseteq \mathbf{P}$, which also implies $\prod_i^{\mathsf{P}} \subseteq \mathbf{P}$ as $\mathbf{P}$ is closed under complementation.
Assume $\sum_{i-1}^{\mathsf{P}} \subseteq \mathbf{P}$. For any $L \in \sum_i^{\mathsf{P}}$, we immediately have

$$(x, u_1) \in L' \Rightarrow L' \subseteq \textstyle\prod_{i-1}^{\mathsf{P}} \Rightarrow L' \in \mathbf{P},$$

where u_1 is the variable quantified by the first quantifier (the first $\exists$). This means there exists a TM M' the decides L' in polynomial time. Also, by the definition of L and L', it is easy to see that

$$x \in L \Leftrightarrow \exists u_1 \text{ s.t. } (x, u_1) \in L'$$

Then plugging M' into the RHS of the above gives:

$$x \in L \Leftrightarrow \exists u_1 \text{ s.t. } M'(x, u_1) = 1,$$

which means $L \in \mathsf{NP}$. Combining it with our assumption $\mathsf{NP} = \mathbf{P}$, we have $L \in \mathbf{P}$. Since our choice of $L \in \sum_i^{\mathsf{P}}$ is arbitrary, we thus proved $\sum_i^{\mathsf{P}} \subseteq \mathbf{P}$, which finishes our induction step. ∎

12.3.2 Circuit Hierarchy and Hard Functions

Theorem 12.3.3 (Existence of hard functions [Sha49])**.** For every $n > 1$, there exists a function $f : \{0,1\}^n \to \{0,1\}$ that cannot be computed by a circuit Cir of size $2n/(10n)$. $\diamondsuit$

Proof. (See Theorem 6.21 in [AB09].) ∎

Similar to the time/space hierarchy theorems (Theorem 12.3.1 and 12.3.2), circuits also have a hierarchy theorem.

Theorem 12.3.4 (Non-Uniform Hierarchy Theorem)**.** For every functions $T, T' : \mathbb{N} \to \mathbb{N}$ with $2n/n > T'(n) > 10T(n) > n$,

$$\mathbf{SIZE}(T(n)) \subsetneq \mathbf{SIZE}(T'(n))$$

$\diamondsuit$

Proof. (See Theorem 6.22 in [AB09].) ∎

12.4 Complete Languages: NP, PSPACE, NL and PH

Around 1971, Cook and Levin independently discovered the notion of NP-completeness and gave examples of combinatorial NP-complete problems whose definition seems to have nothing to do with Turing machines. Soon after, Karp [Kar72] showed that NP-completeness occurs widely and many problems of practical interest are NP-complete, and studied the relations among those problems by Karp reduction.

Theorem 12.4.1: Cook-Levin Theorem [Coo71, Lev73]

Denote by SAT the language of all satisfiable CNF formulae and by $3\text{-}\mathsf{SAT}$ the language of all satisfiable 3-CNF formulae. Then

- SAT is NP-complete.
- $3\text{-}\mathsf{SAT} \leq_{\mathsf{p}} \mathsf{SAT}$.

Proof. SAT is obviously in NP. So we only need to prove it is NP-hard by showing $L \leq_{\mathsf{p}} \mathsf{SAT}$ for any $L \in \mathsf{NP}$. There are 3 typical ways to do this:

(1) Sipser [Sip12] uses tableau argument.

(2) Arora&Barak [AB09] uses oblivious Turing machine.

(3) Prove $\mathsf{CKT\text{-}SAT}$ is NP-hard and $\mathsf{CKT\text{-}SAT} \leq_{\mathsf{p}} \mathsf{SAT}$.

For the first two items, in spite of the difference between the tools use there, these two methods share the same idea of using locality to verify the computation of TMs.

The proof for $3\text{-}\mathsf{SAT} \leq_{\mathsf{p}} \mathsf{SAT}$ can be done by showing that each clause in a CNF can be broken into small segments of 3-variable clauses, by introducing a new variable for each "breaking" operation. For example, consider a 4-variable clause $C = u_1 \vee u_2 \vee v_3 \vee v_4$. One can easily verify the following is true:

$$C \text{ is satisfiable} \quad \Leftrightarrow \quad (u_1 \vee u_2 \vee z) \wedge (\overline{z} \vee v_3 \vee v_4) \text{ is satisfiable}$$

Applying this (poly-time) transformation on each clause of a CNF gives a 3-CNF, finishing the proof. ■

The first language shown to be PSPACE-complete is TQBF. This is a work of Stockmeyer and Meyer [SM73].

Theorem 12.4.1 (PSPACE-Complete Language [SM73])**.** TQBF is PSPACE-complete, where TQBF denotes the set of quantified Boolean formulae that are true. ♢

Proof. To prove that TQBF is in PSPACE, simply design a recursive algorithm (recursive on the quantifiers) to evaluation the formula. Since space can be reused and for the feedback of each level of recursion (the value need to be stored) is just a single bit, all the work can be done in poly space.

To prove that $L \leq_{\mathsf{p}} \mathsf{TQBF}$ for all $L \in \mathsf{PSPACE}$, the main idea is to construct a Boolean formula on the configuration graph of the decider machine for L. (Add details. Check P77 Arora&Barak...) ■

Theorem 12.4.2 (NL-Complete Language)**.** Denote the language PATH as

$$\mathsf{PATH} = \{(G, s, t) \mid \text{ vertex } t \text{ can be reached from } s \text{ in the directed graph } G\ \},$$

Then the following holds:

- PATH is NL-complete
- $\overline{\mathsf{PATH}}$ is NL-complete. (Immerman-Szelepcsényi Theorem [Imm88, Sze87])

$\Diamond$

Before give the proof of Theorem 12.4.2, we remark that the proof actually can be modified to give the following more general (and surprising) result:

Corollary 12.4.1 (Complete-Equivalence of NSPACE)**.** For every space constructible $f(n) > \log n$, $\mathsf{NSPACE}\big(f(n)\big) = \mathsf{coNSPACE}\big(f(n)\big)$. In particular, $\mathsf{NL} = \mathsf{coNL}$. $\Diamond$

Proof for Theorem 12.4.2. As one would expect, the proof uses configuration graph of TM again. (add details according to P80 and P82 of Arora&Barak ...) ■

We now discuss the case of PH. The following two theorem show an interesting facts: while each $\sum_i^{\mathsf{P}}$ does have its own complete language, the class PH does not, unless PH collapses.

Theorem 12.4.3 (Complete-Collapse of PH)**.** If there exists a language L that is PH-complete, then there exists an i such that $\mathsf{PH} = \sum_i^{\mathsf{P}}$. $\Diamond$

Proof. The proof is obvious. ■

Theorem 12.4.4 (Complete Language for $\sum_i^{\mathsf{P}}$)**.** $\sum_i^{\mathsf{P}}\mathsf{SAT}$ is $\sum_i^{\mathsf{P}}$-complete, where $\sum_i^{\mathsf{P}}\mathsf{SAT}$ denotes the following special version of TQBF problem:

$$\textstyle\sum_i^{\mathsf{P}}\mathsf{SAT} = \{\phi \mid \exists u_1, \forall u_2, \ldots, Q_i u_i \quad \phi(u_1, \ldots, u_i) = 1\},$$

where ϕ is a Boolean formula, each u_i is a vector of Boolean variables, and Q_i is $\forall$ or $\exists$ depending on whether i is even or odd respectively. $\Diamond$

12.4.1 Important Languages and Their Implications

- ANE3SAT, 3SAT, CKTSAT, 3-COL
- TAUTOLOGY, GNI, PRIMALITY, IND-SAT, Linear Programing, PRIMES, Factoring (decisional version): See this lecture.
- ST-PATH: for undirected version, Omer Reingold showed a $\log(n)$-space solution; for the undirected version, it is currently unknown whether $\log(n)$-space solutions are possible. See this lecture. Since we know that ST-PATH can be solved in $O(\log(n))$-space (see this vedio).

  Also, this problem is NL complete—it can be solved in non-deterministic $O(\log(n))$-space. This also implies that it can be solved in $O(\log^2(n))$-space, by Theorem 12.5.1.
- Here are some P-complete language w.r.t. log-space reduction: HornSAT, Linear Programming, Circuit Evaluation. They have the following implications:

- $C \in L \Leftrightarrow P \subseteq L$
- $C \in NL \Leftrightarrow P \subseteq NL$
- $C \in NC \Leftrightarrow P \subseteq NC$

- ExactClique, SmallestCirccuit: it is unclear whether we can put these two languages in NP or coNP. But it is easy to see that ExactClique is in $\sum_2^{\mathsf{p}}$ and SmallestCircuit is in $\prod_2^{\mathsf{p}}$. See this lecture.

12.5 Time-Space Trade off

You can trade the size of the TM for its efficiency.

Theorem 12.5.1 (Speed-Up Theorem [HS65])**.** if a function f is computable by a TM M in time $T(n)$ then for every constant $c \geq 1$, f is computable by a TM M' in time $T(n)/c$, where M' possibly has larger state size and alphabet size than M. $\diamondsuit$

Remark 12.5.1. *Note that [HS65] is a very important paper. It proved the first (but a little relaxed) version of the existence of efficient Universal Turing machine (Theorem 12.1.1), the above speed-up theorem, and the time-hierarchy theorem for deterministic computation. Interestingly, it seems to be this paper that starts the use of the term "computation complexity".*

For the trade off between time complexity and space complexity, the following theorem may represents the only non-trial result we currently have. This is rather unsatisfactory since this theorem is not surprising at all.

Theorem 12.5.2. $\mathsf{NSPACE}(S(n)) \subseteq \mathbf{DTIME}(2^{O(S(n))})$ $\diamondsuit$

Proof. (The proof use the configuration graph of Turing machines. P72 on Arora&Barak.)

(This should be simple. Do it soon...) ■

The following theorem reveals the relation between non-deterministic and deterministic power w.r.t. space complexity. It follows an immediately corollary that $\mathsf{NPSACE} = \mathsf{PSPACE}$.

Theorem 12.5.1: Savitch's Theorem [Sav70]

For any space-constructible function $f : \mathbb{N} \to \mathbb{N}$ with $f(n) \geq \log(n)$, the following holds

$$\mathsf{NSPACE}\big(f(n)\big) = \mathbf{PSPACE}\big((f(n))^2\big)$$

Proof. The proof for this theorem follows closely that of 12.4.1. (See P78 of Arora&Barak...) ■

(talk about the trade-off for SAT. P90 of Arora&Barak)

The following theorem reveals that space is a more precious resource than time. This is because, together with space hierarchy theorem (Theorem 12.3.2), it implies that $\mathsf{TIME}\big(t(n)\big) \subsetneq \mathsf{SPACE}\big(t(n)\big)$.

Theorem 12.5.2: [HPV77]

$\mathsf{TIME}(t(n)) \subseteq \mathsf{SPACE}(\frac{t(n)}{\log(t(n))})$

Proof. See this lecture. ■

12.6 Relations among Complexity Classes

(**NL vs L.** ST-PATH is NL complete. Meanwhile, it can be solved in $\log^2(n)$ space. By space hierarchy theorem (Theorem 12.3.2), L is a proper subset of NL.

Theorem 12.6.1: L, NL and P

L ⊂ P, and NL ⊂ P.

Proof. To prove that L ⊂ P: there are only $2^{O(\log n)}$ many different configurations of a decider for a language in L. These can be brute-forced in polynomial time.

To prove that NL ⊂ P: writing down all the $2^{O(\log n)}$ possible configurations of a non-deterministic log space machine. We want to know whether there is a path form the starting config to the accepting config. This is exactly the PATH problem, which can be sovled in polynomial time. ■

Is L or NL a proper subset of P? If the answer is unknown, what is the critical language (known in NL or L, but not known in P)?

)

Theorem 12.6.2: Levin Recution

If $\mathbf{P} = \mathsf{NP}$, then for every $L \in \mathsf{NP}$ and every $x \in L$, there exists a polynomial-time TM M such that $R_L(x, M(x)) = 1$.

Proof. The proof consists of two steps:

(1) Show such a machine for SAT.

(2) For any $L \in \mathsf{NP}$, show a Levin reduction to SAT.

For the 2nd item, we note that the reduction used in the proof of Cook-Levin theorem (Theorem 12.4.1) is already a Levin reduction. So we only need to do the 1st item.

Assume we have a decider for a SAT. We can then do the following for each variables in order: for the i-th variable v_i, test whether ϕ is still satisfiable with assignment $v_i = 1$ and $v_i = 0$ to figure out the correct value for v_i. If there are n variables in total, such test costs $2n$ calls to the assumed SAT decider, thus can be done in poly time. ■

Theorem 12.6.1. $\mathbf{P} = \mathsf{NP} \Rightarrow \mathsf{EXP} = \mathsf{NEXP}$ ◊

Proof. Hint: use "padding" argument. ■

The following result, Ladner's theorem, shows a surprising fact: if $\mathbf{P} \neq \mathsf{NP}$, there must be some language lying in between $\mathbf{P}$ and NP-complete languages.

Theorem 12.6.3: Ladner's theorem—NP intermediate languages [Lad75]

Suppose that $\mathbf{P} \neq \mathsf{NP}$. Then there exists a language $L \in \mathsf{NP} \setminus \mathbf{P}$ that is not NP-complete.

Proof. (to be done. P64 on Arora&Barak) ■

Theorem 12.6.4: Manhany's Theorem
if any "sparse language" is NP-Complete, then $P = NP$.

Proof. See this lecture. ■

Theorem 12.6.2. $\mathsf{PH} \subseteq \mathsf{PSPACE}$. Morover, if $\mathsf{PH} \subsetneq \mathsf{PSPACE}$, PH collapses to $\sum_i^{\mathsf{p}}$ for some i. ◊

Proof. The first part is obvious. The second part follows as a simple corollary of Theorem 12.4.3 plus Theorem 12.4.1. ■

We next show a seemingly straightforward result. However, its proof is non-trivial and gives another approach to prove Cook-Levin Theorem (Theorem 12.4.1). We give more details in Remark 12.6.1

Theorem 12.6.3. $\mathbf{P} \subsetneq \mathsf{P}_{/\mathsf{poly}}$. ◊

Proof. This proof uses oblivious Turing machine. There can at most be polynomially many snapshots of a poly time oblivious Turing machine. And the transition between each two adjacent snapshots can be verified by a constant size circuit due to the locality of such TM. Thus, in total, the computation can be done by a poly size circuit that sequentially verifies the adjacent snapshots.

To see this subset relation is proper, consider the unary halting problem. ■

Remark 12.6.1. *The idea of the proof already implies that* $\mathsf{CKT\text{-}SAT}$ *is* $\mathsf{P}_{/\mathsf{poly}}$*-hard. If we can show* $\mathsf{CKT\text{-}SAT} \leq_{\mathsf{p}}$ *3-*SAT*, we then have another proof for Cook-Levin theorem. Actually, converting a circuit to 3-CNF is easy with the following rules to convert each type of gate:*

- AND *Gate:* $z_1 = z_2 \wedge z_3 \quad \Leftrightarrow \quad (\overline{z}_1 \vee \overline{z}_2 \vee z_3) \wedge (\overline{z}_1 \vee z_2 \vee \overline{z}_3) \wedge (\overline{z}_1 \vee z_2 \vee z_3) \wedge (z_1 \vee \overline{z}_2 \vee \overline{z}_3)$
- OR *Gate:* $z_1 = (z_2 \vee z_3) \quad \Leftrightarrow \quad (z_1 \vee \overline{z}_2) \wedge (z_1 \vee \overline{z}_3) \wedge (\overline{z}_1 \vee z_2 \vee z_3)$
- NOT *Gate:* $z_1 = \overline{z}_2 \quad \Leftrightarrow \quad (z_1 \vee z_2) \wedge (\overline{z}_1 \vee \overline{z}_2)$
- *For output wire* y *of* $\mathsf{CKT\text{-}SAT}$*, add* (y) *as the 3-CNF clause.*

We have already seen that $\mathbf{P} \subsetneq \mathsf{P}_{/\mathsf{poly}}$ in Theorem 12.6.3. The following result of Karp and Lipton [KL82] provides some evidence for the conjecture that $\mathbf{P} \neq \mathsf{NP}$.

Theorem 12.6.4 (Karp-Lipton Theorem [KL82])**.** $\mathsf{NP} \subseteq \mathsf{P}_{/\mathsf{poly}}$ implies $\mathsf{PH} = \sum_2^{\mathsf{p}}$. ◊

Proof. According to Theorem 12.3.2, it is sufficient if we can show that $\mathsf{NP} \subseteq \mathsf{P}_{/\mathsf{poly}}$ implies $\prod_2^{\mathsf{p}} \subseteq \sum_2^{\mathsf{p}}$. To do that, it suffices to show $\prod_2^{\mathsf{p}}\mathsf{SAT} \in \sum_2^{\mathsf{p}}$.

Recall that

$$\phi \in \textstyle\prod_2^{\mathsf{p}}\mathsf{SAT} \quad \Leftrightarrow \quad \forall u_1 \in \{0,1\}^n, \exists u_2 \in \{0,1\}^n \quad \phi(u_1, u_2) = 1. \tag{12.1}$$

Define the following language for tuples of Boolean formula ϕ and a variable $u_1 \in \{0,1\}^n$:

$$\mathcal{L}_1 = \{(\phi, u_1) \mid \exists u_2 \in \{0,1\}^n \text{ s.t. } \phi(u_1, u_2) = 1\}$$

Obviously, $\mathcal{L}_1 \in \mathsf{NP}$. Since we assume $\mathsf{NP} \subseteq \mathsf{P}/\mathsf{poly}$, there exists a poly-size circuit family $\{C_n\}_{n\in\mathbb{N}}$ deciding $\mathcal{L}_1$. In another word, $\{C_n\}_{n\in\mathbb{N}}$ is a (family of) decider for the SAT problem of formulea of the form $\phi(u_1, \cdot)$.

Recall that in the proof of Theorem 12.6.2, we showed how to efficiently construct a witness extractor from the decider of SAT. Thus, there exists a poly-size circuit family $\{C'_n\}_{n\in\mathbb{N}}$ such that $\phi\big(u_1, C'_n(\phi, u_1)\big) = 1$ if (ϕ, u_1) is in $\mathcal{L}_1$ (i.e. $\phi(u_1, \cdot)$ is satisfiable). Note that if $\{C'_n\}_{n\in\mathbb{N}}$ is of size $p(n)$, we can describe C'_n using a binary string of size $q(n) = \mathsf{poly}\big(p(n)\big)$, which is also some polynomial on n.

We then denote the following language:

$$\phi \in \mathcal{L}_2 \quad \Leftrightarrow \quad \exists w \in \{0,1\}^{q(n)}, \forall u_1 \in \{0,1\}^n \quad M'(\phi, u_1, w) = 1 \tag{12.2}$$

where M' is a TM such that on input (ϕ, u_1, w), it interprets w as the description of a C'_n and outputs 1 iff $\phi\big(u_1, C'_n(\phi, u_1)\big) = 1$.

Obviously, $\mathcal{L}_2 \in \sum_2^{\mathsf{p}}$. Morover, with a little thinking, one can see that the expressions (12.1) and (12.2) essentially desribe the same language, i.e. $\mathcal{L}_2 = \sum_2^{\mathsf{p}}\mathsf{SAT}$. Thus, $\prod_2^{\mathsf{p}}\mathsf{SAT} \in \sum_2^{\mathsf{p}}$. This closes our proof. ■

A similar result to Theorem 12.6.4 (also appeared in [KL82], but was attributed to Meyer) can be proved for EXP.

Theorem 12.6.5 (Meyer's Theorem [KL82])**.** $\mathsf{EXP} \subseteq \mathsf{P}/\mathsf{poly}$ implies $\mathsf{EXP} = \sum_2^{\mathsf{p}}$. In particular, $\mathsf{EXP} \subseteq \mathsf{P}/\mathsf{poly}$ implies $\mathbf{P} \neq \mathsf{NP}$. ♢

Proof. (It again uses oblivious Turing and snapshots. Do it later ... P102 Arora&Barak.) ■

Chapter 13

Quantum Complexity Theory

13.1 The Basics

Xiao: Remark that unlike BPP, BQP (and QMA) is defined as a class of *promise* languages. There are several reasons why people prefer to do it in this way: Xiao!

- The real problems appear in physics (or those physicists care about) are usually promise problems.
- The promise BQP has complete problems. Probably the most famous one is solving linear equation systems (or inverting a given matrix). But it seems hard to imagine a complete problem for BPP (or BQP when defined without promise).

Second Nature for Quantum Complexity:

- $\mathsf{BQP} \subseteq \mathsf{EXP}$: this is easy to see. Just use exponential time to simulate quantum computation.
- $\mathsf{BQP} \subseteq \mathsf{PSPACE}$: this is also not hard to see—write down the computation, and then use "Feynman's path integral" to convert the computation to the summation of exponentially many complex numbers, which can be computed in polynomial space.
- $\mathsf{BQP} \subseteq \mathsf{QMA}$: to do...
- $\mathsf{BQP} \subseteq \mathsf{PP}$: This is not that obvious. There are three ways to show it.
  1. one can show $\mathsf{QMA} \subseteq \mathsf{PP}$ via the so-called "strong error reduction", the claim then follows from the fact that $\mathsf{BQP} \subseteq \mathsf{QMA}$;
  2. $\mathsf{BQP} \subseteq \mathsf{QMA} \subseteq \mathbf{P}^{\mathsf{QMA}[\log]}$. Then, using the "hierarchical voting" technique, one can show that $\mathbf{P}^{\mathsf{QMA}[\log]} \in \mathsf{PP}$
  3. $\mathsf{BQP} \subseteq \mathbf{PostBQP} = \mathsf{PP}$.
- $\mathsf{QIP} = \mathsf{PSPACE}$. This is established by the breakthrough work of [JJUW10].
- $\mathsf{QIP} = \mathsf{QIP}(3)$: established by [KW00].
- Note that if $\mathsf{QMA} = \mathsf{PP}$, then $\mathsf{PH} = \mathsf{PP}$, which is unlikely to happen.
- There exists a quantum oracle U such that $\mathsf{QMA}^U \neq \mathsf{QCMA}^U$ (by [AK07]). In contrast, finding a *classical* oracle A such that $\mathsf{QMA}^A \neq \mathsf{QCMA}^A$ remains a notorious open problem to this day. The following is quoted from [Aar16, Section 5.2]: "Very recently, Fefferman and Kimmel [FK18] presented interesting progress on this problem. Specifically, they separated QMA from QCMA relative to a classical oracle of an unusual kind. A standard classical oracle receives an input of the form $|x, y\rangle$ and outputs $|x, y \oplus f(x)\rangle$, whereas theirs receives an input of the form $|x\rangle$ and outputs $|\pi(x)\rangle$, where π is some permutation. Furthermore, their oracle is probabilistic, meaning that the permutation π is chosen from a probability distribution, and can be different at each invocation."

- The situation for $\mathsf{BQP/poly}$ vs $\mathsf{BQP/qpoly}$ is very similar to that above case of QMA vs QCMA. That is, there exists a quantum oracle U such that $\mathsf{BQP/poly}^U \neq \mathsf{BQP/qpoly}^U$ using a proof analogous to [AK07]); but it remains open whether there is a classical oracle separating $\mathsf{BQP/poly}$ and $\mathsf{BQP/qpoly}$.
- $\mathsf{PostBQP} = \mathsf{PP}$ by [Aar05].
- $\mathsf{BQP/qpoly} \subseteq \mathsf{PostBQP/poly} = \mathsf{PP/poly}$: established by [Aar04].
- $\mathsf{BQP/qpoly} \subseteq \mathsf{QMA/poly}$ by [AD10]. Namely, a quantum computer with polynomial-size quantum advice can be simulated by QMA with polynomial-size classical advice). Indeed, something stronger is true: polynomial-size trusted quantum advice can be simulated using polynomial-size trusted classical advice together with polynomial-size untrusted quantum advice.
- $\mathsf{QMA/qpoly} \subseteq \mathsf{BQPSPACE/qpoly} \subseteq \mathsf{PostBQPSPACE/poly} = \mathsf{PSPACE/poly}$, established by [Aar06].

The following Figure 13.1 is a informative figure taken from [Aar16, Figure 5.1].

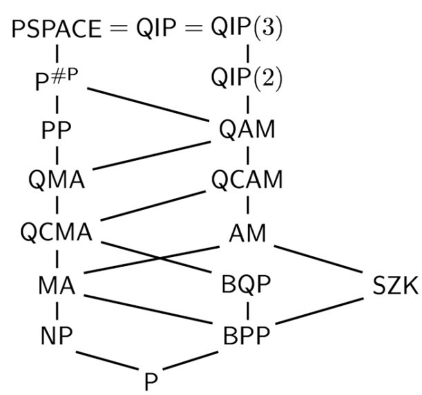

Figure 13.1: Some relevant complexity classes, partially ordered by known inclusions

13.2 Hidden Subgroup Problem and Friends

Xiao: alk about the hidden subgroup problem (HSP) and its variants ([KKNY05, Page 5] contains a good introduction): Xiao!

- HSP is easy on Abelian groups [Kit96]: Use generalized quantum Fourier transform (QFT). But currently there is no known efficient algorithm to solve this problem on non-Abelian groups. Two types of non-Abelian groups on which HSP is hard are symmetric groups and dihedral groups.
- On symmetric groups, the problem is referred to as SHSP. It can be reduced to the problem that given a quantum state of a particular form, try to find the hidden subgroup H. A special case of the latter problem can be further reduced to the DIST problem formalized in [HRT03].

- As argued in [KKNY05], the DIST problem is closed related to the QSCD_{ff} problem, based which [KKNY05] builds a restricted form of PKE. These problem is closed related to the graph isomorphism (GI) problem and the graph automorphism (GA) problem (see this blog article for a good survey of GI).
- On symmetric groups, the problem is referred to as DHSP. It can be reduced to the problem of dihedral coset problem (DCP), which is known to be at least as hard as LWE. In the special case of the group $\mathbb{Z}_N$ where $N = 2^n$, DCP can be reduced to its distinction version called DCSP.

Chapter 14

Proof Systems: Bridging Crypto and Complexity Theory

14.1 PCP Theorem

Theorem 14.1.1: The PCP Theorem
$\mathsf{NP} \subseteq \mathsf{PCP}_{1,\frac{1}{2}}[O(\log n), O(1)]$.

14.1.1 Exponential-Size PCP

Linear PCP. The first step is to construct a *linear PCP* (LPCP) for the (NP complete) language of quadratic equation satisfiability. More concretely, one can show that $\mathsf{QUAD\text{-}EQ} \in \mathsf{LPCP}_{1,\frac{3}{4}}[n^2, O(n+m), 4]$, where n is the number of variables and m is the number of equations. The core idea in this proof is the design of a tensor product. Namely, if the verifier is given a oracle claimed to be a tensor product $x \otimes x$ between the same vector, it should be able to check it.

Exponential-Size PCP for NP. LPCP guarantees that once the verifier gets access to linear oracle, soundness is guaranteed. To build the final PCP by employing the LPCP verifier, we now need to enforce the prover to use a linear oracle. Put it in another way, we need to develop a method for the verifier for linearity testing: if the prover gives a non-linear function as the oracle, the verifier must catch it with good probability. Assume we have such a linearity test, then the PCP can be built in the following way:

- The verifier performs the linearity test [BLR90] to check that the oracle is really a linear function.
- If the last check passes, the verifier can safely assume that the oracle is a real linear function. It then simply runs the LPCP verifier;

The soundness $\varepsilon_{\mathsf{PCP}}$ of this PCP is upper bounded by $\max\{\varepsilon_{\mathsf{LPCP}}, \varepsilon_{\mathsf{LIN}}\}$, where $\varepsilon_{\mathsf{LIN}}$ is the soundness error of the linearity test.

However, the are two caveat when we implement the above idea:

1. Ideally, we want to have a linearity test such that if the oracle is not a linear function, the verifier accepts with probability $< \varepsilon_{\mathsf{LIN}}$. But this is impossible unless the verifier checks the whole true table of the function. For example, we can have a non-linear function that differs with some linear function on only a single input. Thus, to ensure the verifiers' efficiency, we have to tolerate some slackness. The linearity test we will have can only guarantee that: if a function is far (say 10%-far, in terms of hamming distance of the truth table) from any linear

function, the verifier can be fooled with probability $< \varepsilon_{\mathsf{LIN}}$.[1] This introduces some "middle land" to the above soundness analysis: a malicious prover can generate a non-linear function that is only $< 10\%$-far from some linear function. Nevertheless, this middle-land case can be handled by union bound, introducing only a $q \cdot \frac{1}{10}$ additional error term to $\varepsilon_{\mathsf{PCP}}$, where q is the number of the verifier's queries in the whole execution. Rigorously, assuming that there is a linear function $\langle \alpha, \cdot \rangle$ that is 10%-close to the maliciously generated oracle $\widetilde{\pi}$, when we run the LPCP verifier w.r.t. $\widetilde{\pi}$, we have:

$$\begin{aligned}
\Pr\left[V^{\widetilde{\pi}}_{\mathsf{LPCP}}(x) = 1\right] &\le \Pr\left[V^{\widetilde{\pi}}_{\mathsf{LPCP}}(x) = 1 \;\middle|\; \begin{array}{l}\text{All queries land in}\\ \text{good locations}\end{array}\right] + \Pr\left[\begin{array}{l}\text{At least one } \mathsf{LPCP} \text{ query}\\ \text{lands in bad locations}\end{array}\right] \\
&= \Pr\left[V^{\langle \alpha, \cdot \rangle}_{\mathsf{LPCP}}(x) = 1 \;\middle|\; \begin{array}{l}\text{All queries land in}\\ \text{good locations}\end{array}\right] + \Pr\left[\begin{array}{l}\text{At least one } \mathsf{LPCP} \text{ query}\\ \text{lands in bad locations}\end{array}\right] \\
&\le \varepsilon_{\mathsf{LPCP}} + q \cdot \frac{1}{10} \qquad (14.1)
\end{aligned}$$

However, the above bound is not informative if $q \cdot \frac{1}{10}$ is close (or equal) to 1. This can be fixed by repetition: if we repeat each query of the underlying LPCP t times (with fresh randomness), we can drive the term in **??** down to $\varepsilon_{\mathsf{LPCP}} + q \cdot \frac{1}{10^t}$.

2. The second caveat is about **??**. The $\frac{1}{10}$ terms is due to the (ideal-case) fact that the (linear-PCP) verifier's query is uniformly distributed over all the positions. However, in the underlying linear PCP, the verifier's query is not uniformly. But this can be fixed by *self-correction*: every time the V_{LPCP} want to query a position z, it samples a r uniformly at random, and queries both position r and $z + r$. Then it computes $\pi(z) = \pi(r) + \pi(r + z)$ (due to the linearity of the linear PCP π). Now we can safely say that both r and $r + z$ are random queries. However, note that soundness is broken if one of these two queries lands in bad locations, which happens with probability $\frac{2}{10}$ (if $\frac{1}{10}$ of the oracle is bad). Thus, with this self-correction (and the aforementioned repetition), the term in **??** should be $\varepsilon_{\mathsf{LPCP}} + q \cdot \frac{2}{10^t}$.

Remark 14.1.2: On the BLR Linearity Test

The BLR test is very simple: to test the linearity of a function f on $\{0,1\}^n$, just pick $x, y \xleftarrow{\$} \{0,1\}^n$ and check if $f(x) + f(y) = f(x + y)$. All the hard work lies in its soundness analysis. The original [BLR90] paper showed that $\Pr\left[V^f_{\mathsf{LIN}} = 0\right] \ge \min\{\frac{2}{9}, \frac{\delta(f)}{2}\}$, where δf is the fractional hamming distance between f and the closest linear function to it. Their proof used only elementary probability theory argument in an elegant way. Later, relying on tools from Boolean Fourier Analysis, [BCH+95] improved the soundness analysis of BLR test by showing that $\Pr\left[V^f_{\mathsf{LIN}} = 0\right] \ge \delta(f)$. Put it in another way, if we use BLR test with soundness error $\varepsilon_{\mathsf{LIN}}$, we can make sure that the target function is $\varepsilon_{\mathsf{LIN}}$-close to some linear function.

On the Soundness Error. The above analysis says that: for $\widetilde{\pi}$ of a false statement $x \notin \mathcal{L}$,

- if $\widetilde{\pi}$ is $\varepsilon_{\mathsf{LIN}}$-far from any linear function, V will accept (mistakenly) with probability $\le \varepsilon_{\mathsf{LIN}}$ (due to Remark 14.1.2); (note that here we only consider the soundness error of the linearity test. This is because if $\widetilde{\pi}$ manages to pass the linearity test, the LPCP test is not reliable at

[1] For the linearity test we will use, this error is actually the inverse of the "slackness" factor: if a function is $\varepsilon_{\mathsf{LIN}}$-far from linear, the verifier can be fooled with with probability $< \varepsilon_{\mathsf{LIN}}$. See Remark 14.1.2.

all. To the extreme, if $\widetilde{\pi}$ is $\varepsilon_{\mathsf{LIN}}$-far from any linear function, the $V^{\widetilde{\pi}}_{\mathsf{LPCP}}(x)$ may accept with probability 1.)

- if $\widetilde{\pi}$ is $\varepsilon_{\mathsf{LIN}}$-close from some linear function (the linearity test is not reliable), the above analysis tells us that V will accept (mistakenly) with probability $\leq \varepsilon_{\mathsf{LPCP}} + q \cdot 2 \cdot \varepsilon^t_{\mathsf{LIN}}$.

Thus, the soundness error of the above PCP is $\varepsilon_{\mathsf{PCP}} \leq \max\{\varepsilon_{\mathsf{LIN}}, \varepsilon_{\mathsf{LPCP}} + q \cdot 2 \cdot \varepsilon^t_{\mathsf{LIN}}\}$.

On the Complexity. The above analysis is a generic compiler from LPCP to PCP. It shows that

$$\mathsf{LPCP}_{1,\varepsilon_{\mathsf{LPCP}}}[\ell, r, q] \subseteq \mathsf{PCP}_{1,\varepsilon_{\mathsf{PCP}}}[2^\ell, 2\ell + 2qt\ell, 3 + qt], \text{ where } \varepsilon_{\mathsf{PCP}} \leq \max\{\varepsilon_{\mathsf{LIN}}, \varepsilon_{\mathsf{LPCP}} + q \cdot 2 \cdot \varepsilon^t_{\mathsf{LIN}}\}$$

We can set $\varepsilon_{\mathsf{LIN}} = \frac{1}{2}$ and set $t = O(\log q)$ such that $q \cdot 2 \cdot \varepsilon^t_{\mathsf{LIN}}$ is an arbitrarily small constant, e.g. $\frac{1}{100}$. Since we know that $\mathsf{QUAD\text{-}EQ} \in \mathsf{LPCP}_{1,\frac{1}{2}}[\mathsf{poly}(n), \mathsf{poly}(n), O(1)]$, the above parameter setting gives us that:

$$\mathsf{NP} \subseteq \mathsf{PCP}_{1,\frac{1}{2}+\frac{1}{100}}[\exp(n), \mathsf{poly}(n), O(1)].$$

As a historical remark, this exponential size PCP with constant number of queries is the inner PCP in the work [ALM+92, ALM+98].

14.1.2 Polynomial-Size PCP

14.2 PCP of Proximity

Resources

- The first work that formalized PCPP was [BGH+04], where PCPP was defined w.r.t. pair languages. It also contains a discussion about pair languages vs standard languages, and PCP vs PCPP.
- [DK12] contains the same formalism as in [BGH+04]. It indicates that in the [BGH+04] construction, the PCPP proof oracle can be constructed efficiently.
- Section A.5.1 of [GOSV14] also contains a clean formalism of PCPP. It basically summarized the things in [BGH+04] and [DK12].
- [IW14] has a informal description of PCPP w.r.t. standard NP languages. This is the only definition I found that did not use pair languages.

14.3 Interactive Proofs and Arthur-Merlin Games

(A good starting point for this topic is the introduction of [GS86].)

14.3.1 Multi-Prover Interactive Proofs

Chapter 15

Cryptographic Reductions and Impossibility Results

15.1 The [RTV04] Taxonomy

Let us first present the formal definition for cryptographic primitives.

Definition 15.1.1: Cryptographic Primitives [RTV04]

A primitive P is a pair (F_P, R_P), where F_P is a set of functions $f : \{0,1\}^* \mapsto \{0,1\}^*$ and R_P is a relation over pairs (f, M) where $f \in R_P$ and M is a *(possibly inefficient)* Turing machine.

- We say that a function f *implements* P if $f \in F_P$. Additionally, we say that a function f *efficiently implements* P if $f \in F_P$ and f is computable by a PPT machine.
- A machine M P-*breaks* the implementation $f \in F_P$ if the pair $(f, M) \in R_P$. A *secure implementation* of P is a function $f \in F_P$ such that no PPT machine P-breaks f.

We say that a primitive P exists if there exists an *efficient and secure implementation* f of P.

We next define 3 most common reductions and explain the relation among them. In the following, we will use the following two (equivalent) phases alternatively:

- a reduction from Q to P;
- a construction of Q from P.

Both of them mean that the existence of P (the base primitive) implies the existence of Q (the target primitive). (Note that the role of P and Q as presented here is reversed compared to [RTV04]. This is inconsistent with [Yer11, BBF13])

15.1.1 Fully-Black-Box Reductions

Definition 15.1.2 is called "fully-black-box" because it uses the (possibly inefficient) adversary Adv breaking Q in a black-box way to break P. In particular, S must work for any such adversary, even an inefficient one. Actually, most of known constructions in cryptography satisfy this very strong requirement.

Definition 15.1.2: Fully-Black-Box Reductions [RTV04]

There exists a fully-black-box reduction from primitive $Q = (F_Q, R_Q)$ to primitive $P = (F_P, R_P)$, if there exist PPT oracle machines G and S such that:

- **Correctness:** For every implementation $f \in F_P$, $G^f \in F_Q$ (i.e., G^f implements Q);

- **Security:** For every implementation $f \in F_P$ and every (possibly inefficient) machine Adv, if Adv Q-breaks G^f, then $S^{\mathsf{Adv},f}$ P-breaks f.

On Uniformity. Definition 15.1.2 only captures uniform fully-black-box reductions. The reason is the S in the definition is by default a uniform (oracle) Turing machine. However, many cryptographic reductions are actually non-uniform, e.g., [GMR89, ILL89, GMW91, GO94, LPV08, GKP18][1]. To capture non-uniform reductions, a naive attempt is to require S to be a non-uniform machine. However, this does not quite solve the problem because, in non-uniform crypto reductions, the non-uniform advice may depend on the adversary; but the S (even if it is non-uniform) in Definition 15.1.2 is universal for all Adv. There is a paper by Chung et al. [CLMP13] trying to address this issue.

15.1.2 Semi-Black-Box Reductions

Semi-black-box reductions were proposed with the hope to capture reductions that could use the code of the adversary.[2]

Definition 15.1.3: Semi-Black-Box Reductions [RTV04]

There exists a Semi-black-box reduction from primitive $Q = (F_Q, R_Q)$ to primitive $P = (F_P, R_P)$, if there exist PPT oracle machines G such that:

- **Correctness:** For every implementation $f \in F_P$, $G^f \in F_Q$ (i.e., G^f implements Q);
- **Security:** For every implementation $f \in F_P$, if there exists a PPT oracle machine Adv such that Adv^f Q-breaks G^f, then there exits a PPT oracle machine S such that S^f P-breaks f.

Fully-Black-Box vs. Semi-Black-Box. First, note that full-black-box reductions are also semi-black-box ones. We claim it in Lemma 15.1.4, whose proof is left as an exercise (see the proof of Lemma 15.1.7 for an example). They have the following differences:

- In semi-black-box reductions, the security reduction S no longer gets the Q-adversary as an oracle. But note that S can depend on Adv. Indeed, since Adv is PPT now, S can make non-black-box use of it, i.e., use the code of Adv (except for its oracle part f).
- Different from Definition 15.1.2, the Adv in Definition 15.1.3 is only a PPT machine. So it may no longer be able to evaluate the possibly inefficient implementation f. Thus f is given to Adv as an oracle in Definition 15.1.3.

Lemma 15.1.4:

If there exists a full-black-box reduction from Q to P, then there also exists a semi-black-box reduction from Q to P.

[1] Interestingly, the non-uniform proof in [ILL89] was eventually made uniform by [Hås90].

[2] Actually, the notion of semi-black-box does not quite achieve this goal. See the weakly-black-box notion and related comments in [RTV04]).

15.1.3 Relativizing Reductions (and "Two-Oracle" Reductions)

Relativizing reductions are an important type of reduction. They generalize fully-black-box reductions in the sense that every fully-black-box construction is also a relativizing one (Lemma 15.1.7). Thus, impossibility results for relativizing constructions are stronger. Indeed, several works (e.g.,[IR89, Sim98, GKM+00]) proved impossibility results for fully-black-box constructions by ruling out relativizing constructions. There are also impossibility results for fully-black-box reductions *without* ruling out relativizing reductions, e.g., [HR04, Haj18, HY20].[3]

We remark that there are two ways to formalize the existence of primitives relative to some oracle (thus two ways to define relativizing reductions). But Lemma 15.1.7 holds w.r.t. both versions of relativizing reductions.

Definition 15.1.5: Existence relative to Oracle

There are two ways to define existence relative to an oracle:

1. A primitive P is said to exist relative to O if there is an $f \in F_p$ which can be implemented by a PPT oracle machine with oracle O, and there is no PPT oracle machine $\mathsf{Adv}^{\mathsf{O}}$ that P-breaks f.

2. This is identical to the above except that $\mathsf{Adv}^{\mathsf{O}}$ can be computationally-unbounded, but restricted to polynomially many oracle queries.

Definition 15.1.6: Relativizing Reductions [RTV04]

There exists a relativizing reduction from primitive $Q = (F_Q, R_Q)$ to primitive $P = (F_P, R_P)$, if for any oracle $\mathsf{O} : \{0,1\}^* \mapsto \{0,1\}^*$, if P exists relative to O then so does Q. (See Definition 15.1.5.)

Lemma 15.1.7: Fully-Black-Box $\Rightarrow$ Relativizing [RTV04]

If there exists a full-black-box reduction from Q to P, then there also exists a relativizing reduction from Q to P.

Proof. Fix any oracle O w.r.t. which P exists. That is, there is a PPT-computable f such that f^{O} implement P, and no PPT (resp. computationally-unbounded) oracle machine $\mathsf{Adv}^{\mathsf{O}}$ making polynomially-many oracle queries can P-break f^{O}. We remark that the oracle machine Adv could be either PPT or computationally-unbounded (see Definition 15.1.5); as long as it only makes polynomially-many oracle queries to O, this proof will work.

If there is a fully-black-box reduction from Q to P, then it follows from Definition 15.1.2 that there exists PPT oracle machines G and S such that

1. there is a PPT oracle machine G such that $G^{f^{\mathsf{O}}}$ implements Q;

2. if there is a (potentially inefficient) Adv Q-breaks $G^{f^{\mathsf{O}}}$, then $S^{\mathsf{Adv}, f^{\mathsf{O}}}$ P-breaks f^{O}.

To finish this proof, we only need to show that there is no PPT (resp. computationally-unbounded) oracle machine $\widetilde{\mathsf{Adv}}^{\mathsf{O}}$ that Q-breaks $G^{f^{\mathsf{O}}}$ while making only polynomially many queries to O.

[3] This is because these works use the so-called "two-oracle" technique, which does not necessarily imply relativizing separation. It seems that two-oracle separation and relativizing separation are in comparable (but I haven't thought about this formally), though both of them imply fully-black-box separation, which is cryptographers' real concern.

Assume for contradiction that there exits such an $\widetilde{\mathsf{Adv}}^{\mathsf{O}}$. It then follows from Item 2, by treating $\widetilde{\mathsf{Adv}}^{\mathsf{O}}$ as the Adv there, that $S^{\widetilde{\mathsf{Adv}}^{\mathsf{O}}, f^{\mathsf{O}}}$ P-breaks f^{O}.

Remark 15.1.1. *We emphasis that the $\widetilde{\mathsf{Adv}}^{\mathsf{O}}$ can be treated as the Adv in Item 2 because the Adv in Definition 15.1.2 could be computationally-unbounded. In contrast, this is not true for semi-black-box reduction (Definition 15.1.3). Therefore, the current proof cannot be used to show that semi-black-box constructions are also relativizing constructions.*

Now, observe that the machine $S^{\widetilde{\mathsf{Adv}}^{\mathsf{O}}, f^{\mathsf{O}}}$ can be interpreted as an oracle machine $\widetilde{S}^{\mathsf{O}}$:

- $\widetilde{S}^{\mathsf{O}}$ execute the code of $S^{\widetilde{\mathsf{Adv}}^{(\cdot)}, f^{(\cdot)}}$ internally. It forward the oracle queries made by $\widetilde{\mathsf{Adv}}^{(\cdot)}$ and $f^{(\cdot)}$ to its oracle O.

Also, note that if $\widetilde{\mathsf{Adv}}^{\mathsf{O}}$ is PPT (resp. conputationally-unbounded) making polynomially-many queries to O, then $\widetilde{S}^{\mathsf{O}}$ is PPT (resp. conputationally-unbounded) making polynomially-many queries to O. This contradicts our assumption that no PPT (resp. computationally-unbounded) oracle machine $\mathsf{Adv}^{\mathsf{O}}$ can P-break f^{O} while making only polynomially-many oracle queries. ■

On the Efficiency of the Relativizing Adversary. Remark 15.1.1 explains why Semi-BB reduction may not be a relativizing one. This is why the FBB impossibility in [IR89] is *unconditional*, but their Semi-BB impossibility is conditioned on $\mathbf{P} \neq \mathsf{NP}$.[4]

Actually, the unconditional FBB impossibility in [IR89] did not rely on the fact that Lemma 15.1.7 holds w.r.t. inefficient relativizing adversaries. Their instead took the following approach: they first proved their Semi-BB impossibility, which ensures that the adversary is efficient by assuming $\mathbf{P} = \mathsf{NP}$. Then, they observed that their proof techniques relativizes. Therefore, the same results holds w.r.t. the new oracle $\mathsf{O}' = (\mathsf{O}, \mathsf{PSPACE})$, where O is the original oracle they used in the Semi-BB impossibility proof. Now, the PSPACE oracle embedded in O' could take care of all the inefficient operations for the adversary. Thus, they reached the same contradiction using only efficient adversaries, in the oracle world where the oracle is O'.

Some Comments on Reductions

Another paradigm appeared in [GGKT05] (the merged version of [GT00, GGK03]). Their approach can be demonstrated by their separation of "efficient" PRG from TDP. To show that, they argue in the following way: Given a length-preserving random oracle, if an "efficient" PRG can be constructed, then $\mathbf{P} \neq \mathsf{NP}$. They argued that such a result ruled out the **weak-black-box** reduction from "efficient" PRG to TDP, which is a stronger impossibility result than fully/semi black-box separation. I have some questions:

- How to argue formally that their claim rules out the weak black-box separation?
- Is it true that they even rule out some reduction that is more general than weak black-box reduction. Is there a formal definition/name for this more-general reduction?
- Is it true that Barak's non-black-box ZK uses a weak-black-box reduction, but not a semi-black-box reduction?

[4] It is also worth noting that [RTV04] managed to remove this computational assumption for the semi-black-box case of [IR89].

15.2 Polynomial-Time Reductions with Expected Polynomial-Time Adversaries

The Problem. To prove the security of cryptographic objects, we usually need to conduct polynomial time reductions. For example, consider a primitive A that is constructed (solely) from another primitive B which is secure by assumption (for concreteness, think of A as a commitment scheme and B as an one-way permutation). To prove that A achieves some desired security property, the canonical approach is to assume, for the sake of contradiction, that there exists a probabilistic polynomial-time (PPT) adversary Adv, which is able to break the target security property of A. Then, the proof is done if one can show an efficient way (i.e. the "reduction") to make use of Adv to break the security of B.

Everything is good if the security of B holds against the class of adversaries for which we try to prove the security of A. For example, assume that B is secure against all PPT adversaries. Then the security of A against all PPT adversaries will be established once we can finish the reduction in PPT. This type of arguments extends to other class of adversaries. For example, similar results hold for the class of sub-exponential adversaries, with the reduction being sub-exponential time.

But things become a little bit tricky if we want to prove the security of A against a class of adversaries which is stronger than the ones that are ruled out by the security of B. This would not be a reasonable concern if the power of the two classes of adversaries differs too much. Indeed, no one will hope to construct sub-exponentially secure commitments from one-way permutations that are only polynomially secure. However, this difference on power could sometimes be so small that we have to pay attention. One such setting is:

> B is secure against PPT adversaries, but we want to prove the security of A against *expected* PPT adversaries.

The above setting appeared in the literature quite often, e.g., Claim 3 in [GK96], Lemma 4.3 in [BJY97], and Proposition 3.3.30 in Feige's thesis [Fei90][5].

The trick is to employ an interesting combination of averaging argument and Markov's inequality. Though the papers mentioned above are already very well-written with enough details, there are still several steps that may not be straightforward to a beginner. Thus, I decided to take a note here to present a thorough derivation.

The Solution: truncating the execution. To illustrate the core of the this technique, I will show the following lemma about OWFs (though all the aforementioned papers are about zero-knowledge). One can easily extend this argument to proper contexts as he/she needs.

Lemma 15.2.1:
Assume f is an OWF against PPT adversaries. Then it is also an OWF against expected PPT adversaries.

Proof. In this proof, I will gloss over the details that can be inferred from the context easily, such as the length of the pre-images/images of f.

Assume, for the sake of contradiction, that there is an expected PPT machine Adv that breaks the one-wayness of f. We will build a machine Adv' that runs in strictly polynomial time and (still) breaks the one-wayness of f.

[5] Noticeably, [Fei90] devoted the whole Chapter 3 to this issue, presenting a beautiful and thorough discussion.

Let λ denote the security parameter. W.l.o.g., assume that the expected running time of Adv is the polynomial $T(\lambda)$. It breaks the one-wayness of f, which means that there exist a polynomial $P(\lambda)$ such that for infinitely many $\lambda \in \mathbb{N}$, Adv inverts f with probability at least $\frac{1}{P(\lambda)}$. In the remaining part of this section, I will drop λ from $T(\lambda)$ and $P(\lambda)$ to make the presentation succinct.

The machine Adv' is constructed by "truncating" the executions of Adv that go beyond $2TP$ steps. In the following, we argue that Adv' also breaks the one-wayness of f.

First, it follows from Markov's inequality that the truncated executions count for only a small portion. More formally, let random variable X denote the running time of Adv. According Markov's inequality, we have:

$$\Pr[X \geq 2TP] \leq \frac{1}{2P}.$$

Then, by an averaging argument, one can show that in the "un-truncated" executions, Adv (the de facto Adv') can still invert f with probability at least $\frac{1}{2P}$. Formally, let the $\mathsf{Win}(\lambda)$ denote the even that Adv wins in the security game for the one-wayness of f, with security parameter set to λ. We then have:

$$\Pr[\mathsf{Win}] = \Pr\left[\mathsf{Win}|X < 2TP\right] \cdot \Pr\left[X < 2TP\right] + \Pr\left[\mathsf{Win}|X \geq 2TP\right] \cdot \Pr\left[X \geq 2TP\right]$$

We now prove that the above equation implies that

$$\Pr\left[\mathsf{Win}|X < 2TP\right] \geq \frac{1}{2P}. \tag{15.1}$$

Assume for contradiction that $\Pr\left[\mathsf{Win}|X < 2TP\right] < \frac{1}{2P}$. Continuing the above equation with this assumption, we have:

$$\begin{aligned}
\frac{1}{P} = \Pr[\mathsf{Win}] &< \frac{1}{2P} \cdot \Pr\left[X < 2TP\right] + \Pr\left[\mathsf{Win}|X \geq 2TP\right] \cdot \frac{1}{2P} \\
&\leq \frac{1}{2P} \cdot 1 + 1 \cdot \frac{1}{2P} \\
&= \frac{1}{P},
\end{aligned}$$

which implies a contradiction as it says $\frac{1}{P} < \frac{1}{P}$. Thus, Equation (15.1) holds. This finishes our proof as $\Pr\left[\mathsf{Win}|X < 2TP\right]$ is exactly the probability that Adv' wins in the security game. ■

When to NOT use the truncating argument. I want to highlight a restriction of the above "truncating" argument: the winning probability of Adv will drop by a constant fraction. In the above example, the winning probability of Adv is $\geq \frac{1}{P}$, but the winning probability of Adv' can only be shown to be $\geq \frac{1}{2P}$. While this is usually not a problem in crypto reductions, this does restrict its applicability in analyses for ZK simulators or knowledge extractors.

For example, to prove ZK property, we usually construct a simulator Sim that runs in expected polynomial time, and outputs a simulated view indistinguishable from (in other words, negligibly-close to) a real one. One may wondering if it is possible to use the above truncating argument to construct a strictly-polynomial time Sim' that is just as good as Sim. But this does not work because the output of Sim' will not be negligibly-close to that of Sim (due to the constant fraction drop). Therefore, the output of Sim' will not be negligibly-close to the real view anymore.[6] It is

[6] Indeed, this limitation is inherent. [BL02] shows that constant-round zero-knowledge protocols admitting strictly-polynomial-time black-box simulation are possible only for languages in BPP.

also worth noting that this truncating argument can be used in the ε-zero-knowledge setting—ε-ZK with expected PPT/QPT simulation are equivalent to those with strictly PPT/QPT simulation. This is because, for ε simulation, we have the freedom to set the proper bound when using Markov inequality to matching the targeted ε, which is an inverse polynomial on the security parameter.

Remark 15.2.1. *Actually, [BL02] provides a different explanation for why it does work to truncate the expected PPT simulator. That argument take specific constructions of simulators as an example, thus not general (But that argument suffices to show that one can not take an* arbitrary *(but effective) expected PPT simulator, truncate it, and then expect it to work). The explanation given above does not make any assumption on how the given expected PPT simulator works. It shows better why it is hard to use Markov inequality to make the truncating argument work.*

Some Afterthoughts. The above discussion seems to give us more confidence about the thought that "expected polynomial time" is indeed a reasonable relaxation. This is good news due to the following fact—expected polynomial time is actually inherent in all the fully-black-box reductions (in the terminology of [RTV04]) for zero-knowledge property or arguments/proofs of knowledge, if one insists on constant-round constructions from standard assumptions [BL02].

However, the following another-side view may stop you from celebrating the good news. The above argument seems to say that allowing the adversary to run in expected polynomial time does not make much difference. For example, Lemma 15.2.1 essentially says that requiring one-wayness to hold against all expected PPT adversaries would eventually result in the same definition of OWFs as the standard one. If so, then why are cryptographers trying so hard to distinguish between the strict polynomial time and the expected one?

The short answer is—expected polynomial-time simulation/extraction is not closed under composition. Elaborating on this point could require another long article. For those who are interested, see the introduction section of the insightful work of Barak and Lindell [BL02] and the references there.

Chapter 16

Miscellaneous

16.1 Some Superfluous Notes on Negligible Functions

16.1.1 On the Order of Quantifiers

From Katz-Lindell Xiao: citation: Xiao!

Definition 16.1.1. A function $f : \mathbb{N} \to \mathbb{R}_{\geq 0}$ is negligible if

$$\forall p(\cdot), \exists N \in \mathbb{N}, \forall n > N, f(n) < \frac{1}{p(n)}.$$

$\diamondsuit$

From Goldreich Xiao: citation: Xiao!

Definition 16.1.2. A function $f : \mathbb{N} \to \mathbb{R}_{\geq 0}$ is negligible if for every positive polynomial $p(\cdot)$ and all sufficiently large n's, it holds that $f(n) < \frac{1}{p(n)}$. $\diamondsuit$

As a person who constantly overthinks, I was lost in the following two interpretations for Definition 16.1.2 when I saw it for the first time:

1. $\forall p(\cdot), \exists N \in \mathbb{N}, \forall n > N, f(n) < \frac{1}{p(n)}$.
2. $\exists N \in \mathbb{N}, \forall p(\cdot), \forall n > N, f(n) < \frac{1}{p(n)}$.

The first interpretation comes by replacing "all sufficiently large n" with "$\exists N \in \mathbb{N}$ s.t. $\forall n > N$" literally in Definition 16.1.2. It is identical to Definition 16.1.1, and should be the correct interpretation.

At least at the first glance, the second one also seems to be a possible interpretation. I think the reason is rooted in the ambiguity of the phrase "all sufficiently large n". It hides the quantifiers! This is also why I feel uncomfortable when people do not use rigorous mathematical notations when they could. (But this is anyway a personal opinion. It is arguably true that Definition Definition 16.1.2 does not cause a similar confusion as I have to most "regular" readers.)

However, the second interpretation is erroneous as it results in an unsatisfiable definition (except for $f(n) = 0$). The reason is that once N is pre-fixed, no matter now fast $f(\cdot)$ approaches 0, you can always find a large enough polynomial $p(\cdot)$ as such that $f(N) > \frac{1}{p(N)}$. Therefore, in the definition of negligibility, the "threshold" N must depend on $p(\cdot)$.

The above discussion leads to an interesting question:

- Is it problematic if we don't have an universal threshold N? Negligible functions are usually used to guarantee certain security level. Will there be any problems if its behavior depends on adversaries' (polynomial) running time?

A similar problem appears for the order that we quantify the negligble function and the adversary:

1. $\exists \mathsf{negl}(\cdot), \forall \mathsf{Adv}$ ...
2. $\forall \mathsf{Adv}, \exists \mathsf{negl}(\cdot)$, ...

[Bellare97] showed that the above two versions are actually equivalent. But the question I asked above if about where we put the threshold N. And we have to settle on the first interpretation as the second one is wrong. Xiao: But what does it mean in application when we use neglgible functions to capture security level? Xiao!

16.1.2 On the Threshold

Katz-Lindell's definition for OWFs:

- for all PPT Adv, there exists a negligible function negl such that:

$$\Pr[\mathsf{Win}(1^n)] \le \mathsf{negl}(n).$$

They didn't quantify the security parameter n. But it is clear from the context that they mean the following:

- for all PPT Adv, there exists a negligible function negl such that for all $n \in \mathbb{N}$, it holds that:

$$\Pr[\mathsf{Win}(1^n)] \le \mathsf{negl}(n).$$

Again, I have the following overthinker's interpretation:

- for all PPT Adv, there exists a negligible function negl and an $N \ge \mathbb{N}$ such that for all $n > N$, it holds that:

$$\Pr[\mathsf{Win}(1^n)] \le \mathsf{negl}(n).$$

But the above two versions should be equivalent due to the property of negligible functions.

However, an interesting point is: the above claim does not hold any more once the $\mathsf{negl}(\cdot)$ gets instantiated by some concrete negligible function. In this setting, the second one is the correct definition. Meanwhile, the first interpretation would be unsatisfiable. Indeed, the underlying OWFs are from some computational assumptions. For every n, there always exist some adversary who runs in (very large but still) polynomial time and is able to breach the underlying assumption. The second interpretation resolves the above problem by inserting the threshold N, which essentially allow the concrete negligible function to depend on the target adversary.

The second interpretation actually appeared in Bellare's paper [Bellare97] with a notation called "eventually smaller than" to capture it.

This problem will appear wherever we want to talk about concrete functions. For example, if we want to say two ensembles are $\delta(n)$-computationally-close, where $\delta(\cdot)$ is some concrete function (e.g., $\delta(n) = \frac{1}{n^3}$), we have to capture it as:

- For all PPT distinguisher Adv, there exists an $N \in \mathbb{N}$ such that for all $n > N$, $\Pr[\mathsf{Adv} \text{ wins }] \le \delta(n)$.

If we try to cast it in the form of version 2 above, i.e.

- For all PPT distinguisher Adv and for all $n \in \mathbb{N}$, $\Pr[\mathsf{Adv} \text{ wins }] \le \delta(n)$.

This is obviously unsatisfiable.

16.1.3 On the Summation of Negligible Functions

Somewhat surprisingly, the sum of polynomially many negligible functions may not be negligible. Here is a counter example due to Bellare [Remark 3.3, Bellare97].

Consider the following summation of polynomially many *different* negligible functions:

$$f(\lambda) = \sum_{i=1}^{\lambda} \varepsilon_i(\lambda), \text{ where } \varepsilon_i(\lambda) := \begin{cases} 1 & \lambda \le i \\ 0 & \lambda > i \end{cases}.$$

Obviously, for any fixed i, $\varepsilon_i(\lambda)$ is negligible (on λ). But $f(\lambda)$ is not negligible (as a function on λ).

Though this issue is often ignored, it is usually not a real technical issue since negligible functions in the summation are usually upper bounded by a constant number of negligible functions because the number of assumptions being used is usually a constant.

Acknowledgments

I want to express my gratitude to a group of super knowledgeable and patient people, who help me a lot to build a solid understanding of the materials herein. They are Nai-Hui Chia, Kai-Min Chung, Mohammad Hajiabadi, Giulio Malavolta, Omkant Pandey, and Takashi Yamakawa.

Bibliography

[Aar04] Scott Aaronson. Limitations of quantum advice and one-way communication. In *19th Annual IEEE Conference on Computational Complexity (CCC 2004), 21-24 June 2004, Amherst, MA, USA*, pages 320–332. IEEE Computer Society, 2004. doi:10.1109/CCC.2004.1313854. 136

[Aar05] Scott Aaronson. Quantum computing, postselection, and probabilistic polynomial-time. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 461(2063):3473–3482, 2005. 136

[Aar06] Scott Aaronson. Qma/qpoly \subseteq pspace/poly: De-merlinizing quantum protocols. In *21st Annual IEEE Conference on Computational Complexity (CCC 2006), 16-20 July 2006, Prague, Czech Republic*, pages 261–273. IEEE Computer Society, 2006. doi:10.1109/CCC.2006.36. 136

[Aar16] Scott Aaronson. The complexity of quantum states and transformations: From quantum money to black holes. *CoRR*, abs/1607.05256, 2016. URL: http://arxiv.org/abs/1607.05256, arXiv:1607.05256. 135, 136

[AB09] Sanjeev Arora and Boaz Barak. *Computational complexity: a modern approach*. Cambridge University Press, 2009. 74, 77, 117, 128, 129, 172

[ABOEM17] Dorit Aharonov, Michael Ben-Or, Elad Eban, and Urmila Mahadev. Interactive proofs for quantum computations, 2017. arXiv:1704.04487. 72

[AD10] Scott Aaronson and Andrew Drucker. A full characterization of quantum advice. In Leonard J. Schulman, editor, *42nd Annual ACM Symposium on Theory of Computing*, pages 131–140, Cambridge, MA, USA, June 5–8, 2010. ACM Press. doi:10.1145/1806689.1806709. 136

[ADRS15] Divesh Aggarwal, Daniel Dadush, Oded Regev, and Noah Stephens-Davidowitz. Solving the shortest vector problem in 2^n time using discrete Gaussian sampling: Extended abstract. In Rocco A. Servedio and Ronitt Rubinfeld, editors, *47th Annual ACM Symposium on Theory of Computing*, pages 733–742, Portland, OR, USA, June 14–17, 2015. ACM Press. doi:10.1145/2746539.2746606. 104

[AHU19] Andris Ambainis, Mike Hamburg, and Dominique Unruh. Quantum security proofs using semi-classical oracles. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part II*, volume 11693 of *Lecture Notes in Computer Science*, pages 269–295, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Heidelberg, Germany. doi:10.1007/978-3-030-26951-7_10. 54

[Ajt96] Miklós Ajtai. Generating hard instances of lattice problems (extended abstract). In *28th Annual ACM Symposium on Theory of Computing*, pages 99–108, Philadephia, PA, USA, May 22–24, 1996. ACM Press. doi:10.1145/237814.237838. 105

[Ajt98] Miklós Ajtai. The shortest vector problem in L2 is NP-hard for randomized reductions (extended abstract). In *30th Annual ACM Symposium on Theory of Computing*, pages 10–19, Dallas, TX, USA, May 23–26, 1998. ACM Press. doi:10.1145/276698.276705. 104

[AK07] Scott Aaronson and Greg Kuperberg. Quantum versus classical proofs and advice. In *22nd Annual IEEE Conference on Computational Complexity (CCC 2007), 13-16 June 2007, San Diego, California, USA*, pages 115–128. IEEE Computer Society, 2007. doi:10.1109/CCC.2007.27. 135, 136

[AKS01] Miklós Ajtai, Ravi Kumar, and D. Sivakumar. A sieve algorithm for the shortest lattice vector problem. In *33rd Annual ACM Symposium on Theory of Computing*, pages 601–610, Crete, Greece, July 6–8, 2001. ACM Press. doi:10.1145/380752.380857. 104

[ALM+92] Sanjeev Arora, Carsten Lund, Rajeev Motwani, Madhu Sudan, and Mario Szegedy. Proof verification and hardness of approximation problems. In *33rd Annual Symposium on Foundations of Computer Science*, pages 14–23, Pittsburgh, PA, USA, October 24–27, 1992. IEEE Computer Society Press. doi:10.1109/SFCS.1992.267823. 140

[ALM+98] Sanjeev Arora, Carsten Lund, Rajeev Motwani, Madhu Sudan, and Mario Szegedy. Proof verification and the hardness of approximation problems. *Journal of the ACM (JACM)*, 45(3):501–555, 1998. 140

[AP13] Jacob Alperin-Sheriff and Chris Peikert. Practical bootstrapping in quasilinear time. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013, Part I*, volume 8042 of *Lecture Notes in Computer Science*, pages 1–20, Santa Barbara, CA, USA, August 18–22, 2013. Springer, Heidelberg, Germany. doi:10.1007/978-3-642-40041-4_1. 109

[AR04] Dorit Aharonov and Oded Regev. Lattice problems in NP cap coNP. In *45th Annual Symposium on Foundations of Computer Science*, pages 362–371, Rome, Italy, October 17–19, 2004. IEEE Computer Society Press. doi:10.1109/FOCS.2004.35. 104

[AV19] Prabhanjan Ananth and Vinod Vaikuntanathan. Optimal bounded-collusion secure functional encryption. In Dennis Hofheinz and Alon Rosen, editors, *TCC 2019: 17th Theory of Cryptography Conference, Part I*, volume 11891 of *Lecture Notes in Computer Science*, pages 174–198, Nuremberg, Germany, December 1–5, 2019. Springer, Heidelberg, Germany. doi:10.1007/978-3-030-36030-6_8. 123

[Axl15] Sheldon Axler. *Linear algebra done right*. Springer, 2015. 41, 44

[BBF13] Paul Baecher, Christina Brzuska, and Marc Fischlin. Notions of black-box reductions, revisited. In Kazue Sako and Palash Sarkar, editors, *Advances in Cryptology – ASIACRYPT 2013, Part I*, volume 8269 of *Lecture Notes in Computer Science*, pages 296–315, Bengalore, India, December 1–5, 2013. Springer, Heidelberg, Germany. doi:10.1007/978-3-642-42033-7_16. 141

[BCH+95] Mihir Bellare, Don Coppersmith, Johan Håstad, Marcos A. Kiwi, and Madhu Sudan. Linearity testing in characteristic two. In *36th Annual Symposium on Foundations of Computer Science*, pages 432–441, Milwaukee, Wisconsin, October 23–25, 1995. IEEE Computer Society Press. doi:10.1109/SFCS.1995.492574. 139

[BCM+18] Zvika Brakerski, Paul Christiano, Urmila Mahadev, Umesh V. Vazirani, and Thomas Vidick. A cryptographic test of quantumness and certifiable randomness from a single quantum device. In Mikkel Thorup, editor, *59th Annual Symposium on Foundations of Computer Science*, pages 320–331, Paris, France, October 7–9, 2018. IEEE Computer Society Press. doi:10.1109/FOCS.2018.00038. 106, 172

[BDD+02] Michael J Bremner, Christopher M Dawson, Jennifer L Dodd, Alexei Gilchrist, Aram W Harrow, Duncan Mortimer, Michael A Nielsen, and Tobias J Osborne. Practical scheme for quantum computation with any two-qubit entangling gate. *Physical review letters*, 89(24):247902, 2002. 67

[BF10] Niek J. Bouman and Serge Fehr. Sampling in a quantum population, and applications. In Tal Rabin, editor, *Advances in Cryptology – CRYPTO 2010*, volume 6223 of *Lecture Notes in Computer Science*, pages 724–741, Santa Barbara, CA, USA, August 15–19, 2010. Springer, Heidelberg, Germany. doi:10.1007/978-3-642-14623-7_39. 71, 86, 171

[BFNW93] Lźszló Babai, Lance Fortnow, Noam Nisan, and Avi Wigderson. Bpp has subexponential time simulations unlessexptime has publishable proofs. *Computational Complexity*, 3(4):307–318, 1993. 2

[BFO08] Alexandra Boldyreva, Serge Fehr, and Adam O'Neill. On notions of security for deterministic encryption, and efficient constructions without random oracles. In David Wagner, editor, *Advances in Cryptology – CRYPTO 2008*, volume 5157 of *Lecture Notes in Computer Science*, pages 335–359, Santa Barbara, CA, USA, August 17–21, 2008. Springer, Heidelberg, Germany. doi:10.1007/978-3-540-85174-5_19. 99, 100, 172

[BGH+04] Eli Ben-Sasson, Oded Goldreich, Prahladh Harsha, Madhu Sudan, and Salil P. Vadhan. Robust PCPs of proximity, shorter PCPs and applications to coding. In László Babai, editor, *36th Annual ACM Symposium on Theory of Computing*, pages 1–10, Chicago, IL, USA, June 13–16, 2004. ACM Press. doi:10.1145/1007352.1007361. 140

[Bha96] R. Bhatia. *Matrix Analysis*. Graduate Texts in Mathematics. Springer New York, 1996. URL: https://books.google.com/books?id=F4hRy1F1M6QC. 65

[BJY97] Mihir Bellare, Markus Jakobsson, and Moti Yung. Round-optimal zero-knowledge arguments based on any one-way function. In Walter Fumy, editor, *Advances in Cryptology – EUROCRYPT'97*, volume 1233 of *Lecture Notes in Computer Science*, pages 280–305, Konstanz, Germany, May 11–15, 1997. Springer, Heidelberg, Germany. doi:10.1007/3-540-69053-0_20. 78, 145

[BL02] Boaz Barak and Yehuda Lindell. Strict polynomial-time in simulation and extraction. In *34th Annual ACM Symposium on Theory of Computing*, pages 484–493, Montréal, Québec, Canada, May 19–21, 2002. ACM Press. doi:10.1145/509907.509979. 146, 147

[BLMZ18] James Bartusek, Tancrède Lepoint, Fermi Ma, and Mark Zhandry. New techniques for obfuscating conjunctions. Technical report, Cryptology ePrint Archive, Report 2018/936, 2018. https://eprint. iacr. org ..., 2018. 99

[BLR90] Manuel Blum, Michael Luby, and Ronitt Rubinfeld. Self-testing/correcting with applications to numerical problems. In *22nd Annual ACM Symposium on Theory of Computing*, pages 73–83, Baltimore, MD, USA, May 14–16, 1990. ACM Press. doi:10.1145/100216.100225. 138, 139

[BPR12] Abhishek Banerjee, Chris Peikert, and Alon Rosen. Pseudorandom functions and lattices. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 719–737, Cambridge, UK, April 15–19, 2012. Springer, Heidelberg, Germany. doi:10.1007/978-3-642-29011-4_42. 107

[BR93] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In Dorothy E. Denning, Raymond Pyle, Ravi Ganesan, Ravi S. Sandhu, and Victoria Ashby, editors, *ACM CCS 93: 1st Conference on Computer and Communications Security*, pages 62–73, Fairfax, Virginia, USA, November 3–5, 1993. ACM Press. doi:10.1145/168588.168596. 95

[Bra12] Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical GapSVP. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology – CRYPTO 2012*, volume 7417 of *Lecture Notes in Computer Science*, pages 868–886, Santa Barbara, CA, USA, August 19–23, 2012. Springer, Heidelberg, Germany. doi:10.1007/978-3-642-32009-5_50. 106

[BV93] Ethan Bernstein and Umesh V. Vazirani. Quantum complexity theory. In *25th Annual ACM Symposium on Theory of Computing*, pages 11–20, San Diego, CA, USA, May 16–18, 1993. ACM Press. doi:10.1145/167088.167097. 42

[BY22] Zvika Brakerski and Henry Yuen. Quantum garbled circuits. In *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing*, pages 804–817, 2022. 66

[BZ13] Dan Boneh and Mark Zhandry. Secure signatures and chosen ciphertext security in a quantum computing world. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013, Part II*, volume 8043 of *Lecture Notes in Computer Science*, pages 361–379, Santa Barbara, CA, USA, August 18–22, 2013. Springer, Heidelberg, Germany. doi:10.1007/978-3-642-40084-1_21. 63

[CCY20] Nai-Hui Chia, Kai-Min Chung, and Takashi Yamakawa. A black-box approach to post-quantum zero-knowledge in constant round. *CoRR*, abs/2011.02670, 2020. URL: https://arxiv.org/abs/2011.02670, arXiv:2011.02670. 65

[CFM21] Geoffroy Couteau, Pooya Farshim, and Mohammad Mahmoody. Black-box uselessness: Composing separations in cryptography. In James R. Lee, editor, *ITCS 2021: 12th Innovations in Theoretical Computer Science Conference*, volume 185, pages 47:1–47:20, Virtual Conference, January 6–8, 2021. LIPIcs. doi:10.4230/LIPIcs.ITCS.2021.47. 16

[CGGM00] Ran Canetti, Oded Goldreich, Shafi Goldwasser, and Silvio Micali. Resettable zero-knowledge (extended abstract). In *32nd Annual ACM Symposium on Theory of Computing*, pages 235–244, Portland, OR, USA, May 21–23, 2000. ACM Press. doi:10.1145/335305.335334. 2

[CKKS17] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yong Soo Song. Homomorphic encryption for arithmetic of approximate numbers. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology – ASIACRYPT 2017, Part I*, volume 10624 of *Lecture Notes in Computer Science*, pages 409–437, Hong Kong, China, December 3–7, 2017. Springer, Heidelberg, Germany. doi:10.1007/978-3-319-70694-8_15. 34, 110

[CKPR01] Ran Canetti, Joe Kilian, Erez Petrank, and Alon Rosen. Black-box concurrent zero-knowledge requires omega (log n) rounds. In *33rd Annual ACM Symposium on Theory of Computing*, pages 570–579, Crete, Greece, July 6–8, 2001. ACM Press. doi:10.1145/380752.380852. 1

[CLMP12] Kai-Min Chung, Edward Lui, Mohammad Mahmoody, and Rafael Pass. Unprovable security of two-message zero knowledge. Technical report, Department of Computer Science, Cornell University, Ithaca, NY, 2012. 78

[CLMP13] Kai-Min Chung, Huijia Lin, Mohammad Mahmoody, and Rafael Pass. On the power of nonuniformity in proofs of security. In Robert D. Kleinberg, editor, *ITCS 2013: 4th Innovations in Theoretical Computer Science*, pages 389–400, Berkeley, CA, USA, January 9–12, 2013. Association for Computing Machinery. doi:10.1145/2422436.2422480. 142

[CLZ22] Yilei Chen, Qipeng Liu, and Mark Zhandry. Quantum algorithms for variants of average-case lattice problems via filtering. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 372–401. Springer, 2022. 115

[CMSZ21] Alessandro Chiesa, Fermi Ma, Nicholas Spooner, and Mark Zhandry. Post-quantum succinct arguments. *Electron. Colloquium Comput. Complex.*, 28:38, 2021. URL: https://eccc.weizmann.ac.il/report/2021/038. 65

[Coo71] Stephen A Cook. The complexity of theorem-proving procedures. In *Proceedings of the third annual ACM symposium on Theory of computing*, pages 151–158. ACM, 1971. 129

[Coo73] Stephen A Cook. A hierarchy for nondeterministic time complexity. *Journal of Computer and System Sciences*, 7(4):343–353, 1973. 127

[CW79] J Lawrence Carter and Mark N Wegman. Universal classes of hash functions. *Journal of computer and system sciences*, 18(2):143–154, 1979. 97

[Dam88] Ivan Damgård. Collision free hash functions and public key signature schemes. In David Chaum and Wyn L. Price, editors, *Advances in Cryptology – EUROCRYPT'87*, volume 304 of *Lecture Notes in Computer Science*, pages 203–216, Amsterdam, The Netherlands, April 13–15, 1988. Springer, Heidelberg, Germany. doi:10.1007/3-540-39118-5_19. 95

[Deu85] David Deutsch. Quantum theory, the church–turing principle and the universal quantum computer. *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences*, 400(1818):97–117, 1985. 42

[DF04] David Steven Dummit and Richard M Foote. *Abstract algebra*, volume 3. Wiley Hoboken, 2004. 31, 44

[DGH+19] Nico Döttling, Sanjam Garg, Mohammad Hajiabadi, Daniel Masny, and Daniel Wichs. Two-round oblivious transfer from cdh or lpn. Cryptology ePrint Archive, Report 2019/414, 2019. https://eprint.iacr.org/2019/414. 84

[DGI+19] Nico Döttling, Sanjam Garg, Yuval Ishai, Giulio Malavolta, Tamer Mour, and Rafail Ostrovsky. Trapdoor hash functions and their applications. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part III*, volume 11694 of *Lecture Notes in Computer Science*, pages 3–32, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Heidelberg, Germany. doi:10.1007/978-3-030-26954-8_1. 107

[DJ92] David Deutsch and Richard Jozsa. Rapid solution of problems by quantum computation. *Proceedings of the Royal Society of London. Series A: Mathematical and Physical Sciences*, 439(1907):553–558, 1992. 42

[DJ01] Ivan Damgård and Mats Jurik. A generalisation, a simplification and some applications of Paillier's probabilistic public-key system. In Kwangjo Kim, editor, *PKC 2001: 4th International Workshop on Theory and Practice in Public Key Cryptography*, volume 1992 of *Lecture Notes in Computer Science*, pages 119–136, Cheju Island, South Korea, February 13–15, 2001. Springer, Heidelberg, Germany. doi:10.1007/3-540-44586-2_9. 93

[DK12] Dana Dachman-Soled and Yael Tauman Kalai. Securing circuits against constant-rate tampering. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology – CRYPTO 2012*, volume 7417 of *Lecture Notes in Computer Science*, pages 533–551, Santa Barbara, CA, USA, August 19–23, 2012. Springer, Heidelberg, Germany. doi:10.1007/978-3-642-32009-5_31. 140

[DORS03] Yevgeniy Dodis, Rafail Ostrovsky, Leonid Reyzin, and Adam Smith. Fuzzy extractors: How to generate strong keys from biometrics and other noisy data. Cryptology ePrint Archive, Report 2003/235, 2003. https://eprint.iacr.org/2003/235. 99, 100, 101, 102

[DRS04] Yevgeniy Dodis, Leonid Reyzin, and Adam Smith. Fuzzy extractors: How to generate strong keys from biometrics and other noisy data. In Christian Cachin and Jan Camenisch, editors, *Advances in Cryptology – EUROCRYPT 2004*, volume 3027 of *Lecture Notes in Computer Science*, pages 523–540, Interlaken, Switzerland, May 2–6, 2004. Springer, Heidelberg, Germany. doi:10.1007/978-3-540-24676-3_31. 100, 101, 102

[Fei90] Uriel Feige. *Alternative models for zero knowledge interactive proofs*. PhD thesis, Weizmann Institute of Science, Rehovot, Israel, 1990. 145

[FK18] Bill Fefferman and Shelby Kimmel. Quantum vs. classical proofs and subset verification. In Igor Potapov, Paul G. Spirakis, and James Worrell, editors, *43rd International Symposium on Mathematical Foundations of Computer Science, MFCS 2018, August 27-31, 2018, Liverpool, UK*, volume 117 of *LIPIcs*, pages 22:1–22:23. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018. URL: https://doi.org/10.4230/LIPIcs.MFCS.2018.22, doi:10.4230/LIPICS.MFCS.2018.22. 135

[FLS90] Uriel Feige, Dror Lapidot, and Adi Shamir. Multiple non-interactive zero knowledge proofs based on a single random string (extended abstract). In *31st Annual Symposium on Foundations of Computer Science*, pages 308–317, St. Louis, MO, USA, October 22–24, 1990. IEEE Computer Society Press. doi:10.1109/FSCS.1990.89549. 85, 171

[FS87] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *Advances in Cryptology – CRYPTO'86*, volume 263 of *Lecture Notes in Computer Science*, pages 186–194, Santa Barbara, CA, USA, August 1987. Springer, Heidelberg, Germany. doi:10.1007/3-540-47721-7_12. 95

[Gen09] Craig Gentry. Fully homomorphic encryption using ideal lattices. In Michael Mitzenmacher, editor, *41st Annual ACM Symposium on Theory of Computing*, pages 169–178, Bethesda, MD, USA, May 31 – June 2, 2009. ACM Press. doi:10.1145/1536414.1536440. 99

[GG98] Oded Goldreich and Shafi Goldwasser. On the limits of non-approximability of lattice problems. In *30th Annual ACM Symposium on Theory of Computing*, pages 1–9, Dallas, TX, USA, May 23–26, 1998. ACM Press. doi:10.1145/276698.276704. 104

[GGK03] Rosario Gennaro, Yael Gertner, and Jonathan Katz. Lower bounds on the efficiency of encryption and digital signature schemes. In *35th Annual ACM Symposium on Theory of Computing*, pages 417–425, San Diego, CA, USA, June 9–11, 2003. ACM Press. doi:10.1145/780542.780604. 144

[GGKT05] Rosario Gennaro, Yael Gertner, Jonathan Katz, and Luca Trevisan. Bounds on the efficiency of generic cryptographic constructions. *SIAM journal on Computing*, 35(1):217–246, 2005. 144

[GHLS15] Sevag Gharibian, Yichen Huang, Zeph Landau, and Seung Woo Shin. Quantum hamiltonian complexity. *Foundations and Trends® in Theoretical Computer Science*, 10(3):159–282, 2015. URL: http://dx.doi.org/10.1561/0400000066, doi:10.1561/0400000066. 72

[GIK+23] Riddhi Ghosal, Yuval Ishai, Alexis Korb, Eyal Kushilevitz, Paul Lou, and Amit Sahai. Hard languages in np ∩ conp and nizk proofs from unstructured hardness. In *Proceedings of the 55th Annual ACM Symposium on Theory of Computing*, pages 1243–1256, 2023. 85, 171

[Gir23] Steven M. Girvin. Introduction to quantum error correction and fault tolerance. *SciPost Physics Lecture Notes*, jun 2023. URL: https://arxiv.org/abs/2111.08894, doi:10.21468/scipostphyslectnotes.70. 72

[GK96] Oded Goldreich and Ariel Kahan. How to construct constant-round zero-knowledge proof systems for NP. *Journal of Cryptology*, 9(3):167–190, June 1996. 145

[GKM+00] Yael Gertner, Sampath Kannan, Tal Malkin, Omer Reingold, and Mahesh Viswanathan. The relationship between public key encryption and oblivious transfer. In *41st Annual Symposium on Foundations of Computer Science*, pages 325–335, Redondo Beach, CA, USA, November 12–14, 2000. IEEE Computer Society Press. doi:10.1109/SFCS.2000.892121. 143

[GKP18] Sanjam Garg, Susumu Kiyoshima, and Omkant Pandey. A new approach to black-box concurrent secure computation. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018, Part II*, volume 10821 of *Lecture Notes in Computer Science*, pages 566–599, Tel Aviv, Israel, April 29 – May 3, 2018. Springer, Heidelberg, Germany. doi:10.1007/978-3-319-78375-8_19. 142

[GLM23] Vipul Goyal, Xiao Liang, and Giulio Malavolta. On concurrent multi-party quantum computation. *Cryptology ePrint Archive*, 2023. 86, 171

[GM84] Shafi Goldwasser and Silvio Micali. Probabilistic encryption. *Journal of Computer and System Sciences*, 28(2):270–299, 1984. 92

[GMR89] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989. 142

[GMW91] Oded Goldreich, Silvio Micali, and Avi Wigderson. Proofs that yield nothing but their validity or all languages in NP have zero-knowledge proof systems. *Journal of the ACM*, 38(3):691–729, 1991. 142

[GO94] Oded Goldreich and Yair Oren. Definitions and properties of zero-knowledge proof systems. *Journal of Cryptology*, 7(1):1–32, December 1994. doi:10.1007/BF00195207. 142

[Gol08] O. Goldreich. *Computational Complexity: A Conceptual Perspective*. Cambridge University Press, 2008. URL: https://books.google.com/books?id=c-LlnQEACAAJ. 121

[GOS06a] Jens Groth, Rafail Ostrovsky, and Amit Sahai. Non-interactive zaps and new techniques for NIZK. In Cynthia Dwork, editor, *Advances in Cryptology – CRYPTO 2006*, volume 4117 of *Lecture Notes in Computer Science*, pages 97–111, Santa Barbara, CA, USA, August 20–24, 2006. Springer, Heidelberg, Germany. doi:10.1007/11818175_6. 124

[GOS06b] Jens Groth, Rafail Ostrovsky, and Amit Sahai. Perfect non-interactive zero knowledge for NP. In Serge Vaudenay, editor, *Advances in Cryptology – EUROCRYPT 2006*, volume 4004 of *Lecture Notes in Computer Science*, pages 339–358, St. Petersburg, Russia, May 28 – June 1, 2006. Springer, Heidelberg, Germany. doi:10.1007/11761679_21. 124

[GOSV14] Vipul Goyal, Rafail Ostrovsky, Alessandra Scafuro, and Ivan Visconti. Black-box non-black-box zero knowledge. In David B. Shmoys, editor, *46th Annual ACM Symposium on Theory of Computing*, pages 515–524, New York, NY, USA, May 31 – June 3, 2014. ACM Press. doi:10.1145/2591796.2591879. 140

[Got10] Daniel Gottesman. An introduction to quantum error correction and fault-tolerant quantum computation. In *Quantum information science and its contributions to mathematics, Proceedings of Symposia in Applied Mathematics*, volume 68, pages 13–58, 2010. The original paper appeared as arXiv:0904.2557. 70, 72

[GPV08] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In Richard E. Ladner and Cynthia Dwork, editors, *40th Annual ACM Symposium on Theory of Computing*, pages 197–206, Victoria, BC, Canada, May 17–20, 2008. ACM Press. doi:10.1145/1374376.1374407. 106

[GR08] Venkatesan Guruswami and Atri Rudra. Explicit codes achieving list decoding capacity: Error-correction with optimal redundancy. *IEEE Transactions on Information Theory*, 54(1):135–150, 2008. 118, 172

[Gro96] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *28th Annual ACM Symposium on Theory of Computing*, pages 212–219, Philadephia, PA, USA, May 22–24, 1996. ACM Press. doi:10.1145/237814.237866. 42

[GS86] Shafi Goldwasser and Michael Sipser. Private coins versus public coins in interactive proof systems. In *18th Annual ACM Symposium on Theory of Computing*, pages 59–68, Berkeley, CA, USA, May 28–30, 1986. ACM Press. doi:10.1145/12130.12137. 140

[GS08] Jens Groth and Amit Sahai. Efficient non-interactive proof systems for bilinear groups. In Nigel P. Smart, editor, *Advances in Cryptology – EUROCRYPT 2008*, volume 4965 of *Lecture Notes in Computer Science*, pages 415–432, Istanbul, Turkey, April 13–17, 2008. Springer, Heidelberg, Germany. doi:10.1007/978-3-540-78967-3_24. 31

[GSW13] Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013, Part I*, volume 8042 of *Lecture Notes in Computer Science*, pages 75–92, Santa Barbara, CA, USA, August 18–22, 2013. Springer, Heidelberg, Germany. doi:10.1007/978-3-642-40041-4_5. 106

[GT00] Rosario Gennaro and Luca Trevisan. Lower bounds on the efficiency of generic cryptographic constructions. In *41st Annual Symposium on Foundations of Computer Science*, pages 305–313, Redondo Beach, CA, USA, November 12–14, 2000. IEEE Computer Society Press. doi:10.1109/SFCS.2000.892119. 144

[GVW12] Sergey Gorbunov, Vinod Vaikuntanathan, and Hoeteck Wee. Functional encryption with bounded collusions via multi-party computation. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology – CRYPTO 2012*, volume 7417 of *Lecture Notes in Computer Science*, pages 162–179, Santa Barbara, CA, USA, August 19–23, 2012. Springer, Heidelberg, Germany. doi:10.1007/978-3-642-32009-5_11. 123

[Haj18] Mohammad Hajiabadi. Enhancements are blackbox non-trivial: Impossibility of enhanced trapdoor permutations from standard trapdoor permutations. In Amos Beimel and Stefan Dziembowski, editors, *TCC 2018: 16th Theory of Cryptography Conference, Part I*, volume 11239 of *Lecture Notes in Computer Science*, pages 448–475, Panaji, India, November 11–14, 2018. Springer, Heidelberg, Germany. doi:10.1007/978-3-030-03807-6_17. 78, 143

[Hås90] Johan Håstad. Pseudo-random generators under uniform assumptions. In *22nd Annual ACM Symposium on Theory of Computing*, pages 395–404, Baltimore, MD, USA, May 14–16, 1990. ACM Press. doi:10.1145/100216.100270. 142

[HHPS11] Shai Halevi, Danny Harnik, Benny Pinkas, and Alexandra Shulman-Peleg. Proofs of ownership in remote storage systems. In Yan Chen, George Danezis, and Vitaly Shmatikov, editors, *ACM CCS 2011: 18th Conference on Computer and Communications Security*, pages 491–500, Chicago, Illinois, USA, October 17–21, 2011. ACM Press. doi:10.1145/2046707.2046765. 95, 96

[HL18] Justin Holmgren and Alex Lombardi. Cryptographic hashing from strong one-way functions (or: One-way product functions and their applications). In Mikkel Thorup, editor, *59th Annual Symposium on Foundations of Computer Science*, pages 850–858, Paris, France, October 7–9, 2018. IEEE Computer Society Press. doi:10.1109/FOCS.2018.00085. 95

[HLWW13] Carmit Hazay, Adriana López-Alt, Hoeteck Wee, and Daniel Wichs. Leakage-resilient cryptography from minimal assumptions. In Thomas Johansson and Phong Q. Nguyen, editors, *Advances in Cryptology – EUROCRYPT 2013*, volume 7881 of *Lecture Notes in Computer Science*, pages 160–176, Athens, Greece, May 26–30, 2013. Springer, Heidelberg, Germany. doi:10.1007/978-3-642-38348-9_10. 99

[Hoe63] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, March 1963. URL: http://www.jstor.org/stable/2282952? 86

[HPSS08] Jeffrey Hoffstein, Jill Pipher, Joseph H Silverman, and Joseph H Silverman. *An introduction to mathematical cryptography*, volume 1. Springer, 2008. 104

[HPV77] John Hopcroft, Wolfgang Paul, and Leslie Valiant. On time versus space. *Journal of the ACM (JACM)*, 24(2):332–337, 1977. 132

[HR04] Chun-Yuan Hsiao and Leonid Reyzin. Finding collisions on a public road, or do secure hash functions need secret coins? In Matthew Franklin, editor, *Advances in Cryptology – CRYPTO 2004*, volume 3152 of *Lecture Notes in Computer Science*, pages 92–105, Santa Barbara, CA, USA, August 15–19, 2004. Springer, Heidelberg, Germany. doi:10.1007/978-3-540-28628-8_6. 78, 143

[HR07] Ishay Haviv and Oded Regev. Tensor-based hardness of the shortest vector problem to within almost polynomial factors. In David S. Johnson and Uriel Feige, editors, *39th Annual ACM Symposium on Theory of Computing*, pages 469–477, San Diego, CA, USA, June 11–13, 2007. ACM Press. doi:10.1145/1250790.1250859. 104

[HRT03] Sean Hallgren, Alexander Russell, and Amnon Ta-Shma. The hidden subgroup problem and quantum computation using group representations. *SIAM J. Comput.*, 32(4):916–934, 2003. doi:10.1137/S009753970139450X. 136

[HS65] Juris Hartmanis and Richard E Stearns. On the computational complexity of algorithms. *Transactions of the American Mathematical Society*, 117:285–306, 1965. 127, 131

[HS66] Fred C Hennie and Richard Edwin Stearns. Two-tape simulation of multitape turing machines. *Journal of the ACM (JACM)*, 13(4):533–546, 1966. 122

[HY20] Akinori Hosoyamada and Takashi Yamakawa. Finding collisions in a quantum world: Quantum black-box separation of collision-resistance and one-wayness. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2020, Part I*, volume 12491 of *Lecture Notes in Computer Science*, pages 3–32, Daejeon, South Korea, December 7–11, 2020. Springer, Heidelberg, Germany. doi:10.1007/978-3-030-64837-4_1. 143

[HZ11] Teiko Heinosaari and Mário Ziman. *The mathematical language of quantum theory: from uncertainty to entanglement*. Cambridge University Press, 2011. 53

[ILL89] Russell Impagliazzo, Leonid A. Levin, and Michael Luby. Pseudo-random generation from one-way functions (extended abstracts). In *21st Annual ACM Symposium on Theory of Computing*, pages 12–24, Seattle, WA, USA, May 15–17, 1989. ACM Press. doi:10.1145/73007.73009. 99, 142

[Imm88] Neil Immerman. Nondeterministic space is closed under complementation. *SIAM Journal on computing*, 17(5):935–938, 1988. 130

[IP01] Russell Impagliazzo and Ramamohan Paturi. On the complexity of k-sat. *Journal of Computer and System Sciences*, 62(2):367–375, 2001. 2

[IR89] Russell Impagliazzo and Steven Rudich. Limits on the provable consequences of one-way permutations. In *21st Annual ACM Symposium on Theory of Computing*, pages 44–61, Seattle, WA, USA, May 15–17, 1989. ACM Press. doi:10.1145/73007.73012. 16, 143, 144

[IW14] Yuval Ishai and Mor Weiss. Probabilistically checkable proofs of proximity with zero-knowledge. In Yehuda Lindell, editor, *TCC 2014: 11th Theory of Cryptography Conference*, volume 8349 of *Lecture Notes in Computer Science*, pages 121–145, San Diego, CA, USA, February 24–26, 2014. Springer, Heidelberg, Germany. doi:10.1007/978-3-642-54242-8_6. 140

[JJUW10] Rahul Jain, Zhengfeng Ji, Sarvagya Upadhyay, and John Watrous. QIP = PSPACE. In Leonard J. Schulman, editor, *Proceedings of the 42nd ACM Symposium on Theory of Computing, STOC 2010, Cambridge, Massachusetts, USA, 5-8 June 2010*, pages 573–582. ACM, 2010. doi:10.1145/1806689.1806768. 135

[JLS21] Aayush Jain, Huijia Lin, and Amit Sahai. Indistinguishability obfuscation from well-founded assumptions. In Samir Khuller and Virginia Vassilevska Williams, editors, *STOC '21: 53rd Annual ACM SIGACT Symposium on Theory of Computing, Virtual Event, Italy, June 21-25, 2021*, pages 60–73. ACM, 2021. 126

[Jor70] Camille Jordan. *Traité des substitutions et des équations algébriques*, volume 1. Gauthier-Villars, 1870. 50, 65

[Jor75] Camille Jordan. Essai sur la géométrie à n dimensions. *Bulletin de la Société mathématique de France*, 3:103–174, 1875. iii, 65

[Jor93] Camille Jordan. *Cours d'analyse de l'École polytechnique*, volume 1. Gauthier-Villars et fils, 1893. 66

[Kan83] Ravi Kannan. Improved algorithms for integer programming and related lattice problems. In *15th Annual ACM Symposium on Theory of Computing*, pages 193–206, Boston, MA, USA, April 25–27, 1983. ACM Press. doi:10.1145/800061.808749. 104

[Kar72] Richard M Karp. Reducibility among combinatorial problems. In *Complexity of computer computations*, pages 85–103. Springer, 1972. 129

[Kho04] Subhash Khot. Hardness of approximating the shortest vector problem in lattices. In *45th Annual Symposium on Foundations of Computer Science*, pages 126–135, Rome, Italy, October 17–19, 2004. IEEE Computer Society Press. doi:10.1109/FOCS.2004.31. 104

[Kil92] Joe Kilian. A note on efficient zero-knowledge proofs and arguments (extended abstract). In *24th Annual ACM Symposium on Theory of Computing*, pages 723–732, Victoria, BC, Canada, May 4–6, 1992. ACM Press. doi:10.1145/129712.129782. 96

[Kil94] Joe Kilian. On the complexity of bounded-interaction and noninteractive zero-knowledge proofs. In *35th Annual Symposium on Foundations of Computer Science*, pages 466–477, Santa Fe, NM, USA, November 20–22, 1994. IEEE Computer Society Press. doi:10.1109/SFCS.1994.365744. 95

[Kit96] Alexei Y. Kitaev. Quantum measurements and the abelian stabilizer problem. *Electron. Colloquium Comput. Complex.*, (3), 1996. URL: https://eccc.weizmann.ac.il/eccc-reports/1996/TR96-003/index.html. 136

[KK20] Nathan Keller and Ohad Klein. Proof of tomaszewski's conjecture on randomly signed sums, 2020. arXiv:2006.16834. 83

[KKNY05] Akinori Kawachi, Takeshi Koshiba, Harumichi Nishimura, and Tomoyuki Yamakami. Computational indistinguishability between quantum states and its cryptographic application. In Ronald Cramer, editor, *Advances in Cryptology – EUROCRYPT 2005*, volume 3494 of *Lecture Notes in Computer Science*, pages 268–284, Aarhus, Denmark, May 22–26, 2005. Springer, Heidelberg, Germany. doi:10.1007/11426639_16. 136, 137

[KL82] Richard M. Karp and Richard J. Lipton. Turing machines that take advice. *L'Ensignement Mathématique*, 28:191–210, 1982. 133, 134

[KL14] Jonathan Katz and Yehuda Lindell. *Introduction to modern cryptography*. Chapman and Hall/CRC, 2014. 28

[KLM06] Phillip Kaye, Raymond Laflamme, and Michele Mosca. *An Introduction to Quantum Computing*. Oxford University Press, 2006. 41, 67, 71

[KO97] Eyal Kushilevitz and Rafail Ostrovsky. Replication is NOT needed: SINGLE database, computationally-private information retrieval. In *38th Annual Symposium on Foundations of Computer Science*, pages 364–373, Miami Beach, Florida, October 19–22, 1997. IEEE Computer Society Press. doi:10.1109/SFCS.1997.646125. 92, 93, 171

[KOS18] Dakshita Khurana, Rafail Ostrovsky, and Akshayaram Srinivasan. Round optimal black-box "commit-and-prove". In Amos Beimel and Stefan Dziembowski, editors, *TCC 2018: 16th Theory of Cryptography Conference, Part I*, volume 11239 of *Lecture Notes in Computer Science*, pages 286–313, Panaji, India, November 11–14, 2018. Springer, Heidelberg, Germany. doi:10.1007/978-3-030-03807-6_11. v, 35, 119

[Kre89] E. Kreyszig. *Introductory Functional Analysis with Applications*. Wiley Classics Library. Wiley, 1989. URL: https://books.google.co.jp/books?id=nZmpQgAACAAJ. 8, 13, 22, 23, 24

[KRS09] Robert Konig, Renato Renner, and Christian Schaffner. The operational meaning of min-and max-entropy. *IEEE Transactions on Information theory*, 55(9):4337–4347, 2009. URL: https://arxiv.org/abs/0807.1338. 71

[Kup05] Greg Kuperberg. A subexponential-time quantum algorithm for the dihedral hidden subgroup problem. *SIAM Journal on Computing*, 35(1):170–188, 2005. 2

[KW00] Alexei Kitaev and John Watrous. Parallelization, amplification, and exponential time simulation of quantum interactive proof systems. In *32nd Annual ACM Symposium on Theory of Computing*, pages 608–617, Portland, OR, USA, May 21–23, 2000. ACM Press. doi:10.1145/335305.335387. 135

[KW17] Sam Kim and David J. Wu. Watermarking cryptographic functionalities from standard lattice assumptions. In Jonathan Katz and Hovav Shacham, editors, *Advances in Cryptology – CRYPTO 2017, Part I*, volume 10401 of *Lecture Notes in Computer Science*, pages 503–536, Santa Barbara, CA, USA, August 20–24, 2017. Springer, Heidelberg, Germany. doi:10.1007/978-3-319-63688-7_17. 107

[KW19] Sam Kim and David J. Wu. Watermarking PRFs from lattices: Stronger security via extractable PRFs. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part III*, volume 11694 of *Lecture Notes in Computer Science*, pages 335–366, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Heidelberg, Germany. doi:10.1007/978-3-030-26954-8_11. 107

[Lad75] Richard E Ladner. On the structure of polynomial time reducibility. *Journal of the ACM (JACM)*, 22(1):155–171, 1975. 132

[Lev73] Leonid Anatolevich Levin. Universal sequential search problems. *Problemy peredachi informatsii*, 9(3):115–116, 1973. 129

[Lin17] Yehuda Lindell, editor. *Tutorials on the Foundations of Cryptography*. Springer International Publishing, 2017. doi:10.1007/978-3-319-57048-8. 119

[LLL82] Arjen Klaas Lenstra, Hendrik Willem Lenstra, and László Lovász. Factoring polynomials with rational coefficients. *Mathematische Annalen*, 261(4):515–534, 1982. 104

[LPR10] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In Henri Gilbert, editor, *Advances in Cryptology – EUROCRYPT 2010*, volume 6110 of *Lecture Notes in Computer Science*, pages 1–23, French Riviera, May 30 – June 3, 2010. Springer, Heidelberg, Germany. doi:10.1007/978-3-642-13190-5_1. 109

[LPR13] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. A toolkit for ring-LWE cryptography. In Thomas Johansson and Phong Q. Nguyen, editors, *Advances in Cryptology – EUROCRYPT 2013*, volume 7881 of *Lecture Notes in Computer Science*, pages 35–54, Athens, Greece, May 26–30, 2013. Springer, Heidelberg, Germany. doi:10.1007/978-3-642-38348-9_3. 109

[LPV08] Huijia Lin, Rafael Pass, and Muthuramakrishnan Venkitasubramaniam. Concurrent non-malleable commitments from any one-way function. In Ran Canetti, editor, *TCC 2008: 5th Theory of Cryptography Conference*, volume 4948 of *Lecture Notes in Computer Science*, pages 571–588, San Francisco, CA, USA, March 19–21, 2008. Springer, Heidelberg, Germany. doi:10.1007/978-3-540-78524-8_31. 142

[Mar21] T Mart. How do i introduce schrödinger equation during the quantum mechanics course? *Physics Education*, 56(2):025012, 2021. 72

[Mel24] Antonio Anna Mele. Introduction to haar measure tools in quantum information: A beginner's tutorial. *Quantum*, 8:1340, May 2024. URL: http://dx.doi.org/10.22331/q-2024-05-08-1340, doi:10.22331/q-2024-05-08-1340. 13

[MG02] Daniele Micciancio and Shafi Goldwasser. *Complexity of lattice problems - a cryptograhic perspective*, volume 671 of *The Kluwer international series in engineering and computer science*. Springer, 2002. 115

[Mic98] Daniele Micciancio. The shortest vector in a lattice is hard to approximate to within some constant. In *39th Annual Symposium on Foundations of Computer Science*, pages 92–98, Palo Alto, CA, USA, November 8–11, 1998. IEEE Computer Society Press. doi:10.1109/SFCS.1998.743432. 104

[Mil01] Peter Bro Miltersen. Derandomizing complexity classes. *COMBINATORIAL OPTIMIZATION-DORDRECHT-*, 9(2):843–941, 2001. 2

[MLDS+13] Martin Müller-Lennert, Frédéric Dupuis, Oleg Szehr, Serge Fehr, and Marco Tomamichel. On quantum rényi entropies: A new generalization and some properties. *Journal of Mathematical Physics*, 54(12):122203, 2013. 91, 171

[MM11] Daniele Micciancio and Petros Mol. Pseudorandom knapsacks and the sample complexity of LWE search-to-decision reductions. In Phillip Rogaway, editor, *Advances in Cryptology – CRYPTO 2011*, volume 6841 of *Lecture Notes in Computer Science*, pages 465–484, Santa Barbara, CA, USA, August 14–18, 2011. Springer, Heidelberg, Germany. doi:10.1007/978-3-642-22792-9_26. 106

[MP12] Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 700–718, Cambridge, UK, April 15–19, 2012. Springer, Heidelberg, Germany. doi:10.1007/978-3-642-29011-4_41. 106

[MP13] Daniele Micciancio and Chris Peikert. Hardness of SIS and LWE with small parameters. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013, Part I*, volume 8042 of *Lecture Notes in Computer Science*, pages 21–39, Santa Barbara, CA, USA, August 18–22, 2013. Springer, Heidelberg, Germany. doi:10.1007/978-3-642-40041-4_2. 106

[MR04] Daniele Micciancio and Oded Regev. Worst-case to average-case reductions based on Gaussian measures. In *45th Annual Symposium on Foundations of Computer Science*, pages 372–381, Rome, Italy, October 17–19, 2004. IEEE Computer Society Press. doi:10.1109/FOCS.2004.72. 106

[MS16] Carl A Miller and Yaoyun Shi. Robust protocols for securely expanding randomness and distributing keys using untrusted quantum devices. *Journal of the ACM (JACM)*, 63(4):1–63, 2016. 91, 171

[Mun84] James R. Munkres. *Elements of algebraic topology*. Addison-Wesley, 1984. 12

[MV10] Daniele Micciancio and Panagiotis Voulgaris. A deterministic single exponential time algorithm for most lattice problems based on voronoi cell computations. In Leonard J. Schulman, editor, *42nd Annual ACM Symposium on Theory of Computing*, pages 351–358, Cambridge, MA, USA, June 5–8, 2010. ACM Press. doi:10.1145/1806689.1806738. 104

[NC11] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, USA, 10th edition, 2011. 46, 47, 54, 59, 60, 61, 64, 67, 70, 71

[NW94] Noam Nisan and Avi Wigderson. Hardness vs randomness. *Journal of computer and System Sciences*, 49(2):149–167, 1994. 74

[NWZ09] Daniel Nagaj, Pawel Wocjan, and Yong Zhang. Fast amplification of QMA. *Quantum Inf. Comput.*, 9(11&12):1053–1068, 2009. doi:10.26421/QIC9.11-12-8. 65

[NY89] Moni Naor and Moti Yung. Universal one-way hash functions and their cryptographic applications. In *21st Annual ACM Symposium on Theory of Computing*, pages 33–43, Seattle, WA, USA, May 15–17, 1989. ACM Press. doi:10.1145/73007.73011. 95, 97

[NZ96] Noam Nisan and David Zuckerman. Randomness is linear in space. *Journal of Computer and System Sciences*, 52(1):43–52, 1996. 101

[Pai99] Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In Jacques Stern, editor, *Advances in Cryptology – EUROCRYPT'99*, volume 1592 of *Lecture Notes in Computer Science*, pages 223–238, Prague, Czech Republic, May 2–6, 1999. Springer, Heidelberg, Germany. doi:10.1007/3-540-48910-X_16. 93

[Pas03] Rafael Pass. Simulation in quasi-polynomial time, and its application to protocol composition. In Eli Biham, editor, *Advances in Cryptology – EUROCRYPT 2003*, volume 2656 of *Lecture Notes in Computer Science*, pages 160–176, Warsaw, Poland, May 4–8, 2003. Springer, Heidelberg, Germany. doi:10.1007/3-540-39200-9_10. 2

[Pas04] Rafael Pass. Alternative variants of zero-knowledge proofs. Master's thesis, Royal Institute of Technology, Stockholm, Sweden, 2004. 2

[Pei09] Chris Peikert. Public-key cryptosystems from the worst-case shortest vector problem: extended abstract. In Michael Mitzenmacher, editor, *41st Annual ACM Symposium on Theory of Computing*, pages 333–342, Bethesda, MD, USA, May 31 – June 2, 2009. ACM Press. doi:10.1145/1536414.1536461. 106

[Pei15] Chris Peikert. A decade of lattice cryptography. Cryptology ePrint Archive, Report 2015/939, 2015. https://eprint.iacr.org/2015/939. 103, 104, 109

[PW08] Chris Peikert and Brent Waters. Lossy trapdoor functions and their applications. In Richard E. Ladner and Cynthia Dwork, editors, *40th Annual ACM Symposium on Theory of Computing*, pages 187–196, Victoria, BC, Canada, May 17–20, 2008. ACM Press. doi:10.1145/1374376.1374406. 100

[Reg04] Oded Regev. A subexponential time algorithm for the dihedral hidden subgroup problem with polynomial space. *arXiv preprint quant-ph/0406151*, 2004. 2

[Reg05] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In Harold N. Gabow and Ronald Fagin, editors, *37th Annual ACM Symposium on Theory of Computing*, pages 84–93, Baltimore, MA, USA, May 22–24, 2005. ACM Press. doi:10.1145/1060590.1060603. 106, 107

[Ren08] Renato Renner. Security of quantum key distribution. *International Journal of Quantum Information*, 6(01):1–127, 2008. URL: https://arxiv.org/abs/quant-ph/0512258. 42, 72, 91, 171

[RK05] Renato Renner and Robert König. Universally composable privacy amplification against quantum adversaries. In Joe Kilian, editor, *TCC 2005: 2nd Theory of Cryptography Conference*, volume 3378 of *Lecture Notes in Computer Science*, pages 407–425, Cambridge, MA, USA, February 10–12, 2005. Springer, Heidelberg, Germany. doi:10.1007/978-3-540-30576-7_22. 71

[RTV04] Omer Reingold, Luca Trevisan, and Salil P. Vadhan. Notions of reducibility between cryptographic primitives. In Moni Naor, editor, *TCC 2004: 1st Theory of Cryptography Conference*, volume 2951 of *Lecture Notes in Computer Science*, pages 1–20, Cambridge, MA, USA, February 19–21, 2004. Springer, Heidelberg, Germany. doi:10.1007/978-3-540-24638-1_1. v, 141, 142, 143, 144, 147

[Sah99] Amit Sahai. Non-malleable non-interactive zero knowledge and adaptive chosen-ciphertext security. In *40th Annual Symposium on Foundations of Computer Science*, pages 543–553, New York, NY, USA, October 17–19, 1999. IEEE Computer Society Press. doi:10.1109/SFFCS.1999.814628. ii, 36, 170

[Sav70] Walter J Savitch. Relationships between nondeterministic and deterministic tape complexities. *Journal of computer and system sciences*, 4(2):177–192, 1970. 131

[Sch87] Claus-Peter Schnorr. A hierarchy of polynomial time lattice basis reduction algorithms. *Theoretical computer science*, 53(2-3):201–224, 1987. 104

[Sha49] Claude E Shannon. The synthesis of two-terminal switching circuits. *The Bell System Technical Journal*, 28(1):59–98, 1949. 128

[SHL65] Richard Edwin Stearns, Juris Hartmanis, and Philip M Lewis. Hierarchies of memory limited computations. In *6th Annual Symposium on Switching Circuit Theory and Logical Design (SWCT 1965)*, pages 179–190. IEEE, 1965. 128

[Sho94] Peter W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *35th Annual Symposium on Foundations of Computer Science*, pages 124–134, Santa Fe, NM, USA, November 20–22, 1994. IEEE Computer Society Press. doi:10.1109/SFCS.1994.365700. 42

[Sho99] Peter W Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM review*, 41(2):303–332, 1999. 104

[Sho08] Victor Shoup. *A computational introduction to number theory and algebra*. Version 2, 2008. URL: https://www.shoup.net/ntb/. 34, 94

[Sim94] Daniel R. Simon. On the power of quantum computation. In *35th Annual Symposium on Foundations of Computer Science*, pages 116–123, Santa Fe, NM, USA, November 20–22, 1994. IEEE Computer Society Press. doi:10.1109/SFCS.1994.365701. 42

[Sim98] Daniel R. Simon. Finding collisions on a one-way street: Can secure hash functions be based on general assumptions? In Kaisa Nyberg, editor, *Advances in Cryptology – EUROCRYPT'98*, volume 1403 of *Lecture Notes in Computer Science*, pages 334–345, Espoo, Finland, May 31 – June 4, 1998. Springer, Heidelberg, Germany. doi:10.1007/BFb0054137. 143

[Sip12] Michael Sipser. *Introduction to the Theory of Computation*. Cengage learning, 2012. 129

[SM73] LJ Stockmeyer and AR Meyer. Word problems requiring exponential time: preliminary report, in5th symp. on theory of computing'. 1973. 129

[SW05] Amit Sahai and Brent R. Waters. Fuzzy identity-based encryption. In Ronald Cramer, editor, *Advances in Cryptology – EUROCRYPT 2005*, volume 3494 of *Lecture Notes in Computer Science*, pages 457–473, Aarhus, Denmark, May 22–26, 2005. Springer, Heidelberg, Germany. doi:10.1007/11426639_27. 36

[Sze87] Róbert Szelepcsényi. The method of forcing for nondeterministic automata. *Bull. European Associ. Theoret. Compur. Sci*, 33:96100, 1987. 130

[Tao11] T. Tao. *An Introduction to Measure Theory*. Graduate studies in mathematics. American Mathematical Society, 2011. URL: https://books.google.com/books?id=HoGDAwAAQBAJ. 14, 16

[Tha20] Justin Thaler. Proofs, arguments, and zero-knowledge, 2020. Lecture Notes for COSC 544 at Georgetown University,. URL: http://people.cs.georgetown.edu/jthaler/ProofsArgsAndZK.html. 123

[TL17] Marco Tomamichel and Anthony Leverrier. A largely self-contained and complete security proof for quantum key distribution. *Quantum*, 1:14, 2017. 72

[Tom12] Marco Tomamichel. *A framework for non-asymptotic quantum information theory*. PhD thesis, Eidgenössische Technische Hochschule (ETH) Zürich, Zürich, Switzerland, 2012. URL: https://arxiv.org/abs/1203.2142. 72

[Tom16] Marco Tomamichel. *Quantum Information Processing with Finite Resources.* Springer International Publishing, 2016. URL: https://arxiv.org/abs/1504.00233, doi:10.1007/978-3-319-21891-5. 71

[Vad12] Salil P. Vadhan. Pseudorandomness. *Foundations and Trends® in Theoretical Computer Science*, 7(1–3):1–336, 2012. 98

[VV12] Umesh V. Vazirani and Thomas Vidick. Certifiable quantum dice: or, true random number generation secure against quantum adversaries. In Howard J. Karloff and Toniann Pitassi, editors, *44th Annual ACM Symposium on Theory of Computing*, pages 61–76, New York, NY, USA, May 19–22, 2012. ACM Press. doi:10.1145/2213977.2213984. 91, 171

[Wat18] John Watrous. *The theory of quantum information.* Cambridge university press, 2018. 59

[WC81] Mark N. Wegman and J. Lawrence Carter. New hash functions and their use in authentication and set equality. *Journal of computer and system sciences*, 22(3):265–279, 1981. 97, 98, 172

[Wee10] Hoeteck Wee. Black-box, round-efficient secure computation via non-malleability amplification. In *51st Annual Symposium on Foundations of Computer Science*, pages 531–540, Las Vegas, NV, USA, October 23–26, 2010. IEEE Computer Society Press. doi:10.1109/FOCS.2010.87. 2, 170

[Wil11] Mark M. Wilde. From classical to quantum shannon theory. *CoRR*, abs/1106.1445, 2011. URL: http://arxiv.org/abs/1106.1445, arXiv:1106.1445. 53, 58, 59, 60, 71

[Wil17] Mark M. Wilde. *Quantum Information Theory.* Cambridge University Press, 2017. doi:10.1017/9781316809976. 71

[Wol21] Ramona Wolf. Quantum key distribution protocols. In *Quantum Key Distribution*, pages 91–116. Springer, 2021. 91, 171

[Yao82] Andrew Chi-Chih Yao. Theory and applications of trapdoor functions (extended abstract). In *23rd Annual Symposium on Foundations of Computer Science*, pages 80–91, Chicago, Illinois, November 3–5, 1982. IEEE Computer Society Press. doi:10.1109/SFCS.1982.45. 80, 171

[Yer11] Arkady Boris Yerukhimovich. *A study of separations in cryptography: new results and new models.* PhD thesis, University of Maryland, College Park, Maryland, USA, 2011. 141

[YZ22] Takashi Yamakawa and Mark Zhandry. Verifiable quantum advantage without structure, 2022. URL: https://arxiv.org/abs/2204.02063, doi:10.48550/ARXIV.2204.02063. 118, 172

[Zha19] Mark Zhandry. How to record quantum queries, and applications to quantum indifferentiability. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part II*, volume 11693 of *Lecture Notes in Computer Science*, pages 239–268, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Heidelberg, Germany. doi:10.1007/978-3-030-26951-7_9. 63

FiXme Information

List of Corrections